

Manual de Segurança da Informação

Do básico ao avançado — fundamentos, ataque, defesa e governança

Vitor Mattos

30/07/2026

Índice

Apresentação	1
Para quem é este manual	1
Como o manual está organizado	2
Seções fixas de cada capítulo	3
Política editorial	3
Licença	3
I. Volume 1 — Fundamentos da Segurança da Informação	5
1. O que é segurança da informação	7
1.1. Uma definição de trabalho	7
1.2. Dado, informação e ativo: o que exatamente se protege	8
1.3. Segurança não é um estado, é um processo	9
1.4. Pessoas, processos e tecnologia	10
1.5. O adversário não é só o hacker	10
1.6. Quem ataca, e por que o pequeno também é alvo	11
1.7. Os princípios que sustentam qualquer defesa	12
1.8. O vocabulário mínimo	13
1.9. Como se sabe se está funcionando	13
1.10. Brasil em Foco	14
1.11. No Campo	15
1.12. Resumo	16
2. Os pilares: confidencialidade, integridade e disponibilidade	17
2.1. As três propriedades, com precisão	17
2.2. Confidencialidade	17
2.3. Integridade	19
2.4. Disponibilidade	20
2.5. As três colidem	21
2.6. A ordem de prioridade muda com o contexto	22
2.7. Além da tríade	22
2.8. Brasil em Foco	23
2.9. No Campo	23
2.10. Resumo	25
3. Autenticidade, não repúdio e responsabilização	27
3.1. Quatro coisas diferentes que se confundem	27
3.2. Autenticidade	28
3.3. Não repúdio	29
3.4. Responsabilização	30
3.5. Como as seis propriedades se encaixam	31
3.6. Brasil em Foco	32
3.7. No Campo	33

3.8. Resumo	35
4. Ativos, ameaças, vulnerabilidades e risco	37
4.1. A cadeia que liga os quatro termos	37
4.2. Ativos: o inventário é o alicerce	38
4.3. Ameaças: quem ou o quê, e com que capacidade	39
4.4. Vulnerabilidades: quatro tipos, não um	39
4.5. Risco: juntando tudo	41
4.6. Brasil em Foco	43
4.7. No Campo	44
4.8. Resumo	45
5. Modelagem de ameaças: STRIDE, árvores de ataque e abuse cases	47
5.1. O que é, e quando fazer	47
5.2. Pergunta 1: desenhar o sistema	48
5.3. Pergunta 2: o que pode dar errado	49
5.4. Outras abordagens, e quando usá-las	51
5.5. Pergunta 3: o que fazer, e pergunta 4: fizemos bem?	51
5.6. Brasil em Foco	52
5.7. No Campo	53
5.8. Resumo	54
6. Gestão de risco: análise qualitativa e quantitativa	55
6.1. O processo	55
6.2. Análise qualitativa	56
6.3. Análise quantitativa	57
6.4. Lidando com a incerteza: intervalos em vez de pontos	58
6.5. Escolher entre as duas	60
6.6. Tratamento e risco residual	60
6.7. Brasil em Foco	61
6.8. No Campo	62
6.9. Resumo	63
7. Ética, legalidade e a fronteira do escopo autorizado	65
7.1. A linha é jurídica, não técnica	65
7.2. O mapa legal brasileiro	66
7.3. Autorização: como ela é feita de verdade	67
7.4. Divulgação de vulnerabilidade	68
7.5. Bug bounty e programas de recompensa	69
7.6. Ética profissional além da lei	69
7.7. Onde praticar sem risco	70
7.8. Brasil em Foco	70
7.9. No Campo	71
7.10. Resumo	72
II. Volume 2 — Redes e Sistemas: a Base Técnica	75
8. A pilha TCP/IP sob a ótica do atacante e do defensor	77
8.1. Duas pilhas, um vocabulário	77
8.2. Camada 2: onde a confiança é total	78
8.3. Camada 3: endereços que ninguém verifica	79

8.4. Camada 4: o aperto de mão e seus abusos	80
8.5. O pecado original: confiança presumida	82
8.6. Onde cada defesa mora	83
8.7. Brasil em Foco	83
8.8. No Campo	84
8.9. Resumo	85
9. Endereçamento, roteamento e NAT	87
9.1. Endereçamento IPv4 e a matemática que não dá para evitar	87
9.2. IPv6: o que muda de verdade	89
9.3. Roteamento: como o pacote encontra o caminho	89
9.4. NAT: o que ele faz e o que ele não faz	90
9.5. Brasil em Foco	92
9.6. No Campo	92
9.7. Resumo	94
10. Protocolos de aplicação e seus riscos: HTTP, DNS, SMTP e SNMP	95
10.1. O padrão comum: texto claro e confiança presumida	95
10.2. HTTP: simples por fora, complexo por dentro	96
10.3. DNS: a lista telefônica que qualquer um pode adulterar	97
10.4. SMTP: o protocolo que confia no remetente	99
10.5. SNMP: a porta dos fundos do parque de equipamentos	100
10.6. O inventário de protocolos: o exercício que ninguém faz	100
10.7. Brasil em Foco	101
10.8. No Campo	101
10.9. Resumo	103
11. Fundamentos de sistemas operacionais: processos, permissões e memória	105
11.1. O anel de proteção: a fronteira que sustenta tudo	105
11.2. Processos: identidade, herança e contexto	106
11.3. Permissões: dois modelos para o mesmo problema	107
11.4. Memória: onde a exploração acontece	107
11.5. Isolamento: quando o processo não basta	109
11.6. Lendo o que a máquina conta	110
11.7. Brasil em Foco	110
11.8. No Campo	111
11.9. Resumo	113
12. Linux essencial para segurança	115
12.1. O sistema de arquivos como mapa mental	115
12.2. Como a máquina arranca — e onde a persistência mora	116
12.3. Usuários, grupos e o que o sudo realmente permite	116
12.4. SSH: a porta principal	117
12.5. Registros: o que o sistema conta	118
12.6. Rede vista de dentro da máquina	119
12.7. Os quinze comandos da triagem	119
12.8. Endurecimento: a lista curta que vale a pena	120
12.9. Brasil em Foco	120
12.10. No Campo	121
12.11. Resumo	123

13. Windows e Active Directory essencial	125
13.1. Como o Windows pensa	125
13.2. Active Directory: o mapa da confiança	126
13.3. Autenticação: NTLM, Kerberos e a memória da estação	126
13.4. Os caminhos de escalada que existem sem falha nenhuma	128
13.5. Registros e detecção: o que ligar antes de precisar	129
13.6. Brasil em Foco	130
13.7. No Campo	130
13.8. Resumo	132
14. Virtualização e montagem do laboratório de estudos	133
14.1. Virtualização: o que a fronteira realmente vale	133
14.2. O laboratório mínimo	134
14.3. O que instalar, em que ordem	135
14.4. Plataformas de treino: o complemento	136
14.5. Um roteiro de exercícios que percorre o volume	137
14.6. Brasil em Foco	137
14.7. No Campo	138
14.8. Resumo	140
III. Volume 3 — Criptografia Aplicada	141
15. História e princípios da criptografia	143
15.1. O vocabulário mínimo	143
15.2. Da substituição à análise de frequência	144
15.3. O princípio de Kerckhoffs	145
15.4. Shannon: sigilo perfeito e seu preço	145
15.5. Confusão, difusão e o que uma boa cifra faz	146
15.6. O que o adversário pode fazer	146
15.7. As três famílias de primitivas	147
15.8. Aleatoriedade: a matéria-prima	147
15.9. O que a criptografia não resolve	148
15.10. Brasil em Foco	149
15.11. No Campo	149
15.12. Resumo	151
16. Criptografia simétrica: AES, modos de operação e AEAD	153
16.1. O que uma cifra de bloco faz — e o que ela não faz	153
16.2. Modos de operação: onde se decide o destino	154
16.3. Cifrar não é autenticar	156
16.4. AEAD: a resposta certa	157
16.5. Como escolher, em uma tabela	159
16.6. Brasil em Foco	159
16.7. No Campo	160
16.8. Resumo	162
17. Funções hash, HMAC e integridade	163
17.1. O que uma função hash faz	163
17.2. A linha do tempo, e onde estamos	164
17.3. Hash não é criptografia, e hash não é anonimização	165
17.4. Length extension: a peculiaridade que matou o MAC caseiro	165

17.5. MAC e HMAC: integridade com autenticidade	166
17.6. Derivar chave e guardar senha: usos opostos	167
17.7. Árvores de Merkle: integridade que escala	168
17.8. Brasil em Foco	168
17.9. No Campo	169
17.10Resumo	171
18.Criptografia assimétrica: RSA, Diffie-Hellman e curvas elípticas	173
18.1. O problema que a assimétrica resolve	173
18.2. Diffie-Hellman: acordar uma chave em público	174
18.3. RSA: cifrar e assinar com o mesmo par	175
18.4. Curvas elípticas: mesma segurança, chaves menores	176
18.5. Sigilo encaminhado: por que “efêmero” importa	177
18.6. Como tudo se combina, na prática	177
18.7. Brasil em Foco	178
18.8. No Campo	179
18.9. Resumo	180
19.PKI, certificados digitais e cadeias de confiança	183
19.1. O problema que a PKI resolve	183
19.2. O certificado X.509	184
19.3. A cadeia de confiança	185
19.4. Como um certificado é emitido	185
19.5. A falha estrutural, e as três salvaguardas	186
19.6. Revogação: a parte que quase não funciona	187
19.7. PKI interna e autenticação mútua	187
19.8. Brasil em Foco	188
19.9. No Campo	189
19.10Resumo	191
20.TLS na prática: handshake, versões e configuração segura	193
20.1. O que o TLS garante, e onde ele fica	193
20.2. O handshake do TLS 1.3	194
20.3. Como ler uma suíte de cifras	196
20.4. O que cada ataque histórico ensinou	196
20.5. Configuração segura, em concreto	197
20.6. Brasil em Foco	198
20.7. No Campo	199
20.8. Resumo	200
21.Erros clássicos: criptografia mal empregada	203
21.1. Onde a criptografia realmente falha	203
21.2. Categoria 1 — Escolha errada de primitiva	203
21.3. Categoria 2 — Aleatoriedade e nonce	204
21.4. Categoria 3 — Integridade ausente ou mal feita	205
21.5. Categoria 4 — Gestão de chaves	205
21.6. Categoria 5 — Validação que não valida	206
21.7. Categoria 6 — Vazamento por canal lateral	207
21.8. Categoria 7 — Escopo e expectativa	207
21.9. O guia rápido de decisão	208
21.10. Brasil em Foco	208
21.11. No Campo	209

21.12	Resumo	211
IV.	Volume 4 — Identidade, Autenticação e Controle de Acesso	213
22.	Modelos de controle de acesso: DAC, MAC, RBAC e ABAC	215
22.1.	O vocabulário mínimo	215
22.2.	DAC — o dono decide	216
22.3.	MAC — a organização decide, e ninguém pode contrariar	217
22.4.	RBAC — permissões viajam por papel, não por pessoa	218
22.5.	ABAC — a decisão vira uma expressão	220
22.6.	Os princípios que atravessam todos os modelos	221
22.7.	Como os modelos convivem de verdade	222
22.8.	Brasil em Foco	222
22.9.	No Campo	223
22.10	Resumo	225
23.	Senhas: armazenamento, políticas e ataques	227
23.1.	Por que a senha ainda existe	227
23.2.	Armazenamento: o único jeito certo	227
23.3.	Ataques: o que realmente acontece com a base vazada	230
23.4.	Política de senha: o que a evidência mudou	231
23.5.	O ciclo de vida completo	233
23.6.	Gerenciadores de senha	234
23.7.	Brasil em Foco	234
23.8.	No Campo	235
23.9.	Resumo	237
24.	Autenticação multifator e fatores de autenticação	239
24.1.	Os fatores, e por que a categoria importa	239
24.2.	A hierarquia real de segurança	240
24.3.	TOTP: como funciona e onde falha	240
24.4.	Por que SMS é o pior fator que ainda vale a pena ter	241
24.5.	Fadiga de notificação: o ataque contra o fator “bom”	242
24.6.	FIDO2, WebAuthn e passkeys	242
24.7.	Biometria: entender o que ela é	244
24.8.	Recuperação: onde o MFA quase sempre vaza	244
24.9.	Autenticação adaptativa	245
24.10.	Brasil em Foco	245
24.11.	No Campo	246
24.12	Resumo	247
25.	Federação de identidade: SAML, OAuth 2.0 e OpenID Connect	249
25.1.	Os papéis e o vocabulário	249
25.2.	SAML 2.0: XML, assinatura e o navegador como intermediário	250
25.3.	OAuth 2.0: delegação, não autenticação	251
25.4.	OpenID Connect: a camada que faltava	253
25.5.	Comparação e escolha	255
25.6.	Os riscos que a federação cria	255
25.7.	Brasil em Foco	256
25.8.	No Campo	256
25.9.	Resumo	258

26. Gestão de identidades e de acessos privilegiados (IAM e PAM)	259
26.1. O ciclo de vida da identidade	259
26.2. Governança: além do provisionamento	260
26.3. Acesso privilegiado: onde o erro custa mais	261
26.4. Métricas que revelam a saúde do programa	263
26.5. Brasil em Foco	264
26.6. No Campo	265
26.7. Resumo	267
27. Arquitetura Zero Trust	269
27.1. O que o modelo de perímetro pressupunha	269
27.2. O que Zero Trust é, e o que não é	269
27.3. Os componentes	270
27.4. Os pilares e um caminho de implantação	271
27.5. ZTNA: o que muda em relação à VPN	272
27.6. Microsegmentação e identidade de carga de trabalho	273
27.7. Os riscos que o modelo cria	273
27.8. Brasil em Foco	274
27.9. No Campo	274
27.10. Resumo	276
 V. Volume 5 — Segurança de Redes	 279
28. Arquitetura segura e segmentação de redes	281
29. Firewalls: tipos, políticas e boas práticas	283
30. VPNs: IPsec, WireGuard e acesso remoto seguro	285
31. IDS, IPS e detecção em rede	287
32. Segurança de DNS: ataques, DNSSEC e DoH	289
33. Segurança do roteamento e BGP: sequestros, vazamentos e RPKI	291
34. Segurança em redes sem fio e enlaces de rádio	293
35. Ataques de negação de serviço e estratégias de mitigação	295
 VI. Volume 6 — Endpoints e Sistemas Operacionais	 297
36. Hardening de servidores Linux	299
37. Hardening de Windows e Active Directory	301
38. Antivírus, EDR e XDR	303
39. Gestão de vulnerabilidades e ciclo de patches	305
40. Segurança de dispositivos móveis e BYOD	307
41. Criptografia de disco e proteção de dados em repouso	309

VII. Volume 7 — Segurança de Aplicações	311
42. Ciclo de desenvolvimento seguro e DevSecOps	313
43. OWASP Top 10: panorama e como usar	315
44. Injeção: SQL, NoSQL e comandos de sistema	317
45. Cross-site scripting e ataques no lado do cliente	319
46. Falhas de autenticação e gestão de sessão	321
47. Falhas de controle de acesso: IDOR, SSRF e escalonamento	323
48. Segurança de APIs REST e GraphQL	325
49. SAST, DAST, SCA e automação de testes de segurança	327
VIII. Volume 8 — Nuvem e Contêineres	329
50. Modelo de responsabilidade compartilhada	331
51. IAM em nuvem e os erros de configuração mais caros	333
52. Segurança de contêineres e de imagens	335
53. Segurança em Kubernetes	337
54. Infraestrutura como código e segurança de pipeline	339
55. Segredos, cofres e gestão de chaves	341
IX. Volume 9 — Testes de Invasão	343
56. Metodologias e definição de escopo: PTES, OSSTMM e NIST SP 800-115	345
57. Reconhecimento passivo e OSINT	347
58. Varredura e enumeração	349
59. Análise de vulnerabilidades	351
60. Exploração: conceitos e anatomia de uma falha	353
61. Pós-exploração, persistência e movimentação lateral	355
62. Teste de invasão em aplicações web na prática	357
63. Relatório técnico e comunicação de achados	359
X. Volume 10 — Fator Humano	361
64. Engenharia social: os princípios psicológicos da persuasão	363

65. Phishing, spear phishing e fraude do e-mail corporativo	365
66. Fraudes digitais no Brasil: Pix, clonagem e golpes por mensageria	367
67. Segurança física e ataques presenciais	369
68. Cultura de segurança e programas de conscientização	371
 XI. Volume 11 — Malware e Inteligência de Ameaças	 373
69. Taxonomia de malware	375
70. Anatomia de um ataque de ransomware	377
71. Análise estática em laboratório	379
72. Análise dinâmica e sandboxing	381
73. MITRE ATT&CK e o mapeamento de TTPs	383
74. Inteligência de ameaças e indicadores de comprometimento	385
 XII. Volume 12 — Blue Team, SOC e Detecção	 387
75. Estrutura e operação de um SOC	389
76. Logging: o que registrar, por quanto tempo e por quê	391
77. SIEM: arquitetura, ingestão e correlação	393
78. Escrita de regras de detecção com Sigma e YARA	395
79. Threat hunting: caçando o que a regra não pegou	397
80. Deception: honeypots, honeytokens e canários	399
81. Métricas e maturidade da detecção	401
 XIII. Volume 13 — Resposta a Incidentes e Forense Digital	 403
82. O ciclo de resposta a incidentes	405
83. Preparação: plano, papéis e playbooks	407
84. Contenção, erradicação e recuperação	409
85. Fundamentos de forense digital e cadeia de custódia	411
86. Forense de disco e sistemas de arquivos	413
87. Forense de memória	415
88. Forense de rede e análise de tráfego	417

XIV	Volume 14 — Continuidade e Resiliência	419
89.	Backup: estratégias e a regra 3-2-1-1-0	421
90.	Recuperação de desastres e continuidade de negócio	423
91.	Testes de continuidade e exercícios de mesa	425
92.	Alta disponibilidade e resiliência de infraestrutura	427
93.	Recuperação após um ataque de ransomware	429
XV.	Volume 15 — Governança, Risco e Conformidade	431
94.	Governança de segurança e o papel do CISO	433
95.	ISO/IEC 27001 e 27002	435
96.	NIST Cybersecurity Framework e CIS Controls	437
97.	LGPD na prática: bases legais, incidentes e a ANPD	439
98.	Marco Civil, Lei 12.737 e o direito penal digital brasileiro	441
99.	Obrigações regulatórias de provedores de internet	443
100.	PCI-DSS e requisitos setoriais	445
101.	Auditoria, conformidade e gestão de riscos de terceiros	447
XVI	Volume 16 — Fronteira e Carreira	449
102.	Segurança de sistemas de IA e modelos de linguagem	451
103.	Criptografia pós-quântica	453
104.	Segurança de OT, IoT e sistemas embarcados	455
105.	Segurança da cadeia de suprimentos de software	457
106.	Privacidade, anonimato e vigilância	459
107.	Carreira, certificações e aprendizado contínuo	461
	Referências	463

Apresentação

Este manual cobre segurança da informação do básico ao avançado, em português, com foco em construir entendimento real em vez de decorar siglas. Ele nasce da premissa de que segurança não é uma disciplina isolada: é a consequência de entender profundamente como redes, sistemas operacionais, criptografia e pessoas realmente funcionam — e onde essas engrenagens se desencaixam.

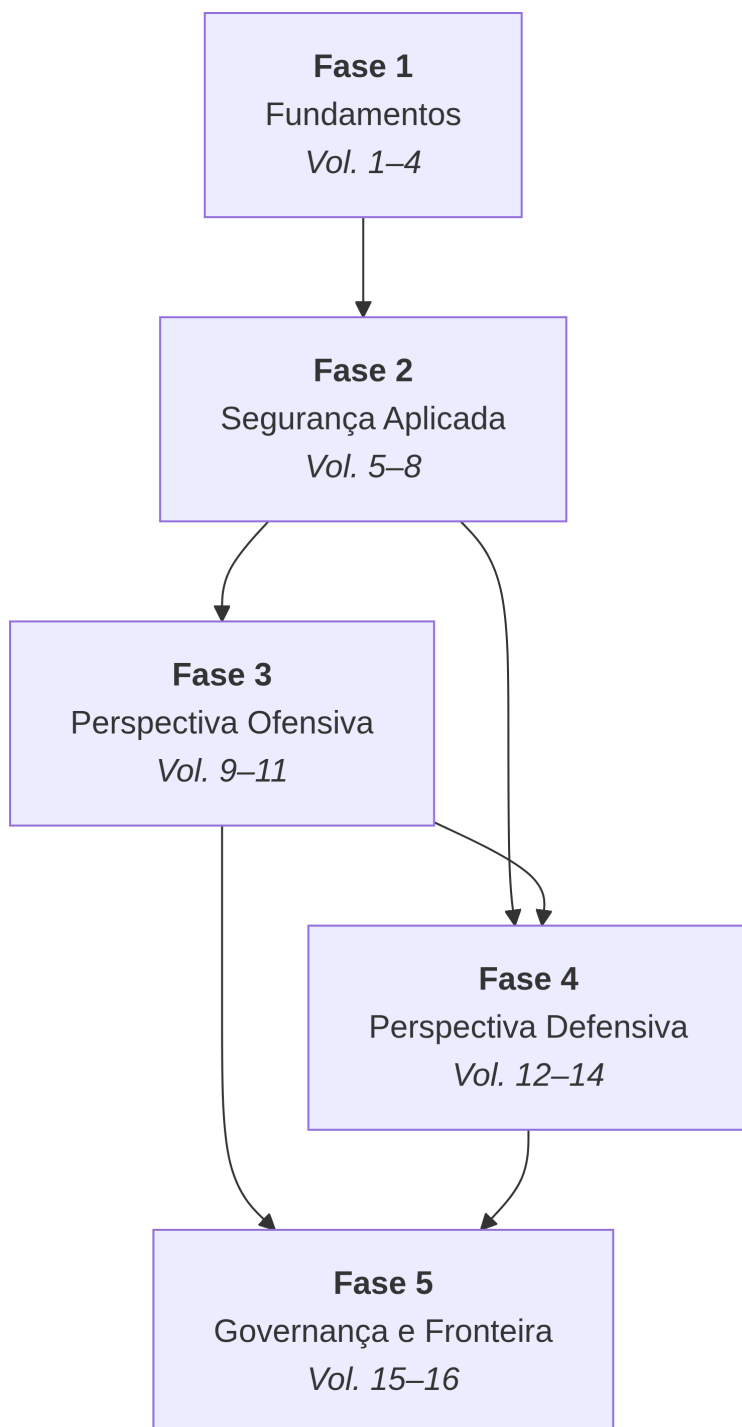
São **107 capítulos** distribuídos em **16 volumes** e **5 fases**, escritos para serem lidos em sequência por quem está começando, e consultados por assunto por quem já atua na área.

Para quem é este manual

- Quem está entrando na área e quer uma trilha ordenada em vez de tutoriais soltos
- Profissionais de infraestrutura e redes que precisam incorporar segurança ao dia a dia
- Desenvolvedores que querem entender as falhas que produzem sem perceber
- Estudantes preparando-se para certificações (Security+, CEH, CISSP, OSCP) que precisam da base conceitual antes do material de prova

Não é preciso conhecimento prévio de segurança. É preciso, sim, disposição para acompanhar os fundamentos técnicos do Volume 2 antes de correr para as partes mais chamativas.

Como o manual está organizado



Dependências entre as fases do manual

A **Fase 1** estabelece o vocabulário e a base técnica: o que é risco, como funcionam redes e sistemas, criptografia e controle de acesso. A **Fase 2** aplica isso aos domínios concretos — redes, endpoints, aplicações e nuvem. As fases **3** e **4** apresentam os dois lados da mesma moeda: como se ataca e como se detecta e responde. A **Fase 5** fecha com governança, conformidade brasileira e os temas de fronteira.

Ataque e defesa aparecem nesta ordem por um motivo pedagógico: não se detecta o que não se

compreende. Mas as duas fases são complementares, não sequenciais — quem só quer defender pode ler a Fase 4 antes da 3, aceitando algumas referências para frente.

Seções fixas de cada capítulo

Todo capítulo traz duas seções obrigatórias ao final:

i **Brasil em Foco** — como o tema se conecta à legislação e às instituições brasileiras: LGPD e ANPD, Marco Civil, Lei 12.737, Anatel, CERT.br, Gov.br, Banco Central e Pix.

💡 **No Campo** — a tradução do conceito para a operação real de infraestrutura: o que muda no dia a dia de quem administra rede, servidores e clientes.

Política editorial

! Escopo, ética e legalidade

Este manual é material educacional com orientação defensiva. Técnicas ofensivas são apresentadas para que se possa **entender, detectar e mitigar** ataques — nunca como receita operacional.

Na prática, isso significa: explicamos mecanismos, usamos ferramentas públicas e consagradas em laboratório isolado, e mostramos as contramedidas correspondentes. Não publicamos exploits armados, payloads prontos para uso, nem código de malware funcional.

Testar sistemas sem autorização formal e por escrito é crime no Brasil (Lei 12.737/2012, art. 154-A do Código Penal). O Volume 9 trata escopo e autorização como pré-requisito, não como formalidade. Monte seu laboratório (Capítulo 14) e pratique apenas nele ou em plataformas que autorizam explicitamente.

Licença

Conteúdo sob licença [CC BY-SA 4.0](#). Código e scripts sob licença MIT. Contribuições, correções e sugestões são bem-vindas via issues no repositório.

Parte I.

**Volume 1 — Fundamentos da
Segurança da Informação**

1. O que é segurança da informação

Numa terça-feira de março, às 6h40, o técnico de plantão de um provedor regional atende o primeiro chamado do dia: um assinante não consegue emitir a segunda via da fatura. Às 7h10 são doze chamados. Às 7h30 fica claro que o problema não é o portal de faturas — é o sistema de gestão inteiro, o ERP que emite boleto, controla contrato e guarda o cadastro de quatro mil assinantes. A tela de login abre. A senha do administrador não é aceita. No servidor, os arquivos do banco de dados continuam todos lá, com uma extensão nova grudada no fim. Na raiz do disco há um arquivo de texto com um endereço de carteira de criptomoeda e um prazo.

Vale olhar primeiro o que **não** falhou naquela manhã. O link de saída estava perfeito, com 6 ms de latência para o PTT. Nenhum alarme do monitoramento disparou, porque o monitoramento vigiava latência, perda de pacote, potência óptica e temperatura de rack — e os quatro estavam normais. O firewall de borda seguia bloqueando exatamente aquilo que alguém havia configurado para bloquear, sem errar um pacote. Toda a infraestrutura funcionou conforme especificada. E a empresa parou por seis dias.

O caminho, reconstruído depois, foi banal. Uma conta de VPN criada dois anos antes para um consultor, com senha reaproveitada de outro serviço que havia vazado, sem segundo fator. De dentro da VPN, uma porta de área de trabalho remota alcançável e uma conta de administrador local repetindo a mesma senha em três servidores. O backup existia — em um compartilhamento de rede montado permanentemente como unidade Z: no próprio servidor do ERP, e por isso foi cifrado junto.

No mesmo mês, essa empresa teve outros dois incidentes que ninguém chamou de incidente. Um notebook do comercial foi furtado de dentro de um carro, com uma planilha de 1.200 assinantes — nome, CPF, endereço, telefone — em disco sem cifragem. E um técnico desligado em janeiro seguiu com acesso administrativo à OLT por 40 dias, porque o desligamento passou pelo RH e pela folha, mas não por ninguém que soubesse quais credenciais aquela pessoa carregava.

Os três episódios são incidentes de segurança da informação. Nenhum dos três se resolve comprando um equipamento. É desse desconforto que a disciplina nasce.

1.1. Uma definição de trabalho

A norma ISO/IEC 27000 define segurança da informação como a **preservação da confidencialidade, da integridade e da disponibilidade da informação** (International Organization for Standardization, 2018a). É uma frase curta e bem mais densa do que parece.

Preservação implica continuidade: não é estado que se alcança e se declara, é algo que se mantém contra uma tendência natural de degradação. *Informação* — não servidor, não rede, não software. E as três propriedades, que o próximo capítulo abre em detalhe:

- **Confidencialidade** — a informação só é acessível a quem tem direito a ela.
- **Integridade** — a informação está correta e completa, e alterações não autorizadas são impedidas ou, no mínimo, detectáveis.
- **Disponibilidade** — quem tem direito à informação a alcança quando precisa.

1. O que é segurança da informação

Os três pilares são necessários, mas não bastam: uma transação bancária exige também que a origem seja comprovadamente quem afirma ser (autenticidade) e que depois não possa negar tê-la feito (não repúdio) — assunto do capítulo seguinte ao próximo.

A definição operacional que este manual carrega é esta:

Segurança da informação é o conjunto de decisões — técnicas, organizacionais e humanas — que mantém a informação disponível para quem tem direito a ela, íntegra para quem depende dela e inalcançável para quem não deveria chegar até ela, dentro de um custo que a organização consegue sustentar de forma contínua.

A última cláusula é a que separa segurança de teatro de segurança: um controle que a empresa não consegue pagar, operar e manter por três anos não é controle, é linha de relatório.

Vale notar desde já que os três pilares **entram em conflito entre si**. Cifrar o backup protege a confidencialidade e cria um risco novo de disponibilidade: chave perdida significa backup inútil. Exigir troca de senha a cada 30 dias parece reforçar o controle de acesso e, na prática, produz **Verão2026!** escrito num post-it colado no monitor da recepção. Segurança da informação é, em boa medida, escolher esses tradeoffs conscientemente em vez de tropeçar neles.

1.2. Dado, informação e ativo: o que exatamente se protege

10.20.0.42 é um dado. “Endereço de gerência da OLT do distrito de Santa Rita, acessível apenas pela VLAN 900” é informação: o dado mais o contexto que o torna útil — e, no mesmo movimento, útil para um atacante.

Ativo é qualquer coisa que tenha valor para a organização, e separá-los em dois níveis é prático:

- **Ativos primários** — a informação em si e os processos de negócio que dependem dela. O cadastro de assinantes. A base de contratos. A capacidade de faturar no dia 5.
- **Ativos de suporte** — o que sustenta os primários: hardware, software, rede, instalações, e as pessoas. O servidor do ERP, a fibra do backbone, a torre de rádio, o técnico que sabe reconfigurar a OLT.

A inversão que a disciplina propõe é esta: o objeto de proteção é o ativo primário, mas os controles quase sempre se aplicam aos de suporte. Ninguém cifra “o cadastro de assinantes” em abstrato — cifra-se o disco, restringe-se a conta, segmenta-se a VLAN.

E a informação não fica parada: percorre um ciclo, e cada etapa tem controles próprios (Figura 1.1).

O descarte é a etapa esquecida com mais frequência, e a que produz vazamentos mais silenciosos: o disco do servidor trocado por defeito que vai para o ferro-velho com o banco de dados legível; o roteador de borda vendido em marketplace com a configuração completa da rede, incluindo chaves de BGP. Nenhum desses casos tem atacante, exploração ou malware. Todos são falhas de confidencialidade.

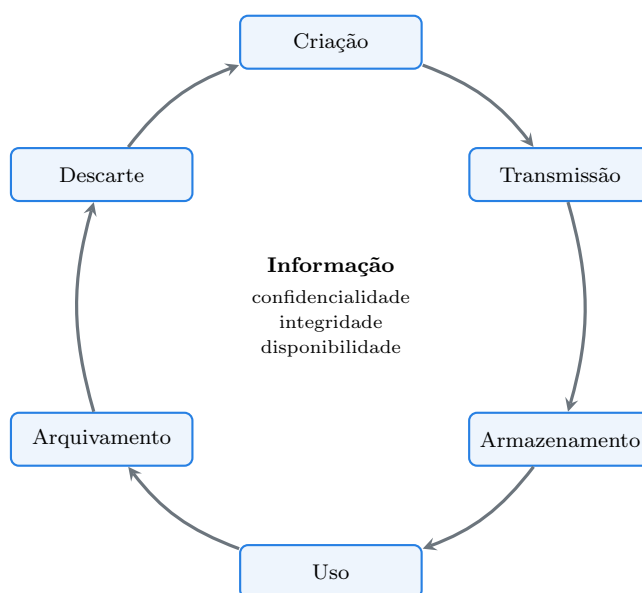


Figura 1.1.: O ciclo de vida da informação. Um controle que só cobre uma etapa deixa as outras cinco expostas.

1.3. Segurança não é um estado, é um processo

A frase é de Bruce Schneier e virou clichê justamente porque resolve uma confusão persistente (Schneier, 2000). Segurança absoluta é alcançável: desliga-se o equipamento, retira-se o disco, lacra-se num cofre. O sistema fica perfeitamente seguro e perfeitamente inútil. Todo sistema real opera num ponto intermediário, e o nome desse ponto é **risco residual**.

O que a disciplina persegue não é risco zero, é risco residual *conhecido e aceito conscientemente* — decidido por quem tem autoridade, documentado e revisitado. A diferença entre uma empresa madura e uma imprudente raramente está no número de controles: está em saber, ou não saber, o que ficou descoberto.

Existe uma razão econômica para isso, esboçada em Figura 1.2.

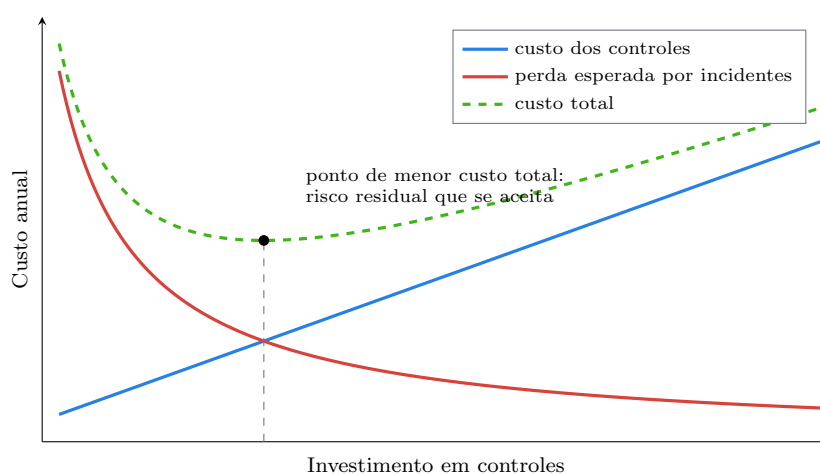


Figura 1.2.: A lógica econômica do controle. Passado certo ponto, cada real gasto em segurança compra menos redução de perda esperada do que custa.

1. O que é segurança da informação

O gráfico é uma idealização didática: na vida real não se conhecem os valores das curvas com precisão, e é por isso que existe gestão de risco como método — tema do sexto capítulo deste volume. O que importa é a *forma*: há um ponto além do qual gastar mais destrói valor, e um ponto abaixo do qual economizar destrói a empresa.

Há uma segunda razão para tratar segurança como processo: **as configurações envelhecem**. Um servidor perfeitamente endurecido em 2023 acumula, até 2026, bibliotecas com falhas conhecidas, cifras que deixaram de ser aceitáveis e regras de firewall abertas “temporariamente” para um teste que ninguém fechou. Toda base de regras com cinco anos contém alguma liberação cuja justificativa ninguém sabe mais explicar, e que ninguém remove por medo de derrubar algo.

1.4. Pessoas, processos e tecnologia

O incidente da abertura é um exemplo didático de por que a tecnologia isolada não fecha a conta. Vale reler o caso separando as camadas:

Tabela 1.1.: As três camadas do incidente de abertura.

Camada	O que falhou
Tecnologia	VPN sem segundo fator, rede interna plana, senha de administrador local repetida, backup em unidade montada
Processo	desligamento sem revogação de credencial, nenhuma revisão periódica de contas, backup nunca testado por restauração
Pessoas	senha reaproveitada de outro serviço, conta de terceiro esquecida ativa por dois anos

Nenhuma das três falhas de processo custa dinheiro para corrigir; todas custam disciplina, que é mais escassa. E o investimento típico vai no sentido inverso — 90% do orçamento em tecnologia —, o que explica por que empresas bem equipadas continuam sendo comprometidas por caminhos elementares.

1.5. O adversário não é só o hacker

A palavra “ameaça” evoca uma pessoa mal-intencionada, e essa associação estreita indevidamente o campo. Convencionalmente, separam-se três origens:

Deliberadas. Ação intencional de um agente: fraude, extorsão por ransomware, sabotagem, espionagem, furto de equipamento.

Acidentais. Erro humano sem intenção: o comando DROP no banco de produção em vez do de teste; a rota redistribuída por engano que anuncia prefixo de terceiro; o e-mail com anexo de dados pessoais enviado para a lista errada. Em volume de incidentes, esta categoria rivaliza com a primeira — e em algumas organizações a supera.

Ambientais e naturais. Descarga atmosférica na torre de rádio que leva switch, conversor e fonte no mesmo estouro. Falta de energia de nove horas num site remoto cujo banco de baterias tem autonomia de duas. Roçadeira de trator que rompe o ramal de fibra do distrito na época da

colheita. Furto de bateria de nobreak em site sem cerca — que, para quem opera rede rural, é ameaça deliberada e ambiental ao mesmo tempo.

Para quem administra infraestrutura, essa terceira categoria é com frequência a maior fonte de horas de indisponibilidade do ano — e disponibilidade é um dos três pilares. Não é curiosidade taxonômica: o mesmo arcabouço (inventariar ativos, estimar risco, implantar controle, exercitar resposta) serve para o raio e para o ransomware. Plano de resposta que só sabe lidar com atacante é metade de um plano.

1.6. Quem ataca, e por que o pequeno também é alvo

Entre os agentes deliberados, os perfis diferem tanto em capacidade quanto em motivação, e a diferença muda a defesa apropriada:

- **Oportunista automatizado.** Não escolheu você. Varre faixas inteiras de endereços procurando serviço exposto com credencial padrão ou falha conhecida. É a esmagadora maioria do que se vê nos logs.
- **Crime organizado.** Opera como negócio, com divisão de trabalho: quem obtém o acesso inicial vende para quem executa a extorsão, que aluga o programa de cifragem de um terceiro. É a economia que tornou ransomware viável contra empresas de qualquer porte.
- **Pessoa interna.** Do funcionário insatisfeito com acesso privilegiado ao terceirizado negligente que reutiliza senha. Não precisa vencer perímetro nenhum: já está dentro.
- **Ativismo.** Motivação ideológica, alvo escolhido pelo que representa e busca de visibilidade: pichação de site, vazamento seletivo, indisponibilidade em data simbólica.
- **Estado-nação.** Recursos altos, paciência longa, alvo deliberadamente escolhido. Raro como adversário direto de uma empresa média — mas provedores de telecomunicações e cadeias de suprimento são alvos de interesse justamente por serem caminho para outros.

Um servidor SSH recém-exposto coleta tentativas de acesso em minutos, sempre com esta cara:

```
Mar 12 03:11:44 borda sshd[2291]: Failed password for invalid user admin from
↪ 203.0.113.47 port 51344 ssh2
Mar 12 03:11:46 borda sshd[2293]: Failed password for invalid user ubnt from
↪ 203.0.113.47 port 51402 ssh2
Mar 12 03:11:49 borda sshd[2295]: Invalid user support from 198.51.100.12 port
↪ 44120
Mar 12 03:11:52 borda sshd[2297]: Failed password for root from 198.51.100.12
↪ port 44188 ssh2
Mar 12 03:12:03 borda sshd[2301]: Failed password for invalid user pi from
↪ 203.0.113.190 port 33012 ssh2
```

Os nomes de usuário entregam o jogo: **admin**, **ubnt**, **support** e **pi** são credenciais de fábrica de equipamentos de rede e placas embarcadas. Ninguém decidiu atacar aquele endereço — um programa varre a internet inteira, e o endereço apareceu na lista. O custo marginal de tentar em um alvo a mais é zero, e é por isso que “sou pequeno, ninguém vai me atacar” é premissa falsa por construção.

Há um segundo motivo, mais específico. Uma empresa pequena que administra rede guarda dados pessoais em volume (CPF, endereço, meio de pagamento) e opera equipamentos de cliente que, tomados em massa, valem como capacidade de ataque: milhares de roteadores domésticos com credencial padrão são ativo de mercado para quem monta rede de ataque distribuído. Nessa configuração, a infraestrutura não é apenas alvo — é vetor contra os próprios assinantes.

1.7. Os princípios que sustentam qualquer defesa

Em 1975, Saltzer e Schroeder publicaram oito princípios de projeto para proteção da informação (Saltzer; Schroeder, 1975). Cinco décadas depois, com toda a tecnologia trocada, seguem sendo o melhor filtro para avaliar uma decisão de segurança:

1. **Economia de mecanismo.** O desenho mais simples que resolve. Complexidade é onde erro se esconde.
2. **Padrões à prova de falha.** Negar por padrão; permitir por exceção explícita. Se o mecanismo falhar, deve falhar fechado.
3. **Mediação completa.** Toda requisição verificada, sempre. Atalhos de desempenho que reaproveitam decisões antigas de autorização são a origem de uma classe inteira de falhas.
4. **Desenho aberto.** A segurança não deve depender do segredo do desenho, apenas do segredo da chave. Criptografia proprietária “secreta” é sinal de alerta, não de força.
5. **Separação de privilégios.** Exigir mais de uma condição independente para a operação crítica — a lógica das duas chaves do cofre, e a razão de ser da autenticação em dois fatores.
6. **Menor privilégio.** Cada sujeito com o mínimo de permissão necessário, pelo menor tempo necessário. É o princípio mais violado da lista: quase toda rede tem contas de serviço com privilégio administrativo porque “assim funciona”.
7. **Mecanismo menos comum.** Evitar compartilhar mecanismo e estado entre usuários diferentes; o que é compartilhado se torna caminho entre eles.
8. **Aceitação psicológica.** O controle precisa ser utilizável. Controle que atrapalha o trabalho é controle burlado — e a criatividade do usuário para burlar é ilimitada.

Desses princípios decorre a estratégia que organiza tudo o que vem nos volumes seguintes: **defesa em profundidade**. Nenhuma camada é considerada confiável o suficiente para ser a única, e o comprometimento de uma não deve implicar o comprometimento das demais (Figura 1.3).

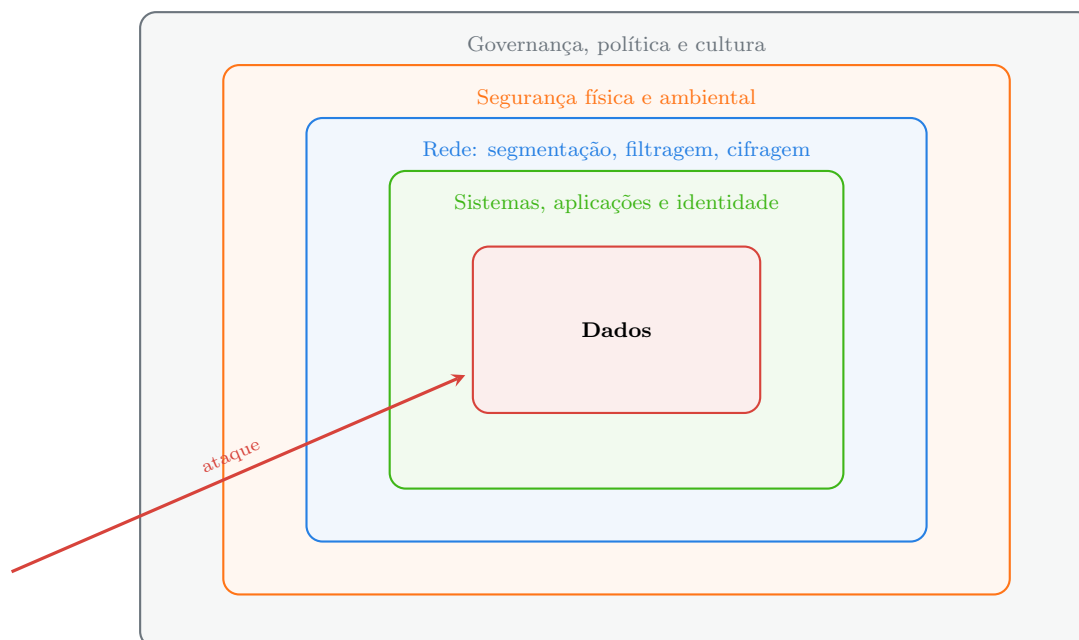


Figura 1.3.: Defesa em profundidade. Cada camada compra tempo e gera registro; nenhuma é suficiente sozinha.

Duas ressalvas importam desde já. Camadas precisam ser **independentes**: cinco que aceitam a mesma senha de administrador são, na prática, uma. E camadas servem para *detectar*, não

só para bloquear — o valor de uma camada intermediária costuma estar menos em barrar o atacante do que em produzir o registro que revela sua passagem enquanto há tempo de reagir. Assumir que o comprometimento vai acontecer e projetar para detectá-lo rápido atravessa toda a parte defensiva deste manual.

1.8. O vocabulário mínimo

Segurança da informação tem vocabulário próprio, em que palavras do uso cotidiano ganham sentido técnico estrito. Confundi-las produz conversas em que ninguém está errado e ninguém se entende. A Tabela 1.2 fixa o mínimo para seguir adiante; o quarto capítulo deste volume aprofunda cada termo.

Tabela 1.2.: Vocabulário mínimo de segurança da informação.

Termo	Sentido técnico	No caso de abertura
Ativo	Aquilo que tem valor e que se quer proteger	O cadastro dos quatro mil assinantes
Vulnerabilidade	Fraqueza que pode ser explorada	VPN sem segundo fator; senha repetida entre servidores
Ameaça	Agente ou evento com potencial de causar dano	Grupo de extorsão que compra acesso inicial
Risco	Combinação de probabilidade e impacto de a ameaça explorar a vulnerabilidade	Alta chance de parada de faturamento por vários dias
Controle	Medida que reduz risco, por prevenção, detecção ou correção	Segundo fator na VPN; backup fora do alcance do servidor
Evento	Qualquer ocorrência observável no ambiente	Uma autenticação falha às 3h11
Incidente	Evento, ou série de eventos, que compromete um dos pilares	A cifragem do banco de dados do ERP
Superfície de ataque	Conjunto de pontos alcançáveis por um adversário	VPN, área de trabalho remota, portal do assinante
Risco residual	O que permanece depois dos controles aplicados	O que se decide conviver, sabendo que existe

Duas dessas distinções têm consequência operacional direta. Uma rede grande gera milhões de *eventos* por dia, e a função de um centro de operações é separar os poucos que são *incidentes*. E como não se elimina a *ameaça* — ninguém acaba com o crime organizado a partir do datacenter —, o trabalho recai sobre reduzir vulnerabilidade e impacto.

1.9. Como se sabe se está funcionando

“Nunca fomos invadidos” não é evidência de segurança; costuma ser evidência de ausência de detecção — comprometimentos que duram meses sem serem percebidos são a regra, e a descoberta vem com frequência de fora: um cliente, um pesquisador, ou o próprio atacante ao pedir resgate.

1. O que é segurança da informação

Medir é difícil justamente porque o resultado desejado é a não ocorrência de algo, mas alguns indicadores sobrevivem ao escrutínio:

- **Tempo até detectar e tempo até conter.** Se a resposta é “não sabemos”, esse é o primeiro problema a resolver.
- **Cobertura de controle sobre inventário.** Percentual de ativos inventariados, de contas com segundo fator, de servidores corrigidos no prazo, de backups cuja restauração foi testada no trimestre. Todas dependem de existir inventário.
- **Superfície exposta.** Quantidade de serviços administrativos alcançáveis da internet. A meta é zero, e o número é fácil de medir.
- **Resultado de exercícios.** Restauração real a partir do backup, simulação controlada de phishing, exercício de mesa. Exercício produz dado; opinião produz sensação.

Métrica que só sobe e nunca gera decisão é métrica de vaidade: contar ataques bloqueados pelo firewall mede o ruído de fundo da internet, não a qualidade da defesa. O Volume 12 retoma o assunto com o rigor devido.

1.10. Brasil em Foco

No Brasil, segurança da informação deixou de ser boa prática e passou a ser obrigação jurídica com nome, prazo e sanção.

A **Lei Geral de Proteção de Dados** (Lei 13.709/2018) é o eixo (Brasil, 2018). Seu art. 46 obriga os agentes de tratamento a adotar medidas de segurança técnicas e administrativas aptas a proteger dados pessoais — redação deliberadamente aberta, que remete ao estado da técnica e à natureza da informação, e não a uma lista fechada de produtos. O art. 48 estabelece o dever de comunicar à Autoridade Nacional de Proteção de Dados e aos titulares afetados a ocorrência de incidente de segurança que possa acarretar risco ou dano relevante. A Resolução CD/ANPD nº 15/2024 regulamentou essa comunicação, fixando prazo de três dias úteis a contar do conhecimento do incidente e definindo o conteúdo mínimo do relato (Autoridade Nacional de Proteção de Dados, 2024). Na prática, a empresa precisa ser capaz de *saber* que foi comprometida, *avaliar* o alcance e *redigir* a comunicação em três dias — impossível sem registro adequado e plano prévio. No caso da abertura, a planilha do notebook furtado é tratamento de dados pessoais, e o furto é potencialmente um incidente comunicável.

O **Marco Civil da Internet** (Lei 12.965/2014) trata da guarda de registros (Brasil, 2014). O art. 13 obriga o provedor de conexão a manter registros de conexão por um ano; o art. 15 obriga o provedor de aplicações constituído como pessoa jurídica e com fins econômicos a guardar registros de acesso por seis meses. Os arts. 10 e 11 impõem sigilo e condicionam a disponibilização a ordem judicial. A leitura é dupla: o registro é obrigação e é ativo sensível — quem guarda log de conexão de milhares de pessoas custodia um alvo de valor.

A **Lei 12.737/2012** inseriu no Código Penal o art. 154-A, que tipifica invadir dispositivo informático alheio para obter, adulterar ou destruir dados, ou instalar vulnerabilidade para obter vantagem ilícita (Brasil, 2012). As penas foram agravadas pela Lei 14.155/2021. Essa é a fronteira jurídica que sustenta a política editorial deste manual: testar sistema de terceiro sem autorização formal e por escrito é crime, ainda que a intenção seja “só verificar”. O sétimo capítulo deste volume trata exclusivamente dessa fronteira.

Para quem opera telecomunicações há camada regulatória adicional: a **Anatel** editou a Resolução nº 740/2020, com requisitos de segurança cibernética para produtos de telecomunicações, o que alcança diretamente as ONUs e roteadores entregues ao assinante (Agência Nacional

de Telecomunicações, 2020). O Volume 15 dedica um capítulo às obrigações regulatórias de provedores.

No lado institucional, o **CERT.br**, mantido pelo NIC.br, é o ponto nacional de tratamento de incidentes em redes conectadas à internet no Brasil: recebe notificações, publica estatísticas e mantém material de boas práticas usado por operadores de rede. Manter contato de abuso atualizado no WHOIS do seu bloco e responder de fato a essas notificações é um dos controles mais baratos e menos praticados do país.

Por fim, o **Banco Central** trata fraude como matéria de segurança regulada: instituições financeiras e de pagamento estão sujeitas a política obrigatória de segurança cibernética (Resolução CMN nº 4.893/2021), e o Pix conta com o Mecanismo Especial de Devolução para casos de fraude — o que faz da engenharia social contra o titular, e não do ataque à instituição, o vetor dominante no país. O Volume 10 volta ao tema.

1.11. No Campo

Traduzindo tudo para a primeira semana de quem assume a responsabilidade por uma infraestrutura, na ordem em que compensa fazer:

1. Inventário, antes de qualquer controle. Não se protege o que não se conhece, e a maior parte das surpresas está em ativo esquecido: o servidor de teste de 2019 ainda ligado com endereço público, a câmera com firmware de fábrica. Vale a varredura do próprio bloco — sua rede, sua autorização:

```
# Descoberta de hosts e serviços no PRÓPRIO bloco de endereços
nmap -sS -Pn --top-ports 200 --open -oA inventario-borda 198.51.100.0/24

# O que este servidor realmente escuta, e qual processo responde
ss -tulnp | sort -k5

# Contas locais com shell de login válido
awk -F: '$7 !~ /(nologin|false)$/ {print $1, $3, $7}' /etc/passwd
```

2. Reduzir a superfície de gerência. Painel de administração acessível da internet é o achado mais comum e mais caro. Winbox, interface web de OLT, SNMP com comunidade `public`, Telnet, área de trabalho remota — nada disso deve estar alcançável do mundo. O padrão é VLAN de gerência separada do tráfego de assinante, alcançável apenas por VPN com segundo fator, com filtro por origem no equipamento além do filtro na borda.

3. Senhas individuais e nomeadas. A senha compartilhada entre a equipe técnica destrói o menor privilégio e a responsabilização de uma vez: quando todos entram como `admin`, o log de auditoria não responde à única pergunta que importa depois de um incidente. Cofre de senhas da equipe, contas nomeadas e revogação como parte formal do desligamento — inclusive chaves SSH, VPN e acesso ao WhatsApp do suporte.

4. Backup que sobrevive ao próprio servidor. O critério é implacável: o backup precisa estar fora do alcance de quem comprometeu o servidor — compartilhamento montado permanentemente não atende, cópia que a máquina de origem pode apagar não atende. E backup nunca restaurado é hipótese, não backup; a restauração de teste trimestral é o que transforma um em outro.

1. O que é segurança da informação

5. Relógio e registro. Sem NTP consistente em todos os equipamentos, os logs não podem ser correlacionados e perdem valor tanto na investigação técnica quanto no atendimento a ordem judicial. É configuração de dez minutos com efeito permanente:

```
timedatectl set-ntp true
timedatectl status | grep -E 'synchronized|NTP service'

# Autenticações falhas por origem, últimas 24 h
journalctl -u ssh --since "24 hours ago" \
  | grep -oE 'from [0-9.]+' | sort | uniq -c | sort -rn | head
```

6. Escrever o plano antes de precisar dele. Uma folha por cenário, resposta a três perguntas: quem decide, a quem se avisa (interno, cliente, ANPD, delegacia), e qual é a primeira ação técnica. Faça para ransomware no sistema de gestão, para vazamento de base de assinantes e para rompimento de fibra do enlace principal. Às 3h da manhã, com o telefone tocando, ninguém improvisa bem.

7. Fechar o ciclo no descarte. Disco que sai da empresa é apagado de forma verificável ou destruído fisicamente. Equipamento devolvido por assinante ou retirado de operação volta a padrão de fábrica antes de ir para o estoque, para venda ou para doação.

Nenhum dos sete itens exige orçamento significativo; todos exigem que alguém seja nomeado responsável — e essa nomeação, mais do que qualquer aquisição, é o que dá início a um programa de segurança.

1.12. Resumo

- Segurança da informação é a **preservação contínua** de confidencialidade, integridade e disponibilidade: processo que se mantém, não estado que se compra. As três propriedades conflitam entre si, e escolher os tradeoffs conscientemente é parte do trabalho.
- O que se protege é a **informação**; servidores, links e equipamentos são ativos de suporte, e é neles que os controles se aplicam — em todo o ciclo de vida, inclusive no descarte, a etapa mais esquecida.
- Ameaça não é sinônimo de atacante: falhas **acidentais** e eventos **ambientais** produzem incidentes de disponibilidade em volume, e cabem no mesmo arcabouço de gestão de risco.
- A maior parte do que se vê nos logs é **varredura automatizada e indiscriminada** — o que invalida a premissa de que porte pequeno confere proteção. Além de alvo, uma infraestrutura mal protegida se torna vetor contra os próprios clientes.
- Os oito princípios de Saltzer e Schroeder, com meio século de idade, seguem sendo o melhor filtro para avaliar decisões; deles decorre a **defesa em profundidade**, com camadas independentes que bloqueiam e também produzem registro.
- No Brasil, o dever de segurança é jurídico: LGPD (arts. 46 e 48) com prazo de três dias úteis para comunicar incidente relevante, Marco Civil para guarda de registros, Lei 12.737/2012 tipificando o acesso não autorizado, mais camada setorial de Anatel e Banco Central.

2. Os pilares: confidencialidade, integridade e disponibilidade

O provedor havia feito tudo certo com o backup. Rotina diária às 2h, retenção de trinta dias, cópia replicada para um segundo site a 40 km, alerta por e-mail em caso de falha do job. Em quatro anos, nenhum alerta.

Em novembro, a equipe descobriu que a tabela de contratos estava com os campos de vencimento embaralhados: assinantes com vencimento no dia 10 apareciam com dia 25, e uma fatia menor tinha vencimento nulo. A causa foi uma migração mal-feita nove meses antes, que rodou uma atualização sem cláusula **WHERE** correta e afetou 3.100 dos 4.000 registros. O sistema não caiu. Ninguém foi invadido. Nada vazou.

E o backup não ajudou: a retenção era de trinta dias, e o dano tinha nove meses. As trinta cópias disponíveis continham, todas, os mesmos dados errados — fielmente preservados, replicados com integridade de arquivo perfeita, disponíveis a qualquer hora. Reconstruir o cadastro exigiu conferir contrato por contrato em papel, um trabalho de sete semanas.

Esse incidente ilustra o ponto central deste capítulo: as três propriedades que definem segurança da informação são **independentes**. Um sistema pode ser exemplar em duas e catastrófico na terceira, e nenhum controle protege as três de graça. Este capítulo abre confidencialidade, integridade e disponibilidade uma a uma — o que cada uma significa com precisão, como cada uma falha, o que a defende e onde as três colidem.

2.1. As três propriedades, com precisão

A tríade CID (em inglês, *CIA triad*) organiza a disciplina desde os anos 1970 e sobrevive porque é ao mesmo tempo curta e completa o bastante para servir de checklist (International Organization for Standardization, 2018a). A Figura 2.1 resume as definições e as tensões.

Vale notar que os três termos têm sentido técnico mais estreito que o do uso corrente. “Confidencial” na linguagem comum sugere sigilo absoluto; em segurança da informação, confidencialidade é um controle *relacional* — sempre em relação a um conjunto definido de sujeitos autorizados, nunca sigilo em abstrato.

2.2. Confidencialidade

Confidencialidade é a garantia de que a informação só é acessível a quem tem direito a ela. A definição embute uma pergunta prévia que costuma ficar sem resposta: *quem* tem direito? Sem classificação de informação e sem uma decisão explícita sobre quem precisa de que acesso, “proteger a confidencialidade” não tem conteúdo operacional.

2. Os pilares: confidencialidade, integridade e disponibilidade

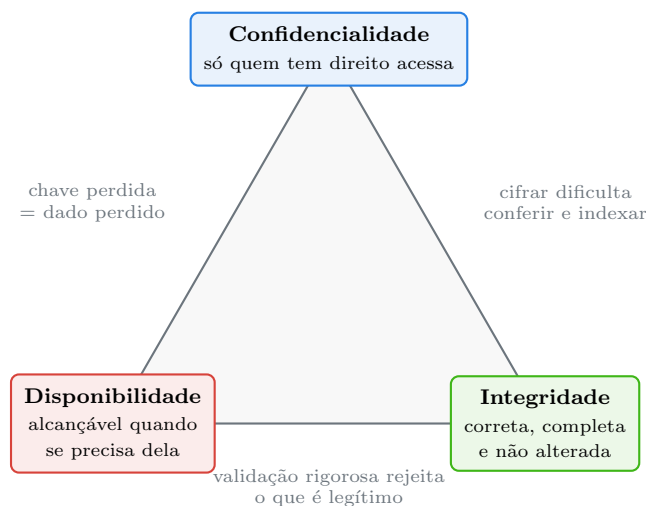


Figura 2.1.: A tríade CID e as tensões entre cada par. Reforçar um vértice quase sempre pressiona os outros dois.

2.2.1. Como a confidencialidade falha

O vazamento por invasão é o caso mais visível e o menos frequente. Os caminhos reais são mais prosaicos:

- **Excesso de permissão.** O compartilhamento de rede com acesso a “todos”, o *bucket* de armazenamento em nuvem com listagem pública, o grupo de mensagens da equipe técnica com cliente dentro. Ninguém invadiu nada: o acesso estava concedido.
- **Perda física.** Notebook furtado, pen drive esquecido, disco de servidor trocado por defeito e devolvido ao fornecedor com os dados legíveis.
- **Canal lateral e metadado.** A informação não vaza, mas se deduz. Um formulário de recuperação de senha que responde “e-mail não cadastrado” confirma quem é cliente. Tráfego cifrado revela, pelo padrão de tamanho e horário dos pacotes, qual serviço está sendo usado. Registro de conexão sem conteúdo ainda revela com quem alguém falou e quando — e é por isso que o Marco Civil trata log como dado sob sigilo.
- **Agregação e inferência.** Nome sozinho é público. Nome com CEP, data de nascimento e plano contratado identifica uma pessoa específica com precisão desconfortável. Bases anonimizadas por remoção de nome frequentemente reidentificam quando cruzadas com outra base.
- **Vazamento pelo canal legítimo.** O log de aplicação que grava o corpo da requisição, senha incluída. A mensagem de erro que devolve o rastro de execução com a *string* de conexão do banco. O relatório em PDF distribuído com texto “tarjado” por retângulo preto sobreposto — que o *pdftotext* recupera integralmente.

2.2.2. O que a defende

Quatro famílias de controle, em ordem de eficácia por real gasto:

1. **Minimização.** Dado que não se coleta não vaza; dado que se apaga no fim do prazo legal não vaza depois. É o único controle cuja eficácia é de 100% e cujo custo é negativo. A LGPD faz disso princípio jurídico (necessidade, art. 6º, III).
2. **Controle de acesso.** Autenticação, autorização e o princípio do menor privilégio. Cobre o caso do excesso de permissão, que é o mais comum. O Volume 4 é inteiro sobre isso.

3. **Cifragem.** Em repouso (disco, banco, backup) e em trânsito (TLS, VPN). Note o que a cifragem realmente faz: transfere o problema do dado para a **chave**. É uma troca excelente — proteger uma chave de 32 bytes é muito mais fácil que proteger 400 GB —, mas é uma troca, não uma solução mágica.
4. **Controles de descarte.** Sobrescrita verificável, apagamento criptográfico em disco autocifrante, destruição física. Fecha a última etapa do ciclo de vida da informação, apresentado no Capítulo 1.

2.3. Integridade

Integridade é a garantia de que a informação está correta e completa, e de que alterações não autorizadas são impedidas ou, no mínimo, detectáveis. Convém separar três níveis que costumam ser confundidos:

- **Integridade de bits.** Os bytes gravados são os bytes lidos. Ataca corrupção silenciosa de disco, erro de transmissão, falha de memória. Ferramentas: soma de verificação, memória com correção de erro, sistemas de arquivos com verificação de bloco (ZFS, Btrfs).
- **Integridade semântica.** O dado faz sentido segundo as regras do negócio. Vencimento entre 1 e 31, CPF válido, contrato ativo com endereço preenchido, saldo que nunca fica negativo sem lançamento correspondente. Ferramentas: restrições no banco, validação de entrada, transações com garantia ACID, conciliação.
- **Integridade de origem.** O dado veio de quem se afirma — que é autenticidade, tema do Capítulo 3, e se apoia em assinatura digital e código de autenticação de mensagem.

O incidente de abertura foi uma falha de integridade **semântica** com integridade de bits perfeita. É a categoria mais perigosa porque não dispara alarme: os sistemas continuam consistentes consigo mesmos, propagando o erro para relatórios, faturas e backups.

2.3.1. Por que a integridade é a mais difícil das três

Confidencialidade e disponibilidade têm sinais externos claros: o vazamento aparece, a queda aparece. A perda de integridade é silenciosa por construção. Some-se a isso o fato de que o dano se **propaga** — dado errado alimenta cálculo, que alimenta relatório, que alimenta decisão — e que a **detecção tardia inviabiliza a correção**, porque o backup íntegro já saiu da janela de retenção.

Daí três consequências práticas frequentemente ignoradas:

- Retenção de backup deve ser dimensionada pelo **tempo esperado até a detecção**, não pelo tempo de recuperação. Trinta dias cobrem exclusão acidental; não cobrem corrupção lenta.
- Vale manter, além das diárias de curto prazo, cópias **mensais e anuais de longa retenção** — o esquema avô-pai-filho existe justamente para isso.
- Conciliação periódica com uma fonte independente (contagem física, extrato bancário, base de outro sistema) é o único mecanismo que detecta erro semântico consistente.

2.3.2. O que a defende

Soma de verificação e hash criptográfico para detecção; assinatura digital para detecção e atribuição; controle de versão e registro de alterações para reconstrução; transações e restrições de integridade referencial para prevenção; armazenamento imutável (WORM, bloqueio de objeto, cópia instantânea com retenção travada) para impedir sobrescrita — que é, hoje, o controle mais eficaz contra ransomware, porque ataca a estratégia do adversário de apagar as cópias antes de cifrar o original.

2.4. Disponibilidade

Disponibilidade é a garantia de que quem tem direito à informação a alcança quando precisa. É a propriedade mais fácil de medir e a que consome mais orçamento de infraestrutura.

A métrica canônica é o percentual de tempo em operação num período. A Figura 2.2 mostra o que cada patamar significa em minutos por ano — e é o argumento mais eficiente que existe contra a promessa de “99,99%” feita sem cálculo.

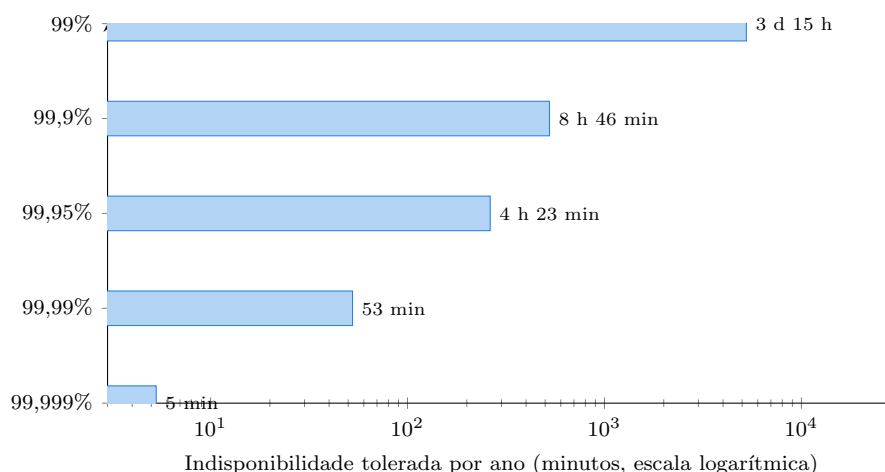


Figura 2.2.: Indisponibilidade anual tolerada por patamar de disponibilidade. Cada nove adicional custa aproximadamente uma ordem de grandeza a mais.

Dois cuidados de leitura. Primeiro, disponibilidade **composta**: se um serviço depende de quatro componentes em série, cada um com 99,9%, a disponibilidade do conjunto é $0,999^4 \approx 99,6\%$ — quatro vezes mais tempo fora do ar que o de cada peça isolada. Segundo, o número sozinho não descreve a experiência: 52 minutos fora do ar distribuídos em paradas de 30 segundos durante a madrugada e 52 minutos contínuos numa terça às 10h são o mesmo 99,99% e situações incomparáveis.

2.4.1. RTO e RPO

Disponibilidade em regime é uma coisa; recuperação depois de um desastre é outra, e tem duas métricas próprias, ilustradas na Figura 2.3.

O **RPO** (*recovery point objective*) é a quantidade máxima aceitável de dados perdidos, expressa em tempo: backup diário às 2h implica RPO de até 24 horas. O **RTO** (*recovery time objective*) é o tempo máximo aceitável até o serviço voltar. Ambos são definidos pela área de negócio e

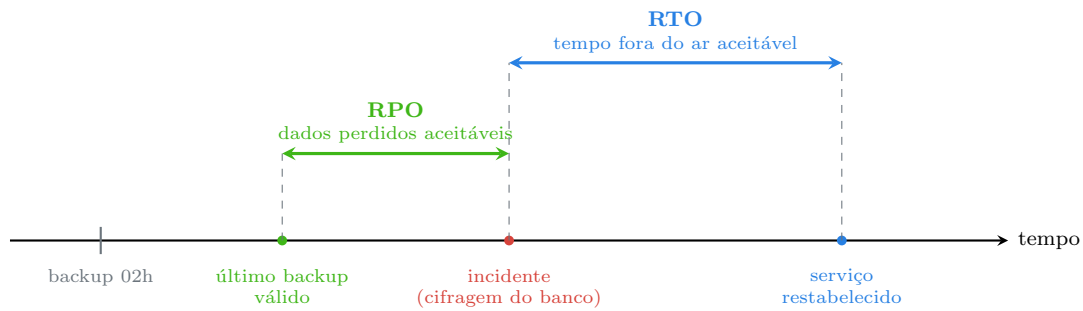


Figura 2.3.: RPO mede quanto trabalho se perde; RTO mede quanto tempo se fica parado. São decisões de negócio, não de infraestrutura.

depois traduzidos em arquitetura — a inversão comum, em que a infraestrutura existente define o RPO por acidente, é a razão de tantas surpresas no dia do incidente.

2.4.2. O que a ameaça e o que a defende

Contra disponibilidade jogam falha de hardware, erro humano, esgotamento de capacidade, dependência externa (energia, link, fornecedor de nuvem), evento ambiental e, por último na lista de frequência, ataque deliberado. Defendem-na redundância sem ponto único de falha, capacidade folgada, backup testado, monitoramento com alarme antes do limite, e — o item mais barato e mais esquecido — **procedimento escrito de recuperação que alguém já executou pelo menos uma vez**.

2.5. As três colidem

A Tabela 2.1 reúne os tradeoffs mais comuns. Reconhecê-los como escolhas, e não como falhas de projeto, é o que separa uma decisão de segurança de um reflexo.

Tabela 2.1.: Conflitos típicos entre os pilares. Nenhum tem resposta universal; todos têm resposta correta *dado o contexto*.

Decisão	Ganha	Pressiona
Cifrar backups	Confidencialidade	Disponibilidade (chave perdida = backup inútil)
Bloquear a conta após 3 erros de senha	Confidencialidade	Disponibilidade (negação de serviço trivial contra usuários)
Validação rigorosa de entrada	Integridade	Disponibilidade (rejeita o legítimo atípico)
Replicação síncrona entre sites	Disponibilidade e integridade	Custo e desempenho; propaga o erro lógico instantaneamente
Retenção longa de log	Integridade e responsabilização	Confidencialidade (mais dado sensível guardado por mais tempo)
Segundo fator em tudo	Confidencialidade	Disponibilidade (celular perdido trava o acesso) e aceitação do usuário

2. Os pilares: confidencialidade, integridade e disponibilidade

Decisão	Ganha	Pressiona
Alta disponibilidade por múltiplas cópias	Disponibilidade	Confidencialidade (mais lugares de onde vazar)

2.6. A ordem de prioridade muda com o contexto

Não existe hierarquia fixa entre os três pilares — existe a hierarquia daquele ambiente, e ela precisa ser declarada, porque é ela que resolve os conflitos da tabela anterior.

Tabela 2.2.: A ordem de prioridade é uma decisão de contexto e precisa estar escrita.

Contexto	Ordem típica	Por quê
Hospital, prontuário eletrônico	$D > I > C$	Prontuário inacessível na emergência custa vidas; o sigilo é gravíssimo, mas cede
Instituição financeira, núcleo bancário	$I > D > C$	Saldo errado destrói a instituição; a operação para até conciliar
Provedor de acesso, rede de transporte	$D > I > C$	O produto vendido é a conectividade; a métrica contratual é o tempo em operação
Provedor de acesso, base cadastral	$C > I > D$	CPF, endereço e meio de pagamento, sob a LGPD
Automação industrial e OT	$D > I > C$	Parada de linha e risco físico a pessoas
Órgão de inteligência e defesa	$C > I > D$	Vazamento é irreversível; indisponibilidade é contornável

Repare que o mesmo provedor aparece duas vezes com ordens opostas. Não é contradição: são **ativos diferentes**, e é exatamente por isso que a classificação da informação precede a escolha de controle.

2.7. Além da tríade

A tríade não é a última palavra. Duas extensões merecem registro.

Donn Parker propôs o **hexágono parkeriano**, acrescentando três propriedades que a tríade não captura bem (Parker, 1998): **posse ou controle** (perder o disco é dano ainda que ele esteja cifrado e nada tenha sido lido), **autenticidade** (a origem é quem afirma ser) e **utilidade** (o dado existe, íntegro e acessível, mas está num formato que ninguém consegue mais abrir — a fita sem leitor, o arquivo do sistema descontinuado). A distinção “posse” é a mais útil na prática: notificar um incidente à ANPD não depende de provar que alguém leu os dados.

A segunda extensão é a que este manual adota: **autenticidade, não repúdio e responsabilização** como propriedades de primeira classe, tratadas no Capítulo 3. Sem elas, não se descreve corretamente nem uma transação financeira nem uma trilha de auditoria.

2.8. Brasil em Foco

As três propriedades aparecem no direito brasileiro com nome próprio e consequência distinta para cada uma.

A **LGPD** trata a tríade explicitamente. O art. 6º lista entre os princípios do tratamento a **qualidade dos dados** (inciso V — exatidão, clareza, relevância e atualização, que é integridade em linguagem jurídica), a **segurança** (inciso VII — proteção contra acessos não autorizados e situações acidentais ou ilícitas de destruição, perda, alteração, comunicação ou difusão, que cobre os três pilares de uma vez) e a **prevenção** (inciso VIII) (Brasil, 2018). O art. 46 impõe o dever de adotar medidas técnicas e administrativas, e o art. 18 dá ao titular o direito de exigir **correção de dados incompletos, inexatos ou desatualizados** — um direito subjetivo de integridade, exercível contra a empresa. No incidente de abertura deste capítulo, cada assinante com vencimento trocado tinha esse direito.

Vale sublinhar o alcance do art. 48: o dever de comunicar incidente à ANPD e aos titulares **não se restringe a vazamento**. A redação fala em incidente de segurança que possa acarretar risco ou dano relevante, e a Resolução CD/ANPD nº 15/2024 detalha o rito e o prazo de três dias úteis (Autoridade Nacional de Proteção de Dados, 2024). Destruição acidental de base e indisponibilidade prolongada de dados pessoais são hipóteses de comunicação — a leitura de que “só notifica quem vaza” é incorreta e comum.

Do lado da **disponibilidade**, quem presta serviço de telecomunicações tem obrigação regulatória direta: a Anatel fixa indicadores de qualidade e disponibilidade no RQUAL (Resolução nº 717/2019), com coleta, meta e possibilidade de sanção (Agência Nacional de Telecomunicações, 2019). Disponibilidade, nesse caso, não é apenas expectativa do cliente — é indicador reportado ao regulador.

Do lado da **integridade**, o Marco Civil exige a guarda de registros de conexão por um ano e de acesso a aplicações por seis meses (arts. 13 e 15), sob sigilo e com disponibilização mediante ordem judicial (Brasil, 2014). Um log só cumpre sua função probatória se for íntegro e se o horário for confiável: sem NTP sincronizado e sem proteção contra alteração, o registro perde valor exatamente quando é necessário. O art. 13, §2º prevê ainda a requisição cautelar de guarda por prazo maior — o que só se atende se o dado ainda existir quando o pedido chegar.

No setor financeiro, a Resolução CMN nº 4.893/2021 exige política de segurança cibernética com controles de confidencialidade, integridade e disponibilidade dos dados e sistemas, além de plano de continuidade e de testes periódicos (Banco Central do Brasil, 2021). Para o Pix, a disponibilidade do arranjo é acompanhada pelo Banco Central com indicadores próprios, e a instituição responde pela indisponibilidade que afete a liquidação.

2.9. No Campo

O que muda concretamente para quem administra rede e servidores, organizado por pilar.

Confidencialidade. Cifragem de disco em todo equipamento que sai do prédio, sem exceção — notebook do comercial incluído. Cifragem também no backup, com a chave guardada em local independente do servidor (cofre de senhas da equipe, com cópia impressa em envelope lacrado, é solução legítima). TLS obrigatório em tudo, inclusive na rede interna. Revisão trimestral de quem tem acesso a quê:

2. Os pilares: confidencialidade, integridade e disponibilidade

```
# Quem consegue autenticar via SSH nesta máquina
grep -E '^(AllowUsers|AllowGroups|PermitRootLogin|PasswordAuthentication)'
↪ /etc/ssh/sshd_config
getent group sudo wheel adm 2>/dev/null

# Compartilhamentos Samba abertos a convidado - achado clássico
grep -rniE 'guest ok *= *yes|public *= *yes' /etc/samba/smb.conf

# Arquivos de configuração legíveis por qualquer usuário (senhas costumam morar
↪ aqui)
find /etc /opt /srv -type f \( -name '*.conf' -o -name '*.env' -o -name '*.ini'
↪ \) \
    -perm -o=r 2>/dev/null | head -40
```

Integridade. Verificação periódica de mídia e de backup, agendada e com alerta em caso de divergência:

```
# ZFS: varredura completa de blocos com verificação de soma
zpool scrub tank && zpool status tank

# Verificação de integridade do repositório de backup (restic)
restic -r /mnt/backup check --read-data-subset=10%

# Conferência de um artefato baixado, antes de instalar
sha256sum -c imagem.iso.sha256

# Saúde do disco antes de ele falhar - atributos que antecipam corrupção
smartctl -A /dev/sda | grep -Ei 'reallocated|pending|uncorrect|crc'
```

Três hábitos valem mais que qualquer ferramenta: **restaurar de verdade** um backup por trimestre e conferir o conteúdo, não só o código de retorno do comando; manter **retenção longa** (mensal por 12 meses, anual por 5) para cobrir detecção tardia; e **conciliar** mensalmente uma amostra do cadastro contra uma fonte independente.

Disponibilidade. Redundância que resolve ponto único de falha real, e não redundância decorativa: dois links do mesmo provedor entrando pela mesma canaleta são um link. Monitorar tendência, não só limiar — disco a 78% crescendo 2% ao dia é um incidente agendado para daqui a onze dias:

```
# Uso de disco por sistema de arquivos, dos mais cheios para os mais vazios
df -h --output=pcent,size,used,target -x tmpfs -x devtmpfs | sort -rn | head

# Reinicializações e quedas registradas
last -x reboot shutdown | head

# Serviços que falharam desde o último boot
systemctl --failed --no-pager
```

E a checagem que fecha o ciclo: escrever, para os três serviços mais críticos, o RTO e o RPO **acordados com quem responde pelo negócio**, comparar com o que a infraestrutura atual

entrega de fato, e levar a diferença para a mesa como decisão consciente de risco — que é o assunto do Capítulo 6.

2.10. Resumo

- Confidencialidade, integridade e disponibilidade são propriedades **independentes**: excelência em duas não compensa falha na terceira, e cada uma falha por caminhos próprios.
- A confidencialidade quase nunca falha por invasão — falha por excesso de permissão, perda física, metadado, inferência e vazamento pelo canal legítimo. Minimizar o dado coletado é o controle mais eficaz e o de menor custo.
- A integridade é a mais difícil das três porque falha em silêncio, propaga o dano e pode ser detectada tarde demais para o backup ajudar. Retenção deve ser dimensionada pelo tempo até a detecção, e conciliação com fonte independente é o que revela erro semântico.
- Disponibilidade é composta e se degrada em série: quatro componentes de 99,9% entregam 99,6%. **RPO** mede dados perdidos, **RTO** mede tempo parado, e ambos são decisão de negócio traduzida depois em arquitetura.
- Os pilares **conflitam**, e a ordem de prioridade depende do contexto e do ativo — hospital, banco e provedor ordenam de formas diferentes, e o mesmo provedor ordena diferente para a rede de transporte e para a base cadastral.
- No Brasil, os três têm âncora normativa: LGPD (arts. 6º, 18, 46 e 48, com comunicação de incidente em três dias úteis e direito de correção), Marco Civil (guarda íntegra de registros), Anatel RQUAL (disponibilidade como indicador regulado) e Resolução CMN 4.893/2021 no setor financeiro.

3. Autenticidade, não repúdio e responsabilização

Numa quinta-feira de junho, o setor financeiro de um provedor recebeu um e-mail do fornecedor de equipamentos ópticos com quem trabalhava havia seis anos. A mensagem vinha na mesma cadeia de respostas de uma negociação real em andamento, com o histórico completo abaixo, a assinatura de sempre e o mesmo tom. Informava mudança de banco e anexava a nota reemitida com os novos dados. O pagamento de R\$ 118 mil saiu no dia seguinte.

O domínio remetente não era `fornecedor.com.br`, e sim `fornecedor-com.br`. Nada mais no e-mail era falso: o histórico era autêntico, porque a caixa de um vendedor do fornecedor havia sido comprometida semanas antes e a conversa fora copiada dali. Não houve invasão do provedor, não houve malware, não houve nenhuma falha nos três pilares do Capítulo 2. Confidencialidade, integridade e disponibilidade da mensagem estavam preservadas: ela chegou íntegra, no horário, a quem devia. O que faltou foi uma propriedade diferente — **a garantia de que a origem é quem afirma ser**.

Dentro do mesmo provedor, na mesma semana, um segundo episódio menor: alguém executou no `ip route 0.0.0.0/0` no roteador de borda às 14h20 de uma quarta e derrubou a saída por dezenove minutos. O log do equipamento registrou o comando com precisão de segundo. Registrou também o usuário: `admin`. Como as sete pessoas da equipe técnica usavam `admin`, o log respondeu o que aconteceu e quando, e não respondeu a única pergunta que faltava.

Este capítulo trata das três propriedades que os pilares clássicos não cobrem: **autenticidade** (a origem é quem afirma ser), **não repúdio** (a origem não pode negar depois) e **responsabilização** (cada ação é atribuível a um sujeito identificado).

3.1. Quatro coisas diferentes que se confundem

Antes das três propriedades, é preciso separar quatro conceitos que a linguagem cotidiana funde e cuja confusão produz erros de arquitetura recorrentes. A Figura 3.1 mostra a cadeia.

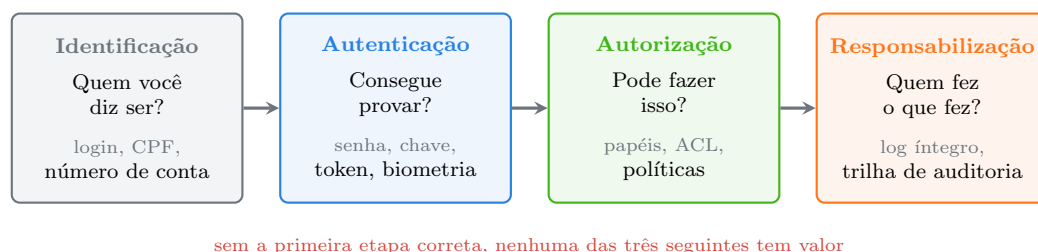


Figura 3.1.: Identificação, autenticação, autorização e responsabilização. Quatro perguntas distintas, quatro mecanismos distintos.

Identificação é a alegação: “sou o usuário `mmatos`”. É pública e não prova nada. **Autenticação** é a verificação dessa alegação contra uma evidência. **Autorização** é a decisão sobre o que aquele

3. Autenticidade, não repúdio e responsabilização

sujeito, já autenticado, pode fazer. E **responsabilização** é a capacidade, *depois*, de atribuir cada ação a um sujeito.

Duas confusões custam caro na prática. A primeira é tratar identificação como autenticação — é o que faz uma central de atendimento liberar mudança cadastral porque a pessoa “sabe o CPF”, quando o CPF é dado de identificação, presente em dezenas de bases e amplamente vazado. A segunda é presumir que autenticação forte resolve a responsabilização: se sete pessoas compartilham a credencial, o segundo fator apenas garante que *alguém do grupo* entrou.

3.2. Autenticidade

Autenticidade é a garantia de que uma entidade, mensagem ou dado é genuinamente o que ou de quem se afirma. Vem em três recortes:

- **Autenticidade de entidade.** Quem está do outro lado da conexão é mesmo aquele servidor, aquele usuário, aquele equipamento. É o que o certificado TLS resolve para o servidor e o que a autenticação resolve para o usuário.
- **Autenticidade de mensagem ou de dado.** Aquele arquivo, e-mail ou pacote realmente foi produzido pela origem alegada. É o que faltou no caso de abertura.
- **Autenticidade de origem em tempo real.** Além de vir da origem certa, a mensagem é *desta* conversa e não a repetição de uma anterior — o problema do frescor, tratado adiante.

Note que autenticidade e integridade caminham juntas mas não são a mesma coisa. Integridade responde “isto foi alterado?”; autenticidade responde “quem produziu isto?”. Um hash SHA-256 publicado ao lado de um arquivo dá integridade contra corrupção acidental, e não dá autenticidade nenhuma: quem alterou o arquivo altera o hash publicado na mesma página. A diferença é a existência de um **segredo** que só a origem legítima possui.

3.2.1. Os dois mecanismos, e por que não são intercambiáveis

Existem duas famílias criptográficas para autenticidade de mensagem, e a diferença entre elas é a diferença entre ter e não ter não repúdio (Figura 3.2).

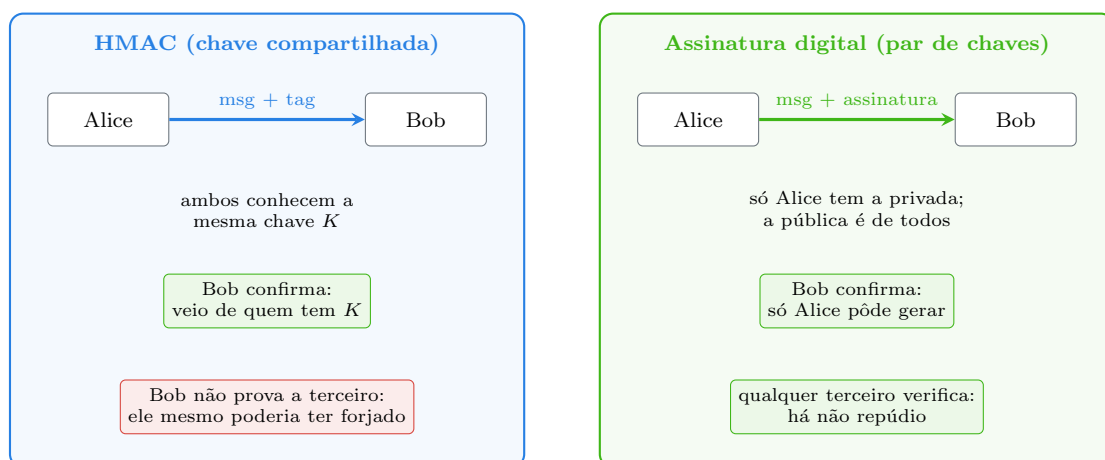


Figura 3.2.: Chave simétrica compartilhada dá autenticidade entre as partes; só o par de chaves assimétrico sustenta não repúdio perante terceiros.

Um **código de autenticação de mensagem** (MAC, tipicamente HMAC) usa uma chave secreta compartilhada entre remetente e destinatário. É rápido, simples e suficiente quando as duas partes confiam uma na outra e o objetivo é apenas detectar adulteração e impostor. Mas, como *ambos* possuem a chave, o destinatário poderia ter produzido a mesma tag — logo não consegue provar a um juiz que a mensagem veio do remetente.

Uma **assinatura digital** usa um par de chaves: a privada, que só a origem possui, gera a assinatura; a pública, que todos podem ter, a verifica. Isso permite que *qualquer terceiro* verifique, e é a base do não repúdio. A mecânica de ambas é assunto do Volume 3.

3.2.2. Autenticidade no e-mail: o caso de abertura resolvido

O correio eletrônico nasceu sem autenticidade nenhuma: o campo **From:** é preenchido livremente por quem envia, como o remetente escrito no verso de um envelope. Três mecanismos foram acrescentados depois:

- **SPF** declara, em registro DNS do domínio, quais servidores estão autorizados a enviar em seu nome. Verifica o envelope, não o **From:** que o usuário vê.
- **DKIM** assina criptograficamente cabeçalhos e corpo da mensagem com uma chave cujo par público está publicado no DNS do domínio. Dá autenticidade de origem e integridade, e sobrevive a encaminhamentos.
- **DMARC** amarra os dois ao domínio que o usuário efetivamente lê, define a política para quem falha na verificação (**none**, **quarantine**, **reject**) e produz relatórios.

E aqui está o ponto desconfortável do caso de abertura: **os três teriam passado**. O domínio **fornecedor-com.br** era real, registrado pelo atacante, com SPF e DKIM próprios corretamente configurados. A mensagem era autêntica — de um domínio que não era o esperado. Autenticidade criptográfica prova a origem; não prova que a origem é a *certa*. É por isso que o controle que resolve esse ataque não é técnico, e sim de processo: **mudança de dados bancários de fornecedor só vale com confirmação por canal independente**, telefone conhecido de cadastro anterior, nunca o número que veio no próprio e-mail.

3.2.3. Frescor: autenticidade não basta

Uma mensagem legítima, capturada e retransmitida depois, continua sendo autêntica — e é um ataque. É a **repetição** (*replay*): o comando de abertura de cancela gravado por rádio e reenviado à noite; a requisição de transferência interceptada e repetida cinco vezes.

Os antídotos são três, e quase todo protocolo sério usa alguma combinação deles: **nonce** (valor único por transação, que o destinatário recusa se repetido), **carimbo de tempo** com janela estreita de aceitação, e **contador de sequência** monotônico. É por isso que a sincronização de relógio (Capítulo 2) é requisito de segurança e não conveniência operacional: com relógios divergindo minutos, a janela de aceitação precisa ser alargada, e a janela alargada é exatamente a brecha para repetição.

3.3. Não repúdio

Não repúdio é a garantia de que quem praticou um ato não consiga negar tê-lo praticado, de forma sustentável perante um terceiro. É a propriedade mais mal-empregada do vocabulário da área, porque exige muito mais do que criptografia.

3. Autenticidade, não repúdio e responsabilização

Para que o não repúdio se sustente, cinco condições precisam valer ao mesmo tempo:

1. **Controle exclusivo da chave.** Se a chave privada estava num servidor a que cinco pessoas tinham acesso administrativo, ou numa planilha, “só o titular poderia ter assinado” é falso. Daí o uso de módulo de segurança em hardware e de cartão com chave não exportável.
2. **Vínculo confiável entre chave e pessoa.** Alguém precisa ter verificado a identidade do titular antes de emitir o certificado, e essa cadeia precisa ser auditável — é o papel de uma infraestrutura de chaves públicas, tema do Volume 3.
3. **Carimbo de tempo confiável.** Sem uma marca temporal de terceira parte, não se distingue uma assinatura feita antes da revogação da chave de outra feita depois.
4. **Integridade do que foi assinado.** A assinatura cobre um documento específico; o documento tem de ser preservado exatamente como assinado.
5. **Registro do processo.** Como a assinatura foi coletada, com que consentimento, sob que interface.

Falhando qualquer uma, o que resta é uma evidência forte — útil, muitas vezes decisiva — mas não uma impossibilidade de negar. Convém então distinguir dois níveis:

- **Não repúdio técnico:** a propriedade matemática de que só quem detém a chave privada poderia ter produzido aquela assinatura.
- **Não repúdio jurídico:** a aceitação, por um julgador, de que aquela pessoa praticou aquele ato. Admite contraprova — a alegação de chave roubada, de coação, de vício de consentimento.

Nenhum sistema entrega o segundo automaticamente. O que a engenharia entrega é a evidência que torna a negativa implausível, e o quanto de implausibilidade se consegue é proporcional ao cuidado com as cinco condições acima.

3.4. Responsabilização

Responsabilização (*accountability*) é a capacidade de atribuir cada ação relevante a um sujeito identificado, depois do fato. É a propriedade que sustenta investigação de incidente, auditoria, dissuasão interna e cumprimento de obrigação regulatória — e é a que o **admin** compartilhado do caso de abertura destrói por completo.

Seus requisitos são quatro:

Contas nomeadas e individuais. Não negociável. Toda credencial pertence a uma pessoa; contas de serviço pertencem a um sistema e têm dono humano registrado. Quando é tecnicamente necessário usar uma conta genérica — equipamento antigo que só aceita um usuário local —, o acesso deve passar por um intermediário que registre quem a usou: servidor de salto, ferramenta de acesso privilegiado, ou ao menos autenticação centralizada via RADIUS/TACACS+.

Registro suficiente. Quem, o quê, quando, de onde, com que resultado. Um log que registra “login bem-sucedido” sem o endereço de origem, ou “configuração alterada” sem dizer o quê, ocupa disco sem produzir responsabilização.

Horário confiável e uniforme. Correlacionar eventos entre firewall, servidor e aplicação exige que os três concordem sobre que horas são, e sobre qual fuso estão reportando.

Integridade do próprio registro. Este é o requisito mais negligenciado: um log que o administrador comprometido pode editar não responsabiliza ninguém. Quem invade um servidor limpa os rastros como parte da rotina. As defesas são o envio imediato para um coletor remoto

ao qual a máquina de origem não tem permissão de escrita retroativa, armazenamento imutável, e o **encadeamento por hash** ilustrado na Figura 3.3.

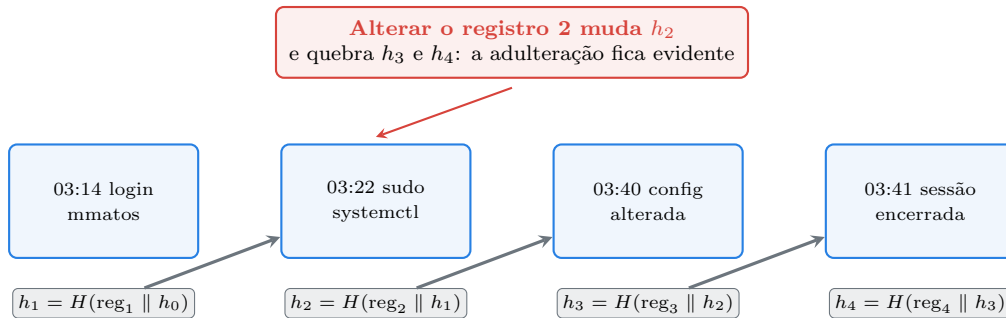


Figura 3.3.: Log encadeado por hash: cada registro incorpora o resumo do anterior, de modo que uma alteração isolada é detectável.

Vale registrar o limite dessa técnica: o encadeamento detecta alteração e remoção no meio da cadeia, mas não impede que um atacante com controle total *trunque* o log a partir de certo ponto e recomece a cadeia. Por isso o encadeamento se combina com publicação periódica da âncora — enviar o hash corrente para fora da máquina a cada intervalo — e com o envio remoto em tempo real. É a mesma lógica de defesa em profundidade do Capítulo 1: nenhuma camada isolada basta.

3.4.1. Segregação de funções

Responsabilização tem uma contraparte preventiva: **segregação de funções**. Quem executa não é quem aprova; quem administra o sistema não é quem audita os logs dele; quem cadastra o fornecedor não é quem libera o pagamento. É a aplicação direta do princípio de separação de privilégios de Saltzer e Schroeder (Saltzer; Schroeder, 1975), e é o que teria interrompido o golpe de abertura em duas frentes: alteração de dado bancário exigindo aprovação de segunda pessoa, e pagamento acima de um limite exigindo dupla assinatura.

Em equipes pequenas a segregação plena é impossível — três pessoas não se dividem em seis papéis. A alternativa aceita é **compensar por detecção**: se a mesma pessoa cadastra e paga, então um terceiro (o sócio, o contador) revisa mensalmente todos os cadastros bancários alterados. A regra sobrevive com custo diferente; o que não sobrevive é fingir que ela existe.

3.5. Como as seis propriedades se encaixam

A Tabela 3.1 reúne a tríade do Capítulo 2 e as três deste capítulo, com o mecanismo típico de cada uma.

Tabela 3.1.: As seis propriedades, a pergunta de cada uma e o mecanismo que a sustenta.

Propriedade	Pergunta que responde	Mecanismo típico
Confidencialidade	Quem consegue ler?	Controle de acesso, cifragem, minimização
Integridade	Foi alterado?	Hash, restrições, transações, armazenamento imutável

3. Autenticidade, não repúdio e responsabilização

Propriedade	Pergunta que responde	Mecanismo típico
Disponibilidade	Está acessível quando preciso?	Redundância, capacidade, backup testado
Autenticidade	Veio mesmo de quem diz?	HMAC, assinatura digital, certificados, DKIM
Não repúdio	A origem pode negar depois?	Assinatura com chave sob controle exclusivo, carimbo de tempo
Responsabilização	Quem fez o quê?	Conta nomeada, log íntegro, relógio sincronizado, segregação

A leitura mais útil dessa tabela é diagnóstica: diante de um incidente, perguntar qual das seis linhas falhou costuma apontar o controle ausente mais rápido do que qualquer outra heurística.

3.6. Brasil em Foco

O Brasil construiu, antes da maioria dos países, um arcabouço jurídico para autenticidade e não repúdio de documento eletrônico — e ele tem detalhes que mudam decisões técnicas.

A **Medida Provisória nº 2.200-2/2001** instituiu a ICP-Brasil e é a peça central (Brasil, 2001). Seu art. 10, §1º estabelece que declarações constantes de documento eletrônico assinado com certificado ICP-Brasil **presumem-se verdadeiras em relação aos signatários**. É presunção relativa — admite prova em contrário —, mas inverte o ônus: quem nega precisa provar que não assinou. O §2º do mesmo artigo é igualmente importante e menos citado: admite outros meios de comprovação de autoria e integridade, **desde que aceitos pelas partes ou pela pessoa a quem for oposto o documento**. Ou seja, assinatura fora da ICP-Brasil vale — só não carrega a presunção legal, e depende de acordo prévio ou de convencimento no caso concreto.

A **Lei nº 14.063/2020** organizou o campo em três níveis (Brasil, 2020):

Tabela 3.2.: Os três níveis de assinatura eletrônica da Lei 14.063/2020.

Nível	O que exige	Uso típico
Simple	Identifica o signatário e anexa dados em formato eletrônico	Interações de baixo risco; login em serviço público
Avançada	Controle exclusivo do signatário, detecção de alteração posterior, vínculo com a identidade — sem exigir ICP-Brasil	Atos de médio risco; assinatura gov.br
Qualificada	Certificado ICP-Brasil	Atos de maior risco, registro público, o que a lei exigir

O **Código de Processo Civil** completa o quadro: o art. 411 dispõe que se considera autêntico o documento quando assinado digitalmente na forma da lei, e o art. 441 admite documentos eletrônicos produzidos e conservados com observância da legislação específica. Na prática forense, log de servidor sem assinatura nem cadeia de custódia não é imprestável, mas seu peso probatório depende inteiramente da credibilidade do processo de coleta — assunto do Volume 13.

Para **responsabilização**, o Marco Civil impõe a guarda de registros (arts. 13 e 15) e submete sua disponibilização a ordem judicial (arts. 10 e 22), exigindo do requerente indícios do ilícito, identificação clara do usuário e período determinado (Brasil, 2014). O provedor precisa, portanto, conseguir responder *a quem pertencia aquele endereço IP naquele instante* — o que, num cenário de CGNAT, depende de registrar também a porta de origem e o horário com precisão. Registro sem porta em rede com NAT em larga escala responde por centenas de assinantes ao mesmo tempo e não individualiza ninguém.

A **LGPD** trata responsabilização como princípio expreso: o art. 6º, X exige a demonstração da adoção de medidas eficazes de cumprimento (*accountability*), e o art. 37 obriga o controlador e o operador a **manter registro das operações de tratamento** que realizarem (Brasil, 2018). Não é registro técnico apenas — é registro que precisa ser exibível à ANPD.

Do lado penal, a **Lei nº 12.737/2012** e a **Lei nº 14.155/2021** alcançam diretamente a falsidade de origem: o estelionato praticado por meio eletrônico ganhou forma qualificada no art. 171, §2º-A do Código Penal, com pena de reclusão de 4 a 8 anos, e o §2º-B agrava quando praticado contra idoso ou vulnerável (Brasil, 2021). A fraude do e-mail corporativo do caso de abertura é exatamente essa figura, e a investigação depende dos registros que o Marco Civil obriga a guardar.

Por fim, para quem opera telecomunicações: a Anatel exige identificação inequívoca do assinante no cadastro, e o **Pix** trouxe ao Banco Central um regime em que a autenticidade da ordem de pagamento é responsabilidade da instituição — com o Mecanismo Especial de Devolução para casos de fraude. É o Volume 10 que retoma o tema pelo lado da engenharia social.

3.7. No Campo

1. Acabar com a conta compartilhada. É a intervenção de maior retorno deste capítulo. Cada pessoa com sua conta em servidores, roteadores, OLTs, sistema de gestão e painel do provedor de nuvem. Em equipamento de rede, centralizar em RADIUS ou TACACS+ resolve simultaneamente a nomeação, a revogação no desligamento e o registro:

```
# Chaves SSH autorizadas - cada linha deve ter dono identificável no comentário
for u in $(awk -F: '$3>=1000 {print $1}' /etc/passwd); do
    f="/home/$u/.ssh/authorized_keys"
    [ -f "$f" ] && awk -v u="$u" '{print u, $NF}' "$f"
done

# Quem usou sudo nas últimas 24 h, e para quê
journalctl _COMM=sudo --since "24 hours ago" -o cat | grep -oP 'USER=\S+ ;
↳  COMMAND=.*'

# Contas com senha vazia ou sem expiração definida
awk -F: '($2 == "" ) {print "SEM SENHA:", $1}' /etc/shadow
```

2. Enviar o log para fora da máquina, no ato. Coletor remoto que a origem não pode reescrever. No **rsyslog**, uma linha basta; o essencial é o coletor **não** aceitar que o cliente apague o que já enviou:

3. Autenticidade, não repúdio e responsabilização

```
# /etc/rsyslog.d/90-remoto.conf - encaminha tudo, com fila em disco se o
↪ coletor cair
*. * action(type="omfwd" target="10.20.0.9" port="6514" protocol="tcp"
      StreamDriver="gtls" StreamDriverMode="1"
      queue.type="LinkedList" queue.filename="fwd"
      ↪ queue.saveOnShutdown="on")
```

3. Sincronizar o relógio em tudo. Servidor, roteador, switch, OLT, câmera, controlador de acesso. Diferença de fuso e horário de verão devem ser tratados registrando em UTC e convertendo na exibição:

```
timedatectl set-timezone America/Sao_Paulo
timedatectl set-ntp true
chronyc tracking | grep -E 'Reference ID|System time|Stratum'
```

4. Autenticar o e-mail do próprio domínio. SPF, DKIM e DMARC publicados, com política progressiva: começar em `p=none` para colher relatórios, corrigir os remetentes legítimos que falharem e só então subir para `quarantine` e `reject`:

```
dig +short TXT provedor.exemplo.br          # SPF
dig +short TXT _dmarc.provedor.exemplo.br    # DMARC
dig +short TXT seletor._domainkey.provedor.exemplo.br # DKIM
```

E o complemento defensivo contra o caso de abertura: registrar os domínios parecidos com o seu (troca de hífen, letras trocadas, `.com` por `.com.br`) e monitorar registros novos semelhantes ao seu domínio, que é o que viabiliza a fraude.

5. Um processo, não um produto, para pagamento. Alteração de dados bancários de fornecedor confirmada por telefone constante do cadastro anterior, nunca pelo contato informado na própria mensagem. Aprovação por segunda pessoa acima de um limite definido. Revisão mensal, por alguém de fora do processo, de todos os cadastros bancários alterados no período.

6. Assinar o que precisa sobreviver a uma disputa. Contrato com assinatura qualificada quando a lei exigir, avançada quando bastar. Pacote e imagem distribuídos para clientes assinados. Em repositório de código, assinar as versões publicadas:

```
# Verificar assinatura de um pacote antes de instalar
gpg --verify release-2.4.tar.gz.asc release-2.4.tar.gz

# Assinar as próprias tags de versão
git tag -s v2.4 -m "versão 2.4" && git tag -v v2.4
```

7. Testar a responsabilização com uma pergunta. Escolha uma alteração de configuração feita há três meses num equipamento de borda e tente responder, apenas com o que está registrado: quem fez, de onde, com que autorização, e quem aprovou. Se a resposta não sair em quinze minutos, o problema não é o log — é o desenho do processo.

3.8. Resumo

- **Identificação, autenticação, autorização e responsabilização** são quatro perguntas distintas. Tratar dado de identificação (CPF, número de contrato) como prova de identidade é a raiz de boa parte da fraude por atendimento.
- **Autenticidade** garante que a origem é quem afirma ser; integridade só garante que nada foi alterado. A diferença é a existência de um segredo detido pela origem — e é por isso que hash publicado ao lado do arquivo não autentica nada.
- **HMAC** dá autenticidade entre duas partes que compartilham a chave; só a **assinatura digital**, com chave privada sob controle exclusivo, sustenta **não repúdio** perante terceiros.
- SPF, DKIM e DMARC autenticam o *domínio*, não a *identidade comercial*: um domínio parecido, registrado pelo golpista e corretamente configurado, passa em todos os três. Contra isso, o controle é de processo — confirmação por canal independente.
- **Não repúdio técnico jurídico**. Exige controle exclusivo da chave, vínculo auditável com a pessoa, carimbo de tempo, preservação do documento e registro do processo; faltando um, resta evidência forte, não impossibilidade de negar.
- **Responsabilização** exige conta nomeada, registro suficiente, relógio sincronizado e log íntegro — encadeado por hash e enviado a um coletor remoto que a máquina de origem não pode reescrever. Conta **admin** compartilhada a destrói inteiramente.
- No Brasil: MP 2.200-2/2001 dá presunção de veracidade à assinatura ICP-Brasil e admite outros meios acordados entre as partes; a Lei 14.063/2020 estabelece os níveis simples, avançada e qualificada; o Marco Civil exige guarda de registros que individualizem o usuário (porta inclusa, sob CGNAT); a LGPD faz da *accountability* princípio e do registro de operações uma obrigação.

4. Ativos, ameaças, vulnerabilidades e risco

Numa sexta-feira à noite, o boletim do CERT.br e metade dos canais técnicos anunciam a mesma coisa: falha crítica de execução remota de código num servidor de aplicação amplamente usado, com exploração já observada na internet, nota máxima de severidade. O coordenador de TI do provedor lê o alerta e faz a única pergunta que importa: **nós temos isso rodando em algum lugar?**

Ninguém sabe responder. Há trinta e dois servidores em produção, três deles herdados de uma aquisição de 2021, dois em nuvem contratados diretamente pelo comercial sem passar pela TI, e um que “acho que só serve o portal antigo, mas ninguém desliga porque tem medo”. A resposta chega no domingo à tarde, depois de uma varredura improvisada: sim, dois servidores, um deles exposto à internet. O tempo entre a pergunta e a resposta — quarenta e uma horas — foi tempo de exposição pura, com exploração ativa acontecendo no mundo.

Essa é a diferença prática entre uma organização que gerencia risco e uma que reage a manchete. E ela começa não pela ferramenta de correção, mas pelo termo mais entediante do vocabulário: **inventário**.

O Capítulo 1 fixou o vocabulário mínimo numa tabela. Este capítulo abre cada um daqueles termos até o ponto em que eles viram método de trabalho: como levantar ativos, como caracterizar ameaças, como classificar vulnerabilidades com os identificadores que a indústria usa, e como combinar as três coisas em uma estimativa de risco que sustente uma decisão.

4.1. A cadeia que liga os quatro termos

Os quatro conceitos não são uma lista: são uma cadeia causal, e cada elo tem um tipo de controle que age sobre ele (Figura 4.1).

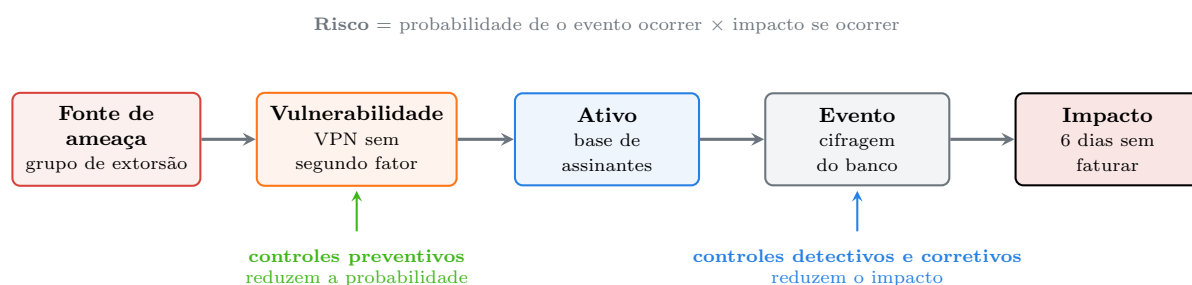


Figura 4.1.: A cadeia do risco. Não se elimina a fonte de ameaça; o trabalho recai sobre a vulnerabilidade (probabilidade) e sobre o impacto.

A leitura dessa cadeia responde a uma pergunta prática recorrente: onde investir? À esquerda, reduzindo a chance de o evento acontecer; à direita, reduzindo o estrago quando acontecer. Programas imaturos gastam quase tudo à esquerda. Programas maduros equilibram, porque a probabilidade nunca chega a zero e o custo marginal de reduzi-la cresce muito mais rápido que o de conter o impacto.

4.2. Ativos: o inventário é o alicerce

Ativo é qualquer coisa que tenha valor para a organização. O Capítulo 1 separou ativos **primários** (a informação e os processos de negócio) de ativos de **suporte** (hardware, software, rede, instalações, pessoas). Falta agora o método: como construir e manter o inventário que responde à pergunta de sexta-feira à noite.

4.2.1. O que precisa constar

Um inventário útil não é uma lista de nomes de máquina. Para cada ativo relevante, cinco campos fazem a diferença entre planilha decorativa e ferramenta de decisão:

Tabela 4.1.: Campos mínimos de um inventário de ativos que sustenta decisão.

Campo	Por que importa
Identificação e localização	Onde está fisicamente e logicamente (endereço, VLAN, provedor de nuvem, conta)
Dono	Pessoa nomeada que decide sobre ele — não “a TI”
Classificação	Qual informação trafega ou repousa ali, e sua sensibilidade
Dependências	De que depende e quem depende dele; alimenta a análise de impacto
Versões e fim de suporte	Sistema operacional, aplicação, firmware — é o campo que responde à pergunta do boletim

O campo **dono** é o que costuma faltar e o que mais paralisa a resposta a incidente: sem alguém nomeado, a decisão de desligar um servidor às 23h de domingo não tem quem a tome.

4.2.2. Como levantar, na prática

Nenhuma fonte isolada é completa; a interseção de várias é o que aproxima da realidade:

- **Varredura ativa** da própria faixa de endereços — encontra o que responde.
- **Escuta passiva** de ARP, DHCP e fluxos (NetFlow/sFlow) — encontra o que existe mas não responde a varredura, e o que aparece só de vez em quando.
- **Fontes administrativas** — faturas do provedor de nuvem, contratos de licença, notas fiscais de compra, registros de domínio. É onde aparecem os dois servidores que o comercial contratou sozinho.
- **Diretório e gerência de configuração** — objetos do Active Directory, agentes instalados, gerenciador de pacotes.
- **A conversa.** Perguntar a cada área “que sistema você usa que a TI talvez não conheça?” produz achados que nenhuma varredura produz, porque nuvem contratada por cartão de crédito não aparece na rede interna.

O que se descobre nesse cruzamento tem nome: **TI sombra** (*shadow IT*) e **ativos órfãos**. São, sistematicamente, os ativos com pior postura de segurança do ambiente, porque não estão em nenhuma rotina de correção, backup ou monitoramento.

4.2.3. Superfície de ataque

Do inventário deriva o conceito operacional mais útil deste capítulo: a **superfície de ataque** é o conjunto de pontos por onde um adversário pode interagir com o sistema. Reduzi-la é o controle de melhor retorno que existe, porque um serviço que não está exposto não precisa ser corrigido com urgência.

Ela tem três dimensões, e a terceira é a esquecida: a superfície de **rede** (portas e serviços alcançáveis), a de **software** (código, bibliotecas, dependências, funcionalidades habilitadas) e a **humana** (quem pode ser alvo de engenharia social, quantas pessoas têm privilégio administrativo, quantos terceiros têm acesso).

4.3. Ameaças: quem ou o quê, e com que capacidade

Ameaça é a causa potencial de um incidente indesejado. O Capítulo 1 apresentou as três origens (deliberada, accidental, ambiental) e os perfis de agente. O que falta é como *usar* isso: caracterizar uma ameaça com três atributos.

Fonte. Quem ou o quê. Para agentes deliberados, o perfil (oportunista automatizado, crime organizado, pessoa interna, ativismo, estado-nação); para os demais, o fenômeno (erro de operação, falha de equipamento, descarga atmosférica).

Capacidade e recursos. O oportunista automatizado usa exploração pública contra falha conhecida e desiste diante de qualquer atrito. O crime organizado compra acesso inicial, tem semanas de paciência e um orçamento. Um estado-nação desenvolve exploração própria para falha desconhecida. A capacidade determina quais controles fazem diferença: bloquear o primeiro é barato; conter o terceiro é outro orçamento.

Motivação e alvo. O oportunista não escolheu você; o crime organizado escolheu seu *setor*; a pessoa interna insatisfeita escolheu *você especificamente*. Isso muda a resposta: contra o indiscriminado, higiene básica resolve; contra o direcionado, é preciso assumir comprometimento e investir em detecção.

Catálogos ajudam a não depender de imaginação. O **MITRE ATT&CK** organiza táticas e técnicas efetivamente observadas em ataques reais, e serve tanto para modelar ameaça quanto para medir cobertura de detecção (MITRE Corporation, 2024); a ISO/IEC 27005 traz listas de fontes de ameaça e de vulnerabilidades típicas (International Organization for Standardization, 2022). O Volume 11 dedica um capítulo ao ATT&CK.

4.4. Vulnerabilidades: quatro tipos, não um

Vulnerabilidade é a fraqueza que pode ser explorada por uma ameaça. A palavra evoca falha de software, e essa associação estreita esconde três quartos do problema:

- **De software.** Erro de implementação ou de projeto no código: estouro de buffer, injeção, desserialização insegura. É a que recebe identificador CVE.
- **De configuração.** O software está correto; o modo como foi configurado, não. Credencial de fábrica, serviço administrativo exposto, permissão excessiva, cifra fraca habilitada, registro desligado. **É a categoria mais frequente em ambientes reais** e a que nenhum boletim avisa, porque não é falha do fornecedor.
- **De processo.** Não há revisão de acessos, o desligamento não revoga credenciais, o backup nunca é testado, não existe procedimento de resposta. Não aparece em varredura alguma.

4. Ativos, ameaças, vulnerabilidades e risco

- **Humana.** Suscetibilidade a engenharia social, senha reaproveitada, aprovação sem conferência. Tema do Volume 10.

Um programa de segurança que só trata a primeira categoria está tratando a minoria dos casos com a maior parte do orçamento.

4.4.1. O ciclo de vida de uma vulnerabilidade e a janela de exposição

Toda vulnerabilidade de software percorre uma linha do tempo, e o que importa nela são os intervalos, não os pontos (Figura 4.2).

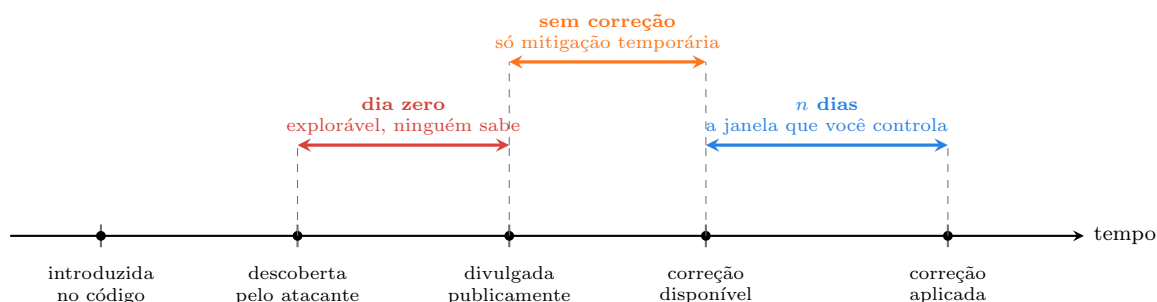


Figura 4.2.: A linha do tempo de uma vulnerabilidade. O intervalo de n dias é o único que depende inteiramente da organização.

O ponto central da figura é político, não técnico. Contra o dia zero há pouco a fazer além de defesa em profundidade e detecção. Mas a janela de n dias — entre a correção existir e a correção estar aplicada — é integralmente responsabilidade da organização, e é onde a esmagadora maioria dos comprometimentos reais acontece. Atacantes não gastam dia zero em provedor regional: usam falhas com correção publicada há meses.

Vale a distinção: **exploração** (*exploit*) é o código ou a técnica que tira proveito da falha; **prova de conceito** demonstra que é possível; **exploração armada** é a versão confiável e automatizada. O intervalo entre divulgação e exploração armada disponível encurtou de meses para dias, e às vezes horas.

4.4.2. Os identificadores que a indústria usa

Cinco siglas aparecem em todo boletim, e cada uma responde a uma pergunta diferente. Confundi-las leva a priorizar errado.

Tabela 4.2.: Os identificadores de vulnerabilidade e a pergunta de cada um.

Sigla	O que é	Pergunta que responde
CVE	Identificador único de uma vulnerabilidade concreta em um produto	<i>Qual</i> falha é essa?
CWE	Catálogo de tipos de fraqueza (ex.: CWE-89, injeção de SQL)	De que <i>classe</i> ela é?

Sigla	O que é	Pergunta que responde
CVSS	Pontuação de severidade de 0 a 10, com métricas base, de ameaça e ambientais	Quão <i>grave</i> ela é?
EPSS	Probabilidade estimada de exploração nos próximos 30 dias	Quão <i>provável</i> é ser explorada?
KEV	Catálogo de vulnerabilidades com exploração confirmada em ataques reais	Já está sendo <i>usada</i> ?

O erro mais comum e mais caro é tratar **CVSS como fila de prioridade**. O CVSS mede severidade *intrínseca*, sob a premissa do pior caso (Forum of Incident Response and Security Teams, 2023). Numa base típica, milhares de itens pontuam 7,0 ou mais — corrigir por ordem de nota significa gastar semanas em falhas que ninguém explora, enquanto uma de nota 6,5 com exploração ativa segue aberta.

A priorização que funciona combina três eixos, ilustrados na Figura 4.3: severidade (CVSS, ajustado pelas métricas ambientais que refletem *seu* contexto), probabilidade de exploração (EPSS e KEV) e **exposição real do ativo** — se o serviço vulnerável está atrás de VPN, sem alcance da internet, a urgência é outra.

4.5. Risco: juntando tudo

Risco é o efeito da incerteza sobre os objetivos (International Organization for Standardization, 2018b) — e, na formulação operacional que a segurança da informação usa, a combinação da probabilidade de um evento e do seu impacto:

$$\text{Risco} = P(\text{evento}) \times \text{Impacto}(\text{evento})$$

A fórmula é útil e é uma simplificação perigosa se levada ao pé da letra, por três motivos que valem registro desde já:

1. **Nem todo impacto é comparável.** Um evento com 1% de chance e R\$ 10 milhões de impacto e outro com 100% de chance e R\$ 100 mil de impacto têm o mesmo valor esperado e exigem decisões completamente diferentes: o segundo é custo operacional, o primeiro é ameaça à existência da empresa.
2. **As distribuições têm cauda pesada.** Perdas por incidente não se distribuem em torno de uma média bem-comportada; o valor esperado subestima sistematicamente o cenário ruim.
3. **A probabilidade raramente é conhecida.** É estimada, e a qualidade da estimativa é o que o Capítulo 6 trata em detalhe.

Duas distinções completam o quadro. **Risco inerente** é o risco antes de qualquer controle; **risco residual** é o que sobra depois deles aplicados — e é o número que a direção precisa conhecer e aceitar formalmente. E o impacto tem camadas: o **direto** (horas de indisponibilidade, custo de recuperação, dado perdido) costuma ser o menor; o **indireto** (multa da ANPD, ação de titulares, perda de assinantes, custo de aquisição para repor a base, reputação junto ao regulador) é que decide a sobrevivência da empresa.

4. Ativos, ameaças, vulnerabilidades e risco

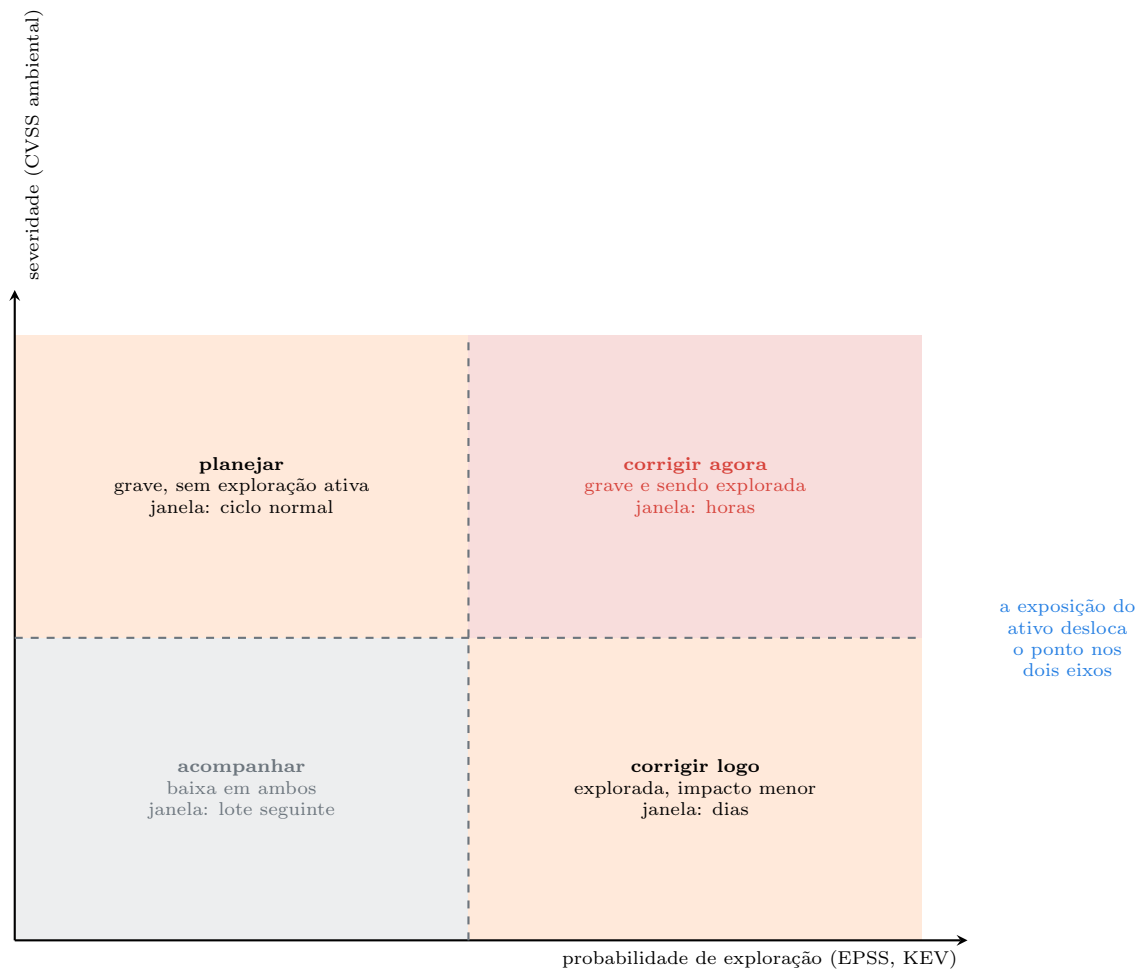


Figura 4.3.: Priorização por severidade e por probabilidade de exploração. A nota isolada do CVSS não distingue o quadrante superior esquerdo do superior direito.

4.5.1. Controles: três funções, três naturezas

Fechando a cadeia, **controle** é a medida que modifica o risco. Duas classificações que se cruzam e organizam qualquer conversa sobre o assunto:

Por **função**: **preventivo** (impede o evento — segundo fator, segmentação, menor privilégio), **detectivo** (revela que ocorreu — registro, alerta, integridade de arquivo), **corretivo** (restaura — backup, plano de resposta), e ainda **dissuasivo** (desestimula — aviso legal, câmera visível) e **compensatório** (substitui um controle inviável por outro de efeito equivalente).

Por **natureza**: **técnico** (firewall, cifragem), **administrativo** (política, procedimento, treinamento, contrato) e **físico** (fechadura, cerca, controle de acesso à sala).

A regra prática que decorre da Figura 4.1: um conjunto de controles só é defensável se cobrir as três funções. Só preventivos significa que, no dia em que um falhar, ninguém fica sabendo; só detectivos significa assistir ao incidente acontecer.

4.6. Brasil em Foco

A **LGPD** transformou inventário e análise de risco em obrigação documental. O art. 37 exige que controlador e operador **mantenham registro das operações de tratamento** de dados pessoais que realizarem — na prática, um inventário de ativos primários com finalidade, base legal, categorias de titulares, prazo de retenção e compartilhamentos (Brasil, 2018). O art. 38 faculta à ANPD determinar a elaboração de **relatório de impacto à proteção de dados pessoais**, documento que descreve os tratamentos, os riscos às liberdades civis e as medidas de mitigação — ou seja, exatamente a cadeia deste capítulo, escrita.

O art. 50 introduz as **regras de boas práticas e governança**, que a lei sugere considerando “a natureza, o escopo, a finalidade, a probabilidade e a gravidade dos riscos” — a linguagem é de gestão de risco, não de lista de controles obrigatórios. Isso tem consequência: no Brasil, não existe uma lista fechada de medidas que, cumprida, garante conformidade. O que se demonstra é o **processo**, e o processo começa por saber o que se tem.

Para agentes de tratamento de **pequeno porte**, a Resolução CD/ANPD nº 2/2022 flexibilizou obrigações, e a ANPD publicou guia orientativo de segurança da informação para esse público, com um conjunto mínimo de medidas — leitura recomendada para provedores regionais e empresas de suporte, por ser o parâmetro que a própria autoridade declara esperar (Autoridade Nacional de Proteção de Dados, 2023).

Do lado da telecomunicação, a Resolução Anatel nº 740/2020 exige que produtos de telecomunicações atendam a requisitos de segurança cibernética — sem senha padrão universal, com mecanismo de atualização de firmware e com correção de vulnerabilidades conhecidas (Agência Nacional de Telecomunicações, 2020). Para quem entrega ONU e roteador ao assinante, isso significa incluir o **parque de equipamentos de cliente no inventário**: são dezenas de milhares de ativos sob responsabilidade do provedor, e um firmware vulnerável em massa é simultaneamente risco ao assinante, risco à rede e questão regulatória.

No setor financeiro, a Resolução CMN nº 4.893/2021 exige classificação de dados e sistemas por relevância, avaliação de riscos e controles proporcionais, além de exigir a gestão de risco de **serviços de terceiros** relevantes (Banco Central do Brasil, 2021) — o que puxa fornecedores e integradores para dentro do escopo.

Institucionalmente, o **CERT.br** publica estatísticas de incidentes notificados e mantém material de boas práticas para redes; para órgãos federais, o **CTIR Gov** emite alertas de vulnerabilidade

4. Ativos, ameaças, vulnerabilidades e risco

e coordena resposta. Acompanhar essas fontes, mais o catálogo de vulnerabilidades exploradas conhecidas mantido pela CISA norte-americana, é o mínimo para não descobrir por manchete.

4.7. No Campo

1. Construir o inventário em uma semana, não em um projeto de seis meses. A versão de 20% que existe vale infinitamente mais que a versão completa que nunca fica pronta. Uma planilha com os cinco campos da Tabela 4.1, preenchida com o cruzamento de varredura, faturas e conversa, já responde à pergunta de sexta-feira à noite:

```
# Ativos que respondem na rede, com serviço e versão detectados
nmap -sS -sV -Pn --top-ports 500 --open -oA inventario 10.20.0.0/22

# Aprendizado passivo: quem já falou na rede recentemente
ip neigh show | awk '$4=="lladdr" {print $1, $5}' | sort -V

# Concessões de DHCP ativas - pega o que não estava ligado durante a varredura
grep -E 'lease|binding state active' /var/lib/dhcp/dhcpd.leases | head -40
```

2. Registrar dono e classificação, não só endereço IP. É o que transforma inventário em ferramenta de decisão. Sem dono nomeado, nenhuma decisão urgente sai do lugar.

3. Medir a superfície exposta e persegui-la até zero. Serviço administrativo alcançável da internet é o achado mais comum e mais caro. A meta não é reduzir: é zerar.

```
# O que este servidor escuta, em quais interfaces, e qual processo responde
ss -tulnp | awk 'NR==1 || $5 !~ /^(127\.|\[::1\])/ '

# Conferir de fora: o que o mundo enxerga do seu bloco
nmap -Pn -p- --open -T3 198.51.100.7

# Regras de firewall que permitem qualquer origem
iptables -S | grep -E '^-A (INPUT|FORWARD)' | grep -v ' -s '
```

4. Priorizar correção por exploração ativa, não por nota. A fila é: está no catálogo de exploração conhecida e o ativo está exposto → hoje. EPSS alto → esta semana. CVSS alto sem sinal de exploração e ativo interno → no ciclo mensal.

```
# Pacotes com atualização de segurança pendente (Debian/Ubuntu)
apt-get -s upgrade | grep -i security | awk '{print $2}' | sort -u

# Versões instaladas x fim de suporte - o campo que responde ao boletim
hostnamectl | grep -E 'Operating System|Kernel'
lsb_release -d 2>/dev/null; cat /etc/os-release | grep -E '^VERSION'
```

5. Manter um registro de risco de uma página. Dez linhas bastam: ativo, ameaça, vulnerabilidade, impacto estimado, probabilidade estimada, controle atual, controle proposto, dono, prazo, decisão. É o insumo do Capítulo 6 e o documento que a ANPD pede quando pergunta como a empresa avaliou seus riscos.

6. Incluir o parque de equipamentos de assinante. Modelo, versão de firmware, se aceita atualização remota, se tem credencial padrão. Dezenas de milhares de dispositivos com a mesma falha são o maior ativo de risco de um provedor — e o de maior potencial de dano a terceiros.

7. Assinar as fontes de alerta e definir quem lê. Boletins do CERT.br, avisos do fabricante dos equipamentos em uso, catálogo de vulnerabilidades exploradas. Um responsável nomeado, revisão semanal, e um caminho definido para escalar o que for urgente fora do horário. Alerta que chega numa caixa que ninguém abre é o mesmo que alerta que não chegou.

4.8. Resumo

- Os quatro termos formam uma **cadeia causal**: a fonte de ameaça explora uma vulnerabilidade de um ativo, produzindo um evento com impacto. Controles preventivos agem sobre a probabilidade; detectivos e corretivos, sobre o impacto — e um programa defensável cobre as duas pontas.
- **Inventário é o alicerce.** Sem dono nomeado, classificação e versões registradas, a pergunta “nós temos isso?” não tem resposta rápida, e a demora é exposição pura. TI sombra e ativos órfãos são sistematicamente os piores do ambiente.
- Vulnerabilidade não é só falha de software: as de **configuração, de processo e humanas** somam a maioria dos casos reais e não aparecem em boletim nem em varredura.
- A janela de *n dias* — entre a correção existir e estar aplicada — é a única inteiramente sob controle da organização, e é onde ocorre a maior parte dos comprometimentos. Dia zero é raro contra alvos comuns.
- **CVE, CWE, CVSS, EPSS e KEV** respondem perguntas diferentes. Priorizar pela nota do CVSS é o erro caro mais comum: a fila correta combina severidade ajustada ao contexto, probabilidade de exploração e exposição real do ativo.
- **Risco = probabilidade × impacto** é útil como organizador e enganoso como número único: impactos catastróficos e recorrentes não são intercambiáveis, as distribuições têm cauda pesada, e o impacto indireto costuma superar o direto.
- No Brasil, a LGPD converte inventário (art. 37) e avaliação de risco (arts. 38 e 50) em obrigação demonstrável; a Anatel 740/2020 traz o parque de equipamentos do assinante para dentro do escopo; e a Resolução CMN 4.893/2021 estende a gestão de risco aos fornecedores relevantes.

5. Modelagem de ameaças: STRIDE, árvores de ataque e abuse cases

O provedor lançou o portal do assinante em quatro meses de desenvolvimento. Funcionava: segunda via de fatura, alteração de plano, abertura de chamado, histórico de consumo. Autenticação por CPF e senha, sessão com expiração, tráfego em HTTPS com certificado válido, senhas armazenadas com hash forte. Os testes de aceitação passaram todos.

Três semanas depois do lançamento, um assinante mandou uma mensagem para o suporte, sem qualquer má intenção: ele havia percebido que a URL da fatura terminava em `/fatura/48213`, trocou o número por `48212` e apareceu a fatura de outra pessoa — nome, CPF, endereço, valor. Testou mais alguns. Todos abriam.

A falha é conhecida, tem nome (referência direta insegura a objeto), e leva talvez vinte minutos para corrigir. O que custou caro não foi a correção: foi que 4.000 assinantes ficaram com dados pessoais expostos por três semanas, e que o incidente virou comunicação à ANPD.

O detalhe que interessa a este capítulo é outro. Nenhum teste teria pego isso, porque todos os testes verificavam que **o sistema faz o que deve fazer**. Ninguém verificou que ele **não faz o que não deve**. A pergunta “e se o usuário trocar o número na URL?” não custa nada, não exige ferramenta e leva menos de um minuto — desde que alguém a faça. Fazer sistematicamente essa classe de pergunta, antes de construir, é o que se chama modelagem de ameaças.

5.1. O que é, e quando fazer

Modelagem de ameaças é o exercício estruturado de antecipar como um sistema pode ser atacado, antes de ele existir ou antes de ele mudar. Adam Shostack a resume em quatro perguntas (Shostack, 2014), e a virtude do método está inteiramente na disciplina de responder as quatro, na ordem:

1. **O que estamos construindo?** Um desenho do sistema que todos concordam que está correto.
2. **O que pode dar errado?** A parte criativa, guiada por um catálogo para não depender de inspiração.
3. **O que vamos fazer a respeito?** Cada ameaça vira uma decisão com dono e prazo.
4. **Fizemos um bom trabalho?** Revisão do próprio exercício.

A terceira e a quarta perguntas são as que separam modelagem de conversa. Um modelo que produz uma lista de ameaças e nenhuma decisão registrada não mudou nada.

Sobre **quando**: o retorno é máximo quando ainda é barato mudar — ao desenhar um sistema novo, ao alterar arquitetura, ao integrar um terceiro, ao expor algo que antes era interno, ao mudar quem pode acessar o quê. Não é ritual anual; é gatilho por mudança. E não precisa ser pesado: uma sessão de noventa minutos com quadro branco, a pessoa que desenvolve, a que opera a infraestrutura e alguém do negócio cobre a maior parte do valor.

5.2. Pergunta 1: desenhar o sistema

O formato que melhor sustenta o exercício é o **diagrama de fluxo de dados**, com cinco elementos e uma convenção visual estável:

- **Entidade externa** — quem interage e está fora do seu controle (assinante, integração de terceiro).
- **Processo** — código que executa e transforma dados (aplicação, API, serviço).
- **Armazenamento** — onde os dados repousam (banco, arquivo, fila, cache).
- **Fluxo de dados** — o que trafega, e por qual canal.
- **Fronteira de confiança** — a linha em que o nível de confiança muda. É o elemento mais importante do diagrama.

A regra prática que rende mais: **ameaças se concentram onde os fluxos cruzam fronteiras de confiança**. Um desenho sem fronteiras marcadas não é um modelo de ameaças, é um diagrama de arquitetura. A Figura 5.1 desenha o portal do caso de abertura.

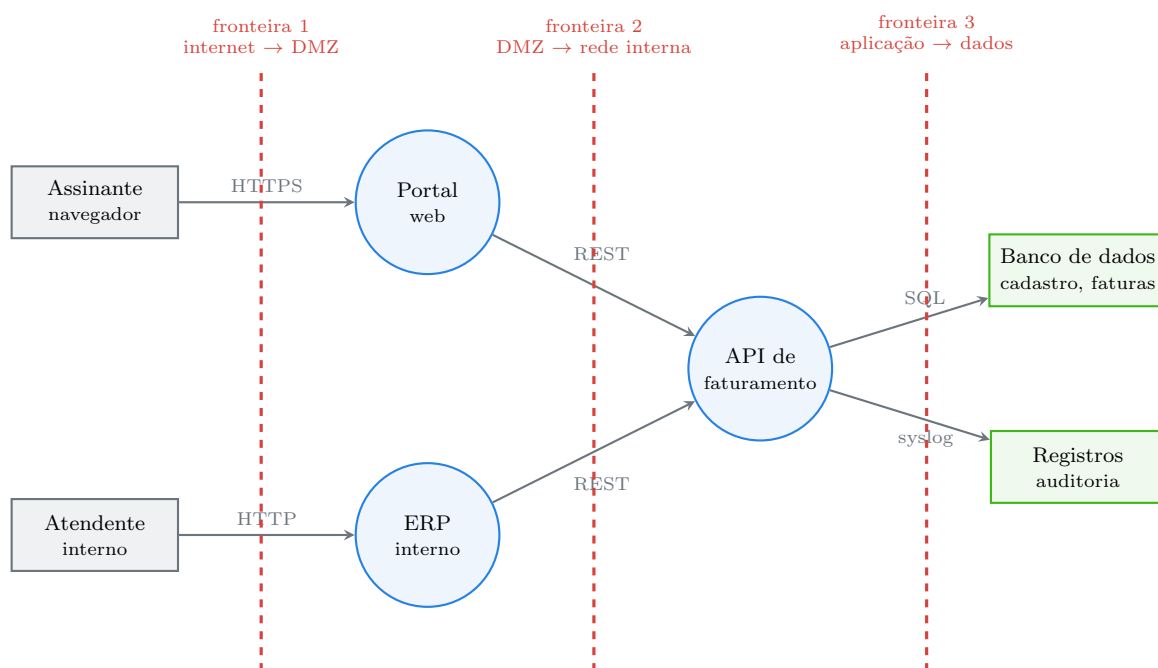


Figura 5.1.: Diagrama de fluxo de dados do portal do assinante. As três linhas vermelhas são onde o esforço de análise se concentra.

Duas dicas de execução. Primeira: desenhe no nível de detalhe em que as decisões são tomadas — um diagrama com trinta caixas não é analisado por ninguém. Segunda: valide o desenho com quem construiu o sistema. É frequente descobrir, nessa validação, um fluxo que não estava no desenho e que ninguém lembrava que existia — e esse fluxo esquecido é, com frequência desconfortável, exatamente onde está o problema.

5.3. Pergunta 2: o que pode dar errado

5.3.1. STRIDE

O **STRIDE** é o catálogo mais usado, e sua elegância está em ser o complemento exato das propriedades de segurança dos capítulos anteriores: cada letra é a violação de uma propriedade.

Tabela 5.1.: STRIDE e sua correspondência com as propriedades de segurança.

Letra	Ameaça	Propriedade violada	Exemplo no portal	Contramedida típica
S	<i>Spoofing</i> — falsificação de identidade	Autenticidade	Entrar como outro assinante com CPF vazado e senha fraca	Autenticação forte, segundo fator, TLS mútuo
T	<i>Tampering</i> — adulteração	Integridade	Alterar o valor da fatura num parâmetro da requisição	Validação no servidor, assinatura, restrições no banco
R	<i>Repudiation</i> — repúdio	Não repúdio	Negar que pediu a mudança de plano	Log íntegro, conta nomeada, carimbo de tempo
I	<i>Information disclosure</i> — vazamento	Confidencialidade	Trocar o número da URL e ver a fatura alheia	Autorização por objeto, minimização, cifragem
D	<i>Denial of service</i> — negação de serviço	Disponibilidade	Esgotar a API com requisições em massa	Limitação de taxa, cota, dimensionamento, filtragem
E	<i>Elevation of privilege</i> — elevação de privilégio	Autorização	Assinante alcançar função de atendente	Menor privilégio, verificação de papel no servidor

A falha do caso de abertura é um **I** clássico — e ela ilustra por que o catálogo funciona: bastava alguém percorrer as seis letras para o fluxo “assinante consulta fatura” e perguntar “como haveria vazamento de informação aqui?”.

A técnica que sistematiza isso chama-se **STRIDE por elemento**: em vez de olhar o sistema inteiro de uma vez, aplicar as letras relevantes a cada elemento do diagrama. Nem todas se aplicam a todos:

- **Entidade externa:** S, R
- **Processo:** todas as seis
- **Armazenamento:** T, I, D (e R, se for o armazenamento dos próprios logs)
- **Fluxo de dados:** T, I, D

Para o portal da Figura 5.1, isso significa cerca de trinta perguntas — algumas descartadas em segundos, outras rendendo achados. É trabalho de uma tarde, e é a razão pela qual a modelagem tem retorno tão desproporcional ao custo.

5.3.2. Árvores de ataque

O STRIDE responde “que categorias de coisa podem dar errado”. A **árvore de ataque**, proposta por Bruce Schneier, responde a uma pergunta diferente e complementar: *dado um objetivo do atacante, quais são todos os caminhos até ele?* (Schneier, 1999)

A raiz é o objetivo; os nós internos são submetas; as folhas são ações concretas. Os nós se combinam de duas formas: **OU** (qualquer filho basta) e **E** (todos os filhos são necessários). A Figura 5.2 monta a árvore para o objetivo mais realista no contexto de um provedor.

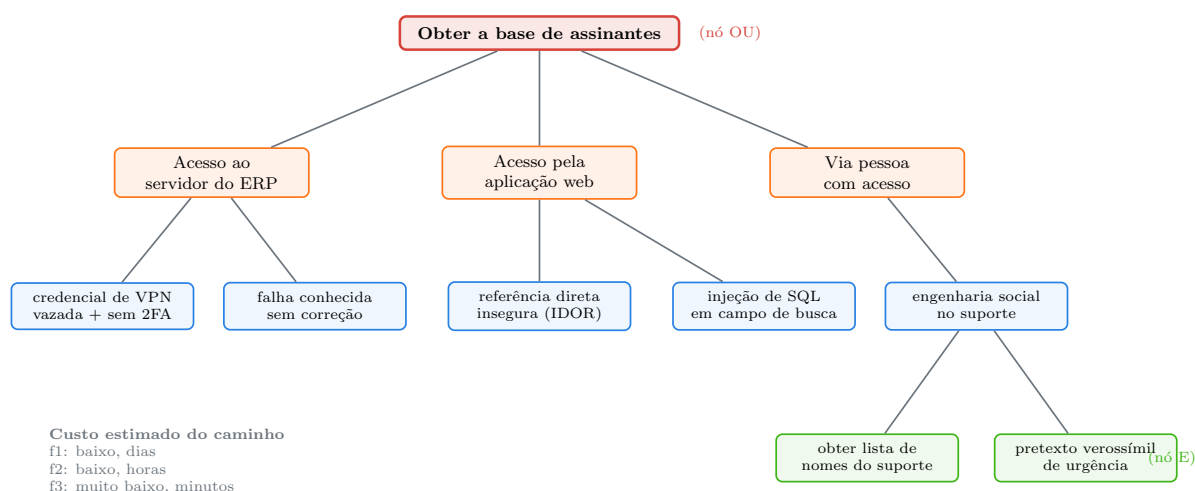


Figura 5.2.: Árvore de ataque para a base de assinantes. O caminho mais barato — não o mais sofisticado — é o que descreve o adversário real.

O valor da árvore aparece quando se anotam as folhas com **custo, tempo e habilidade requerida**. O caminho mais barato define o adversário mais provável, e é ali que o primeiro real de defesa rende mais. No exemplo, gastar em detecção sofisticada enquanto existe uma referência direta insegura explorável em minutos é investir no ramo errado da árvore.

Árvores também mostram onde um único controle corta vários caminhos ao mesmo tempo — segundo fator na VPN elimina f1 e encarece f5. Esse é o tipo de raciocínio que uma lista de vulnerabilidades ordenada por nota nunca produz.

5.3.3. Abuse cases

O terceiro instrumento parte do que a equipe já escreve. Para cada **caso de uso**, escreve-se o **caso de abuso** correspondente: o mesmo ator, com intenção hostil, ou um ator novo que ninguém previu.

Caso de uso. Como assinante autenticado, quero consultar minha fatura para pagá-la.

Caso de abuso. Como assinante autenticado, quero consultar a fatura de outro assinante alterando o identificador na URL.

Critério de aceitação derivado. A requisição de fatura cujo identificador não pertence ao assinante da sessão retorna 404, registra o evento com origem e identificador tentado, e dispara alerta após cinco tentativas.

O ganho dessa formulação é institucional: o caso de abuso vira **critério de aceitação** e depois **teste automatizado**, entrando no mesmo fluxo em que a equipe já trabalha. Segurança que vive numa planilha à parte é esquecida; segurança que vira teste que quebra o build sobrevive.

5.4. Outras abordagens, e quando usá-las

STRIDE, árvores e casos de abuso cobrem a grande maioria das necessidades. Vale conhecer quatro alternativas pelo que cada uma acrescenta:

- **PASTA** — processo de sete estágios centrado em risco de negócio, com simulação de ataque. Mais pesado; adequado quando a modelagem precisa dialogar com análise financeira de risco.
- **LINDDUN** — o análogo do STRIDE para **privacidade**: vinculabilidade, identificabilidade, não repúdio (aqui como problema, não como propriedade desejável), detectabilidade, divulgação, inconsciência e não conformidade. É o instrumento certo para o relatório de impacto exigido pela LGPD, porque olha ameaças ao titular dos dados, não à empresa.
- **OCTAVE** — orientado a organização e ativos críticos, mais que a sistemas; útil em nível de programa.
- **MITRE ATT&CK** — não é método de modelagem, e sim catálogo de táticas e técnicas reais (MITRE Corporation, 2024). Excelente como fonte de folhas para árvores de ataque e como régua de cobertura de detecção. O Volume 11 aprofunda.

Uma nota sobre a **cadeia de ataque** (*kill chain*): modelar o ataque como sequência de etapas — reconhecimento, acesso inicial, execução, persistência, movimentação lateral, exfiltração — tem valor defensivo próprio, porque cada etapa é uma oportunidade de detecção e interrupção. Um adversário precisa vencer todas; o defensor precisa pegar uma.

5.5. Pergunta 3: o que fazer, e pergunta 4: fizemos bem?

Cada ameaça identificada termina em uma de quatro decisões, e a decisão precisa estar **escrita, com dono e prazo**:

- **Mitigar** — implantar controle que reduza probabilidade ou impacto.
- **Eliminar** — remover a funcionalidade ou o dado que cria a exposição. Subestimada: a funcionalidade que ninguém usa e que só existe porque estava no escopo original é o alvo mais fácil.
- **Transferir** — passar o risco a um terceiro (seguro, contrato, fornecedor especializado). Transfere o custo financeiro, nunca a responsabilidade perante o titular dos dados.
- **Aceitar** — decidir conviver com ele, conscientemente, registrado, por quem tem autoridade para isso.

A quarta pergunta de Shostack — *fizemos um bom trabalho?* — se responde com três checagens: todos os elementos do diagrama receberam as letras aplicáveis do STRIDE; alguém que não participou revisou o modelo; e as ameaças mitigadas geraram teste que detecta regressão.

5.5.1. Erros que esvaziam o exercício

- **Modelar uma vez.** O modelo envelhece na primeira mudança de arquitetura. Revisar por gatilho, não por calendário.
- **Modelar sem quem constrói.** Modelo feito só por segurança produz achados que a equipe de desenvolvimento não reconhece como reais, e nada é implementado.
- **Buscar completude.** Não existe lista exaustiva de ameaças. Um modelo que cobre as fronteiras de confiança e os fluxos principais entrega quase todo o valor; perseguir os últimos 5% consome o tempo que faltará para corrigir os 95%.
- **Parar na lista.** Sem decisão, dono e prazo, o documento é um inventário de preocupações.
- **Confundir com teste de invasão.** A modelagem antecipa o que *pode* dar errado no desenho; o teste verifica o que *está* errado na implementação. São complementares e não se substituem — o Volume 9 trata do segundo.

5.6. Brasil em Foco

A LGPD incorporou modelagem de ameaças ao ordenamento sem usar essa expressão. O art. 46, §2º determina que as medidas de segurança deverão ser observadas **desde a fase de concepção do produto ou do serviço até a sua execução** (Brasil, 2018) — é a positivação do princípio de *security e privacy by design*, e a única forma prática de cumpri-la é analisar riscos antes de construir. Uma decisão de arquitetura tomada sem essa análise e que depois causa vazamento é, sob esse dispositivo, falha demonstrável de diligência.

O **relatório de impacto à proteção de dados pessoais** do art. 38 é o produto formal mais próximo de um modelo de ameaças no direito brasileiro. Sua estrutura pede descrição dos tratamentos, dos riscos às liberdades civis e aos direitos fundamentais, e das salvaguardas adotadas — exatamente as perguntas 1, 2 e 3 de Shostack, com o foco deslocado: o risco a analisar é o risco **ao titular dos dados**, não à empresa. É por isso que o LINDDUN é mais adequado que o STRIDE para esse documento: uma funcionalidade pode ser perfeitamente segura para a organização e ainda assim criar vinculabilidade ou identificabilidade indevida para as pessoas.

A distinção tem consequência concreta no caso de abertura. Sob a ótica da empresa, o incidente foi vazamento de dados de clientes. Sob a ótica do titular, foi a exposição de nome, CPF, endereço e valor de fatura de 4.000 pessoas a qualquer um com um navegador — situação que o art. 48 obriga a comunicar à ANPD e aos titulares afetados quando houver risco ou dano relevante, no prazo de três dias úteis fixado pela Resolução CD/ANPD nº 15/2024 (Autoridade Nacional de Proteção de Dados, 2024). Modelar antes custaria uma tarde.

No setor financeiro e de pagamentos, a Resolução CMN nº 4.893/2021 exige que a política de segurança cibernética contemple **avaliação prévia** de novas tecnologias e de mudanças relevantes, e que a instituição avalie riscos antes de contratar serviços de terceiros (Banco Central do Brasil, 2021). Para telecomunicações, a Resolução Anatel nº 740/2020 exige que produtos sejam concebidos com requisitos de segurança, o que pressupõe análise na fase de projeto (Agência Nacional de Telecomunicações, 2020).

Um ponto prático para provedores regionais: a **ANPD reconhece proporcionalidade**. Para agentes de pequeno porte, a autoridade não espera um processo formal de sete estágios — espera evidência de que os riscos foram considerados e as decisões registradas (Autoridade Nacional de Proteção de Dados, 2023). Uma ata de duas páginas por projeto, com o diagrama, as ameaças levantadas e as decisões tomadas, é documento suficiente e mais defensável que a ausência de qualquer registro.

5.7. No Campo

1. A sessão de noventa minutos. Quadro branco, três pessoas (quem desenvolve, quem opera, quem entende o negócio), um sistema por vez. Primeiros 20 minutos: desenhar o fluxo de dados e marcar as fronteiras de confiança. Próximos 45: percorrer STRIDE por elemento nos fluxos que cruzam fronteira. Últimos 25: decidir o que fazer com cada achado, com dono e prazo. Fotografar o quadro e anexar à ata é registro suficiente.

2. Modelar a rede, não só a aplicação. Para quem administra infraestrutura, o exercício mais valioso é sobre o **acesso de gerência**. Fluxo: técnico → VPN → VLAN de gerência → OLT. Aplicando STRIDE ao fluxo que cruza a fronteira internet–VPN:

Tabela 5.2.: STRIDE aplicado ao acesso de gerência de rede.

Letra	Pergunta	Achado típico
S	Alguém consegue se passar por técnico?	Conta compartilhada, sem segundo fator
T	Dá para alterar a configuração em trânsito?	Telnet ou SNMPv2 na gerência
R	Dá para negar que fez a alteração?	Login <code>admin</code> genérico no equipamento
I	Dá para ler configuração ou tráfego?	Comunidade SNMP <code>public</code> habilitada
D	Dá para derrubar a gerência?	Plano de gerência sem separação do de dados
E	Dá para sair de um acesso limitado para total?	Um único nível de privilégio no equipamento

3. Transformar cada achado em verificação executável. O caso de abuso vira teste. Para o incidente de abertura, a verificação é uma linha:

```
# Autentica como assinante A e tenta ler a fatura do assinante B.
# Esperado: 404. Qualquer 200 aqui é falha de autorização por objeto.
curl -s -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer $TOKEN_ASSINANTE_A" \
  https://portal.exemplo.br/api/fatura/48212
```

4. Verificar as fronteiras que você acha que existem. Modelo desenhado é hipótese; a rede é o fato. Vale confirmar que a DMZ não alcança a rede interna e que o servidor de aplicação não alcança o que não precisa:

```
# A partir de um host da DMZ: o que da rede interna responde? (esperado: só o
↪ previsto)
nmap -Pn -n --top-ports 50 --open 10.20.0.0/24

# Rotas e regras que contradizem a fronteira desenhada
ip route show
nft list ruleset | grep -A3 -E 'chain (forward|input)'
```

5. Guardar o modelo junto do código. Um arquivo Markdown no repositório, versionado com o sistema que descreve, com o diagrama e a tabela de ameaças e decisões. Modelo em pasta compartilhada some; modelo no repositório aparece na revisão de código.

6. Reservar dez minutos da revisão de mudança para a pergunta. Antes de subir alteração relevante: *esta mudança cria fluxo novo que cruza fronteira de confiança?* Se sim, as seis letras. Se não, segue. É o formato mais barato de modelagem contínua que existe.

7. Fazer o exercício da funcionalidade removida. Percorrer o sistema perguntando o que pode ser **desligado**: relatório que ninguém abre há um ano, campo coletado e nunca usado, integração de um projeto abandonado, endereço de API sem consumidor. Eliminar é a única mitigação com eficácia de 100% e custo de manutenção zero — e conversa diretamente com o princípio da necessidade da LGPD.

5.8. Resumo

- Testes verificam que o sistema **faz o que deve**; modelagem de ameaças verifica que ele **não faz o que não deve**. Falhas como referência direta insegura passam por qualquer suíte de aceitação e são triviais de antecipar em uma sessão de quadro branco.
- O método são quatro perguntas: o que estamos construindo, o que pode dar errado, o que faremos, e fizemos bem. As duas últimas são o que separa modelagem de conversa.
- No diagrama de fluxo de dados, o elemento decisivo é a **fronteira de confiança** — é onde as ameaças se concentram e onde o esforço de análise deve se concentrar.
- **STRIDE** é o complemento exato das propriedades dos capítulos anteriores: cada letra viola uma delas. Aplicado **por elemento**, converte-se numa lista finita de perguntas concretas.
- **Árvores de ataque** partem do objetivo do adversário e revelam o caminho mais barato — que é o que descreve o atacante real — e os controles que cortam vários ramos de uma vez. **Casos de abuso** convertem achados em critério de aceitação e teste automatizado.
- Cada ameaça termina em mitigar, eliminar, transferir ou aceitar, sempre com dono e prazo. Transferir move o custo financeiro, nunca a responsabilidade perante o titular dos dados.
- No Brasil, o art. 46, §2º da LGPD exige segurança desde a concepção, e o relatório de impacto do art. 38 é o modelo de ameaças formalizado — com o foco deslocado para o risco ao **titular**, o que torna o LINDDUN mais adequado que o STRIDE nesse documento. Para agentes de pequeno porte, a ANPD espera proporcionalidade: registro das decisões, não processo formal pesado.

6. Gestão de risco: análise qualitativa e quantitativa

A reunião tem quarenta minutos e três itens na pauta. O terceiro é a proposta de contratar um segundo link de trânsito, de outra operadora, entrando por rota física distinta: R\$ 4.800 por mês, R\$ 57.600 no ano.

O coordenador de rede é a favor: já houve dois rompimentos de fibra em dezoito meses, um deles de onze horas. O diretor financeiro é contra: é o equivalente a três salários, num ano em que a meta é cortar custo. O sócio pergunta se não dá para negociar um desconto com a operadora atual. A discussão dura vinte minutos, sobe de tom duas vezes, e termina com “vamos deixar para revisar no próximo trimestre” — que é como se registra, em ata, a decisão de aceitar o risco sem nunca ter dito que se estava aceitando um risco.

O que faltou ali não foi um argumento melhor. Faltou um número. Onze horas fora do ar custam quanto? Qual a chance de acontecer de novo no ano? Quanto dessa perda o segundo link elimina de fato? Nenhuma dessas perguntas exige precisão contábil — exigem apenas uma estimativa honesta, com sua incerteza explícita. Com elas na mesa, a conversa deixa de ser sobre quem fala mais alto e passa a ser sobre se R\$ 57.600 compram ou não mais do que custam.

Gestão de risco é o método que produz esses números e as decisões que decorrem deles. Este capítulo cobre o processo, as duas famílias de análise — qualitativa e quantitativa —, suas armadilhas, e como escolher entre elas.

6.1. O processo

As normas convergem para o mesmo ciclo. A ISO 31000 dá o arcabouço geral de gestão de risco (International Organization for Standardization, 2018b), a ISO/IEC 27005 o especializa para segurança da informação (International Organization for Standardization, 2022) e o NIST SP 800-30 detalha a etapa de avaliação (National Institute of Standards and Technology, 2012). A Figura 6.1 resume.

Vale insistir em duas etapas que a prática costuma pular.

A **etapa 1** define os critérios *antes* de olhar qualquer risco específico — escala de impacto, escala de probabilidade, e o **apetite a risco**: quanto a organização está disposta a tolerar. Definir isso depois é como escolher a régua depois de ver o resultado, e o efeito é previsível: os riscos que dão trabalho corrigir acabam classificados como aceitáveis.

A **etapa 4** é distinta da 3 e frequentemente fundida com ela. Analisar é estimar probabilidade e impacto; avaliar é **comparar com o critério** e decidir se está dentro ou fora do aceitável. Sem critério declarado, não há avaliação — há opinião com aparência de método.

6. Gestão de risco: análise qualitativa e quantitativa

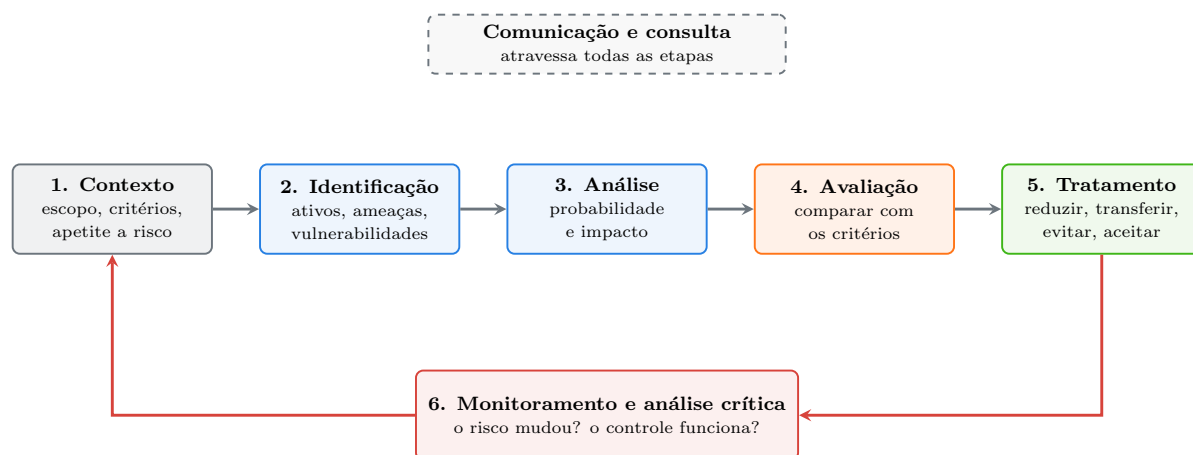


Figura 6.1.: O ciclo de gestão de risco. A sexta etapa é a que quase sempre falta: risco avaliado e nunca revisitado envelhece junto com o ambiente.

6.2. Análise qualitativa

A análise qualitativa classifica probabilidade e impacto em escalas ordinais e combina as duas numa matriz. É rápida, exige poucos dados e comunica bem — por isso domina a prática.

Também é onde mora a maior parte dos erros. E o principal deles não está na matriz: está nas escalas. Uma escala com rótulos vagos — “raro”, “possível”, “provável” — significa coisas diferentes para cada pessoa da sala. A correção é barata: **ancorar cada nível em número e em consequência concreta**.

Tabela 6.1.: Escalas ancoradas de probabilidade e impacto. Os valores são ilustrativos e devem ser calibrados ao porte de cada organização.

Nível	Probabilidade (ancorada)	Impacto (ancorado no contexto do provedor)
1 — Muito baixo	Menos de 1 vez em 10 anos	Até R\$ 5 mil; sem efeito perceptível ao assinante
2 — Baixo	Cerca de 1 vez em 5 anos	R\$ 5 mil a R\$ 30 mil; degradação localizada
3 — Médio	Cerca de 1 vez em 2 anos	R\$ 30 mil a R\$ 150 mil; parada de um serviço por horas
4 — Alto	Cerca de 1 vez por ano	R\$ 150 mil a R\$ 500 mil; parada geral por um dia, notificação à ANPD
5 — Muito alto	Mais de 1 vez por ano	Acima de R\$ 500 mil; risco de continuidade do negócio

Com as escalas ancoradas, a matriz de calor (Figura 6.2) vira instrumento de comunicação legítimo — um mapa que a direção lê em segundos.

6.2.1. As armadilhas da matriz

A matriz é útil e tem quatro problemas conhecidos que convém enfrentar de frente:

Multiplicar números ordinais. “Probabilidade 4 × impacto 3 = 12” trata rótulos de ordem como se fossem quantidades. Não são: o salto de 3 para 4 na escala de impacto da Tabela 6.1 é de R\$ 120 mil, e o de 4 para 5 é aberto e ilimitado. O produto é uma conveniência visual, não uma conta.

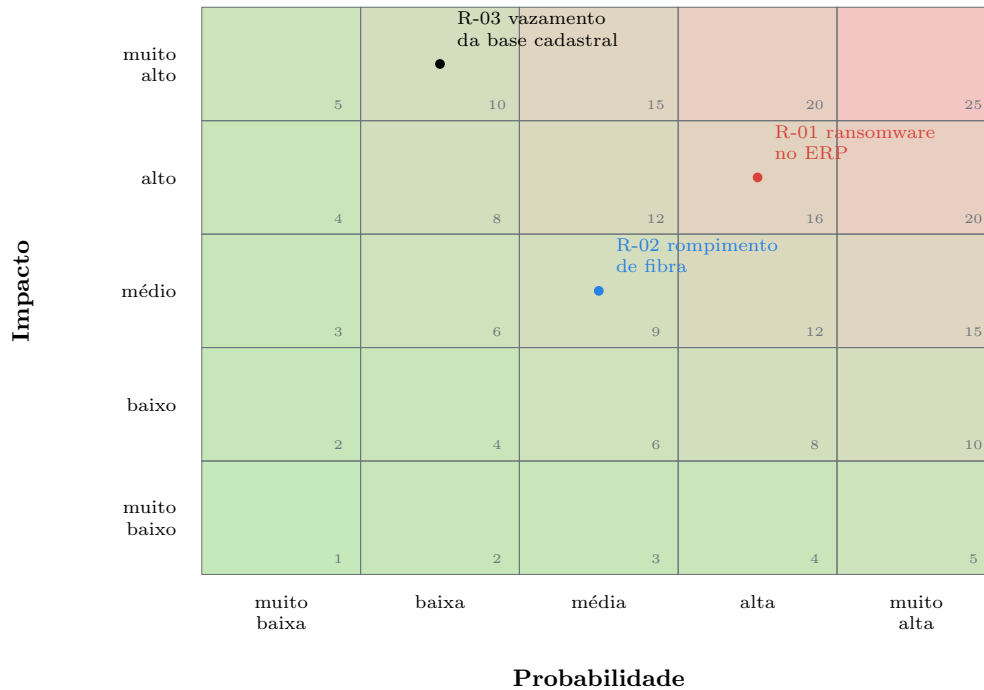


Figura 6.2.: Matriz de risco 5×5 com riscos posicionados. O número em cada célula é o produto dos índices — e é exatamente esse produto que exige cautela.

Nenhuma dessas críticas invalida a análise qualitativa. Elas definem seu uso correto: **triagem**. A matriz separa o que merece atenção do que não merece. Para o que sobe ao topo — e para toda decisão que envolve gastar dinheiro relevante — vale o esforço de quantificar.

6.3. Análise quantitativa

A análise quantitativa estima o risco em unidades monetárias. O vocabulário clássico tem quatro termos:

- **AV** (*asset value*) — valor do ativo.
- **EF** (*exposure factor*) — fração do valor perdida se o evento ocorrer.
- **SLE** (*single loss expectancy*) — perda esperada por ocorrência: $SLE = AV \times EF$.
- **ARO** (*annualized rate of occurrence*) — frequência anual esperada.
- **ALE** (*annualized loss expectancy*) — perda anual esperada: $ALE = SLE \times ARO$.

Aplicando ao caso de abertura. O provedor fatura R\$ 780 mil por mês. Um rompimento de fibra sem link redundante custa, por hora de parada, uma combinação de: crédito contratual e abatimento em fatura, chamados extras no suporte, cancelamentos atribuíveis nos trinta dias seguintes e horas extras da equipe de campo. A equipe estimou R\$ 3.900 por hora, e a duração média histórica em 6 horas.

$$SLE = 6 \text{ h} \times \text{R\$ } 3.900/\text{h} = \text{R\$ } 23.400$$

Com dois eventos em dezoito meses, $ARO \approx 1,33$ por ano:

$$ALE = \text{R\$ } 23.400 \times 1,33 \approx \text{R\$ } 31.100 \text{ por ano}$$

6. Gestão de risco: análise qualitativa e quantitativa

O segundo link não elimina a perda: rompimento continua havendo, mas a comutação leva minutos em vez de horas, e a parada residual foi estimada em 0,4 hora por evento — reduzindo a ALE para cerca de R\$ 2.100. O **retorno sobre o investimento em segurança** é então:

$$\text{ROSI} = \frac{(\text{ALE}_{\text{antes}} - \text{ALE}_{\text{depois}}) - \text{custo}}{\text{custo}} = \frac{(31.100 - 2.100) - 57.600}{57.600} \approx -0,50$$

O número é negativo. Pelo critério estrito de perda esperada, **o segundo link não se paga** — e essa conclusão, que contraria a intuição do coordenador de rede, é exatamente o tipo de resultado que justifica fazer a conta.

Mas a análise não termina aí, e é aqui que a maior parte das apresentações de gestão de risco erra. Três considerações mudam a decisão e precisam ir para a mesa junto com o número:

1. **A média esconde a cauda.** A conta usa 6 horas de duração média. Um rompimento em trecho de difícil acesso, num feriado, com equipe reduzida, é um evento de 30 horas — e a perda cresce mais que proporcionalmente, porque cancelamentos disparam a partir de certo limiar de paciência do assinante.
2. **Há benefício fora do cálculo.** Segundo link com operadora distinta protege também contra falha da operadora atual, permite manutenção programada sem janela noturna e é argumento comercial para contratos empresariais — receita nova que não entra na conta de perda evitada.
3. **O critério pode não ser o valor esperado.** Se uma parada longa ameaça contratos que representam parcela relevante da receita, a empresa pode racionalmente pagar acima do valor esperado para eliminar a cauda. É o mesmo raciocínio de comprar seguro, que tem valor esperado negativo por construção e ainda assim é decisão sensata.

A conclusão prática é que a quantificação **não substitui o julgamento** — ela o organiza. Com os números na mesa, a decisão passa a ser “estamos pagando R\$ 57,6 mil por ano para eliminar R\$ 29 mil de perda esperada e comprar proteção contra o cenário de cauda”, que é uma frase que a direção consegue aprovar ou recusar de forma consciente. Sem eles, decide-se por tom de voz.

6.4. Lidando com a incerteza: intervalos em vez de pontos

O ponto fraco de $\text{ALE} = \text{SLE} \times \text{ARO}$ é fingir precisão que não existe. “R\$ 3.900 por hora” é uma estimativa com incerteza grande, e o resultado herda essa incerteza sem mostrá-la.

A alternativa, defendida por Hubbard e Seiersen (Hubbard; Seiersen, 2016) e formalizada pelo método FAIR (Freund; Jones, 2014), é estimar **intervalos de confiança de 90%** em vez de valores únicos, e propagar a incerteza por simulação. Em vez de “custa R\$ 3.900 por hora”, a pergunta é: *qual intervalo você acredita, com 90% de confiança, que contém o custo horário real?* — digamos R\$ 1.800 a R\$ 9.000.

Sorteando milhares de cenários dentro desses intervalos, obtém-se não um número, mas uma distribuição de perda anual. Vinte linhas de Python bastam:

```
import numpy as np

rng = np.random.default_rng(42)
N = 100_000
```

```
# Intervalos de 90% de confiança, convertidos em lognormais.
# Para uma lognormal, o intervalo [lo, hi] de 90% implica:
def lognormal_90(lo, hi, n):
    mu = (np.log(lo) + np.log(hi)) / 2
    sigma = (np.log(hi) - np.log(lo)) / 3.29 # 3.29 = 2 x 1.645
    return rng.lognormal(mu, sigma, n)

custo_hora = lognormal_90(1_800, 9_000, N) # R$/hora de parada
duracao = lognormal_90(1.5, 30, N) # horas por evento
eventos = rng.poisson(1.33, N) # ocorrências no ano

perda_anual = eventos * custo_hora * duracao

for p in (50, 80, 90, 95, 99):
    print(f"P{p}: R$ {np.percentile(perda_anual, p):>12,.0f}")
print(f"média: R$ {perda_anual.mean():>10,.0f}")
print(f"P(perda > 57.600) = {(perda_anual > 57_600).mean():.1%}")
```

O resultado não é um número, é a **curva de excedência de perda** da Figura 6.3: para cada valor, a probabilidade de a perda anual superá-lo.

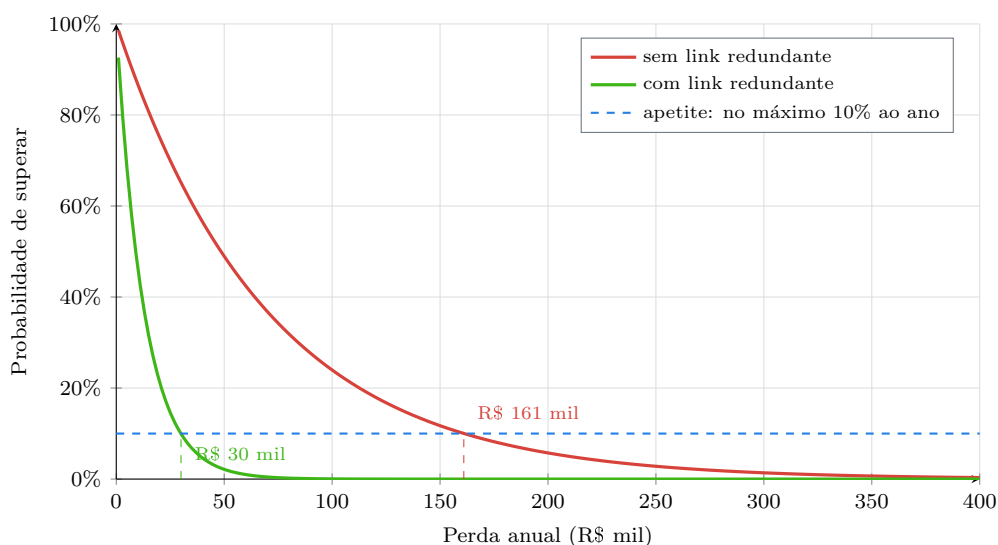


Figura 6.3.: Curva de excedência de perda. Onde a curva cruza a linha do apetite está a perda que se aceita ter 10% de chance de superar no ano.

A leitura é direta e é o que torna o método superior à matriz: com o link redundante, a perda que se tem 10% de chance de superar cai de R\$ 161 mil para R\$ 30 mil. A decisão deixa de ser sobre um valor esperado e passa a ser sobre **onde a cauda é cortada** — que é o que a diretoria de fato quer saber.

Uma objeção previsível: “não temos dados para isso”. A resposta é que o método não exige dados históricos, exige **estimativas calibradas** — e calibração é habilidade treinável. Pessoas treinadas para acertar 90% de suas faixas de 90% produzem intervalos honestos sobre grandezas que nunca mediram. E o ponto decisivo é comparativo: a alternativa não é uma estimativa melhor, é o palpite implícito de quem fala mais alto na reunião.

6.5. Escolher entre as duas

Tabela 6.2.: Quando usar cada abordagem.

Critério	Qualitativa	Quantitativa
Custo do exercício	Horas	Dias
Dado necessário	Julgamento de especialista	Julgamento calibrado, idealmente com histórico
Comparabilidade entre riscos	Frágil (escala ordinal)	Direta (mesma unidade monetária)
Suporta decisão de investimento	Mal	Bem (ROSI, curva de excedência)
Comunica à direção	Bem (mapa visual)	Muito bem (em reais)
Risco de falsa precisão	Baixo	Alto se a incerteza não for mostrada

Na prática, a combinação vence: **qualitativa para triar todo o inventário de riscos, quantitativa para os dez ou quinze do topo** e para toda decisão que envolva gasto relevante. É a proporção que respeita o custo do método.

6.6. Tratamento e risco residual

Analisado e avaliado o risco, restam quatro opções — as mesmas do Capítulo 5, agora com o número que as sustenta:

Reduzir. Implantar controle que diminua probabilidade ou impacto. É a escolha padrão. O critério de aceitação é a comparação entre custo do controle e redução de ALE, temperada pela análise da cauda.

Transferir. Seguro cibernético, cláusula contratual, terceirização para fornecedor especializado. Três ressalvas importam: apólices excluem o que a empresa deixou de fazer (ausência de segundo fator e de correção em dia são exclusões comuns); a indenização cobre prejuízo financeiro, não a obrigação perante a ANPD nem a confiança do assinante; e transferir exige demonstrar controles para conseguir preço razoável — o seguro pressupõe higiene, não a substitui.

Evitar. Não fazer, ou parar de fazer, a atividade que gera o risco. Desativar a funcionalidade pouco usada, não coletar o dado que não é necessário, encerrar a integração legada. É a opção mais subestimada e a única com eficácia integral.

Aceitar. Conviver conscientemente. Só é aceitação legítima se: estiver **registrada**, com o número que a justifica; for decidida por **quem tem autoridade** para comprometer a organização naquele valor; e tiver **data de revisão**. Aceitação sem esses três elementos é omissão com outro nome.

O que sobra depois do tratamento é o **risco residual** — e o produto final de todo o processo é uma lista curta de riscos residuais que a direção conhece e aceitou formalmente. Um programa de segurança maduro não se distingue por ter menos risco, mas por saber qual risco tem.

6.6.1. O registro de risco

O artefato que sustenta tudo isso cabe numa planilha. Dez colunas: identificador, descrição do cenário (ativo + ameaça + vulnerabilidade + consequência), impacto estimado, probabilidade estimada, risco inerente, controles existentes, risco residual, decisão de tratamento, dono, data de revisão.

Duas regras que decidem se o registro vive ou morre: descrever o **cenário completo**, não o rótulo — “ransomware” não é risco, “cifragem do banco do ERP por acesso via VPN sem segundo fator, com parada de faturamento por seis dias” é; e **cada linha tem um dono nomeado**, porque risco de todos é risco de ninguém.

6.6.2. Indicadores que antecipam

Monitoramento (etapa 6) não é reler a planilha em dezembro. São indicadores que se movem antes do incidente: percentual de servidores com correção crítica pendente há mais de 30 dias, contas privilegiadas sem segundo fator, restaurações de backup testadas no trimestre, tempo médio até detecção, serviços administrativos alcançáveis da internet. Cada um deles tem um limite que, ultrapassado, dispara reavaliação do risco correspondente — e essa é a diferença entre gestão de risco e documentação de risco.

6.7. Brasil em Foco

A gestão de risco é a espinha dorsal do modelo brasileiro de conformidade em proteção de dados, e entender isso muda a forma de investir.

A **LGPD** não traz lista fechada de controles obrigatórios. O art. 46 exige medidas “aptas a proteger” os dados, considerando o estado da técnica, a natureza da informação e os riscos; o art. 50 fala em regras de boas práticas que levem em conta “a natureza, o escopo, a finalidade, a probabilidade e a gravidade dos riscos”; o art. 38 prevê o relatório de impacto (Brasil, 2018). É um regime **baseado em risco**: a conformidade se demonstra pelo processo — inventário, avaliação, decisão registrada, revisão — e não pela posse de determinados produtos. Uma empresa com firewall de última geração e nenhum registro de avaliação de risco está em posição pior, diante da ANPD, que outra com controles modestos e um registro de risco bem mantido.

O art. 52, §1º da LGPD fixa os critérios de dosimetria das sanções, entre os quais figuram a **adoção de política de boas práticas e governança**, as **medidas adotadas para reverter ou mitigar** os efeitos do incidente e a **pronta adoção de medidas corretivas** — todos elementos que só existem se houver processo prévio. Na prática, o registro de risco é peça de defesa administrativa.

Para **agentes de pequeno porte**, a Resolução CD/ANPD nº 2/2022 e o guia orientativo da autoridade estabelecem tratamento proporcional (Autoridade Nacional de Proteção de Dados, 2023). A ANPD não espera de um provedor com quarenta funcionários o processo de uma instituição financeira — espera evidência de que os riscos foram identificados e as decisões tomadas de forma consciente. Uma planilha de dez linhas revisada semestralmente atende.

No **setor financeiro e de pagamentos**, a exigência é explícita e mais rígida: a Resolução CMN nº 4.893/2021 obriga política de segurança cibernética com avaliação de riscos, classificação de dados por relevância, controles proporcionais, plano de resposta a incidentes, testes e relatório anual submetido à alta administração — além de gestão de risco sobre serviços relevantes de terceiros (Banco Central do Brasil, 2021). Para telecomunicações, a Resolução Anatel nº 740/2020

6. Gestão de risco: análise qualitativa e quantitativa

exige tratamento de vulnerabilidades em produtos, o que pressupõe avaliação de criticidade (Agência Nacional de Telecomunicações, 2020).

Do lado das normas técnicas, as versões brasileiras estão disponíveis pela ABNT: a **NBR ISO 31000** para o arcabouço de gestão de risco e a **NBR ISO/IEC 27005** para a aplicação à segurança da informação. Adotar a terminologia dessas normas no registro de risco tem valor prático em auditoria e em processo administrativo: usa o vocabulário que o auditor e o regulador esperam.

Sobre **seguro cibernético**, o mercado brasileiro é regulado pela SUSEP e vem crescendo, com apólices que costumam cobrir custo de resposta, perícia, notificação a titulares e defesa administrativa. Duas cautelas: multa administrativa da ANPD tem cobertura limitada ou vedada em várias apólices, e o questionário de contratação funciona como auditoria — declarar controles que não existem compromete o pagamento do sinistro.

6.8. No Campo

1. Escrever os critérios antes de olhar o primeiro risco. Uma página: escala de impacto ancorada em reais e em consequência operacional, escala de probabilidade ancorada em frequência, e a linha do apetite (“não aceitamos risco com mais de 10% de chance de perda anual superior a R\$ 50 mil”). Aprovada pela direção, ela resolve antecipadamente metade das discussões.

2. Levantar dez riscos, não cem. Uma sessão de duas horas com a equipe técnica e alguém do negócio produz os dez cenários que importam. Descrever cada um como cenário completo. Os cem virão depois, se vierem.

3. Estimar o custo da hora parada — o número que falta em quase toda empresa. Ele destrava todas as demais contas:

```
# Insumos para a estimativa, a partir do que já se mede
# 1. Receita por hora
echo "scale=2; 780000 / (30*24)" | bc          # faturamento mensal / horas do mês

# 2. Histórico real de indisponibilidade dos últimos 12 meses
journalctl --list-boots | wc -l
last -x shutdown reboot | head -30

# 3. Volume de chamados por evento de parada (correlacionar com a data)
grep -c "" /var/log/suporte/chamados-2026-*.csv
```

Some receita não faturada, crédito contratual, horas extras, chamados extras e cancelamentos atribuíveis nos trinta dias seguintes. O resultado não precisa ser exato; precisa ser defensável.

4. Quantificar os três do topo, não todos. Para cada um: intervalo de 90% para o custo do evento, intervalo para a frequência, e a simulação de vinte linhas mostrada acima. O resultado em uma frase — “temos 10% de chance de perder mais de R\$ X este ano com este cenário” — é o que a direção compra ou recusa.

5. Levar decisões, não relatórios. A pauta certa tem três colunas: risco, opção recomendada com custo, e o que acontece se não fizermos nada. Uma página, três riscos. Relatório de quarenta páginas não é lido e não gera decisão.

6. Registrar a aceitação por escrito. Quando a decisão for aceitar — e será, com frequência —, uma linha na ata com o valor em risco, quem aceitou e a data de revisão. É proteção do técnico que alertou, é insumo de defesa administrativa, e é o que faz o assunto voltar à pauta antes de voltar como incidente.

7. Acompanhar cinco indicadores, e só cinco. Correções críticas pendentes há mais de 30 dias; contas privilegiadas sem segundo fator; restaurações testadas no trimestre; serviços administrativos alcançáveis da internet; tempo médio até detecção. Painel mensal, cada um com um limite que dispara reavaliação. Mais que cinco vira relatório que ninguém lê.

6.9. Resumo

- O ciclo é **contexto** → **identificação** → **análise** → **avaliação** → **tratamento** → **monitoramento**, com comunicação atravessando tudo. As etapas mais puladas são a primeira (definir critérios e apetite *antes*) e a última (revisitar).
- Na análise **qualitativa**, o erro não está na matriz e sim nas escalas: níveis com rótulos vagos significam coisas diferentes para cada pessoa. Ancorar cada nível em faixa numérica e consequência concreta é a correção mais barata que existe.
- A matriz sofre de multiplicação de ordinais, compressão da cauda, inversão de ordenação e vieses de estimativa. Isso a torna instrumento de **triagem**, não de decisão de investimento.
- A análise **quantitativa** usa $SLE = AV \times EF$ e $ALE = SLE \times ARO$, e o ROSI compara redução de perda esperada com custo do controle. O resultado pode contrariar a intuição — e é para isso que serve.
- Valor esperado esconde a cauda. Estimar **intervalos de 90%** e simular produz a **curva de excedência de perda**, que responde à pergunta que a direção realmente faz: quanto podemos perder no pior ano plausível, e quanto o controle corta disso.
- O tratamento é reduzir, transferir, evitar ou aceitar. Seguro transfere prejuízo financeiro, nunca a responsabilidade perante o titular; **evitar** é a opção mais subestimada; e aceitação só é legítima se registrada, decidida por quem tem autoridade e com data de revisão.
- No Brasil o regime é **baseado em risco**: a LGPD (arts. 38, 46, 50 e 52) demonstra conformidade pelo processo, não pela posse de produtos — e o registro de risco é peça de defesa administrativa. Setor financeiro (Res. CMN 4.893/2021) e telecomunicações (Res. Anatel 740/2020) acrescentam exigências específicas.

7. Ética, legalidade e a fronteira do escopo autorizado

Um técnico de suporte de um provedor está resolvendo um chamado de lentidão. No caminho, abre o portal de um provedor vizinho para comparar como eles apresentam o teste de velocidade. Nota que a URL da área do cliente termina em `?id=1042`. Por hábito profissional — o mesmo hábito que o Capítulo 5 elogia —, troca para 1041. Abre a página de outra pessoa: nome, CPF, plano, endereço de instalação.

O que ele faz nos próximos minutos define de que lado de uma linha ele está. Se fechar a aba e escrever um e-mail ao contato de abuso do provedor vizinho descrevendo o que viu, é um profissional agindo com responsabilidade. Se testar mais dez identificadores para “confirmar a extensão do problema”, já acessou dados de dez pessoas sem autorização. Se baixar a listagem para “provar” o achado, exfiltrou base de dados pessoais de terceiro. Se publicar no grupo de WhatsApp do setor, divulgou vulnerabilidade não corrigida de um concorrente.

A intenção é a mesma nos quatro casos. As consequências jurídicas não são.

Este é o capítulo mais importante do volume para quem pretende trabalhar com segurança, e é o que fecha a Fase 1. Os seis anteriores ensinaram a pensar como quem ataca para defender melhor. Este trata da fronteira que torna esse conhecimento legítimo: **a autorização**.

7.1. A linha é jurídica, não técnica

A confusão mais comum e mais cara na área é achar que a fronteira do lícito é definida pela dificuldade técnica ou pela intenção. Não é. É definida pela **autorização**, e a lei brasileira é explícita.

O art. 154-A do Código Penal, inserido pela Lei 12.737/2012 e agravado pela Lei 14.155/2021, tipifica:

Invadir dispositivo informático de uso alheio, conectado ou não à rede de computadores, com o fim de obter, adulterar ou destruir dados ou informações sem autorização expressa ou tácita do titular do dispositivo ou instalar vulnerabilidades para obter vantagem ilícita.

Três palavras merecem atenção. “**Invadir**” não exige violência, ferramenta sofisticada ou quebra de senha: trocar um número numa URL para acessar o que não é seu é acesso não autorizado. “**Sem autorização**” é o núcleo do tipo — a mesma ação, autorizada, é serviço profissional. E “**de uso alheio**” delimita: o dispositivo do próprio agente, ou aquele que ele foi contratado para testar, está fora.

O que a lei **não** exige também importa: não exige dano, não exige lucro, não exige que os dados sejam usados. A consumação se dá com o acesso.

Daí a regra prática deste manual, que vale para toda a carreira:

7. Ética, legalidade e a fronteira do escopo autorizado

Se você não tem autorização por escrito de quem pode dá-la, não toque. “Só olhei” é acesso. “Não usei nada” é irrelevante. “Foi para ajudar” é atenuante, não excludente.

7.2. O mapa legal brasileiro

O art. 154-A não está sozinho. A Tabela 7.1 reúne os dispositivos que alcançam atividade técnica em segurança da informação — e a coluna da direita é a que mais interessa, porque descreve o que transforma cada conduta em lícita.

Tabela 7.1.: Mapa das condutas técnicas e do que as torna lícitas no direito brasileiro.

Conduta	Enquadramento típico	O que a torna lícita
Acessar sistema de terceiro sem permissão	CP, art. 154-A (invasão de dispositivo informático)	Autorização expressa do titular, por escrito
Interromper serviço de telecomunicações ou de informação de utilidade pública	CP, art. 266, §1º	Janela de manutenção acordada; teste de carga contratado
Interceptar comunicação alheia	Lei 9.296/1996, art. 10	Ordem judicial; ou ser parte da comunicação; ou rede própria com aviso prévio (Brasil, 1996)
Fraude por meio eletrônico	CP, art. 171, §2º-A (estelionato)	Nada a torna lícita
Furto mediante dispositivo eletrônico	CP, art. 155, §4º-B	Nada a torna lícita
Inserir dados falsos em sistema da administração pública	CP, arts. 313-A e 313-B	Nada a torna lícita
Tratar dados pessoais de terceiros sem base legal	LGPD, arts. 7º e 42 (responsabilidade civil e sanção administrativa)	Base legal do art. 7º e finalidade legítima
Divulgar vulnerabilidade não corrigida de terceiro	Sem tipo penal próprio; risco civil e possível concurso com 154-A	Divulgação coordenada, com prazo e consentimento

Quatro observações completam o quadro.

As penas foram agravadas em 2021. A Lei 14.155/2021 elevou a pena do art. 154-A para reclusão de 1 a 4 anos e multa, com aumento se houver obtenção de conteúdo de comunicações privadas, segredos comerciais ou controle remoto do dispositivo — e majorantes adicionais quando praticado contra idoso ou vulnerável (Brasil, 2021). O estelionato eletrônico do art. 171, §2º-A tem reclusão de 4 a 8 anos.

A ação penal. O art. 154-B condiciona a persecução, em regra, a representação da vítima — procede-se mediante ação penal pública incondicionada apenas quando o crime é cometido contra a administração pública ou empresas concessionárias de serviços públicos. Isso significa que o provedor vizinho do caso de abertura decide se representa ou não; não significa que a conduta deixe de ser crime.

A LGPD age em paralelo. Acessar dados pessoais de terceiro sem base legal é tratamento irregular, com responsabilidade civil (art. 42) e sanção administrativa (art. 52) independentes da esfera penal. Quem “só olhou” a fatura de outro assinante tratou dado pessoal sem base legal.

A Convenção de Budapeste está em vigor no Brasil. O Decreto nº 11.491/2023 promulgou a Convenção sobre o Crime Cibernético (Brasil, 2023), que facilita a cooperação internacional e a preservação expedita de dados. O efeito prático é direto: a fronteira nacional deixou de ser obstáculo relevante à investigação, e o argumento de que “o servidor está fora do Brasil” não protege ninguém.

7.3. Autorização: como ela é feita de verdade

Autorização não é uma mensagem dizendo “pode testar”. É um documento, e faltando qualquer dos elementos abaixo, a proteção que ele oferece é parcial:

Tabela 7.2.: Elementos mínimos de uma autorização de teste.

Elemento	Por que é indispensável
Identificação das partes	Quem autoriza e quem executa, com CNPJ/CPF
Escopo positivo	Lista exata de domínios, faixas de IP, aplicações, contas de teste
Escopo negativo	O que está explicitamente fora — sistemas de terceiros, produção crítica, dados reais
Janela temporal	Data e hora de início e fim; fora dela não há autorização
Técnicas permitidas e proibidas	Negação de serviço, engenharia social contra funcionários, exploração destrutiva
Tratamento de dados encontrados	Que se faz ao encontrar dado pessoal: não copiar, registrar apenas evidência mínima, apagar ao fim
Contato de emergência	Quem atender às 3h se algo cair; e o gatilho para parar imediatamente
Sigilo e destinação do relatório	Quem recebe, por quanto tempo se guarda, como se destrói
Assinatura de quem tem poderes	Sócio, diretor ou procurador — não o gerente de TI que “aprovou por e-mail”

Dois pontos que causam problema com frequência.

Quem assina. A autorização precisa vir de quem pode comprometer a pessoa jurídica. O analista de TI que contrata o teste sem procuração não tem poder para autorizar acesso a dados de clientes da empresa. Verificar o contrato social ou a procuração é parte do trabalho, não formalidade.

Escopo com ativo de terceiro. Se o alvo está hospedado em nuvem ou em datacenter de terceiro, a autorização do cliente **não alcança** a infraestrutura do provedor. Os grandes provedores de nuvem publicam políticas de teste que permitem certas atividades sem aviso prévio e proíbem outras — notadamente teste de negação de serviço. Testar fora dessas políticas viola o contrato do cliente com o provedor e pode configurar acesso não autorizado a sistema do provedor.

7. Ética, legalidade e a fronteira do escopo autorizado

A Figura 7.1 condensa a decisão numa sequência que cabe na cabeça.

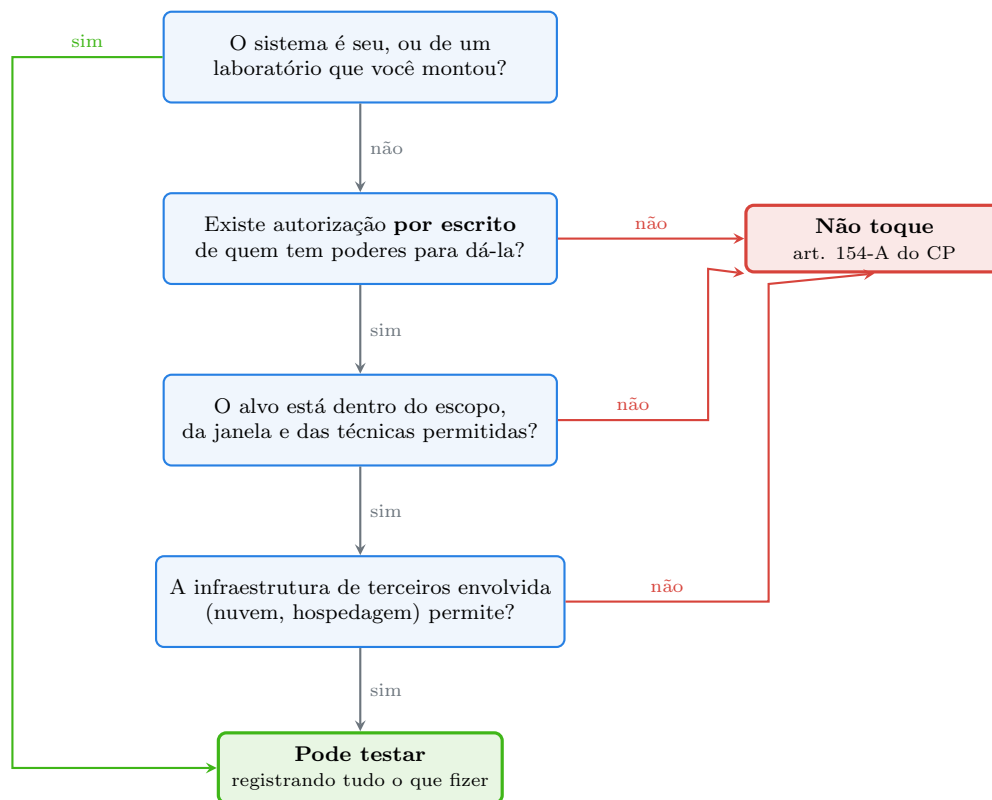


Figura 7.1.: A decisão de testar. Qualquer “não” nas três últimas perguntas leva ao mesmo lugar.

7.4. Divulgação de vulnerabilidade

Encontrada uma falha em sistema de terceiro — por acidente, como no caso de abertura, ou em pesquisa —, existe um caminho estabelecido. As normas ISO/IEC 29147 (divulgação) e 30111 (tratamento interno) descrevem o processo de ambos os lados (International Organization for Standardization, 2018c, 2019), e a Figura 7.2 mostra a linha do tempo típica.

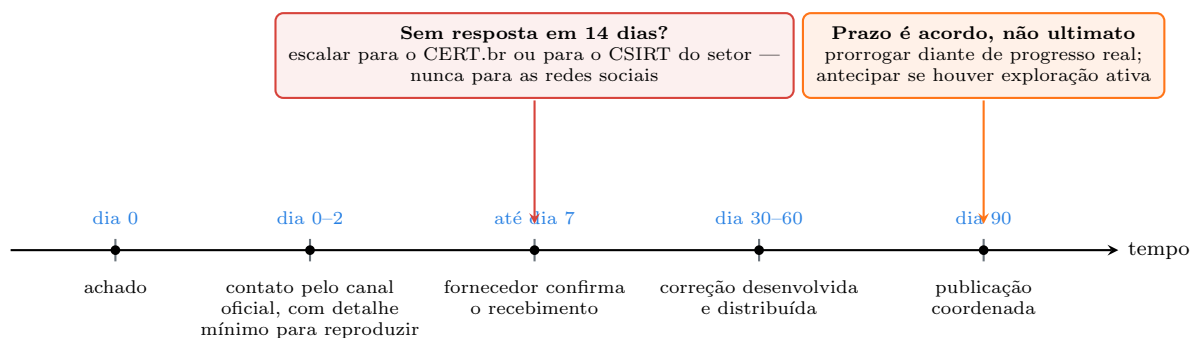


Figura 7.2.: Divulgação coordenada. O prazo de 90 dias é convenção de mercado, não regra jurídica.

Três posturas coexistem no mercado, e vale entender o argumento de cada uma. A **divulgação completa** publica tudo imediatamente, sob o argumento de que só a pressão pública faz fornecedores corrigirem e que defensores precisam da informação para se proteger. A **não divulgação**

nunca publica, e na prática protege o fornecedor às custas dos usuários. A **divulgação coordenada** — o padrão hoje — dá ao fornecedor um prazo razoável para corrigir e então publica, com ou sem correção pronta.

Cuidados que evitam transformar um ato de responsabilidade em problema:

- **Não teste além do mínimo.** Um caso comprova a falha. Dez casos “para confirmar a extensão” é acesso repetido não autorizado, e o relatório não fica melhor por isso.
- **Não copie dados.** Registre a evidência mínima — a URL, o código de resposta, uma captura com os dados tarjados. Baixar a base transforma o achado em exfiltração.
- **Documente o que fez, quando e como.** Se houver questionamento, a diferença entre pesquisador e invasor está nesse registro.
- **Use o canal oficial.** Procure `/.well-known/security.txt` no domínio, a página de contato de segurança, ou o contato de abuso no WHOIS do bloco. Não havendo canal, o CERT.br atua como intermediário.
- **Não peça pagamento.** Comunicar uma falha e sugerir que o silêncio tem preço é extorsão (CP, art. 158), independentemente de como a mensagem for redigida.

Uma nota importante sobre o Brasil: **não existe salvaguarda legal explícita para pesquisa de segurança**. Diferentemente da política do Departamento de Justiça norte-americano de 2022, que orienta não processar pesquisa de boa-fé, aqui a proteção do pesquisador depende de programas privados de recompensa com termos de porto seguro, ou da decisão da vítima de não representar. Reportar de boa-fé é o comportamento correto e ainda assim carrega risco jurídico residual. Quem pesquisa deve conhecer esse risco, minimizá-lo (não ir além do mínimo, documentar, usar canal oficial) e, em achados sensíveis, considerar orientação jurídica antes de fazer contato.

7.5. Bug bounty e programas de recompensa

Programas de recompensa resolvem juridicamente o problema: os termos publicados **são** a autorização, com escopo, técnicas permitidas e, nos bons programas, cláusula de porto seguro comprometendo a empresa a não acionar judicialmente quem cumprir as regras.

Três cuidados práticos: os termos são o limite exato — subdomínio fora da lista está fora, e “achei que estava incluído” não é defesa; testar um alvo do programa hospedado em nuvem ainda exige respeitar as políticas do provedor; e recompensa não é vínculo de trabalho, o que tem implicações fiscais para quem recebe com regularidade.

7.6. Ética profissional além da lei

A lei define o mínimo. A profissão exige mais, e três situações concentram os dilemas reais.

O poder do administrador. Quem administra sistemas pode ler o e-mail de qualquer pessoa da empresa, ver o histórico de navegação, acessar arquivos pessoais em pasta de rede. O acesso é tecnicamente possível e frequentemente autorizado *para fins específicos*. O uso além desses fins — a curiosidade sobre a conversa da colega, o interesse pelo salário do outro setor — é abuso, ainda que nenhum controle o impeça. A régua é simples: *este acesso é necessário para a tarefa que estou executando agora, e eu o registraria sem constrangimento?*

Monitoramento de funcionários. É lícito no Brasil quando há finalidade legítima, comunicação prévia e clara aos empregados, proporcionalidade e limite ao ambiente de trabalho. Não é lícito

7. Ética, legalidade e a fronteira do escopo autorizado

monitorar comunicação pessoal sem ciência, instalar captura de teclas de forma oculta ou vasculhar o dispositivo pessoal de alguém. A LGPD se aplica integralmente à relação de emprego, e a base legal costuma ser o legítimo interesse — que exige teste de proporcionalidade documentado, e não sobrevive a monitoramento excessivo.

O achado incidental. Durante um trabalho autorizado, encontra-se algo fora do escopo: indício de fraude interna, dados de outro cliente misturados, material manifestamente ilegal. A conduta correta é parar, não explorar, registrar o mínimo e comunicar imediatamente o contato definido no contrato. Diante de material de abuso infantil, a comunicação às autoridades é dever, e a preservação da evidência deve seguir orientação jurídica — não se copia, não se apaga, não se investiga por conta própria.

Os códigos de ética profissional da área — o da ISC2 é o mais citado — convergem em quatro deveres: proteger a sociedade e a infraestrutura; agir com honestidade e legalidade; prestar serviço diligente e competente; e promover a profissão (ISC2, 2024). O primeiro dever, quando conflita com os interesses de um cliente específico, prevalece: é ele que sustenta a recusa em ocultar um incidente que afeta terceiros.

7.7. Onde praticar sem risco

Aprender segurança ofensiva exige praticar. O caminho legítimo é abundante e gratuito:

- **Laboratório próprio.** Máquinas virtuais na sua máquina, rede isolada, alvos deliberadamente vulneráveis (DVWA, Metasploitable, Juice Shop, VulnHub). É o Volume 2 que monta esse ambiente.
- **Plataformas de treino.** Hack The Box, TryHackMe, PortSwigger Web Security Academy, PicoCTF. A autorização está nos termos de uso.
- **Competições CTF.** Autorização explícita, alvos preparados, comunidade ativa no Brasil.
- **Programas de recompensa,** respeitando o escopo.

E, para não errar em documentação, exemplos e testes, use os identificadores reservados: `example.com`, `example.org`, e as faixas de endereços de documentação 192.0.2.0/24, 198.51.100.0/24 e 203.0.113.0/24 (RFC 5737). Todo endereço IP que aparece neste manual vem dessas faixas, por isso mesmo.

7.8. Brasil em Foco

O capítulo inteiro é direito brasileiro; esta seção fixa o que muda no dia a dia de quem opera infraestrutura no país.

Provedor tem obrigações que se cruzam com a ética. O Marco Civil impõe guarda de registros (arts. 13 e 15) e, simultaneamente, sigilo (arts. 10 e 11): os dados só podem ser disponibilizados mediante ordem judicial, que precisa conter indícios do ilícito, identificação clara do usuário e período determinado (Brasil, 2014). Entregar registro a quem pede sem ordem — inclusive a delegado, a advogado ou ao “outro provedor que também foi atacado” — é violação de sigilo, com responsabilização civil e administrativa. A conduta correta é exigir a ordem, verificar sua abrangência e responder exatamente o que foi requisitado, nem mais.

Interceptação é linha vermelha. Capturar tráfego de assinante além do necessário para operação técnica esbarra na Lei 9.296/1996, cujo art. 10 tipifica a interceptação sem autorização

judicial (Brasil, 1996). Coleta de metadados de fluxo para engenharia de tráfego é operação legítima; inspeção de conteúdo é outra coisa, e exige base jurídica sólida.

A LGPD alcança o profissional, não só a empresa. O art. 42 estabelece a responsabilidade do controlador e do operador pelos danos causados; o art. 52 lista as sanções administrativas. Um prestador de serviço que trata dados do cliente é operador e responde nessa qualidade — o que torna o contrato de prestação, com cláusulas de finalidade, sigilo e devolução ou eliminação dos dados ao fim, instrumento de proteção do próprio prestador.

Sem porto seguro legal para o pesquisador. Vale repetir por ser a diferença mais relevante em relação a outros ordenamentos: no Brasil, quem reporta de boa-fé depende da representação da vítima e dos termos de programas privados. Não existe, hoje, dispositivo que exclua a tipicidade da conduta do pesquisador. O **CERT.br** é o intermediário estabelecido para notificar quem não tem canal — o que reduz o risco de o contato direto ser lido como ameaça.

A Convenção de Budapeste mudou a cooperação. Com o Decreto nº 11.491/2023, pedidos de preservação expedita e de cooperação internacional passaram a ter rito próprio (Brasil, 2023). Para provedores, isso significa maior volume de requisições e a necessidade de um procedimento interno definido para atendê-las corretamente — o Volume 15 detalha as obrigações regulatórias.

7.9. No Campo

1. Publicar um security.txt. Cinco minutos de trabalho que dão a quem encontrar uma falha no seu sistema um caminho responsável para avisar — e evitam que o achado vire postagem pública. Fica em `/.well-known/security.txt`:

```
Contact: mailto:seguranca@provedor.exemplo.br
Contact: https://provedor.exemplo.br/seguranca
Expires: 2027-12-31T23:59:00.000Z
Preferred-Languages: pt-BR, en
Policy: https://provedor.exemplo.br/politica-de-divulgacao
Acknowledgments: https://provedor.exemplo.br/agradecimentos
```

2. Manter o contato de abuso do bloco atualizado. É por ali que o CERT.br e outros operadores avisam sobre máquinas comprometidas na sua rede. Contato desatualizado significa não saber que seus clientes estão participando de um ataque:

```
# Contato de abuso registrado para o seu ASN e para o seu bloco
whois -h whois.registro.br AS64500 | grep -iE 'abuse|owner|responsible'
whois -h whois.registro.br 198.51.100.0/24 | grep -iE 'abuse|nserver'
```

3. Escrever a política de divulgação antes de precisar dela. Uma página: como reportar, o que a empresa se compromete a fazer (confirmar em X dias úteis, corrigir em Y), o que espera de quem reporta (não acessar dados além do mínimo, não divulgar antes do prazo), e a declaração de que não acionará judicialmente quem seguir as regras. Custa uma tarde e muda inteiramente o desfecho do dia em que alguém encontrar algo.

4. Formalizar a autorização, sempre — inclusive internamente. Vale para o teste contratado de terceiro e para a varredura da própria rede. Um documento de uma página com os elementos da Tabela 7.2, assinado por quem tem poderes, arquivado. A varredura do próprio bloco autorizada por escrito protege o técnico se o tráfego for questionado.

7. Ética, legalidade e a fronteira do escopo autorizado

5. Registrar o uso do acesso privilegiado. Quem administra pode tudo; o registro é o que separa poder de abuso. Sessão gravada ou ao menos registrada, revisão periódica por alguém que não seja o próprio administrador:

```
# Comandos executados com privilégio, por pessoa, nos últimos 7 dias
journalctl _COMM=sudo --since "7 days ago" -o cat \
  | grep -oP 'USER=\S+.*COMMAND=.*' | sort | uniq -c | sort -rn | head -20

# Acessos a arquivos sensíveis via auditd (regra prévia necessária)
auditctl -w /var/lib/erp/dados -p rwa -k dados-assinantes
ausearch -k dados-assinantes --start recent | head -40
```

6. Colocar as cláusulas certas no contrato de terceirizado. Quem presta suporte, quem administra a rede sob contrato, quem desenvolve o portal: finalidade do acesso, dever de sigilo que sobrevive ao fim do contrato, proibição de subcontratar sem anuência, obrigação de comunicar incidente em prazo curto, devolução ou eliminação dos dados ao término, e direito de auditoria. É o mesmo conjunto que a LGPD espera na relação controlador–operador.

7. Comunicar o monitoramento aos empregados. Política de uso aceitável assinada, com o que é monitorado, por quê e por quanto tempo se guarda. Monitoramento comunicado é lícito e eficaz; monitoramento oculto é risco trabalhista, risco sob a LGPD e destruição de confiança de uma vez só.

8. Treinar a resposta ao “encontrei uma falha em vocês”. Quem recebe a mensagem no suporte precisa saber que não deve responder com ameaça, não deve ignorar, e deve encaminhar imediatamente para o responsável definido. A reação hostil ao primeiro contato é o que transforma um aviso gratuito numa divulgação pública.

7.10. Resumo

- A fronteira do lícito é a **autorização**, não a dificuldade técnica nem a intenção. O art. 154-A do Código Penal não exige dano, lucro ou uso dos dados: consuma-se com o acesso não autorizado. Trocar um número numa URL é acesso.
- As penas foram **agravadas pela Lei 14.155/2021**, e o estelionato eletrônico (CP, art. 171, §2º-A) tem reclusão de 4 a 8 anos. Além da esfera penal, a LGPD gera responsabilidade civil e sanção administrativa autônomas.
- Autorização é **documento**, com escopo positivo e negativo, janela temporal, técnicas permitidas, tratamento de dados encontrados, contato de emergência e assinatura de quem tem poderes. Autorização do cliente não alcança a infraestrutura de terceiros que hospeda o alvo.
- Na **divulgação coordenada**, o essencial é não testar além do mínimo, não copiar dados, documentar tudo, usar o canal oficial e jamais condicionar o silêncio a pagamento — o que seria extorsão.
- **O Brasil não tem salvaguarda legal para pesquisa de boa-fé.** A proteção vem de programas privados com porto seguro ou da decisão da vítima de não representar. O CERT.br é o intermediário estabelecido quando não há canal de contato.
- A ética profissional vai além da lei em três frentes recorrentes: o **poder do administrador** (acesso possível não é acesso legítimo), o **monitoramento de empregados** (lícito com finalidade, comunicação prévia e proporcionalidade) e o **achado incidental** (parar, não explorar, comunicar).

- Praticar é obrigatório e há caminho legítimo abundante: laboratório próprio, plataformas de treino, competições e programas de recompensa — mais as faixas de documentação da RFC 5737 para exemplos.

Parte II.

**Volume 2 — Redes e Sistemas: a Base
Técnica**

8. A pilha TCP/IP sob a ótica do atacante e do defensor

Numa terça-feira de manhã, o setor financeiro de uma empresa cliente de um provedor regional começa a reclamar de avisos de certificado inválido ao acessar o internet banking. O navegador diz que a identidade do site não pôde ser verificada. O suporte de primeiro nível manda limpar o cache, trocar de navegador, reiniciar o computador. Nada resolve, e o problema atinge somente as máquinas daquele andar.

O técnico de rede sobe, conecta o notebook na mesma VLAN e captura trinta segundos de tráfego. O que ele vê não tem nada de sofisticado: um mesmo endereço MAC respondendo a consultas ARP para dois endereços IP diferentes — o do gateway e o do servidor de arquivos. Alguém plugou um dispositivo na rede e o colocou entre todas as máquinas do andar e o resto do mundo. O aviso de certificado não era falha do banco: era o único sinal visível de um ataque que já durava dias.

O que torna esse ataque possível não é uma vulnerabilidade de software. É uma decisão de projeto tomada nos anos 1970: o protocolo ARP acredita em qualquer resposta que receba. A pilha TCP/IP inteira foi desenhada para funcionar entre pares que confiavam uns nos outros, e a segurança foi acrescentada depois, camada por camada, remendo por remendo. Entender **onde** a confiança foi presumida é entender onde os ataques moram — e onde as defesas precisam ser instaladas.

Este é o primeiro capítulo do Volume 2, que constrói a base técnica sobre a qual o resto do manual se apoia. O Capítulo 4 e o Capítulo 5 ensinaram a enxergar ativos e ameaças de forma abstrata. Aqui a abstração encosta no fio.

8.1. Duas pilhas, um vocabulário

A literatura usa dois modelos de camadas, e a confusão entre eles atrapalha conversas técnicas todo dia. O **modelo OSI**, de sete camadas, é referência conceitual e nomenclatura: quando alguém diz “problema de camada 2” ou “firewall de camada 7”, está falando OSI. O **modelo TCP/IP**, de quatro camadas, é o que a internet realmente implementa (Kurose; Ross, 2021).

A Figura 8.1 mostra os dois lado a lado, com o que trafega em cada nível e a unidade de dados correspondente.

Vale guardar a correspondência, porque quase todo produto de segurança se descreve em termos de camada: um filtro de pacotes atua em 3 e 4, um WAF em 7, um controle de porta em 2.

8.1.1. Encapsulamento: quem envolve quem

Quando um navegador pede uma página, o dado desce a pilha e cada camada acrescenta seu próprio cabeçalho ao que veio de cima. Chegando ao destino, o processo se inverte. Esse empacotamento sucessivo é o **encapsulamento**, e tem duas consequências diretas para segurança.

8. A pilha TCP/IP sob a ótica do atacante e do defensor

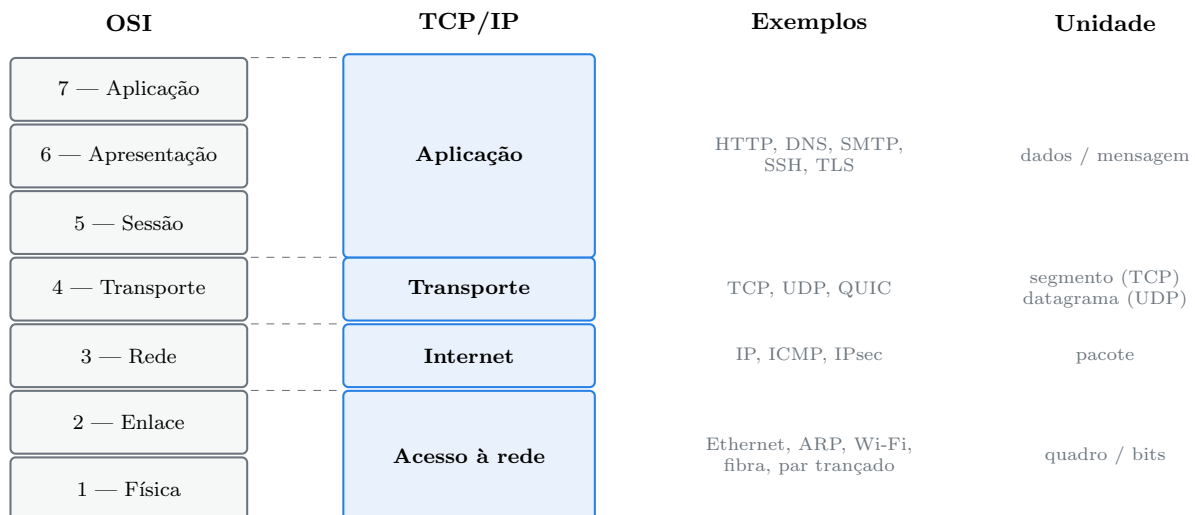


Figura 8.1.: Os dois modelos. O OSI dá o vocabulário; o TCP/IP é o que roda.

A primeira: **um cabeçalho só é lido por quem opera naquela camada**. Um switch comum olha o cabeçalho Ethernet e ignora o resto. Um roteador olha o cabeçalho IP. Um firewall de estado olha até o TCP. Cada equipamento no caminho enxerga uma fatia diferente do mesmo pacote, e a defesa precisa ser posicionada na camada onde o ataque acontece — um firewall de rede não vê injeção de SQL, e um WAF não vê ARP spoofing.

A segunda: **o que uma camada cifra, a de baixo não entende**. O TLS protege o conteúdo da camada de aplicação, mas os cabeçalhos IP e TCP continuam em claro. Quem observa o tráfego não lê a senha digitada, mas sabe com quais servidores a máquina falou, quando, por quanto tempo e com que volume. Esses **metadados** são frequentemente tão reveladores quanto o conteúdo — tema que o Volume 16 retoma ao tratar de privacidade e vigilância.

8.2. Camada 2: onde a confiança é total

A camada de enlace entrega quadros dentro de um mesmo segmento de rede. É a camada com menos verificação de todas e, por isso, a mais explorada por quem já conseguiu um ponto de presença interno.

8.2.1. ARP: o protocolo que acredita em tudo

Para enviar um quadro a um endereço IP na mesma rede, a máquina precisa descobrir o endereço MAC correspondente. O ARP resolve isso com uma pergunta em difusão — “quem tem 192.0.2.1?” — e aceita a primeira resposta que chegar. Não há autenticação, não há verificação, e respostas não solicitadas (*gratuitous ARP*) são aceitas e usadas para atualizar a tabela.

O ataque decorre diretamente disso. O invasor responde às consultas da vítima afirmando ser o gateway, e às do gateway afirmando ser a vítima. As duas tabelas ARP passam a apontar para o MAC dele, e todo o tráfego entre a vítima e o mundo atravessa sua máquina — o cenário de abertura deste capítulo. É o **ataque de intermediário** (*man in the middle*) em sua forma mais barata: não requer exploração de falha nenhuma, apenas acesso à rede local.

O que o invasor consegue depois disso depende do que trafega. Protocolos em claro entregam credenciais direto. Com HTTPS, ele pode tentar *downgrade* — interceptar a primeira requisição

HTTP e impedir o redirecionamento para HTTPS — ou apresentar um certificado próprio, gerando exatamente o aviso que o setor financeiro do caso de abertura via na tela. **O aviso de certificado é o controle funcionando**; o problema é que anos de avisos espúrios ensinaram os usuários a clicar em “prosseguir”.

As defesas moram no switch, não no computador:

- **Dynamic ARP Inspection (DAI)**, que valida respostas ARP contra a tabela de DHCP snooping e descarta as inconsistentes;
- **DHCP snooping**, que constrói essa tabela e bloqueia servidores DHCP não autorizados;
- **Port security**, limitando quantos MACs podem aparecer em cada porta de acesso;
- **HSTS** no lado do servidor, que elimina a janela de *downgrade* ao instruir o navegador a nunca usar HTTP naquele domínio.

8.2.2. Os outros ataques de camada 2

MAC flooding. A tabela CAM do switch — que associa MAC a porta — tem tamanho finito. Inundá-la com milhares de endereços forjados faz o switch, em muitos modelos, passar a inundar quadros por todas as portas, degradando-o ao comportamento de um hub e permitindo captura passiva. *Port security* limita o número de MACs por porta e resolve.

VLAN hopping. Duas variantes. Na primeira, o atacante negocia entroncamento com um switch que tem DTP habilitado e passa a receber o tráfego de todas as VLANs. Na segunda, marcação dupla (*double tagging*), ele insere duas etiquetas 802.1Q; a primeira é removida pelo switch de entrada e o quadro é entregue numa VLAN à qual ele não deveria ter acesso. As defesas são configuração explícita: desabilitar DTP nas portas de acesso, não usar a VLAN 1 para nada e definir uma VLAN nativa sem uso em todos os entroncamentos.

STP e o root bridge forjado. Anunciando-se como raiz da árvore de spanning tree, o atacante reposiciona o fluxo lógico da rede para passar por ele. As proteções chamam-se BPDU Guard e Root Guard, e devem estar ligadas em toda porta de acesso.

DHCP rogue. Um servidor DHCP falso entrega às vítimas o gateway e o DNS do atacante — resultado equivalente ao ARP spoofing, com menos esforço. DHCP snooping bloqueia respostas vindas de portas não confiáveis.

Um padrão emerge: **todas as defesas de camada 2 são configurações do switch que vêm desligadas de fábrica.** Rede montada com o padrão do equipamento é rede sem nenhuma dessas proteções.

8.3. Camada 3: endereços que ninguém verifica

A camada de rede leva pacotes de uma rede a outra. Seu cabeçalho carrega origem, destino, TTL, protocolo encapsulado e informações de fragmentação — e o campo de origem **não é verificado por ninguém no caminho.**

8.3.1. Falsificação de endereço de origem

Escrever um endereço de origem falso num pacote IP é trivial. As consequências dependem do protocolo acima.

Com **UDP**, que não exige resposta, a falsificação funciona plenamente. É a base dos ataques de **amplificação e reflexão**: o atacante envia consultas pequenas a servidores mal configurados — DNS recursivos abertos, NTP com `monlist`, memcached exposto — usando como origem o endereço da vítima. Os servidores respondem à vítima com pacotes dezenas ou centenas de vezes maiores. Um atacante com 1 Gbps de banda gera dezenas de Gbps contra o alvo, e o tráfego chega de milhares de servidores legítimos, o que dificulta o bloqueio por origem.

Com **TCP**, a falsificação é bem mais difícil, porque o atacante não recebe a resposta e precisaria adivinhar o número de sequência inicial escolhido pelo servidor. Pilhas modernas geram esse número de forma imprevisível — o que nem sempre foi verdade, e a história do TCP está cheia de ataques de predição de sequência.

A defesa contra falsificação de origem não é do lado da vítima: é de quem opera a rede de origem. A **BCP 38** (RFC 2827) recomenda que todo operador filtre, na borda, pacotes cuja origem não pertença ao bloco do cliente (Ferguson; Senie, 2000). Um provedor que aplica filtragem de entrada impede que sua rede seja usada como plataforma de reflexão. Um provedor que não aplica é parte do problema — e no Brasil essa é orientação de longa data do CERT.br e do NIC.br.

8.3.2. Fragmentação, ICMP e TTL

Fragmentação. Pacotes maiores que a MTU do enlace são divididos e remontados no destino. Historicamente isso foi usado para evadir inspeção: fragmentos sobrepostos, remontados de formas diferentes pelo IDS e pelo sistema-alvo, permitiam esconder o conteúdo real. Ataques clássicos como o *ping of death* e o *teardrop* exploravam falhas na remontagem. Firewalls e IDS modernos remontam antes de inspecionar, e a orientação prática é descartar fragmentos sobrepostos.

ICMP. Muito além do ping. `Echo` mapeia hosts vivos; `Time exceeded` alimenta o `traceroute` e revela a topologia; `Destination unreachable` com “fragmentation needed” é essencial à descoberta de MTU de caminho; `Redirect` pode alterar a rota de um host. Bloquear ICMP inteiramente é erro comum: quebra a descoberta de MTU e produz conexões que abrem e travam sem explicação. A postura correta é seletiva — permitir os tipos 3 e 11 e o `echo reply`, filtrar `redirect` e limitar a taxa do resto.

TTL. Decrementado a cada salto, existe para matar pacotes em laço. Como efeito colateral, seu valor inicial denuncia o sistema operacional de origem (64 em Linux, 128 em Windows, 255 em muitos equipamentos de rede) e a diferença entre o valor inicial esperado e o observado dá a distância em saltos. É um dos sinais que ferramentas de identificação passiva de sistema operacional usam — e um dos que denunciam um dispositivo intruso se apresentando como outra coisa.

8.4. Camada 4: o aperto de mão e seus abusos

O TCP oferece entrega confiável e ordenada sobre uma camada de rede que não garante nada. Faz isso mantendo **estado** — e estado é recurso finito, o que é exatamente a superfície de ataque.

8.4.1. O handshake de três vias

A Figura 8.2 mostra a abertura normal de conexão e, ao lado, o que o SYN flood faz com ela.

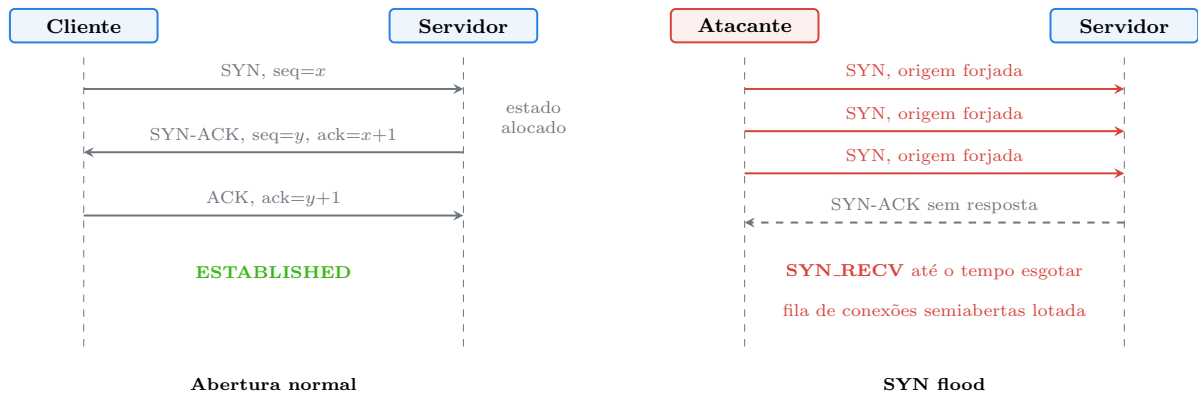


Figura 8.2.: O handshake de três vias e o abuso do estado que ele obriga o servidor a manter.

O ataque é elegante na sua economia: cada SYN custa ao atacante um pacote de algumas dezenas de bytes e custa ao servidor uma entrada de estado mantida por dezenas de segundos. A defesa clássica são os **SYN cookies**, que codificam o estado dentro do próprio número de sequência do SYN-ACK, permitindo ao servidor não alocar nada até que o ACK final chegue. No Linux é `net.ipv4.tcp_syncookies=1`, ligado por padrão nas distribuições atuais.

8.4.2. Flags, RST e a varredura

As bandeiras do cabeçalho TCP são a linguagem que as ferramentas de varredura falam, e as diferentes técnicas de escaneamento — que o Capítulo 14 vai colocar em laboratório e o Volume 9 vai detalhar — são combinações delas.

Tabela 8.1.: Técnicas de varredura e o que cada uma revela.

Técnica	O que envia	Interpretação	Rastro deixado
SYN scan	SYN, e RST antes de completar	SYN-ACK = aberta; RST = fechada	Conexão não completa; pouco log de aplicação
Connect scan	Handshake completo	Conexão bem-sucedida = aberta	Registro em todo log de aplicação
FIN, NULL, Xmas	Flags anômalas sem conexão	Silêncio = aberta ou filtrada; RST = fechada	Falha contra Windows, que sempre responde RST
ACK scan	Apenas ACK	Mapeia regra de firewall, não porta aberta	Discreto, mas informativo
UDP scan	Datagrama vazio ou específico	ICMP porta inalcançável = fechada	Lento; limitação de taxa distorce o resultado

O **RST** merece atenção separada. Um pacote RST forjado, com os números de sequência corretos, derruba uma conexão TCP estabelecida. Quem está no caminho — um provedor,

8. A pilha TCP/IP sob a ótica do atacante e do defensor

um equipamento de inspeção, um atacante em posição de intermediário — consegue injetá-lo. Historicamente foi usado tanto para censura quanto para gestão abusiva de tráfego. A proteção real é criptográfica: o TLS não impede o RST, mas impede que o conteúdo seja lido ou alterado, e protocolos sobre QUIC — que roda em UDP com autenticação de cabeçalho — resistem bem melhor a essa injeção.

8.4.3. UDP e QUIC

O UDP não tem handshake, não tem estado, não tem entrega garantida. Isso o torna adequado para DNS, VoIP, jogos e streaming — e conveniente para amplificação, como já visto. Do ponto de vista defensivo, a ausência de estado significa que o firewall também não tem estado real para acompanhar: o que ele faz é manter uma pseudo-sessão por tempo limitado após ver o primeiro datagrama.

O **QUIC**, padronizado na RFC 9000 e base do HTTP/3, muda o quadro. Roda sobre UDP na porta 443, integra o handshake TLS 1.3 ao estabelecimento da conexão e cifra a maior parte dos próprios metadados de transporte (Iyengar; Thomson, 2021). Para o operador de rede isso significa **menos visibilidade**: números de sequência, confirmações e controle de congestionamento — tudo o que um analisador de tráfego usava para diagnosticar desempenho — deixam de ser legíveis. É a tensão recorrente entre privacidade do usuário e observabilidade da rede, e ela reaparece no Volume 5.

8.5. O pecado original: confiança presumida

Recuando um passo, o padrão fica visível. A Tabela 8.2 lista, camada por camada, o que a pilha original simplesmente presumiu — e o que foi criado depois para consertar.

Tabela 8.2.: A confiança presumida em cada camada e o que veio depois para compensar.

Camada	O que o projeto original presumiu	Ataque decorrente	Remendo posterior
Enlace	Quem está na rede é legítimo	ARP spoofing, DHCP rogue, VLAN hopping	802.1X, DAI, DHCP snooping, port security
Rede	O endereço de origem é verdadeiro	Falsificação, amplificação, reflexão	BCP 38, uRPF, IPsec
Transporte	Os pares seguem o protocolo	SYN flood, injeção de RST	SYN cookies, TLS, QUIC
Aplicação	O nome resolvido é o servidor certo	Envenenamento de DNS, phishing	DNSSEC, TLS com validação de certificado, HSTS

Duas leituras importam. A primeira é histórica e generosa: a ARPANET ligava algumas dezenas de instituições que se conheciam, e autenticar cada quadro seria custo sem benefício. A segunda é operacional e menos generosa: **todos os remendos da coluna da direita são opcionais e a maioria vem desligada**. A internet insegura não é acidente inevitável — é o valor padrão que ninguém mudou.

Essa é também a razão pela qual a arquitetura Zero Trust, tema do Volume 4, ganhou tração: ela parte de negar a premissa de que estar dentro da rede significa alguma coisa.

8.6. Onde cada defesa mora

Encerrando com o mapa que a operação usa. Cada tecnologia de segurança atua numa camada específica e é cega para as demais:

- **Camada 2** — 802.1X (autenticação de porta), MACsec, port security, DAI, DHCP snooping, segmentação por VLAN.
- **Camada 3** — filtragem de pacotes, listas de acesso, IPsec, uRPF, roteamento com RPKI, anti-spoofing na borda.
- **Camada 4** — firewall de estado, SYN cookies, limitação de taxa por fluxo, balanceamento com verificação de saúde.
- **Camada 7** — TLS, WAF, filtro de DNS, gateway de e-mail, proxy com inspeção, autenticação de aplicação.

A pergunta que essa lista responde é a mais comum na resposta a incidentes: *o controle que eu tenho conseguiria enxergar esse ataque?* Um WAF caríssimo não vê um dispositivo intruso fazendo ARP spoofing no andar do financeiro. Um switch com DAI ligado vê — e teria evitado o caso de abertura antes que qualquer aviso de certificado aparecesse.

8.7. Brasil em Foco

Captura de tráfego é atividade regulada, não neutra. O diagnóstico do caso de abertura envolveu capturar pacotes de uma rede corporativa. Isso é lícito quando há autorização de quem opera a rede e finalidade técnica definida, mas o Capítulo 7 já fixou a régua: a Lei 9.296/1996 tipifica a interceptação de comunicação sem autorização judicial (Brasil, 1996), e capturar conteúdo alheio “para diagnosticar” sem autorização escrita é risco real. A prática correta é capturar o mínimo necessário, preferir cabeçalhos a conteúdo e ter autorização documentada antes de conectar o analisador.

A LGPD alcança o arquivo de captura. Um .pcap de rede corporativa contém endereços IP, nomes de máquina, credenciais em protocolos legados e frequentemente conteúdo de e-mail e navegação — dados pessoais no sentido do art. 5º, I da LGPD (Brasil, 2018). Capturas precisam de prazo de retenção, armazenamento restrito e descarte definido. Aquele **captura-financeiro.pcap** guardado por dois anos na pasta compartilhada do suporte é um incidente esperando por um momento oportuno.

BCP 38 é a contribuição mais concreta que um provedor brasileiro pode dar. O NIC.br e o CERT.br mantêm há anos o programa de boas práticas para operadores, e a filtragem anti-spoofing na borda é o item de maior impacto coletivo. Um provedor que não filtra permite que seus clientes participem de ataques de reflexão contra terceiros — inclusive contra outros provedores brasileiros. Há testes públicos que permitem verificar externamente se a própria rede está filtrando.

Anatel e a resiliência da rede. A Resolução 740/2020 estabelece requisitos de segurança cibernética aplicáveis a produtos de telecomunicações e às redes das prestadoras (Agência Nacional de Telecomunicações, 2020). Na prática, configurações básicas de camada 2 e 3 — controle de acesso à administração dos equipamentos, filtragem de borda, registro de eventos — deixaram de ser boa vontade e passaram a compor obrigação regulatória, com fiscalização possível.

Marco Civil e a inspeção de tráfego. O art. 9º do Marco Civil consagra a neutralidade de rede e restringe a discriminação de tráfego a requisitos técnicos indispensáveis e à priorização de serviços de emergência (Brasil, 2014). Bloquear ou degradar tráfego de uma aplicação específica

por decisão comercial é vedado; aplicar mitigação de ataque volumétrico é requisito técnico legítimo. A diferença entre as duas coisas precisa estar documentada na política de gestão de tráfego do provedor.

8.8. No Campo

1. Ligar as proteções de camada 2 que vieram desligadas. É o item de melhor relação entre esforço e resultado em toda a rede de acesso. Em equipamento com sintaxe Cisco-like:

```
! Em todas as portas de acesso
interface range GigabitEthernet0/1-46
  switchport mode access
  switchport nonegotiate           ! desliga DTP, mata VLAN hopping
  switchport port-security
  switchport port-security maximum 2
  switchport port-security violation restrict
  spanning-tree portfast
  spanning-tree bpduguard enable   ! ninguém vira root bridge

! Globais
ip dhcp snooping
ip dhcp snooping vlan 10,20,30
ip arp inspection vlan 10,20,30    ! depende do snooping acima
```

2. Verificar a tabela ARP quando o sintoma for “certificado inválido em vários micros”. Dois IPs com o mesmo MAC é o padrão que denuncia o intermediário:

```
# Linux: agrupa vizinhos por MAC e mostra os repetidos
ip neigh show | awk '$5 != "" {print $5}' | sort | uniq -c | sort -rn | head
```

```
# Windows: MACs que respondem por mais de um IP
Get-NetNeighbor -AddressFamily IPv4 |
  Where-Object State -ne 'Unreachable' |
  Group-Object LinkLayerAddress |
  Where-Object Count -gt 1
```

3. Aplicar filtragem anti-spoofing na borda. Em roteador com sintaxe Cisco-like, o caminho curto é o uRPF; em Linux, uma regra de saída equivalente:

```
interface GigabitEthernet0/0
  description Cliente - bloco 198.51.100.0/24
  ip verify unicast source reachable-via rx
```

```
# Linux: só deixa sair o que tem origem no bloco do cliente
nft add rule inet filter forward iifname "eth1" \
  ip saddr != 198.51.100.0/24 counter drop
```


4. Confirmar que os SYN cookies estão ativos e a fila é adequada. Custa dois comandos e evita indisponibilidade por um ataque de baixo volume:

```
sysctl net.ipv4.tcp_syncookies net.ipv4.tcp_max_syn_backlog net.core.somaxconn
# Conexões semiabertas agora - número alto e crescente é sintoma
ss -tan state syn-recv | wc -l
```

5. Filtrar ICMP com bisturi, não com machado. Bloqueio total quebra a descoberta de MTU e gera chamados insolúveis de “o site abre e trava”:

```
nft add rule inet filter input icmp type { echo-reply, destination-unreachable,
↪ time-exceeded } accept
nft add rule inet filter input icmp type echo-request limit rate 10/second
↪ accept
nft add rule inet filter input icmp type redirect drop
```

6. Fechar os amplificadores da própria rede. Antes de se preocupar em ser vítima, garantir que não se é plataforma. Recursão de DNS aberta ao mundo, NTP com `monlist`, memcached e SNMP expostos:

```
# O que responde recursão vindo de fora? (executar de fora da rede)
dig @198.51.100.53 example.com A +short
# NTP: a consulta de modo 7 deve falhar
ntpq -c rv 198.51.100.10
```

7. Padronizar a captura de tráfego. Um procedimento de uma página: quem autoriza, o que se captura (preferir `-s 96` para limitar a carga útil), onde se guarda, por quanto tempo e como se apaga:

```
# Cabeçalhos apenas, rotação a cada 5 minutos, 10 arquivos no máximo
tcpdump -i eth0 -s 96 -w /var/captura/rede-%Y%m%d-%H%M.pcap -G 300 -W 10 \
'not port 22'
```

8. Documentar a política de gestão de tráfego. Uma página que separa, para o Marco Civil e para a Anatel, o que é requisito técnico (mitigação de ataque, controle de congestionamento) do que seria discriminação comercial. Escrita antes do incidente, é defesa; escrita depois, é justificativa.

8.9. Resumo

- Os dois modelos convivem: **OSI dá o vocabulário** (camada 2, camada 7) e **TCP/IP é o que roda**. Cada equipamento no caminho lê apenas os cabeçalhos da sua camada, e por isso a defesa precisa estar na camada onde o ataque ocorre.
- A **camada 2 não verifica nada**. ARP spoofing, DHCP rogue, MAC flooding, VLAN hopping e root bridge forjado exploram a mesma premissa de que quem está na rede é legítimo. Todas as defesas — DAI, DHCP snooping, port security, BPDU Guard, 802.1X — são configurações de switch **desligadas de fábrica**.

8. A pilha TCP/IP sob a ótica do atacante e do defensor

- O **endereço de origem IP não é verificado por ninguém**. Com UDP isso viabiliza amplificação e reflexão; com TCP é limitado pela imprevisibilidade dos números de sequência. A defesa é a filtragem na borda de quem origina o tráfego (**BCP 38**), não do lado da vítima.
- O TCP mantém **estado**, e estado finito é superfície de ataque: o SYN flood custa dezenas de bytes ao atacante e uma entrada de tabela ao servidor. **SYN cookies** resolvem. Injeção de RST derruba conexões estabelecidas e só é neutralizada por criptografia.
- **QUIC muda o equilíbrio**: cifra os metadados de transporte, ganhando privacidade e reduzindo a visibilidade que operadores usavam para diagnóstico.
- O padrão que atravessa a pilha é a **confiança presumida** por um projeto feito para uma rede entre pares conhecidos. Todos os remendos posteriores são opcionais — a insegurança padrão é escolha de configuração, não fatalidade.
- No Brasil, capturar tráfego exige autorização e cuidado com a Lei 9.296/1996; o **.pcap** resultante é dado pessoal sob a LGPD; e a filtragem anti-spoofing na borda é a contribuição coletiva mais concreta de um provedor.

9. Endereçamento, roteamento e NAT

Um provedor de médio porte cresceu por aquisição. Herdou três redes que usavam 192.168.1.0/24 para a gerência dos equipamentos — a sua, a da empresa comprada em 2021 e a da comprada em 2023. Ninguém reenumerou. A solução foi ir empilhando NAT: cada rede herdada fica atrás de um roteador que traduz seus endereços para um bloco distinto antes de entrar na rede principal.

Funcionou por dois anos. Até a noite em que um servidor de monitoramento passou a registrar tentativas de acesso SSH vindas de 10.20.0.7, um endereço que, dependendo de qual tabela de tradução se consulte, corresponde a três equipamentos diferentes em três cidades diferentes. A equipe levou seis horas para descobrir de qual deles vinha o tráfego, porque o registro do firewall guardava o endereço traduzido e não o original.

O invasor teve seis horas de vantagem. Não por causa de uma falha de software: por causa de um plano de endereçamento que nunca existiu.

Este capítulo trata da camada 3 do Capítulo 8 do ponto de vista de quem projeta e opera: como os endereços são atribuídos, como os pacotes encontram o caminho, e o que o NAT realmente faz — incluindo o que ele não faz, apesar de todo mundo acreditar que faz.

9.1. Endereçamento IPv4 e a matemática que não dá para evitar

Um endereço IPv4 tem 32 bits, escritos em quatro octetos decimais. A **máscara de rede** separa esses 32 bits em duas partes: um prefixo que identifica a rede e um sufixo que identifica o host dentro dela. A notação CIDR escreve o tamanho do prefixo depois da barra — 198.51.100.0/24 significa 24 bits de rede e 8 de host (Fuller; Li, 2006).

A conta que todo profissional precisa fazer de cabeça:

Tabela 9.1.: Prefixos CIDR de uso corrente.

Prefixo	Máscara	Endereços	Hosts utilizáveis	Uso típico
/30	255.255.255.252	4	2	Enlace ponto a ponto
/29	255.255.255.248	8	6	Pequeno bloco de servidores
/28	255.255.255.240	16	14	Sub-rede de DMZ
/24	255.255.255.0	256	254	VLAN padrão de escritório
/22	255.255.252.0	1.024	1.022	Rede de acesso de provedor
/16	255.255.0.0	65.536	65.534	Espaço de agregação

Os dois endereços subtraídos em cada linha são o de **rede** (todos os bits de host em zero) e o de **broadcast** (todos em um). A exceção é o /31, previsto na RFC 3021 para enlaces ponto a ponto, onde os dois endereços são utilizáveis — economia que importa quando um provedor tem centenas de enlaces.

Há três faixas **privadas**, definidas pela RFC 1918, que não são roteáveis na internet pública (Rekhter et al., 1996):

- 10.0.0.0/8 — 16,7 milhões de endereços;
- 172.16.0.0/12 — de 172.16.0.0 a 172.31.255.255;
- 192.168.0.0/16 — 65.536 endereços.

E vale conhecer duas faixas que causam confusão em diagnóstico. 127.0.0.0/8 é *loopback*: nunca sai da máquina. 169.254.0.0/16 é auto-configuração de enlace local — quando uma máquina aparece com um endereço dessa faixa, significa que **o DHCP não respondeu**. É o sintoma, não a causa, e a causa costuma ser cabo, VLAN errada ou servidor DHCP fora do ar.

9.1.1. O plano de endereçamento como controle de segurança

O caso de abertura mostra por que endereçamento é assunto de segurança, e não apenas de conectividade. Um plano de endereçamento bem feito produz três benefícios diretos:

Legibilidade. Ao ver 10.30.20.15 num alerta, quem está de plantão deve conseguir dizer, sem consultar planilha, que se trata da filial 30, VLAN de servidores, host 15. Isso se obtém reservando significado posicional aos octetos: por exemplo, o segundo octeto para o site, o terceiro para a função (10 = gerência, 20 = servidores, 30 = usuários, 40 = visitantes, 50 = câmeras).

Filtragem simples. Se todas as VLANs de gerência de todos os sites estiverem em 10.x.10.0/24, uma única regra 10.0.10.0/16 com máscara não contígua — ou, mais limpo, um bloco agregado dedicado — cobre a regra “gerência não fala com usuário”. Endereçamento aleatório exige uma regra por sub-rede, e listas longas de regras são listas com erro.

Ausência de sobreposição. Redes que se sobrepõem impedem VPN entre sites, impedem fusão de empresas e obrigam ao empilhamento de NAT do caso de abertura. Reservar de saída um bloco /8 privado inteiro e distribuir /16 por site custa nada e evita o problema para sempre.

A Figura 9.1 mostra a estrutura de um plano assim.

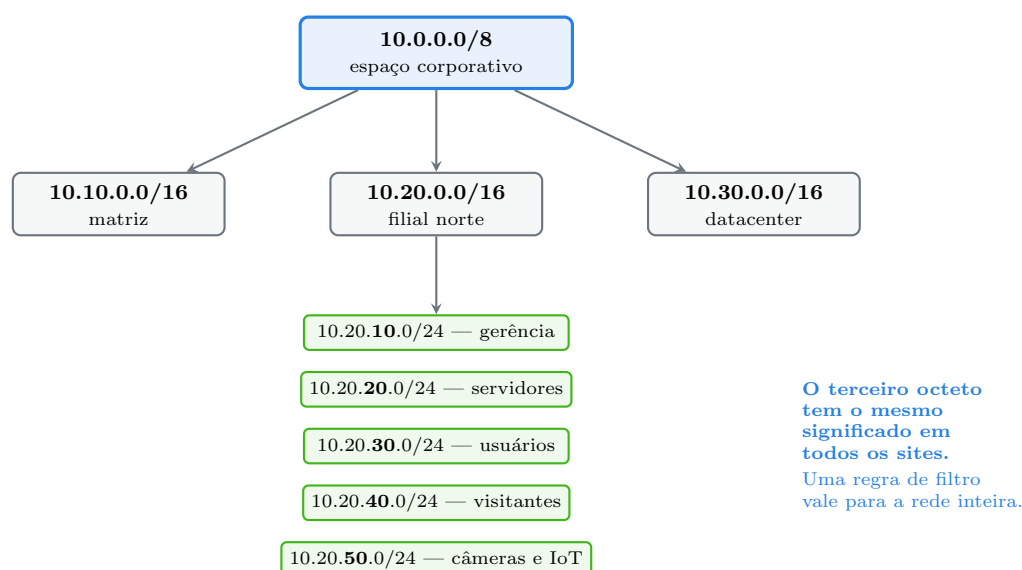


Figura 9.1.: Plano de endereçamento hierárquico. Posição com significado transforma um endereço em informação de diagnóstico.

9.2. IPv6: o que muda de verdade

O IPv6 usa 128 bits, escritos em oito grupos hexadecimais de quatro dígitos, com regras de abreviação: zeros à esquerda de cada grupo podem ser omitidos e uma única sequência de grupos nulos vira `::`. Assim, `2001:0db8:0000:0000:0000:0000:0001` escreve-se `2001:db8::1`. A faixa `2001:db8::/32` é reservada para documentação, e é dela que vêm todos os exemplos deste manual.

Cinco pontos concentram o que importa para segurança:

Não existe NAT, e isso é bom e ruim. Cada dispositivo pode ter endereço global roteável. Bom porque restaura a conectividade fim a fim e simplifica diagnóstico e registro; ruim porque desaparece o efeito colateral de ocultação que muita gente confundia com segurança. Com IPv6, **o firewall precisa ser explícito** — não há mais um NAT bloqueando por acidente o que ninguém liberou.

A varredura muda de natureza. Uma sub-rede IPv6 padrão é /64: 18 quintilhões de endereços. Varrer sequencialmente é inviável. Isso não protege tanto quanto parece, porque atacantes usam DNS, certificados públicos, registros em serviços e padrões previsíveis de atribuição para descobrir hosts ativos. Endereçamento sequencial manual (`::1`, `::2`, `::3`) devolve a varredura à mesa.

Autoconfiguração e o vizinho malicioso. O SLAAC permite que um host se configure a partir de anúncios de roteador (RA). Um RA forjado redireciona todo o tráfego da rede — o equivalente IPv6 do DHCP rogue. A defesa chama-se **RA Guard** e é configuração de switch, tal como as do Capítulo 8. O protocolo NDP substitui o ARP e herda os mesmos problemas de confiança, mitigados por **ND Inspection** e, em ambientes exigentes, por SEND.

Extensões de privacidade. Endereços derivados do MAC identificavam a máquina em qualquer rede. As extensões da RFC 4941 geram endereços temporários rotativos, hoje o padrão nos sistemas operacionais. Para o operador, isso significa que um endereço IPv6 de cliente **muda com frequência**, com impacto direto em registro, correlação e resposta a incidentes.

Pilha dupla é dobro de superfície. O erro mais comum na transição: a organização configura firewall rigoroso em IPv4, habilita IPv6 sem perceber (ele vem ligado por padrão em praticamente tudo) e deixa a política v6 permissiva ou inexistente. **Todo controle precisa existir nas duas pilhas** — regra de firewall, registro, monitoramento, lista de bloqueio. Um caminho v6 aberto anula meses de trabalho no v4.

9.3. Roteamento: como o pacote encontra o caminho

Um roteador recebe um pacote, consulta sua tabela e decide por onde encaminhá-lo. A regra de decisão é o **prefixo mais longo**: entre `0.0.0.0/0`, `198.51.0.0/16` e `198.51.100.0/24`, o pacote para `198.51.100.7` sai pela rota /24, a mais específica.

Rotas entram na tabela por três origens. As **conectadas** aparecem sozinhas quando uma interface recebe endereço. As **estáticas** são escritas à mão — previsíveis, sem tráfego de controle, mas sem reação a falha. As **dinâmicas** vêm de protocolos que trocam informação entre roteadores: OSPF e IS-IS dentro de um domínio administrativo, BGP entre domínios.

9.3.1. O interior: OSPF e amigos

Protocolos de gateway interior propagam mudanças de topologia em segundos e recalculam caminhos automaticamente. Do ponto de vista de segurança, dois riscos:

Vizinhança não autenticada. Um roteador não autorizado que estabeleça adjacência OSPF pode injetar rotas e atrair tráfego. A mitigação tem duas partes: **autenticação de área** com chave compartilhada ou HMAC-SHA, e marcação de todas as interfaces de usuário como **passive-interface**, para que não formem adjacência com nada.

Vazamento de topologia. Um protocolo de estado de enlace distribui o mapa completo da rede a todos os participantes. Quem consegue escutar aprende a topologia inteira de graça — reconhecimento sem enviar um único pacote.

9.3.2. O exterior: BGP e a confiança global

O BGP mantém a internet de pé anunciando quais blocos cada sistema autônomo (AS) alcança. E funciona, essencialmente, na base da palavra: por padrão, um AS que anuncia um prefixo é acreditado pelos vizinhos.

Daí decorrem duas famílias de incidente. O **sequestro de prefixo** (*hijack*) ocorre quando um AS anuncia bloco que não é seu e atrai o tráfego destinado a ele — por erro de digitação ou por intenção, com casos documentados de desvio de tráfego para interceptação e de roubo de criptoativos por sequestro de rotas de DNS. O **vazamento de rota** (*route leak*) ocorre quando um AS repassa a um provedor rotas aprendidas de outro provedor, transformando-se involuntariamente em trânsito para tráfego que não deveria passar por ele — com congestionamento imediato.

As defesas atuais são três, e todas dependem de adoção coletiva: **filtros de prefixo** entre vizinhos, construídos a partir de objetos de roteamento registrados; **RPKI com validação de origem** (ROV), que assina criptograficamente a autorização de um AS para anunciar um prefixo; e **limites de prefixos recebidos**, que derrubam a sessão quando um vizinho anuncia muito além do esperado, contendo vazamentos. O Volume 5 dedica um capítulo ao tema; aqui basta fixar que a autorização de anúncio é um problema de confiança global e ainda parcialmente resolvido.

9.4. NAT: o que ele faz e o que ele não faz

A tradução de endereços de rede reescreve endereços — e frequentemente portas — nos cabeçalhos dos pacotes que atravessam um roteador. Nasceu como paliativo para a escassez de IPv4 e acabou virando peça estrutural de quase toda rede corporativa.

O tipo dominante é o **NAT de sobrecarga** (PAT, ou *masquerade*), em que muitos hosts internos compartilham um endereço público, distinguidos pela porta de origem. A Figura 9.2 mostra a mecânica e a tabela de estado que a sustenta.

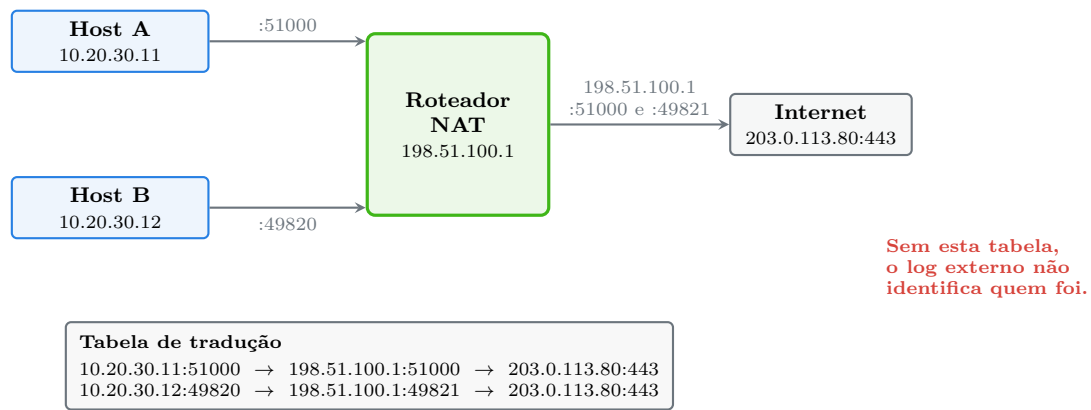


Figura 9.2.: NAT de sobrecarga. A tabela de tradução é o único elo entre o endereço público registrado e o host que originou a sessão.

9.4.1. O mito da segurança por NAT

A crença de que “estar atrás de NAT é estar protegido” é uma das mais persistentes e mais caras da área. O que o NAT realmente oferece é um efeito colateral: como não há tradução prévia para conexões entrantes não solicitadas, elas não têm para onde ser encaminhadas e são descartadas. Isso se parece com um firewall — mas não é um.

O que o NAT **não** faz:

- **Não protege contra nada iniciado de dentro.** Um usuário que abre um anexo malicioso estabelece a conexão de dentro para fora, e o NAT cria a tradução com prazer. Praticamente todo malware moderno funciona assim, exatamente por isso.
- **Não inspeciona conteúdo.** Não vê exploração, não vê exfiltração, não vê comando e controle.
- **Não resiste a técnicas de travessia.** UPnP e NAT-PMP permitem que qualquer aplicação na rede interna abra portas no roteador sem autorização humana. STUN e *hole punching* estabelecem sessões diretas entre dois hosts atrás de NAT. Essas técnicas existem porque aplicações legítimas precisam delas — e servem igualmente ao que não é legítimo.
- **Não impede movimentação lateral.** Dentro da rede traduzida, tudo se comunica livremente, a menos que haja segmentação de verdade.

E o que o NAT **quebra**, em compensação: rastreabilidade (o caso de abertura), geolocalização e reputação por endereço, IPsec fim a fim em modo transporte, protocolos que carregam endereços na carga útil (SIP, FTP ativo) e, no caso do CGNAT, a própria capacidade de responder a um ofício judicial. A postura correta é: **NAT é solução de endereçamento; firewall é solução de segurança.** Quem precisa de proteção configura política explícita de filtragem, com ou sem NAT no caminho.

9.4.2. CGNAT e a obrigação de registrar

Provedores que esgotaram seu espaço IPv4 usam CGNAT: centenas ou milhares de assinantes compartilham um mesmo endereço público, com o espaço 100.64.0.0/10 (RFC 6598) entre o cliente e a borda. É uma segunda camada de tradução, sobre a que o roteador do cliente já faz.

O problema operacional é imediato e recorrente. Chega um ofício informando que, em determinada data e hora, o endereço 198.51.100.42 praticou um ilícito. Esse endereço foi compartilhado por

512 assinantes naquele instante. **Sem o registro de tradução — endereço interno, porta externa, destino, marca temporal com fuso — não há como identificar o assinante**, e a resposta ao ofício é a admissão de que a informação não existe.

A prática consolidada, e que reduz o volume de registro em ordens de magnitude, é a **alocação determinística de blocos de portas**: cada assinante recebe uma faixa fixa de portas no endereço público, e basta registrar essa atribuição em vez de cada sessão. Um registro por assinante por dia substitui milhões de linhas de fluxo.

9.5. Brasil em Foco

O Marco Civil exige o registro que o CGNAT dificulta. Os arts. 13 e 15 da Lei 12.965/2014 obrigam à guarda de registros de conexão por um ano e de registros de acesso a aplicações por seis meses (Brasil, 2014). Com CGNAT, “registro de conexão” só é útil se incluir a **porta de origem** e a **marca temporal precisa** — sem isso, o endereço público sozinho aponta para centenas de assinantes e o provedor fica sem condições de responder. Guardar apenas o endereço é cumprir a obrigação de forma que não serve a ninguém.

Marca temporal e fuso horário derrubam investigações. Registro sem fuso explícito, ou com relógio dessincronizado, produz identificação errada de assinante — com potencial de apontar pessoa inocente. Sincronização NTP obrigatória em todo equipamento que registra, e registro em UTC com fuso declarado, são requisitos práticos, não preciosismo. É o tipo de detalhe que só aparece quando já é tarde.

Sigilo dos registros. Os arts. 10 e 11 do Marco Civil impõem que esses dados só sejam disponibilizados mediante ordem judicial, com indicação do período e do fato apurado. Entregar identificação de assinante a quem pede sem ordem — inclusive a autoridade policial fora do rito — é violação de sigilo, com responsabilização civil e administrativa. O Capítulo 7 tratou da postura correta: exigir a ordem, verificar sua abrangência e responder exatamente o requisitado.

A LGPD sobre a tabela de tradução. Endereço IP associável a pessoa é dado pessoal. Os registros de CGNAT são, portanto, base de dados pessoais sob a LGPD (Brasil, 2018), com necessidade de controle de acesso, registro de quem consulta, prazo de retenção definido (os prazos legais são mínimos, não permissão para guardar indefinidamente) e eliminação ao fim. Um provedor que retém cinco anos de tabela NAT “por precaução” está criando risco, não reduzindo.

Blocos e ASN vêm do registro.br. No Brasil, prefixos IPv4/IPv6 e números de AS são atribuídos pelo NIC.br via registro.br, e o contato de abuso registrado é o canal por onde o CERT.br notifica máquinas comprometidas. Manter esse contato atualizado e monitorado é obrigação prática. **Adotar IPv6 nativo** é, além disso, a única saída estrutural para o problema de rastreabilidade do CGNAT — e o Brasil está entre os países com maior adoção, o que remove a desculpa de que o mercado não está pronto.

9.6. No Campo

1. Escrever o plano de endereçamento antes de precisar dele. Uma planilha com bloco por site, significado fixo do terceiro octeto e faixas reservadas para crescimento. Depois de renumerar uma rede em produção, ninguém precisa ser convencido do valor disso.


```
10.<site>.<função>.<host>/24
```

```
site:      10=matriz  20=filial norte  30=DC  40..=novos
função:    10=gerência 20=servidores 30=usuários 40=visitantes 50=IoT/CFTV
reserva:   60..99 livres para crescimento por site
```

2. Guardar registro de tradução que serve para responder ofício. Com CGNAT, blocos determinísticos de porta reduzem o volume a uma linha por assinante:

```
# Exemplo com nftables: bloco determinístico de portas por assinante
nft add rule ip nat postrouting ip saddr 100.64.0.10 \
    snat to 198.51.100.42:2048-3071

# Verificar sincronização de relógio em quem registra - sem isso, o log mente
timedatectl show -p NTPSynchronized -p Timezone
chronyc tracking | grep -E 'Stratum|System time'
```

3. Auditar a sobreposição de endereços antes de cada aquisição ou VPN nova. Cinco minutos que evitam meses de NAT empilhado:

```
# Lista as redes conectadas e as rotas aprendidas, para conferir contra o plano
ip -4 route show | sort
ip -6 route show | sort
```

4. Fechar a pilha IPv6 com a mesma política do IPv4. O erro mais comum da transição:

```
# As duas tabelas precisam ter política equivalente
nft list ruleset | grep -A5 'table ip filter'
nft list ruleset | grep -A5 'table ip6 filter'

# Se IPv6 não for usado nem gerenciado, desabilitar é melhor que ignorar
sysctl net.ipv6.conf.all.disable_ipv6
```

5. Proteger o protocolo de roteamento interno. Autenticação nas adjacências e interface passiva em tudo que dá para usuário:

```
router ospf 1
 area 0 authentication message-digest
 passive-interface default
 no passive-interface GigabitEthernet0/1    ! só o enlace de trânsito
 interface GigabitEthernet0/1
 ip ospf message-digest-key 1 md5 <chave>
```

6. Ligar RA Guard e DHCPv6 Guard nas portas de acesso. O equivalente IPv6 do DHCP snooping, e igualmente desligado de fábrica:

```
ipv6 nd raguard policy ACESSO
device-role host
interface range GigabitEthernet0/1-46
ipv6 nd raguard attach-policy ACESSO
```

7. Verificar o que está anunciado com o seu ASN. Sequestro de prefixo é detectado por monitoramento externo, não pelo roteador de borda. Ferramentas públicas de olhar-de-vidro (*looking glass*) e serviços de monitoramento de BGP alertam quando um prefixo seu aparece anunciado por outro AS — configurar esse alerta leva minutos e é a única forma prática de descobrir o problema em tempo hábil.

8. Desligar UPnP na borda. Permitir que qualquer aplicação da rede interna abra portas no roteador sem aprovação anula a política de firewall. Onde houver necessidade legítima, encaminhamento explícito e documentado, com revisão periódica da lista.

9.7. Resumo

- Endereçamento é **controle de segurança**, não só conectividade: um plano hierárquico com significado posicional dá legibilidade ao alerta, permite regras de filtro curtas e evita a sobreposição que leva ao NAT empilhado e à perda de rastreabilidade.
- As faixas privadas da RFC 1918 e as de documentação (192.0.2.0/24, 2001:db8::/32) são o vocabulário obrigatório. Um 169.254.x.x na máquina significa **DHCP sem resposta**, não configuração válida.
- No **IPv6**, o que muda para segurança é a ausência de NAT (o firewall precisa ser explícito), a inviabilidade da varredura sequencial, os ataques de RA e NDP equivalentes aos de camada 2, os endereços temporários que dificultam correlação, e o risco de **pilha dupla com política só em v4**.
- O roteamento decide por **prefixo mais longo**. Internamente, autenticar adjacências e marcar interfaces de usuário como passivas. Externamente, o BGP funciona na base da confiança, e as defesas contra sequestro e vazamento — filtros de prefixo, RPKI com validação de origem, limite de prefixos — dependem de adoção coletiva.
- **NAT não é firewall**. Não protege contra o que é iniciado de dentro, não inspeciona conteúdo, é atravessado por UPnP e *hole punching* e não impede movimentação lateral. Em compensação, quebra rastreabilidade, geolocalização, IPsec fim a fim e protocolos que carregam endereços na carga útil.
- Com **CGNAT**, o registro só é útil se incluir porta de origem e marca temporal sincronizada e com fuso declarado — sem isso, o Marco Civil não é atendido de forma útil e um ofício aponta para centenas de assinantes. Blocos determinísticos de portas reduzem o volume de registro em ordens de magnitude.
- Registros de tradução são **dados pessoais sob a LGPD**: acesso controlado, retenção definida e eliminação ao fim do prazo. Adotar IPv6 nativo é a única solução estrutural para o problema de rastreabilidade.

10. Protocolos de aplicação e seus riscos: HTTP, DNS, SMTP e SNMP

O departamento financeiro de uma distribuidora recebe um e-mail do fornecedor habitual informando mudança de conta bancária. O remetente aparece corretamente como `financeiro@fornecedor.exemplo.br`. O texto cita o número do pedido em aberto, o valor exato e o nome da pessoa que costuma assinar. O pagamento de R\$ 180 mil sai naquela tarde.

Duas semanas depois, o fornecedor cobra a fatura. A investigação mostra que o e-mail nunca passou pelo servidor do fornecedor: o campo **From:** foi simplesmente escrito pelo atacante, porque o SMTP permite escrever o que se quiser ali. O domínio do fornecedor não tinha registro SPF, não assinava com DKIM e não publicava política DMARC. A informação sobre o pedido veio de uma caixa postal comprometida meses antes, lida em silêncio.

Nenhum software foi explorado. O ataque consistiu em usar quatro protocolos exatamente como eles foram especificados.

Os dois capítulos anteriores trataram da infraestrutura que move pacotes. Este sobe à camada 7 — onde estão os dados que interessam a quem ataca, e onde a maior parte dos incidentes efetivamente acontece.

10.1. O padrão comum: texto claro e confiança presumida

Os protocolos de aplicação fundamentais da internet foram especificados entre 1971 e 1990, e compartilham três traços que explicam quase todos os seus problemas de segurança.

São textuais e legíveis. Comandos em ASCII, respostas em ASCII. Isso foi decisão deliberada — facilita depuração, permite falar o protocolo à mão com um `telnet` — e significa que qualquer um no caminho lê tudo.

Não autenticam a origem. O SMTP aceita qualquer remetente. O DNS aceita qualquer resposta que case com a pergunta. O HTTP não tem noção de identidade.

Não verificam integridade. Nada impede que um intermediário altere o conteúdo em trânsito.

A resposta histórica foi a mesma em todos os casos: envolver o protocolo original em TLS e acrescentar mecanismos de autenticação por cima. A Tabela 10.1 resume o quadro atual.

Tabela 10.1.: Protocolos de aplicação, portas e o que cada um expõe sem proteção.

Protocolo	Porta em claro	Porta protegida	O que vaza sem proteção	Camada de autenticação
HTTP	80	443 (TLS)	Conteúdo, cookies, credenciais	TLS + HSTS

Protocolo	Porta em claro	Porta protegida	O que vaza sem proteção	Camada de autenticação
DNS	53	853 (DoT), 443 (DoH)	Todo o histórico de navegação	DNSSEC (integridade)
SMTP	25	587/465 (STARTTLS, TLS)	Mensagens, credenciais	SPF, DKIM, DMARC, MTA-STS
IMAP / POP3	143 / 110	993 / 995	Caixa postal inteira	TLS + autenticação forte
FTP	21	990 (FTPS), 22 (SFTP)	Credenciais e arquivos	Substituir por SFTP
Telnet	23	—	Sessão administrativa completa	Substituir por SSH
SNMP v1/v2c	161	161 (SNMPv3)	Inventário e configuração	SNMPv3 com authPriv
LDAP	389	636 (LDAPS)	Diretório e credenciais	LDAPS ou StartTLS

Duas leituras práticas dessa tabela. A primeira: **a coluna do meio ainda está em uso** em quase toda rede corporativa, tipicamente em equipamentos antigos, sistemas legados e impressoras. A segunda: para os quatro protocolos que este capítulo detalha, a defesa não é apenas ligar TLS — cada um tem um problema estrutural próprio que o TLS não resolve.

10.2. HTTP: simples por fora, complexo por dentro

O HTTP é um protocolo de requisição e resposta sem estado (Fielding; Nottingham; Reschke, 2022). O cliente envia um método, um caminho e cabeçalhos; o servidor responde com um código e um corpo. Toda a complexidade — e a superfície de ataque — vem do que foi acrescentado por cima dessa simplicidade.

10.2.1. Os cabeçalhos que importam para segurança

Os cabeçalhos de resposta são o principal controle de segurança que uma aplicação web configura sem tocar no código:

Tabela 10.2.: Cabeçalhos de resposta HTTP com função de segurança.

Cabeçalho	O que faz	Sem ele
Strict-Transport-Security	Obriga HTTPS naquele domínio	Janela de <i>downgrade</i> na primeira visita
Content-Security-Policy	Restringe origens de script, estilo e mídia	XSS explora sem restrição
X-Content-Type-Options: nosniff	Impede adivinhação de tipo	Upload vira script executável
X-Frame-Options / frame-ancestors	Impede enquadramento em outro site	<i>Clickjacking</i>

Cabeçalho	O que faz	Sem ele
Referrer-Policy	Limita o que vaza na navegação	URLs internas com token vazam a terceiros
Set-Cookie: Secure; HttpOnly; SameSite	Protege o cookie de sessão	Roubo por script ou por HTTP; CSRF

O Capítulo 5 chamaria isso de controles que atuam sobre a exploração, não sobre a falha. São baratos, testáveis por ferramenta automática e mesmo assim ausentes na maioria dos sistemas internos — precisamente porque “é só a intranet”.

10.2.2. O estado que o protocolo não tem

Como o HTTP não guarda estado, a sessão é reconstruída a cada requisição por um **cookie**, um cabeçalho **Authorization** ou um token. Isso desloca o problema: quem obtiver esse identificador é o usuário, sem precisar da senha. Daí a importância dos atributos do cookie e da rotação do identificador de sessão após a autenticação — assunto que o Volume 7 aprofunda.

Três abusos merecem menção desde já, porque aparecem em qualquer análise de tráfego:

Métodos indevidamente habilitados. TRACE viabiliza roubo de cabeçalhos por *cross-site tracing*; PUT e DELETE abertos em servidor mal configurado permitem escrita direta. A regra é permitir apenas o que a aplicação usa.

Redirecionamento aberto. Um parâmetro que recebe URL de destino sem validação transforma o domínio confiável em trampolim de phishing: o usuário vê o domínio do banco no link e termina no site do atacante.

Vazamento por resposta. Cabeçalhos Server e X-Powered-By com versão exata, páginas de erro com pilha de execução, comentários em HTML com caminho interno. Nada disso é vulnerabilidade por si; tudo isso encurta o trabalho de reconhecimento.

10.2.3. HTTPS não é opcional, e “só interno” não é argumento

O Capítulo 8 mostrou como um intermediário na rede local intercepta tráfego. Toda aplicação HTTP interna — o painel do switch, o sistema de chamados, a impressora — entrega credenciais a quem esteja nessa posição. Certificados internos assinados por uma CA própria, ou ACME interno automatizado, resolvem o problema sem custo de certificado público. O Volume 3 trata do TLS em detalhe; a decisão de política, porém, é deste nível: **não existe rede confiável o bastante para justificar HTTP em claro.**

10.3. DNS: a lista telefônica que qualquer um pode adulterar

Toda conexão começa por uma consulta DNS. Isso faz do DNS, simultaneamente, o alvo mais estratégico e o sensor mais rico de uma rede.

A Figura 10.1 mostra o caminho de uma consulta e os quatro pontos onde ele pode ser subvertido.

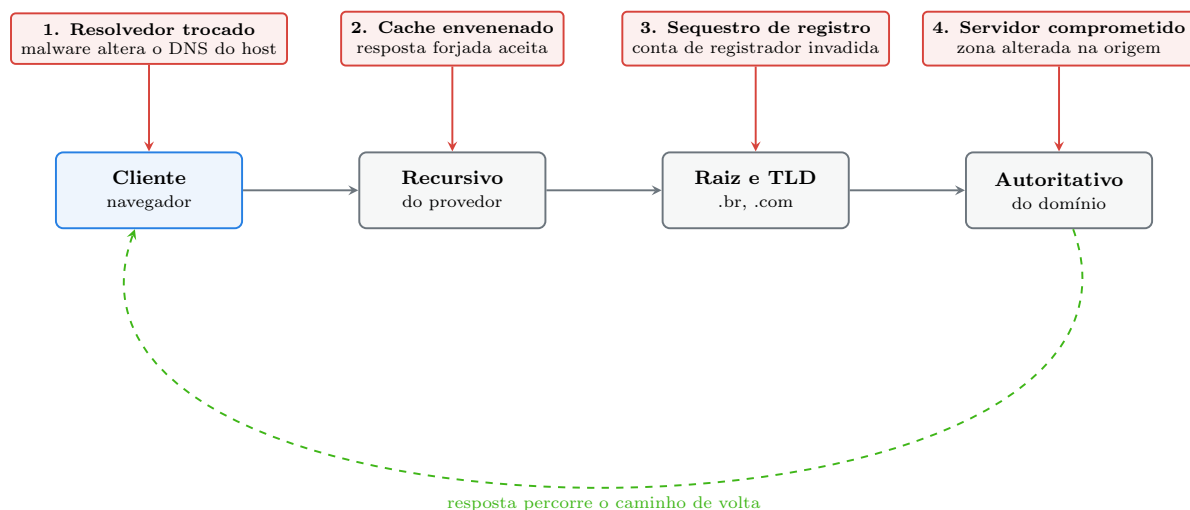


Figura 10.1.: Os quatro pontos de subversão de uma resolução DNS. Só o segundo é mitigado por DNSSEC.

10.3.1. Envenenamento de cache e o que o DNSSEC resolve

O DNS usa UDP por padrão e casa perguntas com respostas por um identificador de 16 bits mais a porta de origem. Um atacante que consiga entregar uma resposta forjada com esses valores corretos antes da resposta legítima insere no cache do recursivo um registro falso — que será servido a todos os clientes daquele recursivo até expirar o TTL. A técnica descrita por Dan Kaminsky em 2008 tornou isso viável a ponto de forçar a adoção emergencial de aleatorização de porta de origem em toda a internet.

O **DNSSEC** resolve o problema na raiz: assina criptograficamente os registros, encadeando a confiança da raiz até o domínio. O que ele **não** faz merece ênfase igual — DNSSEC garante integridade e autenticidade, **não confidencialidade**. A consulta continua visível. Quem observa o tráfego sabe exatamente que sites foram acessados, mesmo com todo o conteúdo em HTTPS.

Confidencialidade veio depois, com **DoT** (DNS sobre TLS, porta 853) e **DoH** (DNS sobre HTTPS, porta 443) (Hoffman; McManus, 2018). E aqui há uma tensão real para quem opera rede: DoH esconde as consultas do próprio operador, quebrando filtros de conteúdo corporativos, bloqueio de domínios maliciosos e visibilidade para resposta a incidentes. Navegadores que habilitam DoH por padrão para um resolver externo tiram do administrador um dos seus melhores sensores. A resposta madura não é bloquear DoH e sim **oferecer DoH interno**: o mesmo benefício de privacidade contra terceiros, com o controle e a visibilidade permanecendo na organização.

10.3.2. O DNS como sensor e como canal de fuga

Do lado defensivo, o registro de consultas DNS é o dado mais custo-eficiente de uma rede: uma única fonte revela acesso a domínios de comando e controle, a sites de phishing recém-registrados, a serviços não autorizados, e denuncia máquinas infectadas antes de qualquer outro sinal.

Do lado ofensivo, o mesmo protocolo é o canal de exfiltração mais confiável que existe. Como quase toda rede permite DNS de saída, dados podem ser codificados em nomes de subdomínio — `ZGFkb3NleGZpbHRyYWRvcw.atacante.exemplo` — e recuperados no servidor autoritativo do

atacante. **Tunelamento DNS** carrega sessões interativas inteiras assim, lentamente e sem chamar atenção. Os indicadores são reconhecíveis: volume anormal de consultas a um único domínio, nomes muito longos, alta proporção de respostas NXDOMAIN, uso intenso de tipos TXT e NULL, entropia elevada nos rótulos.

10.4. SMTP: o protocolo que confia no remetente

O SMTP entrega mensagens numa sequência de comandos textuais. O ponto que explica o caso de abertura deste capítulo é a existência de **dois remetentes** numa mesma mensagem, e o fato de que só o segundo aparece na tela.

O **envelope** (MAIL FROM) é usado no transporte e é o que o SPF verifica. O **cabeçalho** (From:) é o que o cliente de e-mail exhibe. São campos independentes, e nada no protocolo obriga que coincidam. Escrever qualquer coisa no **From:** é comportamento previsto — é assim que listas de discussão e sistemas de notificação funcionam.

Sobre essa base foram construídos três mecanismos que, **juntos**, fecham a falsificação:

SPF publica em DNS quais servidores podem enviar em nome do domínio (Kitterman, 2014). Verifica o envelope, não o **From:** exibido, e quebra ao encaminhar mensagens — daí não bastar sozinho.

DKIM assina criptograficamente cabeçalhos e corpo com uma chave cujo público está no DNS (Crocker; Hansen; Kucherawy, 2011). Sobrevive ao encaminhamento e prova integridade, mas não diz o que fazer quando falha.

DMARC amarra os dois ao **From:** exibido, exigindo **alinhamento**, e publica a política a aplicar em caso de falha — **none** (só relatar), **quarantine** ou **reject** — além de habilitar relatórios agregados (Kucherawy; Zwicky, 2015).

A Figura 10.2 mostra a sequência de verificação no servidor receptor.

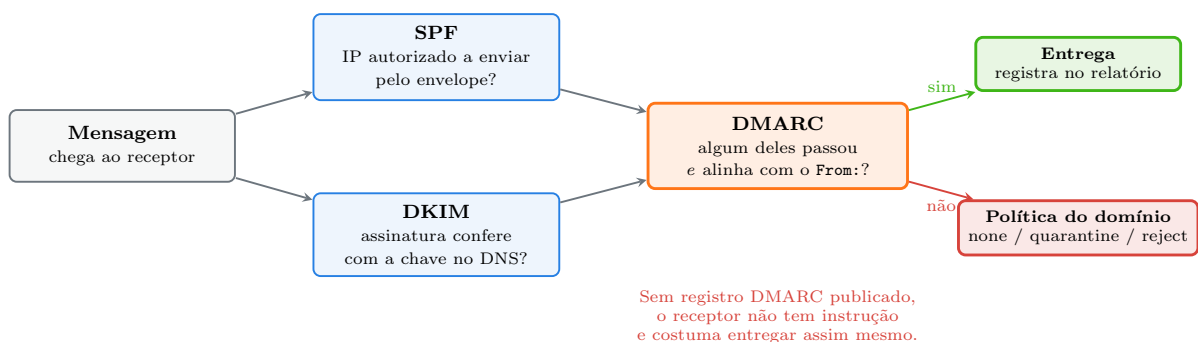


Figura 10.2.: Verificação de autenticidade de e-mail. O alinhamento com o **From:** exibido é a peça que só o DMARC fornece.

A publicação em p=none é o erro mais comum. É o ponto de partida correto — permite observar os relatórios sem quebrar fluxos legítimos —, mas muitas organizações param ali e acreditam estar protegidas. Enquanto a política não avançar para **quarantine** e depois **reject**, o receptor não recebe instrução para rejeitar nada. A publicação de **p=none** protege exatamente zero mensagens.

Vale ainda distinguir dois ataques que a defesa trata de forma diferente. A **falsificação direta do domínio** é o que SPF, DKIM e DMARC bloqueiam. O **domínio parecido** — `fornecedor-exemplo.com.br` no lugar de `fornecedor.exemplo.br`, ou caracteres visualmente similares — passa por todas as verificações, porque é um domínio legítimo do atacante, devidamente autenticado. Contra esse, os controles são outros: monitoramento de registros semelhantes, marcação visual de remetente externo no cliente de e-mail, e processo de negócio que exija confirmação por canal independente para mudança de dados bancários. O caso de abertura teria sido evitado por qualquer um dos três — e o Volume 10 trata do assunto em profundidade.

10.5. SNMP: a porta dos fundos do parque de equipamentos

O SNMP monitora e configura equipamentos de rede. É a fonte dos gráficos de banda e, quando mal configurado, o mapa completo da infraestrutura entregue a quem perguntar.

O problema está nas versões. **SNMPv1 e v2c autenticam por uma *community string*, que trafega em claro** e funciona como senha compartilhada. Duas delas vêm de fábrica em praticamente todo equipamento: `public` para leitura e `private` para escrita. Com leitura, o atacante obtém inventário de interfaces, tabela ARP, tabela de roteamento, versão de firmware, nomes de host, tempo de atividade — reconhecimento interno completo sem tocar em nenhum sistema. Com escrita, reconfigura o equipamento.

O SNMP é ainda um dos vetores clássicos de **amplificação** descritos no Capítulo 8: uma consulta `GetBulk` de poucos bytes gera resposta muitas vezes maior, e o serviço roda sobre UDP.

As três medidas, em ordem de prioridade: **migrar para SNMPv3** com `authPriv` (autenticação SHA e cifragem AES); onde o equipamento legado não suportar v3, **restringir por lista de acesso** ao endereço do coletor e usar apenas leitura, com *community* longa e única por equipamento; e **nunca expor o 161/UDP à internet**, o que ainda ocorre com frequência em roteadores de borda de provedores pequenos.

10.6. O inventário de protocolos: o exercício que ninguém faz

Fecha o capítulo o exercício mais revelador e menos praticado: listar o que efetivamente escuta na rede. Ele produz invariavelmente três descobertas — um serviço que ninguém sabia que estava ligado, um protocolo em claro que deveria ter sido desativado anos atrás, e um sistema exposto à internet por engano.

A ordem de tratamento que funciona é por consequência:

1. **Administração em claro exposta** — Telnet, HTTP de painel, SNMP v2c com escrita. Sessão administrativa capturada é comprometimento imediato.
2. **Autenticação em claro** — FTP, POP3, IMAP, LDAP sem TLS. Credenciais capturadas costumam ser reutilizadas em outros sistemas.
3. **Serviços amplificadores abertos** — DNS recursivo, NTP, SNMP, memcached. Aqui a vítima é um terceiro, mas a responsabilidade e a reputação do bloco são da organização.
4. **Serviços legítimos sem TLS** — o resto.

10.7. Brasil em Foco

A fraude do caso de abertura tem nome e tipo penal. O golpe do boleto e a fraude do e-mail corporativo (*BEC*) configuram estelionato, e desde a Lei 14.155/2021 o art. 171, §2º-A do Código Penal prevê pena de reclusão de 4 a 8 anos quando praticado por meio eletrônico (Brasil, 2021). A pena aumenta quando a vítima é idosa. Para a empresa que pagou, resta a via civil — e a discussão sobre quem tinha o dever de cuidado costuma envolver justamente a ausência de SPF, DKIM e DMARC no domínio do fornecedor.

Domínio .br e o registro.br. Registro de domínio e publicação de SPF, DKIM e DMARC são feitos no painel do registro.br, e a proteção da conta de registrador é crítica: é o ponto 3 da Figura 10.1, e o comprometimento dessa conta permite redirecionar e-mail e site do domínio inteiro. Segundo fator obrigatório nessa conta e nas contas de DNS não é recomendação, é requisito básico.

Registro de consultas DNS é dado pessoal. O histórico de resolução de um assinante revela hábitos, saúde, orientação política e religiosa — dado potencialmente sensível no sentido do art. 5º, II da LGPD (Brasil, 2018). Um provedor que registra consultas precisa de base legal, finalidade declarada, prazo de retenção e controle de acesso. Usar esse registro para fins comerciais, como perfilamento publicitário, exige base legal própria e esbarra também na neutralidade de rede do art. 9º do Marco Civil (Brasil, 2014).

Bloqueio de porta 25 e o combate ao spam. A gestão da porta 25, coordenada no Brasil pelo CGI.br e pelo CERT.br desde 2012, é caso de política pública técnica bem-sucedida: o encaminhamento de mensagens passou a usar a porta 587 com autenticação, e a saída direta pela 25 a partir de redes de acesso residencial foi bloqueada. Provedores que não aplicam a medida veem seus blocos entrarem em listas de bloqueio internacionais, com prejuízo para todos os assinantes — inclusive os que não têm relação alguma com o problema.

Anatel e a segurança do equipamento do assinante. A Resolução 740/2020 alcança os produtos de telecomunicações fornecidos ao cliente (Agência Nacional de Telecomunicações, 2020). Roteadores entregues com senha padrão, painel de administração acessível pela WAN ou SNMP aberto são, além de risco técnico, exposição regulatória do provedor — e foram vetor de campanhas em larga escala de alteração de DNS de assinantes no Brasil, exatamente o ponto 1 da Figura 10.1.

10.8. No Campo

1. Publicar SPF, DKIM e DMARC — e avançar a política. Vinte minutos que fecham a falsificação direta do domínio:

```
; SPF - encerrar com -all, nunca com ?all
exemplo.com.br.      IN TXT "v=spf1 include:_spf.provedor.exemplo.br -all"

; DMARC - começar em none para observar, avançar em semanas
_dmarc.exemplo.com.br. IN TXT "v=DMARC1; p=none;
  ⇨ rua=mailto:dmARC@exemplo.com.br; pct=100"
; depois: p=quarantine; pct=25 → pct=100 → p=reject
```

```
# Conferir o que está publicado hoje
dig +short TXT exemplo.com.br | grep spf1
dig +short TXT _dmarc.exemplo.com.br
dig +short TXT selecao._domainkey.exemplo.com.br
```

2. Levantar o inventário de portas abertas na borda. O exercício que sempre encontra surpresa. Executado a partir de fora, contra o próprio bloco e **com autorização escrita**, conforme o Capítulo 7:

```
# Serviços administrativos e amplificadores expostos no bloco
nmap -sS -Pn -p 21,22,23,25,53,80,110,143,161,389,443,445,3389,8080
↪ 198.51.100.0/24 -oA borda
nmap -sU -Pn -p 53,123,161,1900,11211 198.51.100.0/24 -oA borda-udp
```

3. Checar os cabeçalhos de segurança das aplicações web internas. Vale para o painel do switch tanto quanto para o sistema de chamados:

```
curl -sI https://app.exemplo.com.br | \
  grep -iE
↪ 'strict-transport|content-security|x-content-type|x-frame|referrer-policy|set-cookie'
```

4. Migrar SNMP para v3 e fechar o que sobrar. Prioridade máxima onde houver escrita habilitada:

```
! Substitui community por usuário com autenticação e cifragem
snmp-server group MONITORA v3 priv
snmp-server user coletor MONITORA v3 auth sha <senha-auth> priv aes 128
↪ <senha-priv>
no snmp-server community public
no snmp-server community private
snmp-server host 10.30.20.9 version 3 priv coletor
```

5. Registrar as consultas DNS e vigiar os indicadores de tunelamento. É a fonte de detecção mais barata que existe:

```
# No resolvidor: domínios com muitas consultas e rótulos longos
awk '{print $NF}' /var/log/dns/queries.log \
  | awk -F. '{print $(NF-1)}' "$NF, length($1)}' \
  | sort | uniq -c | sort -rn | head -20

# Proporção de NXDOMAIN por cliente - pico indica DGA ou túnel
grep NXDOMAIN /var/log/dns/queries.log | awk '{print $3}' | sort | uniq -c |
↪ sort -rn | head
```

6. Oferecer DoH e DoT internos em vez de bloquear. Preserva a privacidade do usuário contra terceiros e mantém a visibilidade e o filtro na organização. Na prática: publicar o resolvidor interno como DoH, distribuí-lo por política nos navegadores e bloquear na borda os resolvidores externos conhecidos.

7. Desativar os protocolos em claro com prazo, não com boa intenção. Um inventário com data de desligamento por sistema, comunicado aos donos, e a porta efetivamente fechada no dia marcado. Sem prazo, Telnet sobrevive a três trocas de gerência.

8. Exigir confirmação por canal independente para mudança de dados bancários. É o controle de processo que teria evitado os R\$ 180 mil do caso de abertura, e não custa nada: qualquer alteração de conta de fornecedor é confirmada por telefone, num número já cadastrado — nunca no número que veio no e-mail.

10.9. Resumo

- Os protocolos fundamentais da internet são **textuais, não autenticam origem e não verificam integridade**. TLS resolve confidencialidade e integridade em trânsito, mas cada protocolo tem um problema estrutural próprio que o TLS não alcança.
- No **HTTP**, os cabeçalhos de resposta (HSTS, CSP, nosniff, X-Frame-Options, atributos de cookie) são controles baratos e sistematicamente ausentes em sistemas internos. Como o protocolo não tem estado, quem obtém o identificador de sessão é o usuário.
- No **DNS**, há quatro pontos de subversão — resolvente do host, cache do recursivo, conta no registrador e servidor autoritativo — e o DNSSEC só cobre o segundo. **DNSSEC dá integridade, não confidencialidade**; DoT e DoH dão confidencialidade e, se forem externos, custam ao operador o seu melhor sensor.
- O DNS é ao mesmo tempo a **fonte de detecção mais custo-eficiente** e o canal de **exfiltração** mais confiável. Os indicadores de tunelamento são reconhecíveis: rótulos longos, alta entropia, excesso de NXDOMAIN, uso intenso de TXT.
- No **SMTP** existem dois remetentes, e só o exibido chega ao usuário. SPF verifica o envelope, DKIM assina o conteúdo, e apenas o **DMARC** amarra os dois ao **From**: exibido e publica a política. **p=none não protege nenhuma mensagem** — é ponto de partida, não de chegada. Contra domínio parecido, a defesa é processo, não protocolo.
- **SNMP v1 e v2c entregam o mapa da infraestrutura** a quem souber a *community* — que é **public** na fábrica, trafega em claro e ainda serve de amplificador. SNMPv3 com **authPriv**, lista de acesso e nunca exposto à internet.
- O **inventário de protocolos** é o exercício mais revelador e menos praticado. Trata-se por consequência: administração em claro, autenticação em claro, amplificadores abertos, e só então o resto.

11. Fundamentos de sistemas operacionais: processos, permissões e memória

Um analista abre um chamado de rotina: o servidor de arquivos está com uso de CPU acima do normal desde a madrugada. O `top` mostra um processo chamado `httpd-worker` consumindo 90% de um núcleo. Parece o servidor web. Não é: o servidor web da casa é o `nginx`, e ninguém instalou Apache naquela máquina.

O processo roda como o usuário `www-data`, o mesmo do `nginx`. Foi iniciado às 3h14, tem como pai um processo que já morreu, e seu executável foi apagado do disco — o arquivo continua acessível apenas através de `/proc/<pid>/exe`, marcado como `(deleted)`. O diretório de trabalho é `/tmp`. Ele mantém uma conexão TCP estabelecida com um endereço na Holanda, na porta 443.

Cada um desses detalhes é uma propriedade do sistema operacional: identidade de processo, herança de pai, descritor de arquivo que sobrevive ao arquivo, contexto de usuário, espaço de memória. Quem não conhece esses conceitos vê um processo estranho. Quem conhece lê, em dois minutos, a história inteira de um comprometimento — inclusive como ele começou e o que ele consegue alcançar a seguir.

Os três capítulos anteriores trataram da rede. Os quatro seguintes tratam da máquina. Este é o alicerce conceitual comum a Linux e Windows; o Capítulo 12 e o Capítulo 13 descem ao específico de cada um.

11.1. O anel de proteção: a fronteira que sustenta tudo

Processadores modernos executam código em pelo menos dois níveis de privilégio. Em **modo núcleo** (anel 0), o código acessa qualquer endereço de memória, fala diretamente com o hardware e altera as tabelas que controlam tudo. Em **modo usuário** (anel 3), cada programa vê apenas a memória que o núcleo lhe concedeu e não toca em hardware.

Essa separação é a fundação de toda a segurança de um sistema operacional. Dela decorrem três consequências:

A única passagem entre os modos é a chamada de sistema. Para ler um arquivo, abrir um socket ou criar um processo, o programa precisa pedir ao núcleo. Esse pedido é o ponto onde toda decisão de autorização é tomada — e, não por acaso, é onde antivírus, EDR e ferramentas de auditoria se instalam para observar. O Volume 6 trata de como isso é feito na prática.

Um defeito no núcleo derruba toda a proteção. Uma falha em código de anel 0 — driver, módulo, subsistema de arquivos — dá controle total. É por isso que uma vulnerabilidade de escalonamento local no núcleo é tratada com a mesma gravidade de uma remota: ela transforma qualquer acesso mínimo em domínio da máquina.

O núcleo é a superfície de ataque menos visível. Um *rootkit* que consiga executar em anel 0 mente para todas as ferramentas que rodam em anel 3, inclusive para o antivírus. Daí a importância de mecanismos que verificam a integridade do próprio núcleo — inicialização verificada, assinatura obrigatória de módulos, integridade protegida por virtualização.

11.2. Processos: identidade, herança e contexto

Um **processo** é um programa em execução com seu próprio espaço de endereçamento. Do ponto de vista de segurança, o que importa não é o código que ele executa, e sim o **contexto** em que executa: quem ele é, o que herdou e o que pode alcançar.

11.2.1. A árvore genealógica

Todo processo (exceto o primeiro) é criado por outro, e a relação pai-filho é a estrutura mais reveladora numa investigação. A pergunta que resolve a maioria dos casos é sempre a mesma: *quem chamou isso?*

Um interpretador de comandos cujo pai é um editor de planilhas é anomalia grave — significa que uma macro executou. Um processo de rede cujo pai é um serviço de impressão idem. Regras de detecção comerciais e abertas são, em boa medida, catálogos de pares pai-filho que não deveriam existir; o Volume 12 trata de escrevê-las.

Quando o pai termina antes do filho, o filho é adotado pelo processo inicial do sistema. Isso apaga o rastro da origem — exatamente o que aconteceu no caso de abertura — e é técnica comum tanto de daemons legítimos quanto de código malicioso.

11.2.2. Identidade e o bit que muda tudo

Cada processo carrega identificadores de usuário e de grupo que determinam o que ele pode acessar. O modelo Unix distingue o identificador **real** (quem iniciou) do **efetivo** (o usado nas verificações de permissão). A diferença entre os dois é a base do mecanismo de elevação — e da sua exploração.

O bit **SUID** faz um executável rodar com a identidade do seu dono, não de quem o executou. Existe por necessidade legítima: `passwd` precisa escrever em `/etc/shadow`, que só o root pode alterar. E é, simultaneamente, uma das superfícies de escalonamento local mais exploradas que existem. Um binário SUID de root com qualquer falha — estouro de buffer, injeção de argumento, confiança numa variável de ambiente, chamada a um programa externo por caminho relativo — entrega o sistema.

A regra operacional é direta: **o inventário de binários SUID de uma máquina deve ser conhecido, pequeno e estável**. Um SUID novo que aparece sem explicação é indicador de comprometimento de alta confiança, e é o primeiro lugar onde se olha depois de um acesso suspeito.

O Windows resolve o mesmo problema de forma diferente, com *tokens* de acesso que carregam a identidade, os grupos e uma lista de **privilégios** nomeados. Alguns desses privilégios equivalem, na prática, a ser administrador: `SeDebugPrivilege` permite abrir qualquer processo e ler sua memória; `SeImpersonatePrivilege` permite assumir a identidade de outro usuário; `SeBackupPrivilege` permite ler qualquer arquivo ignorando as permissões. Conceder esses privilégios a uma conta de serviço é conceder administração por caminho indireto — e é assim que boa parte das escaladas em ambiente Windows acontece, como o Capítulo 13 detalha.

11.3. Permissões: dois modelos para o mesmo problema

O controle de acesso a arquivos resolve a mesma questão em Unix e Windows com filosofias distintas. A Figura 11.1 compara as duas.

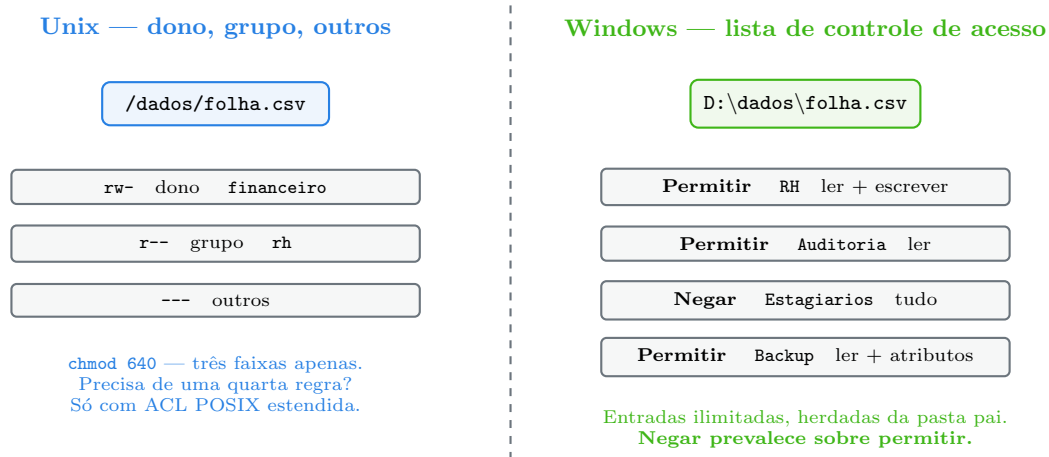


Figura 11.1.: Os dois modelos de permissão. O Unix é simples e enxuto; o do Windows é expressivo e difícil de auditar.

O modelo Unix cabe em nove bits e é fácil de auditar em massa; sua limitação é não expressar “este grupo lê, aquele escreve, aquele outro não vê” sem recorrer a ACLs estendidas. O modelo Windows expressa qualquer política, ao custo de uma complexidade que, na prática, produz permissões efetivas que ninguém consegue enunciar — especialmente com herança em árvores de pastas profundas e grupos aninhados.

Dois detalhes causam a maioria dos erros reais:

A permissão de execução em diretório significa “atravessar”, não “executar”. Sem ela não se acessa nada abaixo, mesmo com leitura no arquivo. Com ela e sem leitura no diretório, acessa-se um arquivo cujo nome já se conhece, mas não se lista o conteúdo — propriedade útil para diretórios que devem ser alcançáveis sem serem enumeráveis.

A permissão do diretório governa a criação e a exclusão, não a do arquivo. Quem tem escrita num diretório pode apagar arquivos dele mesmo sem qualquer permissão sobre esses arquivos. É o motivo de existir o *sticky bit* em `/tmp`: sem ele, qualquer usuário apagaria os arquivos temporários de todos os outros.

E há o erro clássico que aparece em toda auditoria: `chmod 777`. É a resposta rápida para “não estou conseguindo acessar” e significa que qualquer usuário do sistema — inclusive o processo web comprometido do caso de abertura — pode ler, alterar e apagar aquele arquivo. Em diretório com script executado por serviço, é escalonamento de privilégio servido em bandeja.

11.4. Memória: onde a exploração acontece

O núcleo dá a cada processo um espaço de endereçamento virtual próprio, traduzido para memória física por tabelas que só o anel 0 altera. Isso garante isolamento: um processo não enxerga a memória de outro. A Figura 11.2 mostra a organização típica desse espaço.

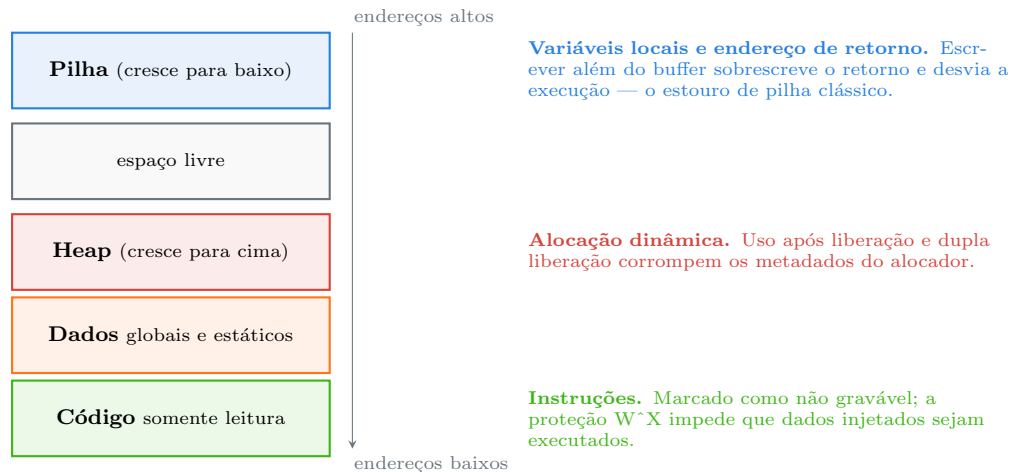


Figura 11.2.: Espaço de endereçamento de um processo e as regiões onde a corrupção de memória ocorre.

11.4.1. Por que a corrupção de memória vira execução de código

Linguagens como C e C++ não verificam limites de vetor. Se um programa reserva 64 bytes para um nome e copia 200, os 136 excedentes sobrescrevem o que estiver adiante — que, na pilha, inclui o **endereço de retorno** da função. Controlando esse endereço, o atacante controla para onde a execução desvia quando a função termina.

Este manual não ensina a construir o desvio — o Capítulo 7 fixou essa fronteira, e o Volume 9 trata do assunto no nível conceitual necessário para entender e detectar. O que interessa aqui é a cadeia de mitigações que a indústria construiu, porque **entender por que cada uma existe é entender por que todas precisam estar ligadas**:

Tabela 11.1.: Mitigações de corrupção de memória e seus limites.

Mitigação	O que faz	O que a contorna
Canário de pilha	Valor sentinela antes do endereço de retorno; alterado, aborta	Vazamento de memória que revela o canário
DEP / NX (W [^] X)	Páginas graváveis não são executáveis	Reúso de código já presente (ROP)
ASLR	Randomiza posições de pilha, heap e bibliotecas	Vazamento de endereço; entropia baixa em 32 bits
PIE	Randomiza também o executável	Idem
RELRO	Torna a tabela de relocação somente leitura	—
CFI	Restringe destinos de desvio indireto	Cadeias dentro dos destinos válidos
Linguagem com memória segura	Elimina a classe inteira de falhas	Blocos unsafe , dependências em C

Duas conclusões práticas. A primeira: **nenhuma mitigação isolada resolve**; cada uma eleva o custo, e a defesa vem da combinação. A segunda: essas proteções precisam ser **ativadas na**

compilação, e software antigo ou binário proprietário mal empacotado frequentemente não as tem. Verificar isso num binário é tarefa de dois comandos, e está no fim do capítulo.

A tendência de fundo é a migração para linguagens com memória segura. Levantamentos consistentes de grandes bases de código — Microsoft, Google, projetos do núcleo Linux — apontam que aproximadamente **dois terços das vulnerabilidades graves são falhas de segurança de memória**, e recomendações oficiais recentes empurram nessa direção para software novo. Não é solução para o que já existe, mas muda a conta para o que vai ser escrito.

11.4.2. Segredos vivem em memória

Consequência menos óbvia e muito explorada: **tudo que um processo usa está em claro na sua memória**, por melhor que seja a criptografia em disco e em trânsito. Senhas digitadas, chaves privadas carregadas, tokens de sessão, o conteúdo descriptografado de um banco — tudo passa por ali.

Por isso o privilégio de depuração é tão sensível: quem pode ler a memória de outro processo lê seus segredos sem quebrar nenhuma cifra. É o mecanismo por trás das ferramentas que extraem credenciais do processo de autenticação do Windows, e por trás da extração de chaves de um processo de servidor web. As defesas são de contenção — restringir quem tem o privilégio, isolar processos sensíveis, usar módulos de hardware que nunca expõem a chave, e limpar buffers sensíveis após o uso.

11.5. Isolamento: quando o processo não basta

A separação entre processos é boa, mas grosseira demais para muitos casos. Um servidor web comprometido tem, por padrão, acesso a todo o sistema de arquivos legível pelo seu usuário. Daí a construção de camadas adicionais:

Redução de privilégio. Cada serviço com seu próprio usuário sem interpretador de comandos e sem diretório pessoal. Barato, eficaz e ainda assim frequentemente ignorado — serviços rodando como root “porque era mais fácil” são achado comum em qualquer auditoria.

Capacidades. O Linux fatiou os poderes de root em dezenas de capacidades independentes. Um servidor que precisa apenas abrir a porta 80 recebe `CAP_NET_BIND_SERVICE` em vez de ser root inteiro. Vale conhecer as que equivalem a root: `CAP_SYS_ADMIN` (chamada de “a nova root” com razão), `CAP_SYS_PTRACE`, `CAP_SYS_MODULE`, `CAP_DAC_OVERRIDE`.

Espaços de nomes e cgroups. A base dos contêineres: o processo vê apenas um subconjunto de PIDs, montagens, interfaces de rede e usuários, com limites de recurso. Isolamento útil, mas **compartilhando o mesmo núcleo** — uma falha de anel 0 atravessa a fronteira. O Volume 8 trata das implicações.

Filtragem de chamadas de sistema. O `seccomp` restringe quais chamadas um processo pode fazer. Se o serviço nunca precisa de `ptrace` nem de `mount`, negá-las reduz o que uma exploração bem-sucedida consegue fazer em seguida.

Controle de acesso obrigatório. SELinux e AppArmor impõem política que o dono do arquivo não pode relaxar — o modelo MAC que o Volume 4 detalha. Curva de aprendizado real, e o único mecanismo que contém um processo root comprometido.

Virtualização. Fronteira mais forte, com núcleo separado por máquina. É o assunto do Capítulo 14.

O princípio que organiza a lista é o do **menor privilégio**, formulado por Saltzer e Schroeder em 1975 (Saltzer; Schroeder, 1975) e citado no Capítulo 1: cada componente recebe exatamente o necessário para sua função, e nada além. A lista acima é a caixa de ferramentas para aplicá-lo de fato.

11.6. Lendo o que a máquina conta

Voltando ao caso de abertura: o que permitiu ler a situação em dois minutos foi saber onde o sistema operacional guarda cada resposta.

Tabela 11.2.: As sete perguntas de triagem de um processo suspeito.

Pergunta	Onde está	O que revela
Quem iniciou este processo?	PID pai, linha de comando do pai	Origem do comprometimento
Sob que identidade ele roda?	UID efetivo, grupos, token	O que ele alcança agora
Qual binário ele executa?	<code>/proc/<pid>/exe</code> , imagem do processo	Executável apagado é sinal forte
Que arquivos ele mantém abertos?	<code>/proc/<pid>/fd</code> , handles	Dados sendo lidos ou escritos
Com quem ele fala?	Sockets do processo	Comando e controle
O que está na memória dele?	Mapas de memória, regiões graváveis e executáveis	Código injetado
Ele sobrevive a um reinício?	Unidades de serviço, tarefas agendadas, chaves de execução	Persistência

Um detalhe importante para a resposta a incidentes, e que o Volume 13 aprofunda: **desligar a máquina destrói a maior parte dessas respostas**. Memória é volátil; conexões desaparecem; o executável apagado, que só existe como descritor aberto, some para sempre. A ordem correta é preservar antes de conter.

11.7. Brasil em Foco

Registro de comando com privilégio e o não repúdio. O Capítulo 3 tratou de responsabilização; aqui ela vira configuração. Contas administrativas compartilhadas — o **root** que três pessoas usam, o **Administrador** cuja senha está num bilhete — destroem a possibilidade de atribuir uma ação a uma pessoa. Isso enfraquece qualquer apuração interna, prejudica a defesa da organização num processo trabalhista e compromete a resposta a requisições judiciais. Acesso nominal com elevação registrada é o requisito mínimo.

Dados pessoais em memória e a notificação de incidente. Se um processo comprometido tinha em memória dados pessoais de clientes, houve tratamento irregular sob a LGPD (Brasil, 2018). A Resolução CD/ANPD nº 15/2024 estabelece prazo e conteúdo da comunicação de incidente de segurança à autoridade e aos titulares (Autoridade Nacional de Proteção de Dados, 2024), e a avaliação sobre “que dados foram alcançados” depende diretamente das evidências da Tabela 11.2 — coletadas **antes** de desligar a máquina. Quem desliga primeiro perde a capacidade de dizer o que aconteceu, e a resposta “não sabemos” não é aceitável perante a ANPD.

Preservação de evidência é obrigação, não zelo. O art. 13 do Marco Civil trata da guarda de registros de conexão (Brasil, 2014); o Código de Processo Penal e a Convenção de Budapeste, promulgada pelo Decreto nº 11.491/2023 (Brasil, 2023), tratam da preservação expedita de dados. Uma organização que recebe pedido de preservação e já reinstalou o servidor comprometido tem um problema jurídico além do técnico. Procedimento escrito de resposta que inclua “coletar memória e estado antes de qualquer contenção destrutiva” resolve os dois.

Software sem manutenção é risco jurídico e não apenas técnico. Sistema que não recebe correção não tem as mitigações da Tabela 11.1 e acumula falhas conhecidas. Sob a LGPD, os arts. 46 e 47 exigem medidas de segurança aptas a proteger os dados; a ANPD já sinalizou, em guia orientativo dirigido a agentes de pequeno porte (Autoridade Nacional de Proteção de Dados, 2023), que manter software atualizado é medida básica esperada. Rodar sistema fora de suporte com dados pessoais é decisão que precisa estar documentada, com controles compensatórios, ou é exposição pura.

11.8. No Campo

1. Fazer a triagem de processo suspeito na ordem certa. As sete perguntas da Tabela 11.2, sem desligar nada:

```
PID=1234
ls -l /proc/$PID/exe          # (deleted) é sinal forte
ls -l /proc/$PID/cwd
tr '\0' ' ' < /proc/$PID/cmdline; echo
grep -E 'Uid|Gid|PPid|Threads' /proc/$PID/status
ls -l /proc/$PID/fd | head -40
ss -tanp | grep "pid=$PID"
grep -E 'rwx' /proc/$PID/maps  # região gravável e executável: suspeita de
↪ injeção
```

```
# Windows: pai, linha de comando e dono do processo
Get-CimInstance Win32_Process -Filter "ProcessId=1234" |
  Select-Object ProcessId, ParentProcessId, CommandLine, ExecutablePath
Get-NetTCPConnection -OwningProcess 1234
```

2. Manter o inventário de binários SUID e comparar periodicamente. Um SUID novo é indicador de alta confiança:

```
# Linha de base, guardada fora da máquina
find / -xdev \( -perm -4000 -o -perm -2000 \) -type f -printf '%m %u %g %p\n' \
  2>/dev/null | sort > /var/lib/baseline/suid.txt

# Comparação na rotina
diff <(find / -xdev \( -perm -4000 -o -perm -2000 \) -type f -printf '%m %u %g
↪ %p\n' \
  2>/dev/null | sort) /var/lib/baseline/suid.txt
```

3. Caçar permissões frouxas antes que alguém as encontre. 777 em diretório de script executado por serviço é escalonamento pronto:

11. Fundamentos de sistemas operacionais: processos, permissões e memória

```
# Arquivos e diretórios graváveis por todos, fora das áreas temporárias
find / -xdev -perm -0002 ! -type l ! -path '/proc/*' ! -path '/tmp/*' \
    ! -path '/var/tmp/*' -printf '%m %p\n' 2>/dev/null | sort | head -40

# Diretórios de escrita geral sem sticky bit - qualquer um apaga arquivo alheio
find / -xdev -type d -perm -0002 ! -perm -1000 2>/dev/null
```

4. Verificar se os binários em produção têm as mitigações ligadas. Software interno compilado há anos costuma não ter:

```
# NX, PIE, RELRO, canário - o ideal é tudo habilitado
readelf -lW /usr/local/bin/app | grep -E 'GNU_STACK|GNU_RELRO'
readelf -hW /usr/local/bin/app | grep Type          # DYN = PIE, EXEC = sem PIE
nm -D /usr/local/bin/app 2>/dev/null | grep -c stack_chk
```

5. Reduzir o privilégio dos serviços com o que o systemd já oferece. Ganho grande, esforço pequeno, sem tocar na aplicação:

```
[Service]
User=app
Group=app
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/var/lib/app
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
SystemCallFilter=@system-service
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
```

6. Restringir quem pode ler a memória de outro processo. Contenção direta contra extração de credenciais:

```
# Linux: só o processo pai pode depurar um filho
sysctl -w kernel.yama.ptrace_scope=1
echo 'kernel.yama.ptrace_scope = 1' > /etc/sysctl.d/10-pttrace.conf
```

```
# Windows: quem tem privilégios equivalentes a administração
whoami /priv | Select-String 'SeDebug|SeImpersonate|SeBackup|SeTakeOwnership'
```

7. Auditar contas de serviço com privilégios perigosos. No Windows, SeDebugPrivilege, SeImpersonatePrivilege e SeBackupPrivilege concedidos a conta de serviço são administração por via indireta. A revisão é anual, no mínimo, e entra na mesma rotina que revisa membros de grupos administrativos.

8. Escrever o procedimento de preservação antes do incidente. Uma página: coletar memória e estado (processos, conexões, descritores abertos) **antes** de conter, onde guardar, quem autoriza, como registrar a cadeia de custódia. Sem isso, a primeira reação do plantão será reiniciar o servidor — e com ele, a evidência.

11.9. Resumo

- A separação entre **modo núcleo e modo usuário** é a fundação de toda segurança de sistema operacional. A chamada de sistema é a única passagem entre eles e o ponto onde a autorização é decidida; uma falha em anel 0 anula todas as demais proteções.
- O que importa num processo é o **contexto**: quem o criou, sob que identidade roda, o que herdou. A relação **pai-filho** é a estrutura mais reveladora numa investigação, e órfãos apagam o rastro da origem.
- **SUID no Unix e privilégios de token no Windows** são os mecanismos legítimos de elevação e as principais superfícies de escalonamento local. O inventário de SUID deve ser pequeno, conhecido e estável; `SeDebugPrivilege`, `SeImpersonatePrivilege` e `SeBackupPrivilege` equivalem a administração.
- Os dois modelos de permissão trocam simplicidade por expressividade. Dois detalhes explicam a maioria dos erros: execução em diretório significa **atravessar**, e é a permissão do **diretório** que governa criação e exclusão. `chmod 777` é escalonamento servido pronto.
- A **corrupção de memória** vira execução de código porque o endereço de retorno é dado gravável. As mitigações — canário, DEP/NX, ASLR, PIE, RELRO, CFI — só funcionam **em conjunto**, precisam ser ativadas na compilação, e cerca de dois terços das falhas graves pertencem a essa classe, que linguagens com memória segura eliminam de origem.
- **Tudo que um processo usa está em claro na memória dele**. Criptografia em disco e em trânsito não protege contra quem tem privilégio de depuração — daí a sensibilidade desse privilégio e a existência de módulos de hardware que nunca expõem a chave.
- O isolamento além do processo é uma escada: usuário próprio, capacidades, espaços de nomes, `seccomp`, MAC (SELinux/AppArmor) e virtualização. Todos são formas de aplicar o **menor privilégio**; contêineres compartilham o núcleo, e uma falha de anel 0 atravessa essa fronteira.
- Numa triagem, sete perguntas resolvem quase tudo — e **desligar a máquina destrói a maior parte das respostas**. Preservar antes de conter é requisito técnico e, no Brasil, também condição para responder à ANPD e a pedidos de preservação.

12. Linux essencial para segurança

Servidor de aplicação de um provedor, sexta-feira de tarde. O monitoramento acusa que a partição raiz encheu. O plantonista entra, roda `du` e encontra 40 GB num arquivo dentro de `/var/tmp` com nome de seis caracteres aleatórios. Apaga o arquivo, o espaço não volta. Reinicia o serviço, o espaço volta — e o arquivo reaparece dez minutos depois.

O que estava acontecendo cabe em três fatos do sistema operacional. O espaço não voltou porque um processo ainda mantinha o arquivo aberto, e no Linux o espaço só é liberado quando o último descritor fecha. O arquivo reapareceu porque havia uma entrada no `cron` de um usuário de serviço, criada três semanas antes. E o processo que escrevia era um minerador, rodando como `www-data`, iniciado por uma falha na aplicação web.

Nenhum desses fatos exige ferramenta especial. Todos são visíveis com comandos que já estão instalados em qualquer distribuição. O problema é que quem não sabe onde olhar apaga o arquivo, vê o espaço “não voltar”, reinicia o servidor e considera o caso encerrado — deixando a persistência intacta.

Este capítulo é o mapa operacional do Linux para quem faz segurança: onde as coisas ficam, como o sistema arranca, como usuários e permissões funcionam de verdade, o que os registros contam e quais são os quinze comandos que resolvem 90% da triagem. O Capítulo 11 deu o modelo conceitual; aqui ele vira caminho de arquivo.

12.1. O sistema de arquivos como mapa mental

Tudo no Linux é um caminho a partir de `/`. Discos, dispositivos, processos, parâmetros do núcleo — tudo aparece como arquivo. Para segurança, os diretórios que importam são poucos:

Tabela 12.1.: Diretórios que importam numa investigação.

Caminho	O que guarda	Por que interessa
<code>/etc</code>	Configuração do sistema	Onde as políticas moram e onde alterações indevidas aparecem
<code>/var/log</code>	Registros	A primeira parada de qualquer investigação
<code>/home</code> , <code>/root</code>	Dados de usuário	<code>.ssh/authorized_keys</code> , histórico de comandos
<code>/tmp</code> , <code>/var/tmp</code> , <code>/dev/shm</code>	Áreas temporárias graváveis por todos	Ponto de largada favorito de código malicioso
<code>/proc</code>	Estado do núcleo e dos processos	A verdade sobre processos, sem intermediários
<code>/etc/systemd</code> , <code>/etc/cron*</code>	Serviços e agendamentos	Onde a persistência é instalada
<code>/usr/bin</code> , <code>/usr/sbin</code>	Binários do sistema	Binário alterado ou novo é achado grave

Caminho	O que guarda	Por que interessa
/opt, /srv	Aplicações e dados de serviço	Onde a aplicação vulnerável costuma estar

Três propriedades do sistema de arquivos explicam comportamentos que confundem quem está começando.

O nome do arquivo não é o arquivo. O que existe de fato é o **inode**, com os dados e os metadados; o nome é apenas um ponteiro num diretório. Por isso apagar não libera espaço enquanto houver processo com o arquivo aberto — foi o que aconteceu no caso de abertura — e por isso um binário malicioso pode continuar executando depois de removido do disco, acessível apenas por `/proc/<pid>/exe`.

Marcas temporais mentem parcialmente. Cada inode guarda **atime** (último acesso), **mtime** (última alteração de conteúdo) e **ctime** (última alteração de metadados). As duas primeiras podem ser reescritas por quem tem permissão; o **ctime** não pode ser definido diretamente e muda quando qualquer coisa muda — inclusive quando alguém tenta forjar as outras duas. Um arquivo cujo **mtime** é de 2019 e cujo **ctime** é de ontem está com data adulterada, e essa discrepância é achado forense de alto valor.

Áreas graváveis por todos são o ponto de partida padrão. `/tmp`, `/var/tmp` e `/dev/shm` são os únicos lugares onde qualquer usuário escreve. Montá-los com `noexec,nosuid,nodev` quebra uma parcela significativa de código malicioso automatizado, que assume poder executar dali.

12.2. Como a máquina arranca — e onde a persistência mora

A sequência é `firmware` → carregador de inicialização (GRUB) → núcleo → `initramfs` → `systemd` → alvo padrão. Em cada etapa há um ponto que interessa à segurança:

- **Firmware e Secure Boot** — assinaturas verificadas antes de qualquer código do sistema executar. É o que impede um *bootkit*.
- **GRUB** — se não tiver senha, quem tem acesso físico edita a linha de comando do núcleo, arranca em modo de usuário único e é root sem senha. Criptografia de disco é o que realmente resolve; senha no GRUB é a mitigação mínima.
- **Núcleo e módulos** — `lsmod` lista o que está carregado. Módulo desconhecido é candidato a *rootkit*. Assinatura obrigatória de módulos fecha essa porta.
- **systemd** — o gerenciador de serviços e, junto com o `cron`, o principal lugar onde persistência é instalada.

A Figura 12.1 reúne os mecanismos de persistência que uma triagem precisa verificar. A lista importa porque **a maioria das respostas a incidente falha aqui**: o processo malicioso é morto, a máquina é reiniciada, e o agendamento que o recria segue intacto.

12.3. Usuários, grupos e o que o sudo realmente permite

As contas vivem em `/etc/passwd` (legível por todos, sem senhas) e as senhas com sal e derivação em `/etc/shadow` (só root). Três leituras rápidas dizem muito sobre a higiene de uma máquina:

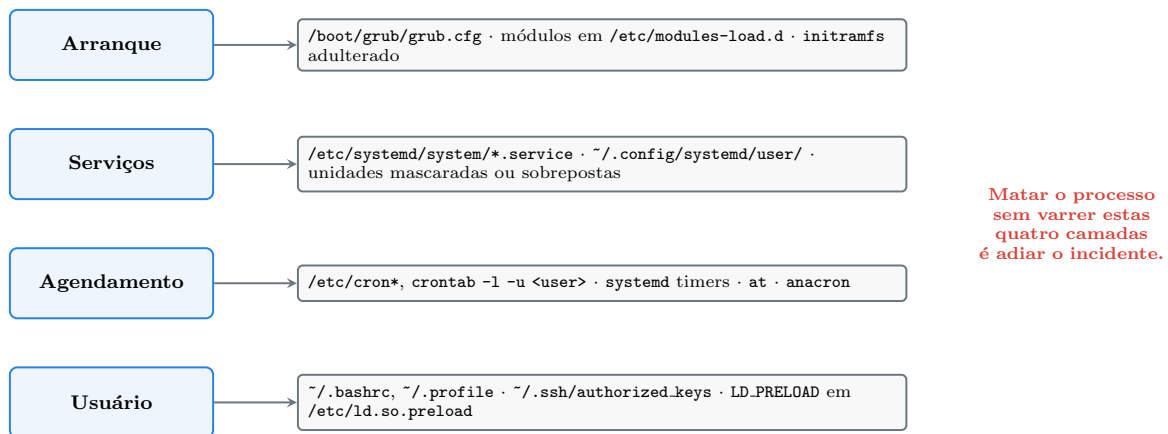


Figura 12.1.: Onde a persistência é instalada em Linux. A varredura precisa cobrir as quatro camadas.

- **Contas com UID 0 além do root.** Deveria haver exatamente uma. Qualquer outra é porta dos fundos.
- **Contas de serviço com interpretador de comandos.** Serviço não precisa de `shell`; `nologin` é o correto.
- **Contas com senha vazia ou sem expiração de credencial.** Achado comum em máquina herdada.

O **sudo** é o mecanismo de elevação, e a maior parte dos problemas está em como ele é configurado. Três armadilhas recorrentes:

NOPASSWD em comando que abre interpretador. Permitir `sudo vim`, `sudo less`, `sudo find`, `sudo awk` ou `sudo tar` sem senha é conceder root, porque todos esses programas executam comandos externos por desenho. A regra: se o binário permitido consegue chamar outro programa, a permissão equivale a acesso total.

Curinga em caminho. `sudo /usr/bin/systemctl restart *` permite muito mais do que reiniciar serviços, porque o argumento não é validado.

Script gravável. `sudo /opt/scripts/backup.sh` com o script gravável pelo usuário é root direto — basta editar o script. A verificação de permissões do alvo de uma regra `sudo` é parte da revisão.

Do lado da observabilidade, o **sudo** registra o que executa, e esse registro é a base da responsabilização discutida no Capítulo 3. Contas administrativas compartilhadas destroem essa cadeia; acesso nominal com elevação registrada é o requisito mínimo.

12.4. SSH: a porta principal

Em servidor Linux, o SSH costuma ser o único serviço administrativo exposto — e por isso concentra a maior parte das tentativas de acesso. A configuração segura cabe em poucas linhas e vale mais do que qualquer camada acrescentada depois:

```
# /etc/ssh/sshd_config - o essencial
PermitRootLogin no
PasswordAuthentication no
```

```

PubkeyAuthentication yes
KbdInteractiveAuthentication no
AllowGroups sysadmin
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
AllowTcpForwarding no
ClientAliveInterval 300
ClientAliveCountMax 2

```

Quatro observações que a experiência de campo confirma. **Chave em vez de senha** elimina de uma vez a força bruta e a reutilização de credencial — e é a mudança de maior impacto isolado. **Mudar a porta padrão** reduz o ruído nos registros, mas não é controle de segurança: uma varredura encontra o serviço em qualquer porta, e o Volume 9 mostra como. **AllowGroups** é melhor que **AllowUsers**, porque a lista não precisa ser editada a cada admissão. E **authorized_keys** é **alvo de persistência**: uma chave acrescentada ali sobrevive a troca de senha, a reinstalação de aplicação e a quase toda resposta a incidente — daí a necessidade de compará-la periodicamente com uma linha de base.

12.5. Registros: o que o sistema conta

O Linux registra em dois lugares que coexistem: o **journal** binário do **systemd** e os arquivos de texto em **/var/log**. Os que importam:

Tabela 12.2.: Fontes de registro em Linux e o que cada uma responde.

Arquivo ou fonte	Conteúdo	O que procurar
journalctl	Tudo do systemd , incluindo o núcleo	Serviço que reinicia em laço; unidade desconhecida
/var/log/auth.log, secure	Autenticação, sudo, SSH	Falhas em rajada seguidas de sucesso
/var/log/syslog, messages	Sistema em geral	Mensagens do núcleo, montagens, USB
lastb, last, w	Tentativas e sessões	Login de origem inesperada ou fora de horário
auditd	Chamadas de sistema com regra	Acesso a arquivo sensível, execução privilegiada

Duas verdades desconfortáveis sobre registros locais. A primeira: **quem vira root apaga ou edita o que quiser**. Registro local é ponto de partida, não prova. A segunda: **a maioria das máquinas não registra o que importa**. Execução de comando, leitura de arquivo sensível e mudança de configuração não aparecem por padrão — é o **auditd** que os produz, e ele exige regras escritas.

A conclusão prática vale para toda a operação: **remessa de registro para fora da máquina, em tempo real**, é o que transforma registro em evidência. Uma máquina comprometida não pode ser a guardiã do seu próprio histórico. O Volume 12 trata de centralização e correlação; a decisão de arquitetura, contudo, é deste nível.

12.6. Rede vista de dentro da máquina

A pergunta mais frequente numa triagem é “o que esta máquina está expondo, e com quem ela fala?”. A resposta vem de três comandos, e a coluna do processo é a parte que interessa:

```
ss -tulpn          # o que escuta, e qual processo
ss -tanp state established  # conexões ativas, com PID
ip -br a; ip r     # endereços e rotas, formato compacto
```

Três achados recorrentes merecem atenção imediata. Um serviço escutando em 0.0.0.0 quando deveria estar em 127.0.0.1 — banco de dados, cache, painel administrativo — é exposição desnecessária e frequentemente involuntária. Uma conexão de saída persistente para endereço desconhecido, mantida por processo que não deveria falar com a internet, é o padrão de comando e controle. E um serviço escutando em porta alta sem processo identificável costuma indicar que a ferramenta está sendo enganada por um *rootkit*.

12.7. Os quinze comandos da triagem

O conjunto abaixo resolve a maior parte de uma primeira análise. Vale decorar a ordem, porque ela vai do mais volátil ao menos volátil:

```
# 1. Quem está conectado agora e o que executou
w; last -20; lastb -20

# 2. Processos por consumo, com árvore genealógica
ps auxf --sort=--%cpu | head -25

# 3. O que escuta e com quem fala
ss -tulpn; ss -tanp state established

# 4. Persistência: serviços, timers e agendamentos
systemctl list-units --type=service --state=running
systemctl list-timers --all
for u in $(cut -d: -f1 /etc/passwd); do crontab -l -u "$u" 2>/dev/null | sed
  ↪  "s/^/$u: /"; done
ls -la /etc/cron.d /etc/cron.* / 2>/dev/null

# 5. Arquivos recentes nas áreas graváveis por todos
find /tmp /var/tmp /dev/shm -type f -mmin -1440 -ls 2>/dev/null

# 6. Binários SUID fora do esperado
find / -xdev -perm -4000 -type f -ls 2>/dev/null

# 7. Chaves de acesso e contas privilegiadas
awk -F: '$3==0 {print $1}' /etc/passwd
find /home /root -name authorized_keys -exec ls -l {} \; -exec cat {} \;

# 8. Descritores abertos de arquivos já apagados - o caso de abertura
ls -l /proc/*/exe 2>/dev/null | grep -i deleted
lsof +L1 2>/dev/null | head -20
```

O último bloco é o que responde à pergunta que abriu o capítulo: `ls -l` lista arquivos abertos cuja contagem de nomes é zero — apagados, mas ainda ocupando espaço e ainda em uso.

12.8. Endurecimento: a lista curta que vale a pena

Endurecimento completo é assunto do Volume 6, com referência a benchmarks reconhecidos (Center for Internet Security, 2024). O conjunto abaixo é o que cabe numa tarde e evita a maioria dos problemas comuns:

Montagem restritiva das áreas temporárias. Uma linha em `/etc/fstab` por partição, e uma classe inteira de código malicioso automatizado deixa de funcionar:

```
tmpfs /tmp      tmpfs defaults,noexec,nosuid,nodev,size=2G 0 0
tmpfs /dev/shm  tmpfs defaults,noexec,nosuid,nodev        0 0
```

Parâmetros de núcleo. Os que fecham reconhecimento e ataques de camada 3 tratados no Capítulo 8:

```
# /etc/sysctl.d/99-hardening.conf
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.tcp_syncookies = 1
kernel.yama.ptrace_scope = 1
kernel.dmesg_restrict = 1
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
```

Superfície mínima. Desinstalar o que não é usado é mais eficaz do que configurar o que não é usado. Cada serviço removido é uma porta que não precisa ser vigiada.

Atualização automática de segurança. Em servidor, o receio de que uma atualização quebre algo é legítimo — e a resposta é ambiente de homologação e janela definida, não adiar indefinidamente. `unattended-upgrades` restrito ao repositório de segurança é o meio-termo que funciona.

Controle de acesso obrigatório ligado. SELinux em `enforcing` ou AppArmor ativo. A tentação de desativar ao primeiro problema é grande e o preço aparece depois: é o único mecanismo que contém um processo root comprometido.

12.9. Brasil em Foco

Registro remoto é o que sustenta a apuração. O Capítulo 3 tratou de não repúdio; em Linux isso significa `auditd` com regras e remessa para fora da máquina. Sem isso, a resposta a um pedido de informação sobre “quem acessou este arquivo e quando” é uma suposição. Para provedores, essa capacidade se soma às obrigações de guarda dos arts. 13 e 15 do Marco Civil (Brasil, 2014) — que tratam de registros de conexão e de acesso a aplicações, e não substituem o registro administrativo interno.

Fuso e sincronização de relógio. Registro sem hora confiável não sustenta correlação entre máquinas nem resposta a ofício. `timedatectl` com NTP ativo, fuso declarado e verificação periódica é requisito prático — e o erro de uma hora por horário de verão mal configurado já bastou para inviabilizar identificação de assinante.

Dados pessoais em `/var/log` e em `/tmp`. Registros de aplicação frequentemente contêm CPF, e-mail, endereço e até senha registrada por engano; áreas temporárias acumulam exportações e relatórios esquecidos. Tudo isso é dado pessoal sob a LGPD (Brasil, 2018), com necessidade de prazo de retenção, controle de acesso e eliminação. A rotação de registros precisa de política escrita, e não apenas do `logrotate` que veio com a distribuição.

Software fora de suporte. Distribuições têm data de fim de vida pública. Rodar servidor sem correções de segurança com dados pessoais é decisão que os arts. 46 e 47 da LGPD alcançam, e que a ANPD, em orientação a agentes de pequeno porte, trata como falha em medida básica esperada (Autoridade Nacional de Proteção de Dados, 2023). O inventário com data de fim de suporte por sistema é documento de conformidade, não só de TI.

CERT.br e o que chega pelo contato de abuso. Máquinas Linux comprometidas em redes brasileiras costumam ser detectadas de fora — por participarem de varreduras, ataques de amplificação ou distribuição de código malicioso — e a notificação chega ao contato de abuso do bloco. Manter esse endereço monitorado, com procedimento de triagem definido, é frequentemente a diferença entre descobrir o comprometimento em horas ou em meses (CERT.br, 2024).

12.10. No Campo

1. Guardar uma linha de base e comparar periodicamente. Sem linha de base, “isso é normal?” não tem resposta:

```
BASE=/var/lib/baseline; mkdir -p $BASE
find / -xdev -perm -4000 -type f -printf '%m %u %p\n' 2>/dev/null | sort >
  ↳ $BASE/suid
ss -tulpn | sort > $BASE/listen
systemctl list-unit-files --state=enabled | sort > $BASE/units
awk -F: '{print $1":"$3":"$7}' /etc/passwd | sort > $BASE/users
# Guardar cópia fora da máquina - em máquina comprometida, a linha de base
  ↳ também é alvo
```

2. Fazer `auditd` registrar o que importa. Sem regras, o `auditd` instalado não produz quase nada de útil:

```
# /etc/audit/rules.d/seguranca.rules
-w /etc/passwd -p wa -k identidade
-w /etc/shadow -p wa -k identidade
-w /etc/sudoers -p wa -k elevacao
-w /etc/sudoers.d/ -p wa -k elevacao
-w /root/.ssh/ -p wa -k chaves
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=1 -k
  ↳ exec-root
```

```
ausearch -k exec-root --start recent | tail -40
```

3. Enviar registro para fora em tempo real. É o que transforma registro em evidência:

```
# /etc/rsyslog.d/10-remoto.conf - TLS e fila em disco para não perder no corte
*. * action(type="omfwd" target="logs.exemplo.local" port="6514"
        protocol="tcp" StreamDriverMode="1" StreamDriverAuthMode="x509/name"
        queue.type="LinkedList" queue.filename="fwd"
↪ queue.saveonshutdown="on")
```

4. Revisar o sudoers procurando as três armadilhas. Comando que abre interpretador, curinga e script gravável:

```
sudo visudo -c # valida a sintaxe antes de qualquer coisa
grep -rE 'NOPASSWD|\\*' /etc/sudoers /etc/sudoers.d/
# Para cada alvo de regra, conferir se o usuário consegue reescrever o binário
```

5. Comparar authorized_keys com a linha de base a cada varredura. Persistência que sobrevive a quase tudo:

```
find /home /root -name authorized_keys -newer /var/lib/baseline/users -ls
```

6. Investigar espaço que não volta depois de apagar arquivo. O caso de abertura, em dois comandos:

```
lsof +L1 | awk '{print $1, $2, $7, $NF}' | sort -k3 -rn | head
ls -l /proc/*/exe 2>/dev/null | grep -i deleted
```

7. Restringir SSH a chave e grupo, e verificar antes de aplicar. Um erro aqui tranca o administrador do lado de fora:

```
sshd -t # valida a configuração
# Manter a sessão atual aberta e testar em outra antes de encerrar
systemctl reload sshd
ss -tanp state established '( sport = :22 )'
```

8. Montar as áreas temporárias com noexec e conferir que pegou. Muda o resultado de boa parte do código malicioso automatizado:

```
findmnt -no OPTIONS /tmp /var/tmp /dev/shm
```

12.11. Resumo

- No Linux, **o nome não é o arquivo**: espaço só é liberado quando o último descritor fecha, e um binário apagado continua executando por `/proc/<pid>/exe`. `lsof +L1` responde à pergunta “apaguei e o espaço não voltou”.
- **ctime não pode ser forjado diretamente**. Um arquivo com `mtime` antigo e `ctime` recente teve as datas adulteradas — achado forense de alto valor.
- Persistência mora em **quatro camadas**: arranque, serviços, agendamento e perfil de usuário. Matar o processo sem varrer as quatro é adiar o incidente.
- No **sudo**, três armadilhas concentram os problemas: `NOPASSWD` em binário que abre interpretador (`vim`, `less`, `find`, `tar`), curinga em argumento e script alvo gravável pelo próprio usuário. Todas equivalem a conceder root.
- **SSH com chave em vez de senha** é a mudança isolada de maior impacto. Mudar de porta reduz ruído, não é controle. `authorized_keys` é alvo de persistência e precisa de linha de base.
- Registro local é **ponto de partida, não prova** — root reescreve o que quiser, e o padrão da distribuição não registra execução de comando nem acesso a arquivo sensível. `auditd` com regras e **remessa remota em tempo real** é o que transforma registro em evidência.
- A triagem cabe em quinze comandos, do mais volátil ao menos: sessões, processos com árvore, sockets com PID, persistência nas quatro camadas, arquivos recentes em áreas graváveis, SUID, contas UID 0 e descritores de arquivos apagados.
- O endurecimento que cabe numa tarde: `noexec,nosuid,nodev` nas áreas temporárias, parâmetros de núcleo, remoção do que não é usado, atualização automática de segurança e MAC em modo restritivo — o único mecanismo que contém um processo root comprometido.

13. Windows e Active Directory essencial

Uma rede corporativa de 300 estações. Um funcionário do setor comercial abre uma planilha recebida por e-mail e habilita as macros. Nada visível acontece. Em quatro horas, o atacante é administrador do domínio inteiro — e nenhuma vulnerabilidade de software foi explorada em nenhum momento do caminho.

A cadeia, reconstruída depois, tem cinco passos. A macro executou como o usuário comercial, uma conta comum. Da memória do processo de autenticação da estação, o atacante extraiu o hash da senha de uma conta de suporte técnico que havia feito login naquela máquina na semana anterior — porque o Windows guardava aquela credencial em memória. Com esse hash, autenticou-se em outras estações sem jamais precisar da senha em texto claro. Numa delas encontrou uma conta de serviço com privilégio de administrador local em todo o parque, porque a instalação de um sistema pedia isso. Essa conta de serviço tinha um SPN registrado, e sua senha — definida em 2019 e nunca trocada — resistiu duas horas a uma quebra a partir do bilhete Kerberos obtido legitimamente por qualquer usuário autenticado. Aquela senha era a mesma de uma conta com direito de replicação no diretório.

Cada passo usou o Active Directory exatamente como ele foi projetado para funcionar. É essa a razão de o AD ser o alvo central em quase todo incidente corporativo relevante: **não é um sistema com falhas, é um sistema com padrões perigosos e uma superfície de confiança enorme.**

O Capítulo 12 mapeou o Linux. Este faz o mesmo para o Windows e para o serviço de diretório que organiza a maior parte das redes corporativas brasileiras.

13.1. Como o Windows pensa

Três conceitos explicam quase todo comportamento de segurança do sistema.

Tudo é um objeto com um descritor de segurança. Arquivos, chaves de registro, processos, serviços, impressoras, objetos do diretório — cada um carrega dono, lista de permissões (DACL) e lista de auditoria (SACL). A consequência prática é que permissões existem em lugares onde ninguém pensa em procurar: a permissão de um serviço pode permitir que um usuário comum altere seu binário; a permissão de uma chave de registro pode permitir que ele altere o que executa no arranque.

A identidade é o SID, não o nome. Cada conta tem um identificador único e imutável. Renomear a conta `Administrador` não muda seu SID, que termina em `-500` — e é por isso que renomear administrador como medida de segurança é teatro: qualquer ferramenta localiza a conta pelo sufixo. Os SIDs bem conhecidos merecem reconhecimento imediato: `-500` (administrador local), `-512` (Admins. do Domínio), `-519` (Admins. da Empresa), `-518` (Admins. do Esquema).

A autorização acontece por token. Ao autenticar, o usuário recebe um token com seu SID, os SIDs de todos os grupos e a lista de privilégios nomeados. Toda verificação posterior consulta esse token — o que produz um efeito que confunde muita gente: **incluir alguém num**

grupo não tem efeito até que ele saia e entre de novo, porque o token só é montado na autenticação.

Os privilégios que equivalem a administração, já introduzidos no Capítulo 11, valem repetição: `SeDebugPrivilege` (ler a memória de qualquer processo), `SeImpersonatePrivilege` (assumir outra identidade), `SeBackupPrivilege` e `SeRestorePrivilege` (ler e escrever qualquer arquivo ignorando permissões), `SeTakeOwnershipPrivilege` e `SeLoadDriverPrivilege` (carregar código em anel 0). Uma conta de serviço com qualquer um deles é administrador por via indireta.

13.1.1. UAC não é fronteira de segurança

O Controle de Conta de Usuário divide o token de um administrador em dois: um restrito para uso comum e um completo, ativado pela elevação. Isso reduz danos acidentais, e a própria Microsoft o documenta como recurso de conveniência — **não como limite de segurança**. Existem caminhos conhecidos de elevação automática que dispensam o consentimento do usuário.

A consequência operacional é a que importa: **usuário administrador local é usuário administrador local, com ou sem UAC**. A separação real é ter contas distintas — uma comum para o dia a dia, outra administrativa usada apenas quando necessário e jamais para navegar ou ler e-mail.

13.2. Active Directory: o mapa da confiança

O AD é um banco de dados hierárquico com autenticação. Cinco termos organizam tudo:

- **Domínio** — fronteira administrativa e de replicação, com seus usuários, grupos e computadores.
- **Floresta** — conjunto de domínios com esquema comum. É a **verdadeira fronteira de segurança**: comprometer um domínio compromete a floresta, porque as relações de confiança internas não são barreiras defensivas.
- **Unidade organizacional (OU)** — contêiner para organizar objetos e aplicar política. Não é grupo de segurança.
- **GPO** — política aplicada a OUs, sites ou domínio, que configura estações e servidores em massa.
- **Controlador de domínio (DC)** — servidor que guarda o `NTDS.dit`, o arquivo com todas as credenciais do domínio. Comprometer um DC é comprometer tudo.

A confusão mais cara aqui é entre floresta e domínio: separar departamentos em domínios distintos “por segurança” não cria isolamento. Se o requisito é isolamento real, a resposta é floresta separada.

13.3. Autenticação: NTLM, Kerberos e a memória da estação

13.3.1. NTLM e o problema de projeto

O NTLM autentica por desafio e resposta, usando o hash da senha. O ponto que sustenta metade dos ataques em rede Windows é este: **o hash funciona como a senha**. Quem o obtém autentica-se sem nunca conhecer a senha em texto claro — a técnica conhecida como *pass the hash*, o segundo passo do caso de abertura.

Além disso, o NTLM não autentica o servidor, o que viabiliza retransmissão: o atacante induz uma máquina a autenticar-se contra ele e repassa a autenticação a um terceiro serviço, agindo como a vítima. Os protocolos de resolução de nomes de contingência — LLMNR, NBT-NS e mDNS — são o gatilho clássico: quando o DNS falha, a estação pergunta em difusão “quem é servidorx?”, e qualquer máquina na rede pode responder “eu sou”. **Desabilitar LLMNR e NBT-NS por GPO é uma das medidas de melhor relação custo-benefício em todo o Windows.**

A direção correta é reduzir o NTLM ao mínimo, exigir assinatura SMB e LDAP (o que quebra a retransmissão) e caminhar para bloqueá-lo — com o cuidado de auditar antes, porque sistemas antigos ainda dependem dele.

13.3.2. Kerberos e seus dois ataques clássicos

O Kerberos é o protocolo padrão no domínio, baseado em bilhetes emitidos por um centro de distribuição de chaves que roda no DC. A Figura 13.1 mostra o fluxo normal e onde os dois abusos mais comuns se encaixam.

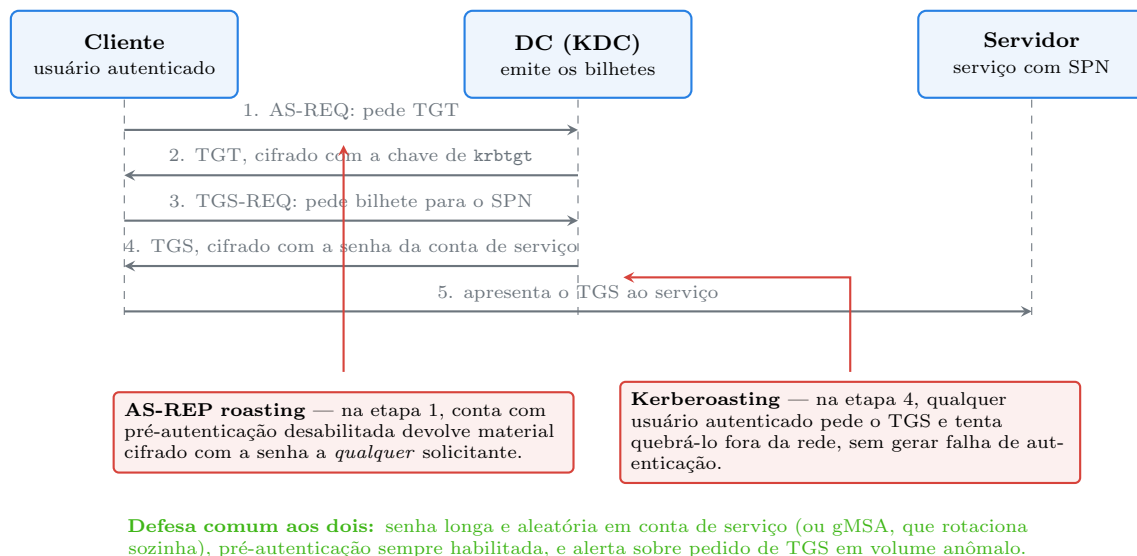


Figura 13.1.: Fluxo Kerberos e os dois abusos que dispensam qualquer vulnerabilidade de software.

O **Kerberoasting** foi o terceiro passo do caso de abertura, e é insidioso porque cada etapa é legítima: pedir um bilhete de serviço é o que todo cliente faz. A tentativa de quebra ocorre **fora da rede**, sem gerar falha de autenticação e sem disparar bloqueio de conta. A defesa, portanto, não é bloquear o pedido — é tornar a quebra inviável, com senhas longas e aleatórias, idealmente através de contas de serviço gerenciadas por grupo (gMSA), que trocam a senha automaticamente.

Dois outros abusos merecem menção, porque aparecem em qualquer relatório de teste de invasão: a **delegação irrestrita**, em que um servidor configurado assim guarda em memória os TGTs de todos que o acessam — incluindo administradores —, e o **bilhete dourado**, forjado a partir do hash da conta `krbtgt`, que dá acesso perpétuo até que essa conta seja trocada duas vezes.

13.3.3. Por que a estação guarda credenciais

O passo que costuma surpreender é o primeiro: como a macro executada por um usuário comum obteve a credencial de um técnico de suporte? Porque o processo de autenticação do Windows mantinha em memória o material de autenticação de todas as sessões ativas ou recentes na máquina — necessário para o *logon* único funcionar sem pedir senha a cada acesso a recurso de rede.

Daí decorrem três controles que se reforçam:

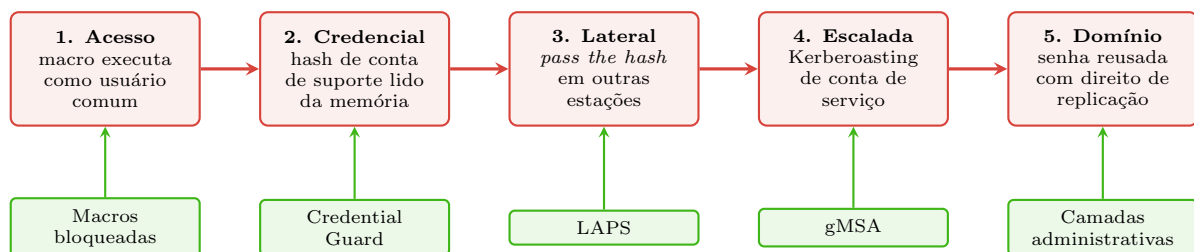
Credential Guard, que isola esses segredos num ambiente protegido por virtualização, inacessível mesmo ao núcleo comprometido. É a mitigação mais efetiva e exige hardware moderno.

Camadas administrativas (o modelo de níveis): a credencial de administrador de domínio **nunca** é usada numa estação de trabalho. Administração de servidores a partir de servidores; administração de domínio a partir de estações administrativas dedicadas. A regra que resume o modelo: *uma credencial só deve ser usada em máquinas do seu nível ou superior*.

LAPS, que dá a cada estação uma senha de administrador local diferente, aleatória e rotacionada, guardada no diretório. Elimina a reutilização que permite pular de máquina em máquina — o segundo passo do caso de abertura.

13.4. Os caminhos de escalada que existem sem falha nenhuma

O que torna o AD difícil de defender é que os privilégios se compõem por relações, e não por listas. Um usuário pode não ser administrador e ainda assim ter um caminho até o domínio através de uma cadeia de permissões individualmente razoáveis. A Figura 13.2 refaz a cadeia do caso de abertura, marcando o controle que interrompe cada elo.



Nenhum passo explorou vulnerabilidade de software. Qualquer um dos cinco controles quebra a cadeia.

Figura 13.2.: A cadeia do caso de abertura e o controle que interrompe cada elo.

Os padrões recorrentes por trás dessa e de outras cadeias:

Tabela 13.1.: Caminhos de escalada em AD que não dependem de vulnerabilidade de software.

Caminho	Como funciona	Correção
Senha reutilizada de admin. local	Mesma senha em todo o parque; hash serve em todas	LAPS
Conta de serviço com privilégio excessivo	Instalação pediu admin. de domínio e ninguém revisou	Conta dedicada, gMSA, privilégio mínimo
Senha em GPO ou em script de logon	SYSVOL é legível por qualquer usuário do domínio	Remover; trocar a credencial exposta

Caminho	Como funciona	Correção
Grupos administrativos inchados	Dezenas de membros herdados de anos	Revisão trimestral; administração temporária
Permissão delegada esquecida	Direito de trocar senha ou escrever em objeto	Auditoria de ACL do diretório
Delegação irrestrita	Servidor acumula TGTs de quem o acessa	Migrar para delegação restrita
Conta com senha sem expiração e sem MFA	Força bruta lenta e discreta ao longo de meses	Política de senha e MFA

A descoberta desses caminhos é automatizável, e é exatamente isso que ferramentas de análise de grafo do diretório fazem — inclusive as usadas defensivamente. Rodar essa análise contra o próprio ambiente, com autorização documentada nos termos do Capítulo 7, é o exercício mais revelador que uma equipe de infraestrutura pode fazer, e costuma encontrar caminhos de cinco saltos que ninguém imaginava.

13.5. Registros e detecção: o que ligar antes de precisar

O Windows registra muito por padrão — e quase nada do que interessa. Os eventos que efetivamente sustentam detecção precisam ser habilitados por política de auditoria e, em alguns casos, por instalação adicional:

Tabela 13.2.: Eventos do Windows que sustentam detecção.

Evento	O que indica
4624 / 4625	Logon bem-sucedido / falho, com o tipo (3 = rede, 10 = RDP)
4648	Logon com credencial explícita — sinal de movimentação lateral
4672	Atribuição de privilégio especial na sessão
4720, 4728, 4732	Conta criada; membro adicionado a grupo global e local
4768 / 4769	Bilhete Kerberos pedido — o 4769 em volume é indício de Kerberoasting
4698	Tarefa agendada criada — persistência
7045	Serviço instalado — persistência e execução remota
1102	Registro de segurança apagado — sempre investigar
4104	Bloco de script PowerShell registrado (exige registro de blocos ligado)

Três configurações mudam o patamar de visibilidade e não custam licença: **registro de linha de comando na criação de processo** (evento 4688 com o comando completo), **registro de blocos e transcrição do PowerShell**, e **aumento do tamanho do registro de segurança**, que no padrão gira em horas numa máquina movimentada e apaga a evidência antes que alguém a leia. Sobre isso, vale a mesma regra do Capítulo 12: **remessa para um coletor externo**,

porque quem vira administrador apaga o registro local — e é justamente esse apagamento que o evento 1102 denuncia.

13.6. Brasil em Foco

Licenciamento e sistema fora de suporte. Parque com Windows sem suporte — e ainda há muito Windows 7 e Server 2008 em operação no país — não recebe correção e não suporta os controles modernos deste capítulo. Sob os arts. 46 e 47 da LGPD (Brasil, 2018), manter dados pessoais em sistema sem manutenção é falha em medida de segurança esperada, e a ANPD já indicou atualização como item básico em sua orientação a agentes de pequeno porte (Autoridade Nacional de Proteção de Dados, 2023). O inventário com data de fim de suporte é documento de conformidade.

Ransomware entra pelo AD. Os ataques que pararam hospitais, prefeituras e empresas brasileiras seguiram, quase sempre, a cadeia deste capítulo: acesso inicial por phishing ou RDP exposto, roubo de credencial em memória, movimentação lateral com hash, escalada até administrador do domínio e, por fim, distribuição da carga por GPO ou por ferramenta de administração remota — usando a própria infraestrutura de gerência contra a organização. Como incidente de segurança com dados pessoais, dispara a comunicação prevista na Resolução CD/ANPD nº 15/2024 (Autoridade Nacional de Proteção de Dados, 2024), com prazo e conteúdo definidos. O Volume 11 trata da anatomia completa.

RDP exposto à internet é o vetor mais comum e o mais evitável. Uma varredura de qualquer bloco brasileiro encontra milhares de portas 3389 abertas. Expor administração remota diretamente é decisão que precisa ser eliminada — VPN, gateway com MFA ou acesso por bastião —, e sua manutenção dificilmente se sustenta numa avaliação de diligência após um incidente.

Registro de acesso e responsabilização. O Capítulo 3 tratou de não repúdio; em ambiente Windows isso significa contas administrativas nominais, jamais compartilhadas, com auditoria ligada e remessa externa. Uma apuração interna ou uma resposta a ofício que dependa de saber “quem executou o quê” fracassa quando três pessoas usam a mesma conta **Administrador**.

Dados pessoais no diretório. O AD guarda nome, matrícula, cargo, telefone, e-mail e frequentemente CPF em atributos personalizados — base de dados pessoais sob a LGPD, com necessidade de controle de acesso. Vale lembrar que **qualquer usuário autenticado lê a maior parte do diretório por padrão**: a lista completa de funcionários, sua estrutura hierárquica e seus contatos estão disponíveis a qualquer conta comprometida.

13.7. No Campo

1. Desabilitar LLMNR e NBT-NS por GPO. Corta a retransmissão NTLM na origem, e é a medida de melhor relação custo-benefício do capítulo:

```
Configuração do Computador > Modelos Administrativos > Rede > Cliente DNS  
→ "Desativar a resolução de nomes multicast" = Habilitado
```

NBT-NS: nas propriedades de TCP/IPv4 de cada adaptador, ou por script na

```
↪ imagem:
```

```
Set-ItemProperty -Path
```

```
↪ 'HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces\*' ` -  
-Name NetbiosOptions -Value 2
```

2. Implantar LAPS. Elimina a senha de administrador local reutilizada em todo o parque:

```
# Windows LAPS (integrado às versões atuais) - conferir aplicação
Get-LapsADPassword -Identity ESTACAO-042 -AsPlainText
Get-CimInstance Win32_ComputerSystem | Select-Object Name, Domain
```

3. Caçar contas de serviço com SPN e senha antiga. É o vetor do Kerberoasting, e a consulta leva segundos:

```
Get-ADUser -Filter 'ServicePrincipalName -like "*" ' `
-Properties ServicePrincipalName, PasswordLastSet, AdminCount |
Select-Object Name, PasswordLastSet, AdminCount,
    ↳ @{n='SPN';e={$_.ServicePrincipalName -join ';' }} |
Sort-Object PasswordLastSet
```

4. Encontrar as contas sem pré-autenticação e sem expiração de senha. Achado de dois minutos que costuma render:

```
# AS-REP roasting
Get-ADUser -Filter 'DoesNotRequirePreAuth -eq $true' -Properties
    ↳ DoesNotRequirePreAuth

# Senha que nunca expira
Get-ADUser -Filter 'PasswordNeverExpires -eq $true -and Enabled -eq $true' |
    Select-Object Name, SamAccountName

# Delegação irrestrita - servidores que acumulam TGT de quem os acessa
Get-ADComputer -Filter 'TrustedForDelegation -eq $true' -Properties
    ↳ TrustedForDelegation
```

5. Revisar os grupos que importam, trimestralmente. Poucos membros, todos justificáveis:

```
foreach ($g in 'Domain Admins','Enterprise Admins','Schema Admins','Account
    ↳ Operators','Backup Operators') {
    "==" $g =="
    Get-ADGroupMember -Identity $g -Recursive | Select-Object -Expand
    ↳ SamAccountName
}
```

6. Procurar senha em SYSVOL. Legível por qualquer usuário do domínio, e ainda aparece:

```
Get-ChildItem \\dominio.local\SYSVOL -Recurse -Include *.xml,*.ps1,*.bat,*.vbs
    ↳ -ErrorAction SilentlyContinue |
    Select-String -Pattern 'cpassword|password\s*=' |
    Select-Object Path, Line -First 40
```

7. Ligar o registro que sustenta detecção. Sem isso, o incidente é reconstruído por adivinhação:

Política de Auditoria Avançada (por GPO):

Logon/Logoff → Logon, Logoff, Bloqueio de conta Êxito e Falha
Acesso a Objetos → Compartilhamento de Arquivos Êxito e Falha
Rastreamento Detalhado → Criação de Processo Êxito
Alteração de Política, Uso de Privilégio Êxito e Falha

Modelos Administrativos > Sistema > Auditoria de Criação de Processo
→ "Incluir linha de comando" = Habilitado

Componentes do Windows > Windows PowerShell

→ Registro de blocos de script = Habilitado
→ Transcrição = Habilitado (destino em compartilhamento restrito)

Tamanho do log Segurança: mínimo 1 GB nas estações, 4 GB nos servidores

8. Aplicar o modelo de camadas administrativas. A regra que mais reduz o alcance de um comprometimento: credencial de administrador de domínio nunca toca estação de trabalho. Estação administrativa dedicada, sem e-mail e sem navegação, e restrição por GPO (“Negar logon local” para contas de domínio nas estações comuns) para que a regra não dependa de disciplina individual.

13.8. Resumo

- No Windows, **tudo é objeto com descritor de segurança** — inclusive serviços e chaves de registro, onde permissões frouxas viram escalonamento. A identidade é o **SID** (renomear a conta administrador não muda o -500), e a autorização vem do **token**, montado na autenticação: mudança de grupo só vale no próximo logon.
- **UAC não é fronteira de segurança.** A separação real é ter contas distintas para uso comum e para administração.
- **A floresta é a fronteira de segurança**, não o domínio. Separar departamentos em domínios não cria isolamento.
- No **NTLM**, o hash funciona como a senha (*pass the hash*), e a ausência de autenticação do servidor viabiliza retransmissão. **Desabilitar LLMNR e NBT-NS** e exigir assinatura SMB e LDAP corta o vetor mais comum.
- **Kerberoasting e AS-REP roasting** abusam de operações legítimas: a quebra ocorre fora da rede, sem falha de autenticação e sem bloqueio de conta. A defesa é senha longa e aleatória em conta de serviço — idealmente **gMSA** — e pré-autenticação sempre habilitada.
- A estação guarda credenciais em memória para o *logon* único funcionar. As três defesas que se reforçam: **Credential Guard**, **modelo de camadas administrativas** (credencial de domínio nunca em estação) e **LAPS** (senha local única por máquina).
- Os caminhos de escalada em AD **não dependem de vulnerabilidade de software** — vêm de senha reutilizada, conta de serviço com privilégio excessivo, senha em SYSVOL, grupos inchados, delegação esquecida. São descobríveis por análise de grafo, e rodar essa análise contra o próprio ambiente é o exercício mais revelador disponível.
- O Windows registra muito e quase nada do que interessa. **Linha de comando na criação de processo, registro de blocos do PowerShell, log de segurança grande e remessa externa** são o mínimo — e o evento **1102** (log apagado) merece alerta sempre.

14. Virtualização e montagem do laboratório de estudos

Um estudante de segurança quer entender ARP spoofing. Leu o Capítulo 8, entendeu a teoria, e agora quer ver acontecer. Instala a ferramenta na própria máquina, aponta para o gateway de casa e executa. Funciona: a conexão da família cai, a irmã reclama que o vídeo travou, e ele não aprende quase nada — porque não tinha como observar os dois lados, não podia repetir em condições controladas e não podia errar sem consequência.

Duas semanas depois, o mesmo estudante monta três máquinas virtuais numa rede isolada: uma vítima, um atacante e um gateway. Executa o mesmo ataque. Captura o tráfego dos três pontos ao mesmo tempo. Vê a tabela ARP da vítima mudar em tempo real. Liga Dynamic ARP Inspection num switch virtual e vê o ataque falhar. Restaura o *snapshot* e repete de outro jeito. Em uma tarde, entende mais do que em um mês de leitura.

A diferença não é conhecimento nem ferramenta: é **ambiente**. O laboratório é o que separa quem sabe descrever um ataque de quem sabe reconhecê-lo, detectá-lo e contê-lo. E ele tem uma segunda função, igualmente importante: **é o único lugar onde a prática é inequivocamente legal**, o que o Capítulo 7 fixou como fronteira da profissão.

Este capítulo fecha o Volume 2 construindo esse ambiente. É também o capítulo mais prático do manual até aqui, e o que torna utilizáveis os próximos volumes.

14.1. Virtualização: o que a fronteira realmente vale

Um **hipervisor** cria máquinas virtuais, cada uma com seu núcleo, sua memória e seus discos virtuais. Há dois tipos, e a diferença importa mais para desempenho do que para segurança: o **tipo 1** roda direto no hardware (Proxmox VE, ESXi, Hyper-V, XCP-ng); o **tipo 2** roda sobre um sistema operacional hospedeiro (VirtualBox, VMware Workstation, Hyper-V no Windows cliente).

Para segurança, o que interessa é a **força do isolamento**, e a comparação com contêineres é o ponto central:

Tabela 14.1.: Máquina virtual e contêiner como fronteiras de isolamento.

Aspecto	Máquina virtual	Contêiner
Núcleo	Um por VM	Compartilhado com o hospedeiro
Fronteira	Hipervisor (superfície pequena)	Chamadas de sistema (superfície grande)
Escape	Raro, alto valor, corrigido com urgência	Mais frequente; erro de configuração basta
Arranque	Segundos a minutos	Milissegundos
Custo de recursos	Alto	Baixo

Aspecto	Máquina virtual	Contêiner
Uso em laboratório	Análise de código malicioso, sistemas completos	Serviços vulneráveis, alvos web

A regra prática que decorre disso: **contêiner para conveniência, máquina virtual para contenção**. Rodar um alvo web deliberadamente vulnerável em contêiner é adequado. Executar código malicioso desconhecido em contêiner **não é** — o Volume 11, ao tratar de análise dinâmica, trabalha com máquina virtual isolada, com instantâneo e sem rede, precisamente por isso.

Vale ainda registrar que o isolamento do hipervisor, embora forte, não é absoluto. Escapes existem e são corrigidos com urgência máxima; e há canais laterais em CPU compartilhada que atravessam fronteiras de VM, o que importa muito em nuvem multi-inquilino — assunto do Volume 8 — e pouco num laboratório doméstico. As orientações do NIST sobre virtualização segura (National Institute of Standards and Technology, 2011) resumem o quadro: a VM é uma fronteira forte, não uma garantia.

14.2. O laboratório mínimo

A pergunta mais comum de quem começa é sobre hardware. A resposta honesta: **8 GB de RAM já permitem estudar**, 16 GB tornam a experiência confortável, 32 GB permitem simular um domínio Windows completo. Disco SSD importa mais que processador, porque o gargalo real é entrada e saída — e um disco de 256 GB dedicado a máquinas virtuais é suficiente para começar.

A Figura 14.1 mostra a topologia que serve de base para tudo o que vem nos próximos volumes.

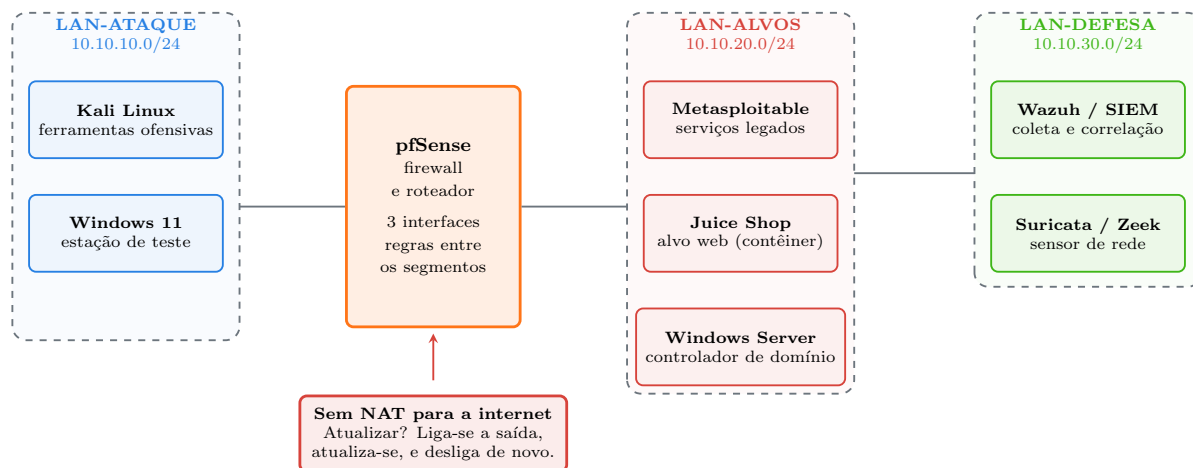


Figura 14.1.: Topologia do laboratório. Três segmentos separados por um firewall, sem rota para a internet no uso normal.

14.2.1. Os três segmentos e por que separá-los

LAN-ATAQUE hospeda as ferramentas ofensivas. **LAN-ALVOS** hospeda os sistemas deliberadamente vulneráveis. **LAN-DEFESA** hospeda a coleta e a detecção. A separação não é preciosismo: ela permite escrever regras de firewall entre os segmentos e **observar o efeito**

delas — que é justamente o que se quer aprender. Um laboratório com tudo numa única rede plana ensina metade.

14.2.2. A regra inegociável do isolamento

Um alvo deliberadamente vulnerável é uma máquina vulnerável. Se ela tiver rota para a internet, será comprometida — não hipoteticamente: sistemas como o Metasploitable são varridos e explorados em minutos quando expostos. E aí o laboratório deixa de ser laboratório e passa a ser um problema seu, do seu provedor e de terceiros.

Quatro regras que não têm exceção:

1. **Rede interna ou apenas hospedeiro**, nunca em ponte com a rede doméstica ou corporativa.
2. **Sem NAT no uso normal**. Precisa atualizar um alvo? Liga a saída, atualiza, desliga. Nunca deixa ligada.
3. **Alvos vulneráveis jamais expostos**, nem “só por cinco minutos para testar de fora”.
4. **Instantâneo antes de qualquer coisa interessante**, e restauração depois. É o que permite errar à vontade.

E uma regra que decorre do Capítulo 7: **o alvo é sempre seu**. O laboratório existe justamente para que a resposta à pergunta “eu tenho autorização?” seja trivialmente sim.

14.3. O que instalar, em que ordem

A tentação de montar tudo de uma vez leva ao abandono. A ordem abaixo entrega valor a cada etapa e cada uma leva algumas horas.

14.3.1. Etapa 1 — Hipervisor e a primeira máquina

Em máquina dedicada, **Proxmox VE** é a melhor escolha: gratuito, robusto, com interface web, instantâneos, contêineres LXC e rede virtual configurável. Em notebook de uso diário, **VirtualBox** ou **VMware Workstation** resolvem sem reinstalar nada.

A primeira VM deve ser uma distribuição Linux comum — Debian ou Ubuntu Server — e não uma distribuição de segurança. O objetivo é praticar o Capítulo 12 num sistema que se administra de verdade: montagens, **systemd**, **auditd**, SSH com chave, **nftables**.

14.3.2. Etapa 2 — Atacante e alvo, uma rede

Kali Linux ou **Parrot OS** de um lado, **Metasploitable 2** do outro, ambos em rede interna. Com apenas essas duas máquinas já se pratica varredura, enumeração, análise de serviço e captura de tráfego — a base dos Volumes 5 e 9.

14.3.3. Etapa 3 — Alvos web

OWASP Juice Shop e **DVWA**, ambos disponíveis como contêiner, cobrem praticamente todo o Volume 7. O Juice Shop é moderno e tem progressão em forma de desafios; o DVWA tem níveis de dificuldade que permitem ver a mesma falha com e sem correção — recurso didático que poucos alvos oferecem.

```
# Dois alvos web em minutos, na rede isolada do laboratório
docker run -d --name juice --network lab-alvos -p 3000:3000
↳ bkimminich/juice-shop
docker run -d --name dvwa --network lab-alvos -p 8080:80 vulnerables/web-dvwa
```

14.3.4. Etapa 4 — Firewall e segmentação

pfSense ou **OPNsense** como roteador entre os segmentos. É aqui que se aprende de verdade o que o Capítulo 8 e o Capítulo 9 descreveram: escrever regra, testar, ver o registro, entender por que o pacote não passou. É também a base do Volume 5.

14.3.5. Etapa 5 — Domínio Windows

Um **Windows Server** como controlador de domínio, com duas estações associadas. Exige 16 GB de RAM para ficar confortável, e é o que torna praticável o Capítulo 13: criar contas de serviço com SPN, observar Kerberoasting no registro de eventos, implantar LAPS, aplicar GPO e ver o efeito. As versões de avaliação são gratuitas por 180 dias e podem ser reinstaladas.

14.3.6. Etapa 6 — Defesa e visibilidade

O passo que a maioria pula, e o que mais ensina. **Wazuh** (ou uma pilha de coleta equivalente) recebendo registros das máquinas; **Suricata** ou **Zeek** observando o tráfego; **Security Onion** se a máquina aguentar, por trazer tudo pronto.

O exercício que fecha o ciclo é sempre o mesmo, e vale mais que qualquer leitura: **executar um ataque conhecido e ir procurar o rastro que ele deixou**. Quem já tentou detectar o próprio ataque entende, de uma vez, por que o Volume 12 existe.

14.4. Plataformas de treino: o complemento

O laboratório próprio ensina a **construir**; as plataformas de treino ensinam a **resolver**. Ambas são necessárias, e todas as citadas têm autorização explícita nos termos de uso:

- **TryHackMe** — trilhas guiadas, boa porta de entrada, com material em progressão.
- **Hack The Box** — máquinas sem orientação, mais próximas da realidade; a academia associada é estruturada.
- **PortSwigger Web Security Academy** — gratuita e a melhor referência prática para o Volume 7.
- **PicoCTF** e competições brasileiras — formato CTF, excelente para fundamentos.
- **Blue Team Labs / CyberDefenders** — o lado defensivo, com investigação de incidentes reais simulados.

A ordem que funciona: fundamentos em plataforma guiada, prática no laboratório próprio para entender a infraestrutura, e só então máquinas sem orientação.

14.5. Um roteiro de exercícios que percorre o volume

Fecha o capítulo — e o volume — um conjunto de exercícios que usa o laboratório para fixar o que os seis capítulos anteriores apresentaram. Cada um leva de uma a três horas.

Tabela 14.2.: Roteiro de exercícios do laboratório para o Volume 2.

Exercício	O que pratica	Capítulo
Capturar um handshake TCP completo e identificar cada flag	Camada 4 na prática	Capítulo 8
Executar ARP spoofing entre duas VMs e observar as tabelas dos dois lados	Camada 2 e MITM	Capítulo 8
Segmentar o laboratório em três VLANs e escrever as regras entre elas	Endereçamento e filtragem	Capítulo 9
Configurar DNS interno e observar o efeito de uma resposta forjada	Resolução e envenenamento	Capítulo 10
Publicar SPF, DKIM e DMARC num domínio de teste e verificar a rejeição	Autenticidade de e-mail	Capítulo 10
Encontrar todos os binários SUID e explicar por que cada um existe	Privilegio e escalada	Capítulo 11
Ligar <code>auditd</code> , executar comandos privilegiados e achá-los no registro	Registro e responsabilização	Capítulo 12
Criar conta de serviço com SPN e senha fraca; observar o pedido de TGS no evento 4769	Kerberos e detecção	Capítulo 13
Enviar todos os registros ao coletor e escrever um alerta que dispare	Visibilidade fim a fim	Volume 12

O último exercício é o mais importante, e por isso está no fim: **um ataque que você executou e não conseguiu detectar é a medida exata da lacuna de visibilidade** da sua operação. Essa constatação, feita no laboratório, custa uma tarde. Feita em produção, custa um incidente.

14.6. Brasil em Foco

O laboratório é a resposta à ausência de porto seguro legal. O Capítulo 7 registrou que o Brasil não tem salvaguarda explícita para pesquisa de segurança de boa-fé: quem reporta depende da decisão da vítima de não representar. Enquanto isso não muda, o ambiente próprio e

isolado é o único lugar onde a prática ofensiva não depende da boa vontade de terceiros. Não é limitação didática — é a diferença entre estudar e cometer o crime do art. 154-A do Código Penal.

Cuidado com o que entra no laboratório. Baixar amostra real de código malicioso para análise é atividade legítima de estudo, e o Volume 11 trata do procedimento. Duas cautelas valem desde já: a amostra pode conter dados pessoais de vítimas — cujo tratamento tem de ser justificado sob a LGPD (Brasil, 2018) — e o isolamento precisa ser real, porque uma amostra que escapa e ataca terceiros gera responsabilidade de quem a executou. Máquina virtual dedicada, sem rede, com instantâneo e descartada ao fim.

Alvo vulnerável exposto tem consequência para o provedor. Uma máquina do laboratório acessível pela internet e comprometida passa a atacar terceiros a partir do bloco do provedor. O resultado prático é notificação do CERT.br ao contato de abuso (CERT.br, 2024), entrada do bloco em listas de reputação e prejuízo para todos os assinantes daquele bloco. Para quem estuda a partir de uma conexão residencial, o contrato de serviço tipicamente veda uso que gere abuso — e a responsabilidade civil por dano a terceiro independe da cláusula contratual.

Licenças e as versões de avaliação. Windows Server e Windows cliente têm versões de avaliação legítimas e gratuitas por prazo determinado, e usá-las em laboratório está dentro dos termos. Recorrer a ativação irregular é violação de direito autoral (Lei 9.610/1998) e crime previsto no art. 184 do Código Penal — e ainda contamina o laboratório com binários de origem desconhecida, o que compromete qualquer conclusão obtida ali.

Documentar o laboratório também é prática profissional. Um documento simples com topologia, endereçamento, finalidade e a declaração de que os alvos são próprios e isolados serve a dois propósitos: organiza o estudo e, se algum tráfego for questionado pelo empregador ou pelo provedor, demonstra a natureza da atividade. É a versão pessoal do princípio de autorização documentada do Capítulo 7.

14.7. No Campo

1. Criar as redes isoladas antes de criar qualquer VM. No Proxmox, três pontes sem interface física associada:

```
# /etc/network/interfaces - pontes isoladas, sem porta física
auto vmbr10
iface vmbr10 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
# LAN-ATAQUE 10.10.10.0/24

auto vmbr20
iface vmbr20 inet manual
    bridge-ports none
# LAN-ALVOS 10.10.20.0/24

auto vmbr30
iface vmbr30 inet manual
    bridge-ports none
# LAN-DEFESA 10.10.30.0/24
```

```
# VirtualBox: rede interna nomeada, sem NAT e sem ponte
VBoxManage modifyvm "alvo-metasploitable" --nic1 intnet --intnet1 "lab-alvos"
VBoxManage modifyvm "kali" --nic1 intnet --intnet1 "lab-ataque"
```

2. Verificar o isolamento antes de ligar o primeiro alvo. Trinta segundos que evitam o problema inteiro:

```
# De dentro do alvo vulnerável: as três devem falhar
ping -c2 8.8.8.8
ping -c2 192.168.0.1          # gateway da rede doméstica
curl -m5 -sI https://example.com
```

3. Automatizar o instantâneo antes de cada sessão. O hábito que transforma o laboratório em ambiente de experimentação real:

```
# Proxmox: instantâneo com data em todas as VMs do laboratório
for id in 101 102 103 104; do
    qm snapshot $id "limpo-$(date +%Y%m%d)" --description "estado antes da
    ↪ sessão"
done
qm rollback 102 limpo-20260730      # voltar em segundos
```

4. Manter uma janela de atualização controlada. Os alvos precisam de correção eventual, e a saída não pode ficar aberta:

```
No pfSense, uma regra desabilitada e pronta:
LAN-ALVOS → any (allow) [desabilitada]
Habilitar, atualizar, desabilitar. Nunca deixar habilitada entre sessões.
```

5. Documentar o laboratório numa página. Topologia, endereçamento por segmento, finalidade, sistema e versão de cada VM, e a declaração de propriedade e isolamento. Atualizada a cada mudança relevante.

6. Praticar a restauração, não só o instantâneo. Instantâneo que nunca foi restaurado é backup que nunca foi testado — a mesma lição que o Volume 14 aplica à continuidade de negócio. Restaurar de propósito, uma vez, e cronometrar.

7. Fechar o ciclo em todo exercício. A regra que multiplica o valor do laboratório: para cada ataque executado, **procurar o rastro**. No registro do alvo, no sensor de rede, no coletor. O que não deixou rastro é lacuna de visibilidade identificada — e é a partir dessa lista que o Volume 12 faz sentido.

8. Registrar o que aprendeu no próprio laboratório. Um arquivo por exercício com o que foi feito, o comando exato, o resultado e o rastro encontrado. Além de fixar o aprendizado, é o embrião do relatório técnico que o Volume 9 vai exigir — e escrever bem é parte do ofício, não acessório.

14.8. Resumo

- **Contêiner para conveniência, máquina virtual para contenção.** O contêiner compartilha o núcleo com o hospedeiro e tem superfície de escape muito maior; código malicioso desconhecido só se executa em VM isolada, com instantâneo e sem rede.
- O laboratório mínimo cabe em **8 GB de RAM**; 16 GB tornam confortável e 32 GB permitem um domínio Windows completo. SSD importa mais que processador.
- A topologia base tem **três segmentos** — ataque, alvos e defesa — separados por um firewall. A separação é o que permite escrever regras e observar seu efeito; rede plana ensina metade.
- Quatro regras de isolamento sem exceção: **rede interna, nunca em ponte**; sem NAT no uso normal; alvos vulneráveis jamais expostos; instantâneo antes de qualquer coisa interessante. Um alvo deliberadamente vulnerável exposto é comprometido em minutos.
- A montagem funciona **em etapas**: hipervisor e Linux comum, atacante e alvo, alvos web em contêiner, firewall e segmentação, domínio Windows, e por fim coleta e detecção.
- **A etapa de defesa é a que a maioria pula e a que mais ensina.** O exercício que fecha o ciclo é executar um ataque conhecido e ir procurar o rastro: o que não deixou rastro é a medida exata da lacuna de visibilidade.
- Plataformas de treino complementam o laboratório próprio — ele ensina a construir, elas ensinam a resolver — e todas trazem a autorização nos termos de uso.
- No Brasil, onde **não há porto seguro legal para pesquisa de boa-fé**, o laboratório próprio é o único ambiente em que a prática ofensiva não depende de terceiros. Documentá-lo é a versão pessoal do princípio de autorização escrita.

Parte III.

Volume 3 — Criptografia Aplicada

15. História e princípios da criptografia

A máquina Enigma não foi quebrada porque alguém descobriu como ela funcionava. Os Aliados sabiam como ela funcionava desde 1932, quando matemáticos poloneses — Marian Rejewski à frente — reconstruíram a lógica interna a partir de material de inteligência e de análise pura. A máquina era conhecida, exemplares foram capturados, o esquema de rotores estava documentado. E mesmo assim as mensagens continuavam secretas, porque o segredo não estava na máquina: estava na configuração diária dos rotores, dos anéis e do painel de conexões.

O que abriu o tráfego alemão foi outra coisa. Foi o operador que repetia a chave de mensagem duas vezes no preâmbulo. Foi o posto meteorológico que transmitia todo dia, no mesmo horário, uma mensagem de estrutura previsível. Foi a unidade que, sem nada a relatar, mandava **Keine besonderen Ereignisse** — “sem ocorrências especiais” — e com isso entregava a Bletchley Park um trecho de texto claro conhecido para casar contra o texto cifrado. Foi a regra de projeto que impedia uma letra de ser cifrada nela mesma, o que eliminava candidatos aos milhares. A matemática do Enigma resistiu razoavelmente bem. O **uso** do Enigma foi o que ruiu.

Essa é a tese deste volume inteiro, e vale enunciá-la agora, antes de qualquer fórmula: **a criptografia moderna quase nunca falha pela matemática**. AES não foi quebrado. RSA com chave adequada não foi quebrado. SHA-256 não foi quebrado. O que se quebra, todos os dias, é o modo de operação escolhido sem entender o que ele garante, o vetor de inicialização reaproveitado, a chave no repositório do Git, o certificado que ninguém valida, o gerador de números aleatórios que não era aleatório. O Capítulo 21 fecha este volume catalogando exatamente esses erros — e nenhum deles é um problema de teoria dos números.

Este capítulo estabelece o vocabulário, os princípios e os limites que tornam os seis capítulos seguintes compreensíveis.

15.1. O vocabulário mínimo

Quatro objetos e duas operações sustentam tudo o que vem depois.

O **texto claro** (M , de *message*) é a informação em sua forma utilizável. O **texto cifrado** (C) é o resultado da transformação. A **chave** (K) é o parâmetro secreto que determina qual transformação, entre todas as possíveis, foi aplicada. O **algoritmo** (ou cifra) é a função pública que combina os três.

Cifrar é $C = E_K(M)$. Decifrar é $M = D_K(C)$. A propriedade que faz o sistema útil é trivial de enunciar e difícil de garantir: $D_K(E_K(M)) = M$ para quem tem K , e obter M a partir de C deve ser inviável para quem não tem.

Inviável é a palavra que carrega o peso. Não significa impossível — significa que o custo de conseguir supera o valor do que se obtém, por uma margem confortável e por tempo suficiente. Uma chave AES de 128 bits admite 2^{128} valores; percorrê-los não é impossível em teoria e é absolutamente impossível em prática, com qualquer energia disponível neste planeta. Já uma chave DES de 56 bits também era “inviável” em 1977 e hoje é quebrada por hardware acessível em horas. **Segurança criptográfica tem prazo de validade**, e reavaliá-la periodicamente é

15. História e princípios da criptografia

parte da operação — assunto que o NIST trata como transição de algoritmos (National Institute of Standards and Technology, 2019).

A Figura 15.1 fixa o modelo com que se raciocina sobre qualquer sistema criptográfico.

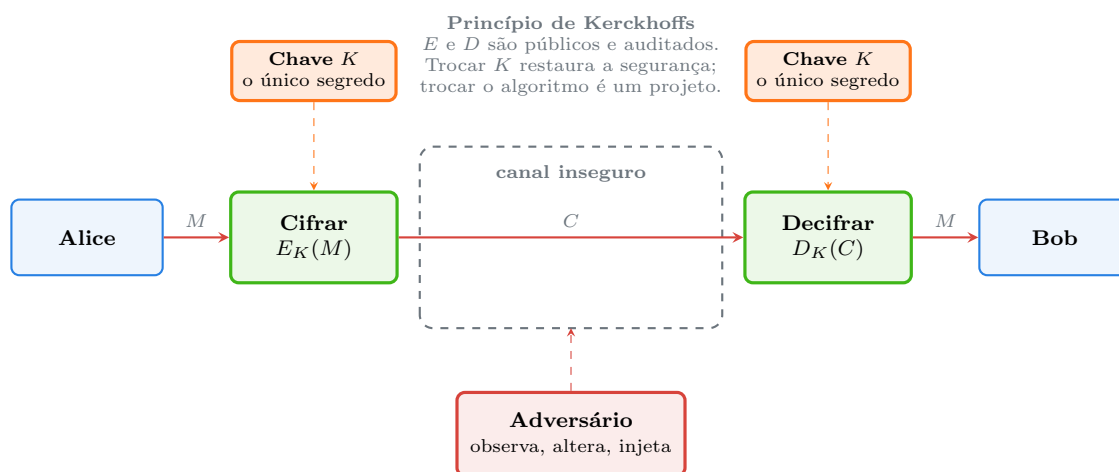


Figura 15.1.: Modelo geral de um sistema criptográfico. O algoritmo é público; a chave é o único segredo.

15.2. Da substituição à análise de frequência

A cifra de César desloca cada letra um número fixo de posições no alfabeto. Com deslocamento 3, ATAQUE vira DWDTXH. O espaço de chaves tem 25 valores úteis, e a cifra é quebrada por inspeção antes do café esfriar.

O interessante não é a fragilidade — é **por que** ela é frágil. A cifra de César é uma **substituição monoalfabética**: cada letra do texto claro corresponde sempre à mesma letra no texto cifrado. Isso preserva integralmente a estatística do idioma. Em português, A e E respondem juntas por mais de um quarto do texto; Q é quase sempre seguido de U; dígramas como DE, RA e ES são frequentíssimos. A substituição embaralha os rótulos e mantém intacta a estrutura por baixo. Contar frequências devolve o alfabeto.

Generalizar a substituição não salva. Uma substituição monoalfabética arbitrária tem $26!$ chaves — cerca de 4×10^{26} , espaço grande o bastante para tornar a força bruta irrelevante. E é resolvida em minutos por análise de frequência, porque o tamanho do espaço de chaves **não é** a medida da segurança. É apenas o piso: um espaço pequeno garante fracasso, um espaço grande não garante nada. A lição sobrevive intacta na criptografia moderna, onde uma chave RSA de 2048 bits oferece segurança comparável à de uma chave simétrica de 112 bits — porque os ataques contra RSA não são busca exaustiva.

A resposta histórica à análise de frequência foi a **substituição polialfabética**: usar vários alfabetos em rodízio. A cifra de Vigenère, do século XVI, faz isso com uma palavra-chave que determina o deslocamento de cada posição. Resistiu por trezentos anos e foi chamada de *le chiffre indéchiffrable*, até que Friedrich Kasiski publicou, em 1863, o método que a desmonta: repetições no texto cifrado revelam o comprimento da chave, e uma vez conhecido esse comprimento a cifra se decompõe em várias cifras de César, cada uma atacável por frequência.

Duas lições permanentes saem daí. A primeira: **repetição vaza informação**. Todo padrão que sobrevive à cifragem é uma alça para o atacante — é exatamente o que torna o modo ECB

inaceitável, como o Capítulo 16 mostra. A segunda: “ninguém quebrou em 300 anos” não é evidência de segurança, é evidência de que ninguém publicou. Confiança em cifra decorre de análise pública sustentada, não de silêncio.

15.3. O princípio de Kerckhoffs

Em 1883, Auguste Kerckhoffs enunciou seis requisitos para cifras militares (Kerckhoffs, 1883). O segundo atravessou o tempo e virou lei de projeto: **o sistema não deve exigir sigilo e deve poder cair nas mãos do inimigo sem inconveniente.**

Ou, na formulação de Shannon: projete supondo que o adversário conhece o sistema. Tudo, menos a chave.

O argumento é prático, não filosófico. Algoritmo secreto não recebe análise independente, e criptografia é uma disciplina em que a única evidência de solidez é a resistência a ataque de gente competente e motivada. Além disso, segredo de algoritmo **não escala e não se recupera**: uma vez vazado — por engenharia reversa, por funcionário ou por captura de equipamento — todo o sistema morre, e substituí-lo é um projeto de infraestrutura. Chave vazada se troca numa tarde.

A história é implacável com quem ignorou isso. A cifra A5/1 do GSM, mantida secreta, foi reconstruída por engenharia reversa e quebrada. O sistema de proteção de DVD, o CSS, caiu em dias após a análise do binário. O MIFARE Classic, usado em bilhetes de transporte no mundo inteiro — inclusive no Brasil —, embutia um algoritmo proprietário chamado Crypto-1 que sucumbiu assim que pesquisadores holandeses reconstruíram sua lógica. Em todos os casos o padrão se repete: o sigilo do algoritmo era o único obstáculo, e obstáculos únicos caem.

Convém separar duas coisas que se confundem. **Segurança por obscuridade como única defesa é falha de projeto**; obscuridade como camada adicional sobre uma base sólida é legítima e às vezes útil. Mover o SSH da porta 22 não é segurança — mas reduz ruído de varredura automatizada e, sobre uma configuração que já usa chave e nega senha, é um ganho operacional barato. O erro está em confundir redução de ruído com controle.

15.4. Shannon: sigilo perfeito e seu preço

Claude Shannon deu à criptografia sua primeira fundamentação matemática (Shannon, 1949) e provou que sigilo absoluto existe — sob condições que quase sempre inviabilizam o uso.

O **one-time pad** combina cada bit da mensagem com um bit de uma chave verdadeiramente aleatória, por XOR. Se a chave for aleatória, tiver o mesmo comprimento da mensagem e for usada **uma única vez**, o texto cifrado não revela absolutamente nada sobre o texto claro além do comprimento. A prova é elegante: para qualquer texto cifrado de n bits existe uma chave que o transforma em qualquer texto claro de n bits. ATACAR e RECUAR são igualmente compatíveis com o que o adversário vê. Não há o que computar; poder computacional infinito não ajuda.

E, no entanto, praticamente ninguém usa. As três condições são o obstáculo:

- **Chave do tamanho da mensagem.** Proteger 10 GB exige distribuir 10 GB de chave. Se existe canal seguro para transportar isso, ele poderia transportar a mensagem.
- **Aleatoriedade verdadeira.** Não pseudoaleatoriedade. Saída de gerador determinístico não serve — o pad herda a entropia da semente, não do seu comprimento.

- **Uso único.** Duas mensagens cifradas com o mesmo pad colapsam: o XOR dos dois textos cifrados é o XOR dos dois textos claros, e a chave desaparece da equação.

Essa terceira condição não é hipótese de sala de aula. O projeto Venona, conduzido pelos Estados Unidos entre 1943 e 1980, decifrou milhares de mensagens diplomáticas soviéticas porque, sob pressão de guerra, blocos de pad foram duplicados e reutilizados. A criptografia estava matematicamente perfeita; a operação, não. É o Enigma outra vez, com outro sotaque.

A criptografia moderna faz então uma troca deliberada: abandona o sigilo perfeito e busca **segurança computacional** — não é impossível quebrar, é inviável com os recursos que existem. Em troca, chaves curtas e fixas protegem volumes arbitrários de dados. É a troca certa, e é a razão de o resto deste volume existir.

15.5. Confusão, difusão e o que uma boa cifra faz

Shannon também nomeou as duas propriedades que uma cifra de bloco precisa entregar, e elas continuam sendo o critério de projeto do AES:

Confusão é tornar opaca a relação entre a chave e o texto cifrado. Cada bit do texto cifrado deve depender de vários bits da chave, de forma não linear. Sem confusão, observar pares de entrada e saída permite montar equações e resolver para a chave.

Difusão é espalhar a estrutura do texto claro por todo o texto cifrado. Mudar **um** bit da entrada deve alterar, em média, metade dos bits da saída — o **efeito avalanche**. Sem difusão, padrões do texto claro sobrevivem à cifragem e a análise estatística volta a funcionar, exatamente como nas cifras clássicas.

O efeito avalanche é fácil de verificar na prática, e vale fazer no laboratório do Capítulo 14: cifrar duas mensagens que diferem em um único caractere e comparar as saídas bit a bit. Numa cifra decente, o resultado é indistinguível de duas saídas independentes.

15.6. O que o adversário pode fazer

Dizer que uma cifra é “segura” sem dizer contra o quê é dizer pouco. A criptografia trabalha com modelos de ataque explícitos, ordenados por poder crescente:

Tabela 15.1.: Modelos de ataque em ordem crescente de poder do adversário.

Modelo	O que o adversário tem	Situação real correspondente
Somente texto cifrado (COA)	Textos cifrados interceptados	Captura passiva de tráfego
Texto claro conhecido (KPA)	Pares claro/cifrado	Cabeçalhos fixos, formatos previsíveis
Texto claro escolhido (CPA)	Pode cifrar o que quiser	Serviço que cifra dados enviados pelo usuário
Texto cifrado escolhido (CCA)	Pode submeter cifrados e observar o resultado	API que decifra e reage ao erro

O padrão moderno exige resistência a **CCA**, o modelo mais forte — e não por preciosismo acadêmico. Um servidor web que decifra o que o cliente envia e responde de maneira distinguível conforme o resultado é, literalmente, o oráculo do modelo CCA. Foi assim que o *padding oracle*

de Vaudenay (Vaudenay, 2002) derrubou implementações de SSL e IPsec, e é por isso que o Capítulo 16 insiste que cifrar sem autenticar é erro de projeto.

Também vale precisar o que significa “**quebrado**” no vocabulário da área, porque a manchete e o artigo raramente coincidem. Uma cifra é considerada academicamente quebrada quando existe qualquer ataque mais rápido que a busca exaustiva, ainda que absurdamente caro. Um ataque a AES-256 que custasse 2^{254} operações seria uma quebra — e não teria consequência prática nenhuma. Já colisões em MD5 saíram do teórico para “segundos num notebook”, e isso tem consequência todo dia. **A pergunta útil não é “foi quebrado?”, é “o ataque é praticável e o meu uso depende da propriedade atacada?”** — o Capítulo 17 mostra por que MD5 é catastrófico para assinatura e irrelevante para detectar corrupção de download.

15.7. As três famílias de primitivas

A criptografia aplicada se organiza em três famílias, e o restante do volume percorre uma a uma. A Figura 15.2 resume o que cada uma entrega em relação aos pilares do Capítulo 2 e do Capítulo 3.

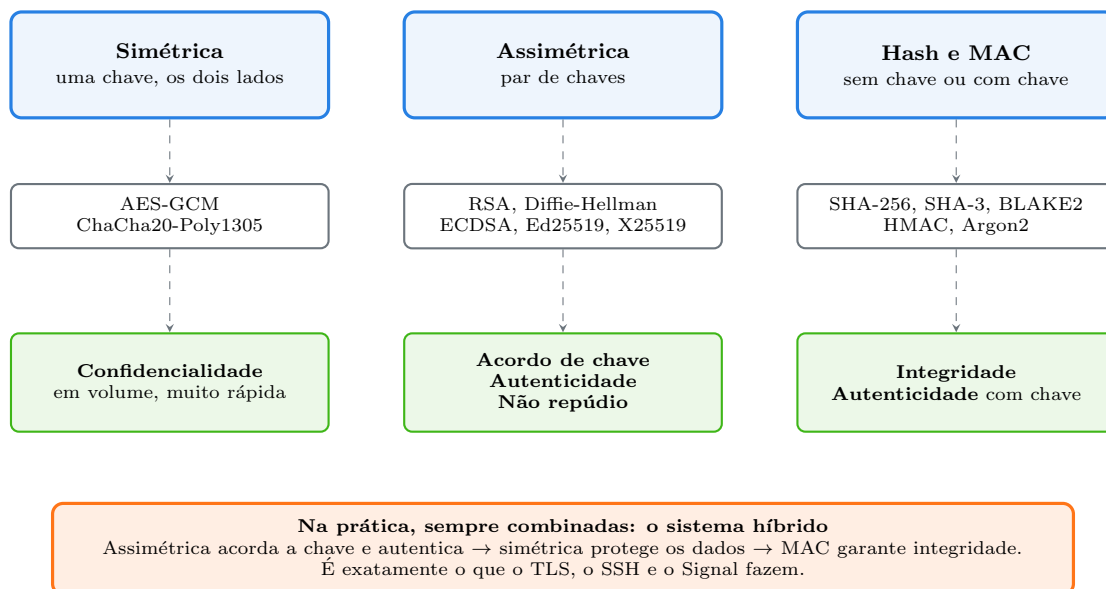


Figura 15.2.: As três famílias de primitivas, o que cada uma entrega e por que nenhuma é usada sozinha.

O ponto central da figura merece destaque: **nenhuma família é usada isolada em sistema real**. A criptografia assimétrica é ordens de grandeza mais lenta que a simétrica e não serve para volume; a simétrica não resolve a distribuição de chave entre partes que nunca se encontraram; nenhuma das duas, sozinha, garante integridade. A **construção híbrida** — assimétrica para acordar a chave e autenticar, simétrica para os dados, MAC para a integridade — é a arquitetura de todo protocolo seguro em uso, do TLS ao SSH, do Signal ao Pix.

15.8. Aleatoriedade: a matéria-prima

Toda chave é, na origem, um número aleatório. Se esse número for previsível, nada do que vem depois importa — e este é, disparadamente, o modo mais comum de um sistema criptograficamente

correto no papel falhar na prática.

A distinção que precisa estar clara é entre dois tipos de gerador. O **PRNG comum** (`rand()` em C, `Math.random()` em JavaScript, `random.random()` em Python) é otimizado para velocidade e boa distribuição estatística, e é **completamente previsível** para quem observa saídas suficientes: o estado interno é reconstruível. O **CSPRNG** é projetado para que, mesmo conhecendo saídas anteriores, seja inviável prever a próxima ou recuperar o estado. No sistema operacional, é `getrandom()` e `/dev/urandom` no Linux, e `BCryptGenRandom` no Windows — alimentados pela entropia física coletada pelo núcleo, como o Capítulo 11 descreveu. As construções recomendadas estão em (National Institute of Standards and Technology, 2015a).

A regra operacional é curta: **para qualquer coisa criptográfica, use a API criptográfica do sistema ou da biblioteca**, nunca o gerador de propósito geral da linguagem. `secrets` em vez de `random` no Python; `crypto.randomBytes` em vez de `Math.random` no Node; `SecureRandom` no Java; `getrandom()` em C.

O caso que melhor ilustra o custo é o do Debian em 2008: uma alteração bem-intencionada removeu do OpenSSL a linha que alimentava o *pool* de entropia, e o gerador passou a depender essencialmente do identificador do processo. O resultado foi um espaço de apenas 32.767 chaves possíveis. Durante quase dois anos, todas as chaves SSH e TLS geradas em sistemas Debian e derivados eram enumeráveis em minutos. Nenhum algoritmo foi quebrado. A entrada estava errada, e isso bastou.

15.9. O que a criptografia não resolve

Fechar o capítulo delimitando o alcance evita a expectativa que produz decisões ruins.

Criptografia não resolve autorização. Cifrar um banco de dados não impede que a aplicação, com credencial legítima, devolva o registro de outro usuário. O dado é decifrado exatamente como deveria; quem errou foi o controle de acesso, tema do Volume 4.

Criptografia em repouso não protege dado em uso. Disco cifrado protege contra roubo do disco e descarte inadequado do hardware. Com o sistema ligado e o volume montado, o dado está claro para todo processo autorizado — e para o invasor que obteve execução. A frase “o banco é criptografado” só faz sentido acompanhada de “em repouso, contra este cenário de ameaça”.

Criptografia não esconde metadados. O conteúdo de uma conexão TLS é opaco; quem falou com quem, quando, por quanto tempo e com que volume, não é. Análise de tráfego funciona sobre canais perfeitamente cifrados, e o Volume 16 mostra o quanto ela revela.

Criptografia não substitui a validação de identidade. Um canal cifrado com quem não se verificou é um canal cifrado com o atacante. Diffie-Hellman sem autenticação estabelece uma chave excelente — com quem estiver no meio. É por isso que o Capítulo 19 existe, e o Capítulo 18 chega ao mesmo ponto por outro caminho.

Criptografia mal aplicada dá pior resultado que a ausência dela, porque cria a convicção de que o problema foi resolvido. Um campo em Base64 chamado `encrypted_token` já produziu mais dano do que o mesmo campo em texto claro produziria — o segundo teria sido revisado.

15.10. Brasil em Foco

Usar criptografia forte é legal, e o Brasil nunca exigiu porta dos fundos. Não há, no ordenamento brasileiro, restrição ao uso de criptografia forte por cidadãos ou empresas, nem obrigação de depósito de chaves. Os bloqueios judiciais do WhatsApp entre 2015 e 2016 — motivados pela impossibilidade técnica de entregar conteúdo protegido por criptografia de ponta a ponta — foram derrubados em instâncias superiores, e o tema tramita no Supremo Tribunal Federal na ADI 5527 e na ADPF 403. O argumento técnico que sustentou a defesa é o mesmo do Capítulo 3: em sistema com sigilo encaminhado, **o provedor não detém as chaves**, e ordem judicial não cria capacidade técnica que não existe.

A LGPD trata criptografia como salvaguarda com efeito jurídico concreto. O art. 46 exige medidas técnicas aptas a proteger dados pessoais, e o art. 48 obriga a comunicação de incidente que possa acarretar risco relevante. A conexão entre os dois é o que importa na prática: dado adequadamente cifrado, com a chave comprovadamente não comprometida, tende a reduzir — e em alguns cenários a afastar — a caracterização de risco relevante ao titular. A Resolução CD/ANPD 15/2024 estruturou o procedimento de comunicação, e a existência de salvaguarda criptográfica documentada pesa na avaliação. Isso transforma registro de configuração, de gestão de chaves e de escopo de cifragem em ativo jurídico, não só técnico (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

Cuidado com “hash é anonimização” — não é. A LGPD distingue dado anonimizado, fora do escopo da lei, de dado pseudonimizado, dentro dela. Aplicar SHA-256 a um CPF **não** anonimiza: o espaço de CPFs válidos tem cerca de $1,1 \times 10^{10}$ elementos, e percorrê-lo inteiro em hardware comum leva minutos. A tabela reversa completa é construível por qualquer pessoa. O Capítulo 17 detalha por que hash de domínio pequeno não protege nada, e o equívoco já apareceu em incidentes brasileiros noticiados como envolvendo “dados anonimizados”.

A assinatura digital tem base legal específica e uma infraestrutura estatal. A MP 2.200-2/2001 instituiu a ICP-Brasil e conferiu presunção de veracidade às assinaturas feitas com certificado emitido em sua cadeia. A Lei 14.063/2020 escalonou as assinaturas eletrônicas em simples, avançada e qualificada — esta última exigindo certificado ICP-Brasil — e definiu para quais atos cada nível serve. É a razão de o e-CNPJ ser obrigatório para NF-e, de o e-CPF aparecer em peticionamento eletrônico e de o gov.br ter níveis de conta com valor jurídico distinto. O Capítulo 18 trata da matemática e o Capítulo 19 da cadeia de confiança que sustenta tudo isso (Brasil, 2001, 2020).

No setor financeiro, criptografia é requisito regulatório explícito. A Resolução BCB 4.893/2021 estabelece a política de segurança cibernética das instituições autorizadas, e o arranjo do Pix impõe autenticação mútua com certificado em toda comunicação com o SPI. Não é boa prática opcional: é condição de participação, verificada em homologação. O Capítulo 20 mostra a configuração de TLS mútuo que materializa essa exigência (Banco Central do Brasil, 2021).

15.11. No Campo

1. Nunca implemente primitiva criptográfica. A regra que mais economiza incidentes. Use `libsodium`, a biblioteca padrão da plataforma ou implementações consagradas e auditadas. Implementação própria falha em canal lateral, em tempo de execução variável e em tratamento de erro — nada disso aparece nos testes de unidade, e tudo isso aparece no ataque.

2. Saiba responder três perguntas sobre qualquer sistema que use criptografia. Qual algoritmo e modo? De onde vem a chave e onde ela vive? O que garante que a mensagem não

15. História e princípios da criptografia

foi alterada? Quem não responde às três não tem sistema criptográfico — tem uma chamada de biblioteca.

3. Inventarie os algoritmos em uso antes de discutir melhorias. Não se gere o que não se conhece:

```
# Certificados: algoritmo de assinatura e tamanho de chave
find /etc/ssl /etc/pki -name '*.crt' -o -name '*.pem' 2>/dev/null | while read
↪ -r c; do
    echo "== $c"
    openssl x509 -in "$c" -noout -subject -dates 2>/dev/null
    openssl x509 -in "$c" -noout -text 2>/dev/null | grep -E 'Signature
↪ Algorithm|Public-Key'
done

# Chaves SSH ainda autorizadas, por tipo
awk '{print $1}' ~/.ssh/authorized_keys 2>/dev/null | sort | uniq -c
```

4. Verifique o efeito avalanche com as próprias mãos. Exercício de dez minutos que ancora a intuição de difusão:

```
printf 'ataque ao amanhecer' | sha256sum
printf 'ataque ao amanhecel' | sha256sum # um caractere de diferença na
↪ entrada
# As saídas não guardam relação visível alguma.
```

5. Trate origem de aleatoriedade como item de revisão de código. Procurar geradores de propósito geral onde deveria haver CSPRNG é uma varredura barata e de retorno alto:

```
grep -rnE 'Math\.random|new Random\(|\[[^s]rand\(|srand\(|random\.random\(' \
--include='*.js' --include='*.java' --include='*.py' --include='*.c' . \
| grep -viE 'test|spec|mock|benchmark'
```

Cada ocorrência num caminho que gere chave, token, identificador de sessão ou senha temporária é achado de severidade alta.

6. Documente a validade dos algoritmos, não só a configuração. Uma tabela com algoritmo, uso, tamanho de chave, data de adoção e data prevista de reavaliação. É o que transforma “estamos seguros” em afirmação verificável, e o que evita descobrir SHA-1 em produção durante uma auditoria. As tabelas de transição do NIST (National Institute of Standards and Technology, 2019) e as recomendações de gestão de chaves (National Institute of Standards and Technology, 2020a) servem de referência.

7. Rejeite algoritmo proprietário sem análise pública. Fornecedor que descreve a criptografia como “algoritmo proprietário de alta segurança” e se recusa a detalhar está descrevendo o oposto do princípio de Kerckhoffs. As perguntas certas em processo de aquisição: qual algoritmo padrão, qual modo, qual tamanho de chave e qual auditoria independente já passou.

8. Separe cifragem de gestão de chaves no desenho e no orçamento. Escolher AES-256-GCM leva cinco minutos; decidir onde a chave nasce, quem a acessa, como ela roda e o que acontece quando vaza é o trabalho de verdade. O Volume 8 trata de cofres e KMS, mas a decisão de arquitetura começa aqui — e chave em variável de ambiente de contêiner não é solução, é adiamento.

15.12. Resumo

- **A criptografia moderna raramente falha pela matemática.** O Enigma caiu por erro operacional, o Venona por reuso de pad, o Debian por entropia removida. Algoritmo é a parte fácil; uso e gestão de chave são onde os sistemas quebram.
- **O princípio de Kerckhoffs** é lei de projeto: algoritmo público e auditado, chave como único segredo. Segredo de algoritmo não escala, não se recupera e nunca recebeu análise independente.
- **Espaço de chaves grande não é sinônimo de segurança.** A substituição monoalfabética tem $26!$ chaves e cai por análise de frequência. O tamanho da chave é piso, não medida.
- O **one-time pad** oferece sigilo perfeito sob três condições — chave aleatória, do tamanho da mensagem e usada uma única vez — que na prática o inviabilizam. A criptografia moderna troca sigilo perfeito por **segurança computacional**, e é a troca certa.
- **Confusão e difusão** são os critérios de projeto de uma cifra de bloco; o efeito avalanche é a difusão observável, e verificável em dois comandos.
- O padrão atual exige resistência ao modelo **CCA**, porque servidores que decifram e reagem a erro são, literalmente, oráculos de decifragem. “Quebrado” na academia e “explorável” na prática são coisas diferentes: a pergunta útil é se o ataque é praticável e se o seu uso depende da propriedade atacada.
- As três famílias — **simétrica, assimétrica e hash/MAC** — nunca aparecem sozinhas em sistema real. Toda arquitetura séria é **híbrida**, e o TLS é o exemplo canônico.
- **Aleatoriedade previsível derruba tudo o que vem depois.** CSPRNG do sistema, sempre; gerador de propósito geral da linguagem, nunca, em caminho criptográfico.
- Criptografia **não** resolve autorização, não protege dado em uso, não esconde metadados e não substitui validação de identidade. Mal aplicada, é pior que ausente, porque encerra a discussão.
- No Brasil, o uso de criptografia forte é livre, a LGPD a trata como salvaguarda com efeito jurídico concreto e a assinatura digital tem infraestrutura estatal própria na ICP-Brasil. **Hash de CPF não é anonimização.**

16. Criptografia simétrica: AES, modos de operação e AEAD

Em outubro de 2013, a Adobe sofreu um vazamento que expôs cerca de 153 milhões de registros de conta. As senhas não estavam em texto claro — estavam cifradas com 3DES, um algoritmo respeitável. E, ainda assim, comunidades da internet montaram em poucos dias um passatempo chamado *the Adobe crossword*: uma grade de palavras cruzadas em que as “definições” eram as dicas de senha dos próprios usuários e as “respostas” eram as senhas, deduzidas coletivamente.

O motivo é que a Adobe usou o modo **ECB**. Nesse modo, blocos idênticos de texto claro produzem blocos idênticos de texto cifrado. Toda conta cuja senha era 123456 tinha exatamente o mesmo texto cifrado — e havia dezenas de milhares delas. Bastou olhar para o texto cifrado mais frequente e, com uma única dica de senha vazada junto, todo o grupo caiu de uma vez. A partir daí a grade se preencheu sozinha, senha após senha, sem que ninguém tivesse quebrado o 3DES nem descoberto a chave.

O algoritmo estava correto. A chave nunca foi revelada. E o vazamento foi total, porque a escolha do **modo de operação** destruiu a segurança que o algoritmo entregava. Esse capítulo trata de por que essa escolha existe, o que cada modo garante, e por que a resposta moderna é não escolher nada disso à mão — é usar AEAD.

16.1. O que uma cifra de bloco faz — e o que ela não faz

Uma **cifra de bloco** é uma função que transforma um bloco de tamanho fixo de texto claro em um bloco do mesmo tamanho de texto cifrado, sob controle de uma chave. Nada mais. O AES opera sobre blocos de **128 bits** (16 bytes), com chaves de 128, 192 ou 256 bits (National Institute of Standards and Technology, 2001).

Dizer isso em voz alta expõe o problema imediato: mensagens reais não têm 16 bytes. Um arquivo tem megabytes; um campo de banco tem tamanho variável; um pacote tem cabeçalho e corpo. **A cifra de bloco, sozinha, não sabe cifrar nada além de 16 bytes.** O que preenche essa lacuna é o **modo de operação** — o protocolo que decide como quebrar a mensagem em blocos, como encadeá-los e o que fazer com o resto que não completa um bloco.

Essa separação é a fonte de quase todos os desastres práticos com criptografia simétrica. O AES é uma primitiva excelente e o modo é onde se erra. E o erro não faz barulho: em ECB o dado *parece* cifrado, o teste de integração passa, o campo no banco vira uma sopa de bytes ilegíveis. Só que a estrutura sobreviveu — e estrutura é tudo o que o atacante precisa, como o Capítulo 15 mostrou a propósito da análise de frequência.

16.1.1. Do DES ao AES, em um parágrafo

O **DES**, adotado em 1977, usava chave de 56 bits — suficiente na época, quebrável por força bruta em horas hoje. A resposta emergencial foi o **3DES**, que aplica DES três vezes com chaves distintas; funciona, é lento, tem bloco de apenas 64 bits e está formalmente aposentado pelo

NIST desde 2001 (National Institute of Standards and Technology, 2019). O substituto veio de uma competição pública aberta em 1997, ganha pelo algoritmo belga **Rijndael**, padronizado como **AES** em 2001. O processo importa tanto quanto o resultado: cinco anos de análise pública e adversarial, com todos os candidatos abertos, é precisamente o princípio de Kerckhoffs aplicado à escolha de padrão.

Internamente, o AES é uma **rede de substituição e permutação**: cada rodada aplica substituição de bytes (a *S-box*, que entrega confusão), deslocamento de linhas e mistura de colunas (que entregam difusão), e soma da subchave da rodada. São 10 rodadas para chave de 128 bits e 14 para 256 bits. Após mais de duas décadas de análise mundial intensa, **não há ataque prático contra o AES completo**. Os ataques que existem são contra implementações — canais laterais de temporização e de cache —, e a resposta a eles é a instrução **AES-NI** do processador, que executa as rodadas em hardware, em tempo constante e uma ordem de grandeza mais rápido.

AES-128 ou AES-256? AES-128 é considerado seguro para praticamente todo uso civil, e é mais rápido. AES-256 custa cerca de 40% mais e oferece margem contra avanços futuros — inclusive a redução quadrática que o algoritmo de Grover traria num computador quântico relevante, tema do Volume 16. Na prática, escolha AES-256 quando houver exigência regulatória ou dado de vida longa; escolha AES-128 quando desempenho importar. Nenhuma das duas é o elo fraco do seu sistema.

16.2. Modos de operação: onde se decide o destino

A Figura 16.1 compara as três construções fundamentais.

16.2.1. ECB: o modo que não deve existir

Electronic Codebook cifra cada bloco de forma independente. É o comportamento ingênuo — e é errado por uma razão simples: blocos de texto claro iguais produzem blocos de texto cifrado iguais. Qualquer estrutura repetida na entrada aparece na saída.

O exemplo canônico do ensino de criptografia é uma imagem *bitmap* cifrada em ECB: os contornos da figura permanecem perfeitamente visíveis, porque regiões de cor uniforme viram blocos repetidos. O caso da Adobe é a mesma falha aplicada a senhas.

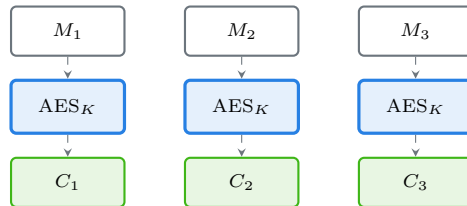
A regra é absoluta: **ECB não tem uso legítimo em sistema real**. Se você o encontrar em código de produção, é um achado, não uma escolha de projeto.

16.2.2. CBC: encadeamento e suas armadilhas

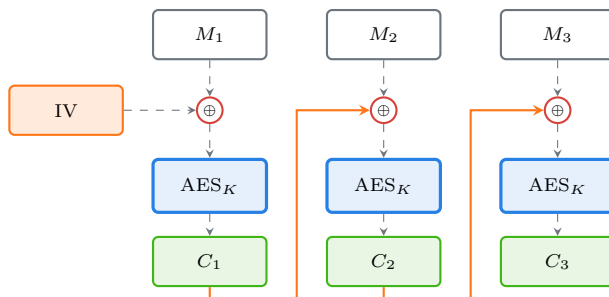
Cipher Block Chaining resolve o problema do ECB combinando, por XOR, cada bloco de texto claro com o texto cifrado do bloco anterior antes de cifrar. O primeiro bloco usa um **vetor de inicialização** (IV) no lugar do bloco anterior. Como o encadeamento propaga qualquer diferença, dois textos claros idênticos cifrados com IVs distintos produzem textos cifrados completamente distintos.

Três exigências acompanham o CBC, e cada uma já derrubou sistemas reais:

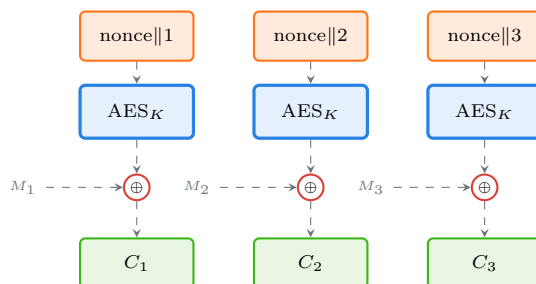
O IV precisa ser imprevisível, não apenas único. IV previsível — um contador, ou o último bloco cifrado da mensagem anterior — permite ao atacante confirmar palpites sobre o texto claro num cenário de texto claro escolhido. Foi exatamente essa a base do ataque BEAST

ECB — blocos independentes

$M_i = M_j \Rightarrow C_i = C_j$. O padrão do texto claro atravessa a cifração intacto. **Nunca usar.**

CBC — encadeado pelo bloco anterior

Sequencial na cifração, paralelizável na decifração. Exige preenchimento. IV precisa ser **imprevisível**. Sem MAC, é vulnerável a oráculo de preenchimento.

CTR — cifra de fluxo a partir de uma cifra de bloco

Totalmente paralelizável, sem preenchimento. Reusar o par (chave, nonce) é **catastrófico**: $C_i \oplus C_j = M_i \oplus M_j$ e a chave sai da equação.

Figura 16.1.: Os três modos fundamentais. O que muda entre eles não é o AES, é o que se faz com ele.

contra o TLS 1.0, que o Capítulo 20 detalha. O IV não é secreto e viaja junto do texto cifrado, mas tem de vir de CSPRNG.

O preenchimento cria um oráculo. Como CBC exige múltiplos exatos do bloco, a mensagem é completada com preenchimento (tipicamente PKCS#7). Se o sistema distinguir, em qualquer canal observável — mensagem de erro, código HTTP, tempo de resposta —, “preenchimento inválido” de “preenchimento válido mas conteúdo inválido”, o atacante decifra a mensagem inteira **sem jamais conhecer a chave**, submetendo textos cifrados modificados e observando as respostas. É o *padding oracle* de Vaudenay (Vaudenay, 2002), e ele reaparece a cada década com nome novo.

CBC não protege integridade. Este é o ponto mais importante e o mais ignorado, e ele merece seção própria.

16.2.3. CTR: a cifra de bloco virando cifra de fluxo

Counter Mode não cifra o texto claro. Cifra um contador — a concatenação de um *nonce* com um número de bloco — e usa o resultado como fluxo de chave, que é combinado por XOR com o texto claro. É elegante: dispensa preenchimento, é totalmente paralelizável, permite acesso aleatório (decifrar o bloco 5000 sem tocar nos anteriores) e usa apenas a função de cifragem do AES.

O preço é uma regra sem exceção: **o par (chave, nonce) nunca pode se repetir**. Reusar o nonce com a mesma chave gera o mesmo fluxo de chave, e dois textos cifrados sob o mesmo fluxo entregam o XOR dos textos claros — a falha do one-time pad reutilizado, do Capítulo 15, aplicada a um sistema moderno. Não é degradação gradual: é colapso.

Esse mesmo defeito, em outra roupagem, foi o que arruinou o **WEP** nas redes sem fio. O WEP usava RC4 com um IV de apenas 24 bits, concatenado à chave. Em rede movimentada, IVs se repetem em horas — e a repetição, somada a fraquezas do RC4, permite recuperar a chave. O Volume 5 volta ao tema; o que importa aqui é a causa: espaço de nonce pequeno demais para o volume de tráfego.

16.2.4. Cifras de fluxo: RC4 morreu, ChaCha20 é o presente

Cifras de fluxo geram diretamente um fluxo de chave, sem passar por blocos. O **RC4** dominou por duas décadas — em WEP, em WPA-TKIP, em TLS — e está proibido desde 2015 (RFC 7465): seus primeiros bytes de saída são estatisticamente enviesados, o que permitiu recuperar cookies de sessão em ataques práticos.

O sucessor é o **ChaCha20**, de Daniel Bernstein (Nir; Langley, 2018). É rápido em software puro, executa naturalmente em tempo constante e não depende de instrução dedicada do processador — o que o torna preferível ao AES em dispositivos sem **AES-NI**, especialmente celulares e sistemas embarcados. É por isso que navegadores móveis frequentemente negociam ChaCha20-Poly1305 em vez de AES-GCM: no hardware deles, é mais rápido e mais seguro contra canal lateral.

16.3. Cifrar não é autenticar

Aqui está o conceito que separa quem entende criptografia aplicada de quem apenas a invoca.

Confidencialidade e integridade são propriedades independentes. Nenhum dos modos anteriores — ECB, CBC ou CTR — impede que um atacante **altere** o texto cifrado. Ele não

conseguirá ler o conteúdo, mas conseguirá modificá-lo de maneira controlada, e o destinatário decifrá-lo silenciosamente o resultado adulterado como se fosse legítimo. Essa propriedade tem nome: **maleabilidade**.

Em CTR é gritante. O texto cifrado é o texto claro combinado por XOR com o fluxo de chave. Inverter o bit n do texto cifrado inverte exatamente o bit n do texto claro decifrado. Um atacante que saiba que os bytes 12 a 16 contêm o valor de uma transferência pode alterá-los sem conhecer nada da chave, e o resultado decifra normalmente. Em CBC o efeito é menos direto, mas igualmente explorável: alterar um bloco de texto cifrado corrompe o bloco correspondente e produz uma mudança **controlada** no bloco seguinte.

A consequência prática:

Texto cifrado que chega sem verificação de integridade não deve ser decifrado. A decifragem de dado não autenticado é o que transforma um canal cifrado em superfície de ataque.

A solução clássica foi acoplar um MAC — tipicamente HMAC, que o Capítulo 17 detalha. E aí surgiu mais uma armadilha: **a ordem importa**.

Tabela 16.1.: Ordem de cifragem e autenticação. Só a primeira é seguramente correta.

Construção	Como funciona	Veredito
Encrypt-then-MAC	Cifra, depois calcula o MAC sobre o texto cifrado	Correta. Verifica antes de decifrar
MAC-then-Encrypt	Calcula o MAC sobre o claro, depois cifra tudo	Frágil. Foi o que expôs o TLS ao padding oracle
Encrypt-and-MAC	Cifra e calcula o MAC sobre o claro, em paralelo	Vaza informação; o MAC do claro é determinístico

A razão de **Encrypt-then-MAC** ser a correta é operacional: ela permite verificar o MAC **antes** de tocar no texto cifrado. Se a verificação falhar, descarta-se a mensagem sem nunca executar a decifragem — e sem preenchimento processado não há oráculo de preenchimento.

Combinar as duas primitivas à mão exige acertar a ordem, usar chaves distintas para cifragem e MAC, e comparar o MAC em tempo constante. São três oportunidades de errar, e a indústria errou nas três. Daí a resposta moderna.

16.4. AEAD: a resposta certa

AEAD — *Authenticated Encryption with Associated Data* — é uma primitiva única que entrega confidencialidade e integridade em uma só operação, com uma só chave e uma só API. Não há ordem para escolher errado, porque não há duas operações a ordenar.

A Figura 16.2 mostra a interface.

Os **dados associados** (A) são a parte menos óbvia e a mais útil. São informações que precisam ser autenticadas mas não cifradas — porque um intermediário precisa lê-las. O cabeçalho de um pacote, o identificador da versão do registro, o número da conta em uma mensagem de roteamento. Ao incluí-los como dados associados, altera-se a etiqueta se alguém os modificar, ainda que permaneçam visíveis. É o mecanismo correto para **amarrar o texto cifrado ao seu contexto** — sem ele, um texto cifrado válido pode ser transplantado para outro registro, outro usuário ou outra conta e continuar decifrando perfeitamente.

16. Criptografia simétrica: AES, modos de operação e AEAD

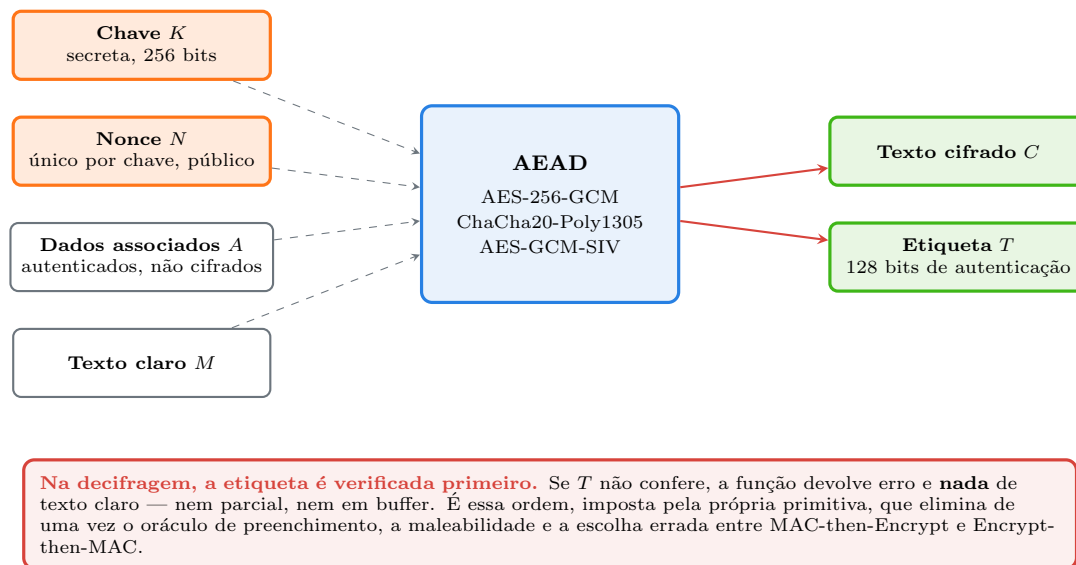


Figura 16.2.: Interface AEAD: quatro entradas, duas saídas, uma decisão a menos para o desenvolvedor errar.

16.4.1. As três construções que importam

AES-GCM é o padrão de fato (National Institute of Standards and Technology, 2007): CTR para a cifragem, mais um MAC baseado em multiplicação no campo de Galois. Muito rápido com AES-NI, e obrigatório no TLS 1.3. Herda do CTR a exigência de nonce único — com um agravante que merece destaque: **em GCM, o reúso de nonce não só revela texto claro como permite recuperar a chave de autenticação**, e a partir daí o atacante forja mensagens válidas à vontade. É a falha mais grave que se pode cometer com GCM.

ChaCha20-Poly1305 (Nir; Langley, 2018) combina a cifra de fluxo ChaCha20 com o autenticador Poly1305. Rápido sem hardware dedicado, naturalmente resistente a canal lateral de temporização. É o parceiro do AES-GCM no TLS 1.3 e a escolha preferencial em dispositivos móveis e embarcados.

AES-GCM-SIV é a rede de segurança. Usa síntese do nonce e do conteúdo para derivar os parâmetros internos, de modo que o reúso de nonce degrada a segurança em vez de destruí-la: o vazamento se limita a revelar que duas mensagens idênticas foram cifradas duas vezes. Custa algum desempenho e uma passada extra sobre os dados. Vale a pena sempre que a unicidade do nonce não puder ser garantida com confiança — sistemas distribuídos sem coordenação, dispositivos que podem ser restaurados de instantâneo, ou qualquer lugar onde um contador possa voltar atrás.

16.4.2. A regra do nonce, e os limites por chave

Nonce em AEAD é o ponto de falha mais frequente, e há três estratégias válidas para gerá-lo:

1. **Contador**, quando existe um único emissor com estado durável. Simples e exato, mas quebra se o estado for restaurado de um instantâneo ou se houver mais de um emissor.
2. **Aleatório de CSPRNG**, com nonce de 96 bits. Pelo paradoxo do aniversário, a probabilidade de colisão só se torna preocupante após cerca de 2^{32} mensagens com a mesma chave — daí o limite prático de **até 2^{32} mensagens por chave** para AES-GCM com nonce aleatório.

3. **Híbrido:** prefixo fixo por instância (identificador do processo ou da máquina) somado a um contador local. É a construção que melhor sobrevive a sistemas distribuídos.

Dois limites adicionais valem estar documentados: AES-GCM não deve cifrar mais de cerca de **64 GB sob a mesma chave e o mesmo nonce**, e o volume total por chave deve ser bem inferior a 2^{64} blocos. Na prática isso significa que **chaves de longa duração precisam de rotação**, e que a derivação de subchaves por sessão — o que o TLS 1.3 faz com HKDF, tema do Capítulo 17 — não é sofisticação, é exigência.

16.5. Como escolher, em uma tabela

Tabela 16.2.: Escolha de cifra simétrica por cenário.

Situação	Escolha	Motivo
Uso geral, servidor com AES-NI	AES-256-GCM	Padrão, acelerado em hardware, ubíquo
Móvel, embarcado, sem AES-NI	ChaCha20-Poly1305	Mais rápido em software, tempo constante
Nonce difícil de garantir único	AES-GCM-SIV	Degrada em vez de colapsar
API de alto nível disponível	libsodium (crypto_secretbox)	Escolhe tudo por você, e escolhe certo
Legado com CBC que não dá para trocar já	CBC + HMAC, encrypt-then-MAC	Correto se e somente se a ordem e as chaves separadas estiverem certas
Qualquer situação	Nunca ECB. Nunca RC4. Nunca DES ou 3DES.	Sem exceção

A linha mais importante é a quarta. Quando existe uma API de alto nível que não expõe modo nem IV nem nonce — como `crypto_secretbox` do `libsodium` ou o `Fernet` da biblioteca `cryptography` do Python —, ela é a escolha certa, porque remove do desenvolvedor toda decisão em que ele poderia errar. Interface que não permite errar vale mais que documentação que explica como não errar.

16.6. Brasil em Foco

A LGPD não nomeia algoritmos, e isso tem uma consequência prática. O art. 46 exige medidas “aptas a proteger” os dados pessoais, sem listar cifras. Na avaliação de um incidente, a ANPD examina se a medida era adequada ao **estado da técnica** e ao risco. Isso quer dizer que usar 3DES ou AES em modo ECB em 2026 não é apenas ruim tecnicamente — é indefensável administrativamente, porque o padrão de mercado está documentado em normas públicas há mais de uma década. A defesa técnica em processo administrativo se apoia em referência normativa: NIST SP 800-38D, ISO/IEC 27002, PCI-DSS. Guardar essa correspondência por escrito, com data, é o que transforma uma decisão de arquitetura em prova (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

Anonimização e pseudonimização passam por criptografia simétrica — e por sua gestão de chaves. A LGPD retira do seu escopo o dado anonimizado, mas mantém o pseudonimizado sob a lei. Um CPF cifrado com AES é dado **pseudonimizado**, não anonimado,

porque quem tem a chave reverte a operação. A consequência é direta: a chave passa a ser o ativo mais sensível do ambiente, e a segregação entre quem opera a base e quem custodia a chave deixa de ser refinamento e vira controle essencial. Se a mesma equipe tem os dois, não há pseudonimização efetiva.

No arranjo do Pix, a exigência é de suíte moderna, não de “criptografia”. A Resolução BCB 4.893/2021 e o manual de segurança do Pix impõem TLS com suítes atuais e autenticação mútua por certificado ICP-Brasil na comunicação com o SPI. Como o TLS 1.3 admite apenas AEAD, participar do arranjo já elimina por construção os modos legados. Para instituições menores que integram via PSTI, a homologação verifica exatamente isso — e reprovar em homologação por suíte obsoleta é um evento comum e evitável (Banco Central do Brasil, 2021).

PCI-DSS v4 é explícito onde a LGPD é principiológica. Quem processa cartão está sujeito ao requisito 3, que trata da proteção do dado de conta armazenado, e ao requisito 4, do dado em trânsito. A norma exige “criptografia forte” com definição própria, veda algoritmos obsoletos e — o ponto que mais reprova auditoria no Brasil — exige **procedimento documentado de gestão do ciclo de vida das chaves**: geração, distribuição, armazenamento, rotação, retirada e substituição em caso de comprometimento. Ter AES-256-GCM não passa na auditoria; ter AES-256-GCM com o ciclo de chaves documentado e executado, passa (PCI Security Standards Council, 2024).

Certificados A1 e A3 mudam onde a chave privada vive, e isso é decisão de arquitetura. No certificado A1 a chave privada é um arquivo protegido por senha, copiável — prático para automação de NF-e, e frágil se o servidor for comprometido. No A3 a chave vive em cartão ou *token* e não é extraível, o que torna a automação mais difícil e o roubo da chave muito mais. A escolha entre eles é, na prática, uma decisão sobre gestão de chaves simétrica ao que este capítulo discutiu: onde a chave nasce, quem a acessa e o que acontece se o hospedeiro cair (Brasil, 2001; Instituto Nacional de Tecnologia da Informação, 2024).

16.7. No Campo

1. Procure ECB no seu código hoje. É a varredura de maior retorno deste capítulo, e leva um minuto:

```
grep -rniE 'AES/ECB|MODE_ECB|aes-[0-9]+-ecb|ECBEncrypt|CipherMode\.ECB' \
  --include='*.java' --include='*.py' --include='*.cs' --include='*.go' \
  --include='*.php' --include='*.js' .

# Java: a armadilha silenciosa. "AES" sem modo explícito É ECB no provider
↪ padrão.
grep -rn 'Cipher.getInstance("AES")' --include='*.java' .
```

A segunda busca merece atenção especial: em Java, `Cipher.getInstance("AES")` seleciona AES/ECB/PKCS5Padding por omissão. É o padrão inseguro mais bem escondido da plataforma, e ainda aparece em código novo.

2. Verifique o efeito do ECB com as próprias mãos. Duas linhas que tornam o problema impossível de esquecer:

```
# 64 bytes de texto claro repetitivo, cifrados em ECB e em CBC
printf 'AAAAAAAAAAAAAAAA%.0s' {1..4} \
  | openssl enc -aes-128-ecb -K 00112233445566778899aabbccddeeff -nosalt | xxd
# Saída: o mesmo bloco de 16 bytes, quatro vezes.

printf 'AAAAAAAAAAAAAAAA%.0s' {1..4} \
  | openssl enc -aes-128-cbc -K 00112233445566778899aabbccddeeff \
    -iv 000102030405060708090a0b0c0d0e0f -nosalt | xxd
# Saída: quatro blocos distintos.
```

3. Prefira sempre a API de alto nível. Quando a biblioteca oferece uma função que não pede modo nem IV, use-a:

```
# Certo: a biblioteca decide modo, nonce e autenticação
from cryptography.fernet import Fernet
chave = Fernet.generate_key()          # guarde no cofre, não no código
token = Fernet(chave).encrypt(b"dado sensível")

# Também certo, quando é preciso controlar o nonce e usar dados associados
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
k = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)                 # CSPRNG, nunca reutilizado com esta
↪ chave
ct = AESGCM(k).encrypt(nonce, b"dado sensível", b"id-registro-4711")
```

O terceiro argumento do `encrypt` são os dados associados. Colocar ali o identificador do registro impede que um texto cifrado válido seja transplantado para outra linha da tabela.

4. Trate a decifragem que falha como evento de segurança, não como erro de aplicação. Etiqueta AEAD inválida significa dado corrompido ou adulterado. Registre com o identificador do registro e do chamador, alerte a partir de um limiar, e **nunca** devolva ao cliente a razão da falha:

```
Errado: 400 {"erro": "authentication tag mismatch at offset 48"}
Errado: 400 {"erro": "invalid padding"}
Certo: 400 {"erro": "requisicao invalida"} + log interno detalhado +
↪ contador de alerta
```

Mensagem de erro específica é o combustível do oráculo. A distinção que o atacante precisa é justamente a que a mensagem detalhada oferece de graça.

5. Escreva o esquema do texto cifrado no início do projeto. Todo dado cifrado deve carregar consigo o que é preciso para decifrá-lo e para migrá-lo depois:

```
[versao:1] [id_chave:2] [nonce:12] [texto_cifrado:n] [etiqueta:16]
```

O campo de versão é o que permite trocar de algoritmo daqui a cinco anos sem reescrever a base inteira. O identificador de chave é o que torna a rotação possível sem uma janela de indisponibilidade. Quem não previu esses dois campos descobre o custo na primeira rotação.

6. Meça antes de assumir que criptografia é cara. Em servidor moderno com AES-NI, AES-GCM roda em vários gigabytes por segundo por núcleo. O argumento de desempenho contra cifrar quase sempre é herdado de uma época em que era verdadeiro:

```
openssl speed -evp aes-256-gcm
openssl speed -evp chacha20-poly1305
grep -o -m1 'aes' /proc/cpuinfo && echo "AES-NI disponível"
```

Se o resultado do ChaCha20 superar o do AES-GCM, o processador não tem aceleração — e a escolha da suíte deve mudar de acordo.

7. Planeje a rotação de chaves antes de precisar dela. Chave de cifragem de dados com prazo definido, chave mestra no cofre ou no KMS, e um procedimento de recifragem que rode em lotes sem indisponibilidade. Documente também o cenário de comprometimento: qual dado precisa ser recifrado, em que ordem, em quanto tempo. As diretrizes do NIST sobre ciclo de vida de chaves (National Institute of Standards and Technology, 2020a) são a referência para essa política.

8. Registre os limites de volume por chave no monitoramento. Um contador de mensagens cifradas por chave, com alerta bem antes de 2^{32} , é barato de implementar e evita a categoria de falha mais difícil de diagnosticar depois: colisão de nonce por volume, num sistema que funcionou perfeitamente por três anos.

16.8. Resumo

- **A cifra de bloco só sabe cifrar um bloco.** Tudo além de 16 bytes depende do **modo de operação** — e é no modo, não no AES, que os sistemas reais quebram.
- **ECB não tem uso legítimo.** Blocos iguais viram textos cifrados iguais, e a estrutura do dado atravessa a cifragem intacta. O vazamento da Adobe em 2013 expôs 153 milhões de senhas sem que ninguém quebrasse o algoritmo.
- **CBC** exige IV imprevisível vindo de CSPRNG, e seu preenchimento cria um oráculo sempre que o sistema distingue erro de preenchimento de erro de conteúdo.
- **CTR** é rápido e paralelizável, e colapsa por completo se o par (chave, nonce) se repetir: os textos cifrados entregam o XOR dos textos claros. Foi essa a falha do WEP.
- **Cifrar não é autenticar.** Todos os modos clássicos são maleáveis: o atacante altera o texto cifrado de forma controlada sem conhecer a chave. Dado não autenticado não deve ser decifrado.
- Ao combinar cifra e MAC manualmente, só **encrypt-then-MAC** com chaves separadas é seguramente correto — e são três oportunidades de errar.
- **AEAD é a resposta moderna:** uma primitiva, uma chave, uma chamada, com a verificação da etiqueta antes de qualquer decifragem. AES-GCM no servidor, ChaCha20-Poly1305 em móvel e embarcado, AES-GCM-SIV quando a unicidade do nonce não for garantível.
- Os **dados associados** amarram o texto cifrado ao seu contexto e impedem que ele seja transplantado para outro registro ou outro usuário.
- **Nonce único é obrigação, não recomendação** — em GCM, reusá-lo permite recuperar a chave de autenticação e forjar mensagens. Chaves têm limite de volume, e por isso rotação e derivação por sessão são exigências.
- No Brasil, LGPD e PCI-DSS convergem no mesmo ponto: escolher o algoritmo é o começo, **documentar e executar o ciclo de vida da chave** é o que passa em auditoria. E dado cifrado é pseudonimizado, não anonimizado — a chave continua sendo o ativo.

17. Funções hash, HMAC e integridade

Em 2012, pesquisadores analisando o código malicioso Flame encontraram algo que não deveria existir: um certificado de assinatura de código aparentemente emitido pela Microsoft, válido segundo toda a cadeia de confiança do Windows, e usado para distribuir a praga através do mecanismo de atualização do sistema. Máquinas comprometidas recebiam o Flame por um canal que o próprio Windows tratava como legítimo.

Não houve invasão da Microsoft. Não houve chave privada roubada. O que houve foi uma **colisão de prefixo escolhido** em MD5. Um serviço de licenciamento da Microsoft ainda assinava certificados usando MD5, e os atacantes — com criptanálise inédita à época e um poder computacional considerável — construíram dois documentos diferentes com o mesmo resumo MD5: um inócuo, submetido para assinatura, e outro com poderes de assinatura de código. A assinatura obtida sobre o primeiro era matematicamente válida sobre o segundo, porque **a assinatura digital não assina o documento, assina o seu resumo**.

Colisões práticas em MD5 eram publicamente conhecidas desde 2004. Foram necessários oito anos para que a consequência aparecesse na forma de um ataque estatal contra a cadeia de atualização do sistema operacional mais usado do mundo. Esse intervalo — entre “quebrado no artigo” e “explorado em campo” — é a razão de este capítulo insistir tanto na diferença entre as propriedades de uma função hash, e em qual delas cada uso realmente depende.

17.1. O que uma função hash faz

Uma **função hash criptográfica** recebe uma entrada de qualquer tamanho e devolve uma saída de tamanho fixo — o **resumo**, ou *digest*. SHA-256 devolve 256 bits, sempre, tanto para um byte quanto para um terabyte.

Duas propriedades básicas vêm de graça: é **determinística** (a mesma entrada sempre produz a mesma saída) e é **rápida de computar**. As três propriedades que a tornam criptográfica são exigências fortes, e cada uma protege um uso diferente:

Resistência à pré-imagem. Dado um resumo h , deve ser inviável encontrar qualquer M tal que $H(M) = h$. É a propriedade de mão única. É o que impede que um resumo armazenado revele a entrada.

Resistência à segunda pré-imagem. Dado um M_1 específico, deve ser inviável encontrar $M_2 \neq M_1$ com $H(M_1) = H(M_2)$. É o que impede substituir **um documento existente** por outro com o mesmo resumo.

Resistência a colisão. Deve ser inviável encontrar **qualquer** par $M_1 \neq M_2$ com o mesmo resumo. Note a diferença: aqui o atacante escolhe as duas mensagens. É a propriedade mais fraca das três, no sentido de ser a primeira a cair — e é exatamente a que o Flame explorou.

Essa gradação é o que torna a pergunta “MD5 está quebrado?” mal formulada. MD5 tem resistência a colisão **destruída** — colisões saem em segundos num notebook. Sua resistência à pré-imagem, no entanto, continua de pé: ninguém sabe, hoje, inverter um resumo MD5 arbitrário. Por isso MD5 permanece aceitável para detectar corrupção acidental de um arquivo baixado (uso

que depende só de pré-imagem, e contra um adversário que não existe) e é **catastrófico** para assinatura digital, certificados, deduplicação com implicação de segurança ou qualquer coisa em que um adversário escolha o conteúdo.

17.1.1. Colisões são inevitáveis; o que importa é o custo

Uma função que mapeia entradas infinitas em saídas de 256 bits necessariamente tem colisões — o princípio da casa dos pombos garante. A segurança está no custo de **encontrar** uma.

E aqui entra o **paradoxo do aniversário**, que é a origem de uma confusão persistente. Para encontrar uma colisão qualquer em uma função de n bits, não são necessárias 2^n tentativas, e sim aproximadamente $2^{n/2}$. Em SHA-256, isso significa 2^{128} — ainda inviável. Mas explica por que **o tamanho do resumo deve ser o dobro do nível de segurança desejado**: para 128 bits de segurança contra colisão, o resumo precisa de 256 bits.

É também por isso que resumos truncados exigem cuidado. Truncar SHA-256 para 64 bits deixa a resistência a colisão em 2^{32} — cerca de quatro bilhões de operações, questão de segundos. Identificadores curtos derivados de hash são convenientes e não são garantia de unicidade contra um adversário.

17.2. A linha do tempo, e onde estamos

Tabela 17.1.: Estado das principais funções hash.

Função	Saída	Situação em 2026	Uso
MD5	128 bits	Colisões em segundos; pré-imagem intacta	Só verificação de corrupção acidental
SHA-1	160 bits	Colisão prática desde 2017; prefixo escolhido desde 2020	Aposentar. Legado apenas
SHA-256 / SHA-512	256 / 512 bits	Sólida; nenhum ataque prático	Padrão para uso geral
SHA-3 (Keccak)	224–512 bits	Sólida; construção interna distinta	Alternativa estrutural ao SHA-2
BLAKE2 / BLAKE3	variável	Sólida; muito rápida em software	Excelente onde desempenho importa

O **SHA-1** merece um parágrafo, porque sua morte foi lenta e instrutiva. Fraquezas teóricas apareceram em 2005. Em 2017, o ataque SHAttered produziu dois PDFs distintos com o mesmo resumo SHA-1 (Stevens et al., 2017) — mas era uma colisão de prefixo *idêntico*, de utilidade prática limitada. Em 2020, o ataque SHA-1 is a Shambles entregou **colisão de prefixo escolhido** por cerca de 45 mil dólares em nuvem, que é o tipo de colisão que forja certificados e assinaturas. Entre a primeira fraqueza teórica e a exploração barata passaram quinze anos — tempo mais que suficiente para migrar, e ainda assim SHA-1 aparece em ambientes de produção hoje.

SHA-2 e SHA-3 não competem, coexistem. SHA-2 usa a construção Merkle–Damgård, a mesma família estrutural de MD5 e SHA-1. Continua sólido, mas herda dessa família uma peculiaridade que será discutida a seguir. SHA-3, escolhido em competição pública em 2015 (National Institute of Standards and Technology, 2015b), usa uma construção completamente

diferente — a *esponja* — e existe justamente como seguro estrutural: se algum dia aparecer um ataque genérico contra Merkle–Damgård, SHA-3 não é afetado. Na prática, use SHA-256 por omissão; SHA-3 e BLAKE são escolhas legítimas, não obrigatórias.

17.3. Hash não é criptografia, e hash não é anonimização

Dois equívocos que aparecem em auditoria com frequência incômoda.

Hash não é criptografia porque não há chave e não há como voltar. Se o dado precisa ser recuperado, hash é a ferramenta errada — o Capítulo 16 tem a certa. “Nós fazemos hash dos dados do cartão para protegê-los” descreve, na melhor hipótese, um sistema que perdeu os dados; na pior, um sistema que os expôs.

Hash não anonimiza domínios pequenos, e essa é a armadilha cara. A função é pública e determinística: qualquer pessoa pode calcular o resumo de todas as entradas possíveis e montar a tabela reversa. O resultado depende inteiramente do tamanho do espaço de entrada:

Tabela 17.2.: Reversibilidade de hash por tamanho do espaço de entrada.

Dado	Entradas possíveis	Tempo para tabela completa
CPF	$\approx 1,1 \times 10^{10}$	Minutos em GPU comum
Telefone celular brasileiro	$\approx 10^{10}$	Minutos
Placa de veículo	$\approx 10^8$	Segundos
E-mail de domínio conhecido	Depende da lista	Horas, com listas vazadas
Chave AES-256 aleatória	2^{256}	Inviável

A conclusão operacional é direta: **hash de identificador nacional não é anonimização, é pseudonimização — e fraca**. O Capítulo 15 já registrou a consequência jurídica no Brasil; a consequência técnica é que, se o objetivo é publicar dado sem reidentificação, hash simples não serve. O que ajuda é hash com chave (HMAC com segredo que nunca é publicado), *salt* por registro, ou técnicas de agregação e ruído que o Volume 16 trata.

17.4. Length extension: a peculiaridade que matou o MAC caseiro

Suponha que alguém queira autenticar mensagens com um segredo compartilhado, e implemente o mais óbvio: $\text{MAC} = \text{SHA256}(\text{segredo} \parallel \text{mensagem})$. É intuitivo, é errado, e o motivo é estrutural.

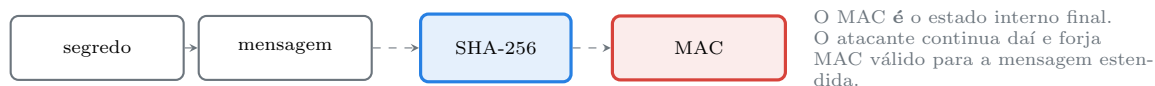
Funções da família Merkle–Damgård — MD5, SHA-1, SHA-2 — processam a entrada em blocos, mantendo um estado interno que é atualizado bloco a bloco. **O resumo final é o estado interno**. Quem conhece o resumo conhece o estado da função ao fim da mensagem, e pode continuar a partir dele.

Na prática, um atacante que veja mensagem e $\text{SHA256}(\text{segredo} \parallel \text{mensagem})$, sem conhecer o segredo, consegue produzir um MAC válido para $\text{mensagem} \parallel \text{preenchimento} \parallel \text{acrescimento}$, para qualquer **acrescimento** que escolher. Ele nem precisa saber o segredo — basta saber seu **comprimento**. Um parâmetro `&admin=false` vira `&admin=false<lixo>&admin=true`, com MAC válido, e a aplicação que lê o último valor do parâmetro obedece.

A Figura 17.1 mostra o problema e a construção que o resolve.

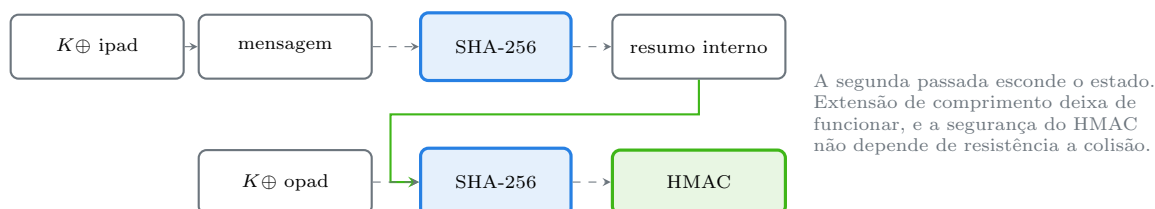
17. Funções hash, HMAC e integridade

MAC ingênuo — vulnerável a extensão de comprimento



O MAC é o estado interno final. O atacante continua daí e forja MAC válido para a mensagem estendida.

HMAC — duas passadas, o estado nunca é a saída



A segunda passada esconde o estado. Extensão de comprimento deixa de funcionar, e a segurança do HMAC não depende de resistência a colisão.

Figura 17.1.: O MAC ingênuo entrega o estado interno da função; o HMAC o esconde atrás de uma segunda passada.

17.5. MAC e HMAC: integridade com autenticidade

Vale separar dois conceitos que o vocabulário corrente mistura:

- **Hash simples** detecta alteração **acidental**. Não detecta alteração maliciosa, porque quem alterou o dado recalcula o resumo.
- **MAC** (código de autenticação de mensagem) usa uma **chave secreta** e detecta alteração **maliciosa**, porque só quem tem a chave produz um MAC válido.

Publicar o SHA-256 de um arquivo no mesmo servidor que hospeda o arquivo protege contra corrupção de transferência e contra praticamente nada mais: quem comprometeu o servidor troca os dois. É por isso que distribuições Linux publicam o arquivo de resumos **assinado** com GPG — a assinatura é o que adiciona autenticidade, e o resumo é apenas o que a assinatura cobre.

O **HMAC** (Krawczyk; Bellare; Canetti, 1997) é a construção padrão de MAC a partir de função hash. Sua definição é duas passadas com a chave misturada por constantes distintas:

$$\text{HMAC}_K(M) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| M))$$

O que importa não é memorizar a fórmula, e sim três consequências práticas:

1. **Resolve a extensão de comprimento**, porque a saída nunca é o estado interno de uma passada sobre a mensagem.
2. **É demonstravelmente seguro sob hipóteses fracas**. HMAC-SHA1 permanece seguro como MAC mesmo com SHA-1 colidido, porque a segurança do HMAC não depende de resistência a colisão. Isso não é motivo para manter HMAC-SHA1 — é motivo para não entrar em pânico ao encontrá-lo.
3. **Usa uma chave**, e essa chave precisa vir de CSPRNG e ter pelo menos o tamanho da saída da função.

Alternativas modernas incluem **KMAC** (baseado em SHA-3) e **Poly1305** (o autenticador do ChaCha20-Poly1305, do Capítulo 16). Para uso geral, HMAC-SHA256 continua sendo a escolha correta e universalmente disponível.

17.5.1. Comparar MAC em tempo constante

Detalhe pequeno, consequência grande. Comparar dois MACs com o operador de igualdade da linguagem quase sempre invoca uma comparação que **retorna assim que encontra o primeiro byte diferente**. Isso torna o tempo de resposta proporcional ao número de bytes iniciais corretos.

Um atacante que consiga medir esse tempo — e a rede moderna, com estatística suficiente, permite —, descobre o MAC **byte a byte**: fixa o primeiro byte testando 256 valores, escolhe o que demorou mais, passa ao segundo. São 256×32 tentativas em vez de 2^{256} .

A correção é usar a função de comparação em tempo constante que toda plataforma oferece: `hmac.compare_digest` no Python, `crypto.timingSafeEqual` no Node, `MessageDigest.isEqual` no Java, `subtle.ConstantTimeCompare` no Go, `CRYPTO_memcmp` no OpenSSL.

17.6. Derivar chave e guardar senha: usos opostos

Duas famílias de funções derivadas de hash são constantemente confundidas, e o critério que as separa é uma inversão completa de objetivo.

KDF de derivação de chave — a exemplo do **HKDF** (Krawczyk; Eronen, 2010) — parte de material com **alta entropia** (um segredo de Diffie-Hellman, uma chave mestra) e produz várias chaves independentes. O objetivo é separação de domínio: uma chave para cifrar, outra para autenticar, outra por sessão, todas derivadas do mesmo segredo sem que uma revele a outra. HKDF opera em duas etapas — *extract* concentra a entropia, *expand* produz o material do tamanho necessário, parametrizado por um rótulo de contexto. É o que o TLS 1.3 usa para derivar todas as chaves do handshake, como o Capítulo 20 mostra. **HKDF é rápido de propósito**, porque a entrada já é imprevisível.

Função de hash de senha — **Argon2**, **sCrypt**, **bcrypt**, **PBKDF2** — parte de material com **baixíssima entropia** (uma senha humana) e precisa ser **deliberadamente lenta e cara**, porque o atacante que obtiver a base vai testar bilhões de candidatos. Argon2id (Biryukov et al., 2021) é a recomendação atual, e sua característica decisiva é o **custo de memória**: exigir centenas de megabytes por tentativa neutraliza a vantagem de GPUs e ASICs, que têm muitos núcleos e pouca memória por núcleo.

O erro fatal é usar a família errada:

Tabela 17.3.: Qual família de função usar em cada situação.

Situação	Correto	Erro comum
Derivar chaves de um segredo de handshake	HKDF-SHA256	Argon2 (lento sem necessidade)
Armazenar senha de usuário	Argon2id, sCrypt, bcrypt	SHA-256, mesmo com sal
Derivar chave a partir de senha (cofre, disco)	Argon2id, PBKDF2 com muitas iterações	SHA-256 iterado à mão
Verificar integridade de arquivo	SHA-256	—

A linha do meio é a que mais aparece em incidente real. **SHA256(senha + sal)** é rápido — e essa é exatamente a propriedade que o torna inadequado. Uma GPU moderna calcula dezenas de

bilhões de SHA-256 por segundo; com Argon2id configurado para 100 ms e 256 MB por tentativa, o mesmo hardware faz algumas dezenas. O Capítulo 15 registrou que hash rápido é ótimo para integridade e péssimo para senha; o Volume 4 aprofunda o tema no capítulo de senhas.

17.7. Árvores de Merkle: integridade que escala

Verificar a integridade de um conjunto grande de dados com um único resumo tem um problema prático: qualquer alteração invalida o resumo inteiro, e descobrir **onde** está a alteração exige recomputar tudo.

A **árvore de Merkle** resolve isso. Cada folha é o resumo de um bloco de dados; cada nó interno é o resumo da concatenação dos resumos de seus filhos; a raiz resume o conjunto inteiro. A Figura 17.2 mostra a estrutura.

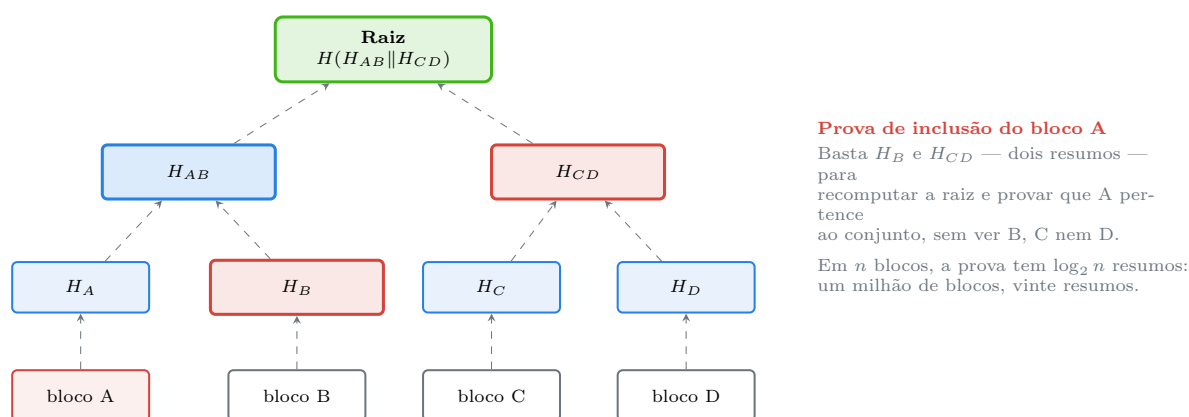


Figura 17.2.: Árvore de Merkle. A prova de inclusão cresce logaritmicamente, não linearmente.

A propriedade que faz a estrutura valer a pena é a **prova de inclusão**: para provar que um bloco pertence ao conjunto, basta apresentar os resumos irmãos ao longo do caminho até a raiz — $\log_2 n$ resumos, não n . Um conjunto de um milhão de blocos gera prova de vinte resumos.

Isso não é curiosidade acadêmica. É o que sustenta:

- O **Git**, em que cada *commit* referencia a árvore por resumo, e o histórico inteiro é uma cadeia de resumos encadeados — razão pela qual reescrever história muda todos os identificadores seguintes.
- O **Certificate Transparency** (Laurie; Langley; Kasper, 2013), que o Capítulo 19 detalha: registros públicos de certificados emitidos, em que a árvore permite provar inclusão e consistência sem baixar o registro inteiro.
- **Backup e sincronização** com deduplicação e verificação incremental (ZFS, **restic**, **rsync** com resumos).
- **Blockchains**, em que a árvore permite validar uma transação sem baixar o bloco completo.

17.8. Brasil em Foco

Hash de CPF não anonimiza, e a distinção tem consequência regulatória. A LGPD retira de seu escopo o dado anonimizado (art. 12) e mantém sob a lei o pseudonimizado. O art. 12, §1º é explícito: não se considera anonimizado o dado cujo processo de anonimização possa ser revertido “com esforços razoáveis”. Com 1,1 bilhão de CPFs possíveis e uma GPU comum, a

reversão de um SHA-256 de CPF não exige esforço razoável — exige alguns minutos. Publicar base “anonimizada por hash” é, na prática, publicar a base, e a ANPD já tratou o tema em seu guia de agentes de tratamento. Quando o objetivo é usar identificador sem expor o titular, o caminho técnico é **HMAC com chave mantida em segredo e sob custódia separada** (Autoridade Nacional de Proteção de Dados, 2023; Brasil, 2018).

Assinatura ICP-Brasil depende do algoritmo de resumo, não só da chave. A assinatura digital assina o resumo do documento, e por isso a política de assinatura da ICP-Brasil especifica quais funções são admitidas. Documentos assinados com SHA-1 em anos anteriores permanecem juridicamente válidos pelo princípio da lei vigente à época, mas **novas assinaturas com resumo obsoleto são rejeitadas pelos validadores oficiais**. Para quem opera emissão de NF-e, petição eletrônico ou o Assinador ITI, o efeito prático é que uma biblioteca desatualizada gera documentos que não passam na validação — e a mensagem de erro raramente aponta para o resumo (Brasil, 2001; Instituto Nacional de Tecnologia da Informação, 2024).

Integridade de log é requisito de investigação, não só de auditoria. O Marco Civil (Lei 12.965/2014) obriga provedores de conexão a guardar registros de conexão por um ano e provedores de aplicação a guardar registros de acesso por seis meses. Registro que possa ter sido alterado tem valor probatório frágil. A prática que resolve é encadeamento por resumo — cada lote de registros carrega o resumo do lote anterior, formando uma cadeia em que qualquer remoção ou alteração quebra a sequência — combinada com carimbo de tempo e, quando o caso justificar, assinatura. O Volume 13 trata de cadeia de custódia; a primitiva que sustenta tudo é esta (Brasil, 2014).

No arranjo do Pix e no Open Finance, HMAC e resumo aparecem em requisito contratual. A verificação de integridade de mensagens, os identificadores de idempotência e o registro de chaves envolvem construções baseadas em hash, com especificação explícita de algoritmo nos manuais operacionais. Instituições que integram por meio de terceiros herdam essas exigências, e a homologação verifica algoritmo e formato — não apenas “usa criptografia” (Banco Central do Brasil, 2021).

Vazamentos brasileiros mostraram o custo do hash rápido em senha. Diversos incidentes divulgados no país expuseram bases com senhas em MD5 ou SHA-1 sem sal, o que equivale, para senhas comuns, a expor as senhas em texto claro: tabelas prontas cobrem as senhas mais usadas, e a reutilização de senha entre serviços transforma um vazamento em comprometimento em cadeia. Sob a ótica do art. 48 da LGPD, base com senha em hash rápido dificilmente sustenta a tese de ausência de risco relevante ao titular.

17.9. No Campo

1. Encontre MD5 e SHA-1 nos caminhos que importam. Nem toda ocorrência é problema — o que interessa é o uso:

```
grep -rniE 'MD5|SHA-?1\b' --include='*.py' --include='*.java' --include='*.go'
↪ \
    --include='*.js' --include='*.cs' --include='*.php' . \
| grep -viE 'test|vendor|node_modules|checksum de download'

# Certificados assinados com algoritmo obsoleto no parque
for c in /etc/ssl/certs/*.pem; do
  alg=$(openssl x509 -in "$c" -noout -text 2>/dev/null | grep -m1 'Signature
↪ Algorithm')
```

17. Funções hash, HMAC e integridade

```
case "$alg" in *md5*|*sha1*) echo "OBSOLETO: $c -> $alg";; esac
done
```

Classifique cada achado por propriedade: se o uso depende de resistência a colisão (assinatura, certificado, identificador com significado de segurança, deduplicação confiável), é achado alto. Se depende só de detecção de corrupção acidental, é dívida técnica de baixa prioridade.

2. Substitua toda comparação de MAC por comparação em tempo constante. Busca de um minuto, correção de dois:

```
grep -rnE
↪ '(hmac|mac|signature|token|digest)\s*==|==\s*(hmac|mac|signature|digest)' \
--include='*.py' --include='*.js' --include='*.go' --include='*.java' . -i
```

```
# Errado: retorna no primeiro byte divergente
if calculado == recebido: ...

# Certo
import hmac
if hmac.compare_digest(calculado, recebido): ...
```

3. Nunca construa MAC concatenando segredo e mensagem. A regra que evita a extensão de comprimento sem exigir que ninguém a entenda:

```
# Errado - vulnerável a length extension
import hashlib
mac = hashlib.sha256(segredo + mensagem).hexdigest()

# Certo
import hmac, hashlib
mac = hmac.new(chave, mensagem, hashlib.sha256).hexdigest()
```

4. Verifique as duas propriedades ao publicar artefatos. Resumo publicado ao lado do arquivo protege contra corrupção; **assinatura** sobre o arquivo de resumos protege contra adulteração:

```
sha256sum -c SHA256SUMS # integridade do conteúdo
gpg --verify SHA256SUMS.asc SHA256SUMS # autenticidade da lista
```

Executar só o primeiro comando é o erro que faz o *checksum* parecer um controle de segurança quando é apenas um controle de qualidade de transferência.

5. Encadeie os registros por resumo onde a integridade tiver valor probatório. Barato de implementar, difícil de retroagir:

```
# Sela diária: cada dia carrega o resumo do dia anterior
ANTERIOR=$(cat /var/log/selo/$(date -d yesterday +%F).sha 2>/dev/null || echo
↪ GENESE)
{ echo "prev=$ANTERIOR"; cat /var/log/auth.log; } \
| sha256sum | awk '{print $1}' > /var/log/selo/$(date +%F).sha
```

O selo deve viver em armazenamento separado e preferencialmente somente-anexação — encadeamento cujo arquivo de selos está no mesmo servidor comprometido não prova nada. O Volume 12 trata do restante.

6. Ajuste o custo do Argon2 ao seu hardware, e reavalie anualmente. Parâmetro de fábrica raramente é adequado. O alvo é o maior custo que a sua carga suporta:

```
from argon2 import PasswordHasher
ph = PasswordHasher(time_cost=3, memory_cost=262144, parallelism=4) # 256 MB
h = ph.hash("senha-do-usuario")
# Meça: o objetivo é 100-500 ms por verificacao no hardware de producao.
```

Registre os parâmetros junto do resumo — todo formato moderno já os embute — para permitir aumentar o custo depois sem invalidar as senhas existentes.

7. Documente qual propriedade cada uso depende. Uma coluna a mais no inventário de algoritmos do Capítulo 15: para cada uso de hash, anote se depende de pré-imagem, de segunda pré-imagem ou de colisão. É o que permite responder, na próxima manchete sobre uma função quebrada, se aquilo afeta ou não o seu sistema — e a resposta costuma ser mais tranquila do que a manchete sugere.

8. Trate identificador derivado de hash como identificador, não como segredo. Resumo truncado para caber em URL, chave de cache ou nome de arquivo é conveniente e não é imprevisível para quem conhece a entrada. Se o valor precisa ser difícil de adivinhar, ele tem de vir de CSPRNG — não de hash de algo que o atacante pode enumerar, como um identificador sequencial ou um e-mail.

17.10. Resumo

- Uma função hash criptográfica tem **três propriedades distintas** — pré-imagem, segunda pré-imagem e colisão — e cada uso depende de uma delas. “Está quebrado?” é pergunta mal formulada; a certa é qual propriedade caiu e se o seu uso depende dela.
- **MD5 tem resistência a colisão destruída e pré-imagem intacta.** Serve para detectar corrupção acidental; é catastrófico para assinatura, como o Flame demonstrou ao forjar um certificado da Microsoft em 2012.
- **SHA-1 está morto para uso adversarial** desde a colisão de prefixo escolhido de 2020, por cerca de 45 mil dólares. SHA-256 é o padrão; SHA-3 e BLAKE são alternativas legítimas, não obrigatórias.
- O **paradoxo do aniversário** exige resumo do dobro do nível de segurança pretendido, e torna resumos truncados perigosos: 64 bits de resumo caem em segundos.
- **Hash não anonimiza domínio pequeno.** A tabela reversa completa de CPFs sai em minutos numa GPU. Para pseudonimizar de verdade, HMAC com chave sob custódia separada.
- Funções Merkle–Damgård expõem o estado interno na saída, e por isso $H(\text{segredo} || \text{mensagem})$ **não é um MAC**: sofre extensão de comprimento. HMAC resolve com duas passadas e não depende de resistência a colisão.
- **Comparação de MAC precisa ser em tempo constante.** Comparação ingênua vaza o MAC byte a byte por temporização.
- **Derivar chave e guardar senha são objetivos opostos:** HKDF é rápido de propósito, porque a entrada já tem entropia; Argon2id é lento e caro em memória de propósito, porque a entrada é uma senha humana. Trocar as famílias é o erro mais comum.

17. Funções hash, HMAC e integridade

- **Árvores de Merkle** dão prova de inclusão logarítmica, e sustentam Git, Certificate Transparency, backup deduplicado e blockchains.
- No Brasil, hash de CPF não é anonimização sob a LGPD, a validade de assinatura ICP-Brasil depende do algoritmo de resumo, e integridade encadeada de registros é o que dá valor probatório à guarda exigida pelo Marco Civil.

18. Criptografia assimétrica: RSA, Diffie-Hellman e curvas elípticas

No 27º Chaos Communication Congress, em dezembro de 2010, o grupo fail0verflow apresentou a recuperação da chave privada com que a Sony assinava todo o software do PlayStation 3. Não foi vazamento interno, não foi invasão de servidor, não foi força bruta contra ECDSA. Foi uma linha de código.

O ECDSA exige que cada assinatura use um valor aleatório novo, chamado k — um nonce, com todas as obrigações que o Capítulo 16 já estabeleceu para nonces. A implementação da Sony usava **um valor constante**. E a matemática do ECDSA é implacável quanto a isso: duas assinaturas produzidas com o mesmo k permitem resolver um sistema de duas equações e recuperar a chave privada diretamente, com aritmética que cabe em vinte linhas de Python. A partir daí, qualquer pessoa podia assinar software que o console aceitaria como oficial. Não havia como revogar: a chave estava gravada no hardware de milhões de aparelhos.

O algoritmo era sólido, a curva era adequada, o tamanho da chave era correto. O gerador de nonce não gerava nada. É o Capítulo 15 outra vez, num contexto de bilhões de dólares.

Este capítulo trata da família de primitivas que resolveu o problema mais antigo da criptografia — como duas partes que nunca se encontraram acordam uma chave secreta — e das armadilhas específicas que ela trouxe junto.

18.1. O problema que a assimétrica resolve

Até 1976, toda criptografia era simétrica, e isso implicava um problema logístico insolúvel: **para conversar em sigilo, as partes precisavam antes compartilhar uma chave por um canal já seguro**. Bancos trocavam chaves por malote. Embaixadas usavam correio diplomático. Redes militares distribuíam livros de código fisicamente.

A escala torna isso impraticável. Uma rede de n participantes que precisem falar dois a dois exige $n(n-1)/2$ chaves distintas: 10 pessoas exigem 45 chaves; 1.000 exigem quase meio milhão; a internet exigiria um número sem sentido. E nada disso resolve o caso que a web tornou cotidiano — um navegador que nunca viu determinado servidor precisa estabelecer sigilo com ele em milissegundos, sem contato prévio.

A resposta veio do artigo de Diffie e Hellman (Diffie; Hellman, 1976), que propôs a ideia central: **duas chaves matematicamente relacionadas, uma pública e uma privada**, tais que conhecer a pública não permita derivar a privada.

A ferramenta que torna isso possível é a **função de mão única com alçapão** — fácil de computar numa direção, inviável de inverter, exceto para quem detém uma informação extra (o alçapão). Multiplicar dois primos de 1024 bits é instantâneo; fatorar o produto de volta é inviável. Elevar um número a uma potência módulo um primo grande é rápido; recuperar o expoente — o logaritmo discreto — não é.

Vale registrar a natureza dessa segurança: **ela é conjectural**. Ninguém provou que fatoração é difícil. Sabe-se que ninguém conseguiu, apesar de séculos de esforço matemático, e que um avanço teórico mudaria o quadro. É uma diferença real em relação ao AES, cuja segurança também não é provada, mas cujo ataque não depende de um problema matemático com nome e história própria. É também a razão de a criptografia pós-quântica existir — um computador quântico relevante resolveria fatoração e logaritmo discreto em tempo polinomial, e o assunto é do Volume 16.

18.2. Diffie-Hellman: acordar uma chave em público

O protocolo Diffie-Hellman faz algo que parece impossível: duas partes trocam mensagens **inteiramente públicas** e terminam compartilhando um segredo que quem observou não consegue calcular.

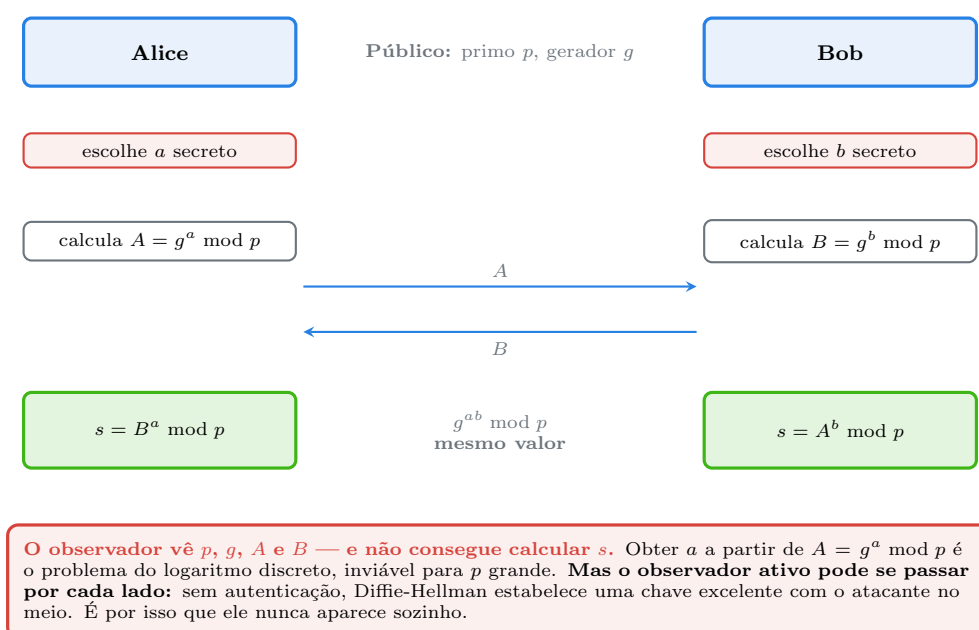


Figura 18.1.: Troca Diffie-Hellman. O segredo comum nunca trafega — e nada nela autentica quem está do outro lado.

O mecanismo é simples de acompanhar. As partes concordam publicamente sobre um primo grande p e um gerador g . Alice escolhe um expoente secreto a e publica $A = g^a \bmod p$; Bob escolhe b e publica $B = g^b \bmod p$. Alice calcula $B^a = g^{ab}$, Bob calcula $A^b = g^{ab}$: o mesmo valor. O observador tem A e B , e para obter g^{ab} precisaria de a ou b — o logaritmo discreto, inviável.

A limitação é tão importante quanto o mecanismo. Diffie-Hellman garante sigilo contra observador passivo e **nada** contra adversário ativo. Um atacante posicionado no caminho executa duas trocas — uma com Alice, outra com Bob —, decifra, lê, recifra e repassa. Ambos os lados acreditam ter um canal seguro, e têm: com o atacante.

A conclusão que estrutura todo o resto do volume: **acordo de chave sem autenticação é inútil na prática**. É por isso que o TLS combina ECDHE com assinatura do servidor, e por isso o Capítulo 19 existe.

Na forma moderna, usa-se **DHE** (efêmero, com expoentes novos a cada sessão) ou, preferencialmente, **ECDHE** sobre curvas elípticas. O adjetivo “efêmero” é o que entrega **sigilo encaminhado**, tratado adiante.

18.3. RSA: cifrar e assinar com o mesmo par

O RSA (Rivest; Shamir; Adleman, 1978) foi o primeiro esquema a fazer as duas coisas — cifrar e assinar — com o mesmo tipo de par de chaves, e por isso dominou por décadas.

A geração é conceitualmente simples: escolhem-se dois primos grandes p e q , calcula-se $n = pq$, e derivam-se um expoente público e (tipicamente 65537) e um expoente privado d . A chave pública é (n, e) ; a privada é (n, d) . A operação básica é exponenciação modular, e a segurança repousa sobre a dificuldade de fatorar n .

Doas advertências práticas valem mais que a matemática:

RSA “de livro-texto” é inseguro. Aplicar a operação diretamente sobre a mensagem é determinístico — cifrar duas vezes o mesmo dado dá o mesmo resultado, o que permite enumerar mensagens curtas — e maleável. Todo uso real exige preenchimento probabilístico: **OAEP** para cifragem e **PSS** para assinatura (Moriarty et al., 2016). O preenchimento antigo, PKCS#1 v1.5, foi a base do ataque de Bleichenbacher (Bleichenbacher, 1998) — um oráculo que revela o texto claro submetendo variações do cifrado e observando as respostas, exatamente a estrutura do modelo CCA do Capítulo 15. Foi corrigido em 1998, ressuscitou em 2017 como ROBOT em produtos de grandes fabricantes, e o Capítulo 20 mostra por que o TLS 1.3 simplesmente removeu o RSA de transporte de chave.

RSA não cifra dados, cifra chaves. A operação é lenta e limitada ao tamanho do módulo — uma chave de 2048 bits cifra no máximo 190 bytes com OAEP. O uso correto é o **esquema híbrido**: gera-se uma chave simétrica aleatória, cifram-se os dados com AES-GCM e cifra-se apenas a chave simétrica com RSA. Quem tenta cifrar um arquivo diretamente com RSA ou está cifrando algo minúsculo ou está fazendo algo errado.

18.3.1. Assinatura: o inverso, e a confusão eterna

Assinar é a operação com a chave **privada**, verificável com a **pública**. E a fonte de confusão mais persistente da área é qual chave faz o quê. A Figura 18.2 resolve de vez.

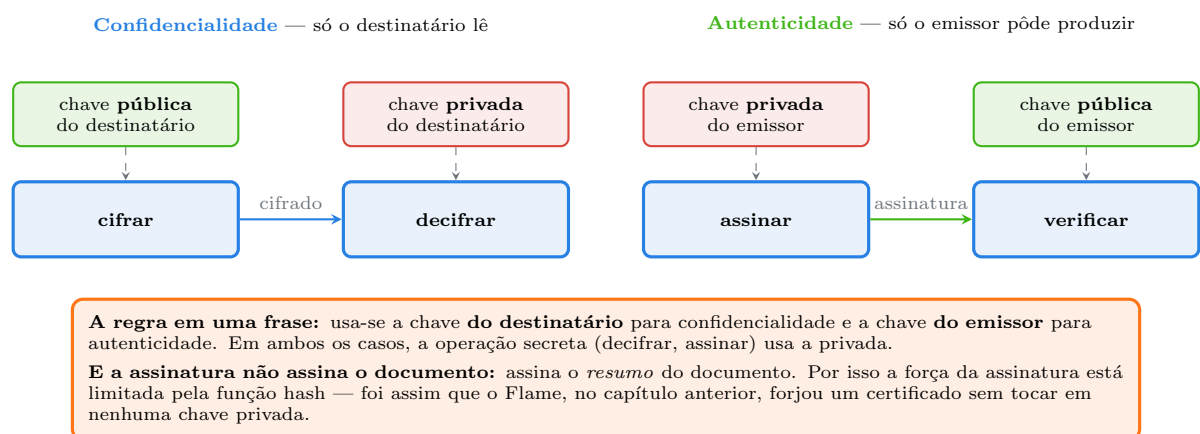


Figura 18.2.: Qual chave faz o quê. Confidencialidade usa a chave do destinatário; autenticidade, a do emissor.

A assinatura digital entrega três propriedades que um MAC não entrega, e a diferença é do Capítulo 3: **autenticidade, integridade e não repúdio**. O MAC prova que a mensagem veio de quem tem a chave — mas as duas partes têm a mesma chave, e por isso nenhuma pode provar

a um terceiro que a outra a produziu. A assinatura, como só o titular da privada pode gerá-la, é oponível a terceiros. É essa propriedade que dá base jurídica à assinatura eletrônica.

18.4. Curvas elípticas: mesma segurança, chaves menores

A criptografia de curvas elípticas (ECC) constrói as mesmas operações sobre a estrutura algébrica de uma curva elíptica em vez de sobre inteiros módulo um primo. O problema difícil correspondente — o logaritmo discreto em curva elíptica — resiste melhor aos algoritmos conhecidos, e a consequência prática é grande:

Tabela 18.1.: Tamanhos de chave de segurança equivalente (National Institute of Standards and Technology, 2020a).

Segurança	Simétrica	RSA / DH	Curva elíptica
80 bits	—	1024 bits	160 bits
112 bits	3DES	2048 bits	224 bits
128 bits	AES-128	3072 bits	256 bits
192 bits	AES-192	7680 bits	384 bits
256 bits	AES-256	15360 bits	512 bits

A tabela explica duas decisões que aparecem o tempo todo em configuração. Primeiro, por que RSA de 2048 bits — ainda o padrão de mercado — entrega apenas 112 bits de segurança, abaixo dos 128 que se considera o alvo moderno; subir para 3072 bits resolve, ao custo de operações sensivelmente mais lentas. Segundo, por que ECC domina em dispositivos móveis, IoT e qualquer lugar com restrição de banda, memória ou bateria: 256 bits de chave elíptica substituem 3072 bits de RSA.

18.4.1. Quais curvas usar

P-256, P-384 (NIST) são as mais suportadas, presentes em todo lugar, obrigatórias em muitos contextos regulados — incluindo a ICP-Brasil. Têm uma objeção conhecida: as constantes de projeto nunca foram plenamente justificadas publicamente, o que gerou desconfiança persistente após o episódio do Dual_EC_DRBG. Não há ataque conhecido contra elas.

Curve25519 e Ed25519, de Daniel Bernstein (Josefsson; Liusvaara, 2017; Langley; Hamburg; Turner, 2016), foram projetadas com parâmetros justificados publicamente e, sobretudo, para serem **difíceis de implementar errado**: resistem naturalmente a canal lateral, dispensam validação de ponto e não têm formatos degenerados. Quando há liberdade de escolha, são a melhor opção — X25519 para acordo de chave, Ed25519 para assinatura. É o padrão do SSH moderno, do Signal, do WireGuard e do TLS 1.3.

Evite curvas exóticas e implementações próprias de curvas. Escolher uma curva sem análise pública consolidada é reencenar o Capítulo 15 com aritmética mais sofisticada.

18.4.2. ECDSA e o problema do nonce

O caso do PlayStation 3 que abre este capítulo não é anedota histórica: a mesma falha reapareceu em carteiras de Bitcoin em dispositivos Android em 2013 (por um defeito no gerador de aleatoriedade do sistema) e continua aparecendo em auditorias.

A raiz é estrutural. **ECDSA exige um nonce k aleatório, secreto e irrepetível a cada assinatura.** Se k vaza, a chave privada sai de uma equação. Se k se repete em duas assinaturas, a chave privada sai de duas equações. Se k é apenas **enviesado** — poucos bits previsíveis —, técnicas de reticulados recuperam a chave a partir de algumas centenas de assinaturas.

Duas defesas existem, e a segunda é a razão de preferir Ed25519:

- **RFC 6979** define ECDSA determinístico: k é derivado por HMAC da chave privada e da mensagem. Elimina a dependência do gerador de aleatoriedade, mantendo compatibilidade com verificadores existentes.
- **Ed25519 é determinístico por projeto.** Não há k a gerar, não há como errar. É o exemplo mais limpo do princípio que percorre este volume: **primitiva que não permite errar vale mais que documentação explicando como não errar.**

18.5. Sigilo encaminhado: por que “efêmero” importa

Considere um servidor que use RSA para transporte de chave — modo em que o cliente sorteia a chave de sessão, cifra com a pública do servidor e envia. Funciona, é rápido, e tem um defeito de longo prazo: **quem gravou o tráfego cifrado hoje e obtiver a chave privada do servidor daqui a cinco anos decifra tudo o que gravou.** Por ordem judicial, por comprometimento do servidor, por descarte inadequado de um disco, por avanço criptanalítico.

O **sigilo encaminhado** (*forward secrecy*) elimina esse risco. Com ECDHE, cada sessão usa um par efêmero, descartado ao fim; a chave privada de longo prazo do servidor serve apenas para **assinar** a troca, provando identidade, e não é suficiente para reconstituir nenhuma chave de sessão. Comprometer o servidor amanhã não abre o tráfego de ontem.

Foi por isso que o TLS 1.3 **removeu completamente** o transporte de chave por RSA (Rescorla, 2018). Não o desaconselhou: eliminou do protocolo. Em TLS 1.2, a diferença aparece no nome da suíte — TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 tem sigilo encaminhado; TLS_RSA_WITH_AES_256_GCM_SHA384 não tem. É a distinção mais importante a saber ler numa lista de suítes, e o Capítulo 20 volta ao ponto.

18.6. Como tudo se combina, na prática

Nenhum sistema real usa uma primitiva isolada. A construção padrão de qualquer canal seguro moderno tem quatro peças, e reconhecê-las torna legível qualquer protocolo:

1. **Acordo de chave efêmero** (ECDHE com X25519 ou P-256) estabelece um segredo compartilhado com sigilo encaminhado.
2. **Assinatura** (Ed25519, ECDSA ou RSA-PSS) sobre a troca prova a identidade de quem está do outro lado — resolvendo o adversário ativo do Diffie-Hellman.
3. **KDF** (HKDF, do Capítulo 17) deriva do segredo compartilhado todas as chaves de que a sessão precisa.
4. **AEAD** (AES-GCM ou ChaCha20-Poly1305, do Capítulo 16) protege os dados em volume.

Cada peça faz uma coisa que as outras não fazem. TLS 1.3, SSH, WireGuard e o protocolo do Signal são variações desse mesmo arranjo — e o que sobra de diferença entre eles é sobretudo gestão de identidade e de estado, não escolha de primitiva.

A peça 2 é a que ainda está em aberto neste volume: **como saber que a chave pública do outro lado é mesmo dele?** Essa é a pergunta que o Capítulo 19 responde.

18.7. Brasil em Foco

A ICP-Brasil é uma infraestrutura de chaves assimétricas com força de lei. A MP 2.200-2/2001 conferiu presunção de veracidade aos documentos assinados com certificado emitido em sua cadeia — o que, no Direito, significa inversão do ônus da prova: quem alega que não assinou é que precisa demonstrar. A base técnica é exatamente o não repúdio descrito neste capítulo, e ele depende de uma premissa que é operacional, não matemática: **a chave privada esteve sob controle exclusivo do titular.** Todo o desenho de custódia da ICP-Brasil existe para sustentar essa premissa (Brasil, 2001; Instituto Nacional de Tecnologia da Informação, 2024).

A escolha entre A1 e A3 é uma decisão sobre onde a chave privada vive. No **A1**, a chave é gerada em software e fica em arquivo `.pfx` protegido por senha, com validade de um ano — copiável, o que viabiliza automação de NF-e e assinatura em servidor, e significa que comprometer o servidor é comprometer a chave. No **A3**, a chave é gerada dentro de um cartão ou *token* criptográfico e **não é extraível**: as operações acontecem no dispositivo, com validade de até três anos. Existe ainda o certificado **em nuvem**, em que a chave fica em módulo criptográfico de hardware (HSM) de um prestador, acionada por autenticação forte — combinação que preserva a não extração e recupera a automação. A decisão é de arquitetura: quanto mais automática a assinatura, mais fraca tende a ser a premissa de controle exclusivo.

A Lei 14.063/2020 escalonou as assinaturas, e o nível determina para que servem. A assinatura **simples** identifica o signatário por meios básicos; a **avanzada** usa certificado não-ICP ou outro meio que assegure integridade e autoria com nível razoável de confiança — é o caso do gov.br; a **qualificada** exige certificado ICP-Brasil e é a única admitida para os atos de maior exigência, como transferência de imóveis e certos atos societários. Para quem projeta sistema, a implicação é que **o nível de assinatura é requisito funcional**, definido pelo ato jurídico pretendido, e não uma escolha de conveniência técnica (Brasil, 2020).

O gov.br popularizou a assinatura avançada e trouxe uma dependência. A assinatura pelo portal usa certificado gerenciado em nuvem pelo governo, com autenticação por conta de nível prata ou ouro. Resolveu o problema de custo — não exige comprar certificado — e criou dependência de disponibilidade do serviço federal, além de deslocar a segurança do documento para a segurança da conta gov.br. Do ponto de vista deste capítulo, o par de chaves não está mais com o titular; está com um custodiante, sob controle de autenticação. É uma troca legítima e explícita, e precisa ser tratada como tal na análise de risco.

Migração de RSA-2048 é assunto de planejamento, não de urgência. Boa parte do parque brasileiro — inclusive certificados ICP-Brasil — usa RSA de 2048 bits, que entrega 112 bits de segurança. Não há ataque prático à vista, e o NIST projeta a transição para depois de 2030 (National Institute of Standards and Technology, 2019). O que vale começar agora é o **inventário**: onde há RSA-2048 e RSA-1024, onde há SHA-1 em assinatura, quais sistemas conseguem lidar com chaves maiores ou com curvas elípticas. Esse mesmo inventário é o insumo da migração pós-quântica, e fazê-lo uma vez serve às duas.

18.8. No Campo

1. Prefira Ed25519 e X25519 sempre que houver liberdade de escolha. Chaves menores, mais rápidas e — o que mais importa — sem nonce a errar:

```
# SSH: gerar par moderno
ssh-keygen -t ed25519 -C "vitor@estacao" -f ~/.ssh/id_ed25519

# Encontrar chaves antigas ou fracas ainda em uso no parque
ssh-keygen -lf ~/.ssh/authorized_keys      # mostra tipo e tamanho de cada
↪ chave
awk '{print $1}' ~/.ssh/authorized_keys | sort | uniq -c    # ssh-dss e rsa
↪ curtas = achado
```

ssh-dss (DSA) deve ser removido sem discussão: está desabilitado por padrão no OpenSSH há anos e tem exatamente o mesmo problema de nonce descrito aqui.

2. Nunca use RSA sem preenchimento moderno. OAEP para cifrar, PSS para assinar:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Certo: OAEP para cifragem
ct = pub.encrypt(chave_simetrica, padding.OAEP(
    mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(),
    ↪ label=None))

# Certo: PSS para assinatura
sig = priv.sign(mensagem, padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()),
    salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
```

E lembre do escopo: o `chave_simetrica` do exemplo é o uso correto. Cifrar o dado em si com RSA é sintoma de desenho errado.

3. Procure PKCS#1 v1.5 e chaves fracas na base de código e no parque. As duas varreduras que mais rendem:

```
grep -rnE 'PKCS1v15|PKCS1_v1_5|RSA/ECB/PKCS1Padding|NoPadding' \
    --include='*.py' --include='*.java' --include='*.go' --include='*.cs' .

# Chaves RSA com menos de 2048 bits em certificados do parque
for c in /etc/ssl/certs/*.pem /etc/pki/tls/certs/*.crt; do
    [ -f "$c" ] || continue
    b=$(openssl x509 -in "$c" -noout -text 2>/dev/null | grep -oE 'Public-Key:
    ↪ \([0-9]+\)' | grep -oE '[0-9]+)'
    [ -n "$b" ] && [ "$b" -lt 2048 ] && echo "FRACA ($b bits): $c"
done
```

RSA/ECB/PKCS1Padding em Java merece nota: apesar do “ECB” no nome, aqui ele não significa modo de operação — significa preenchimento PKCS#1 v1.5, e o achado é o preenchimento.

4. Exija sigilo encaminhado na configuração de TLS. É a suíte que revela, e a leitura é direta:

```
# Suítes efetivamente oferecidas pelo servidor
nmap --script ssl-enum-ciphers -p 443 servidor.exemplo.br

# Qualquer suite TLS_RSA_WITH_* (sem DHE/ECDHE) não tem sigilo encaminhado
openssl s_client -connect servidor.exemplo.br:443 </dev/null 2>/dev/null \
| grep -E 'Protocol|Cipher'
```

5. Trate a chave privada como o ativo que ela é. Cinco controles que valem mais que a escolha do algoritmo: gerar no destino final (nunca gerar em uma máquina e copiar), permissão 0600 e dono correto, senha de proteção onde o formato permitir, HSM ou KMS para chave de assinatura de alto valor, e procedimento escrito de revogação testado antes de precisar dele.

```
# Achados clássicos de auditoria: chave privada legível ou versionada
find / -name '*.key' -o -name '*.pem' 2>/dev/null | while read -r f; do
  p=$(stat -c '%a %U' "$f" 2>/dev/null)
  case "$p" in *4*|*6*|*7*) grep -qs 'PRIVATE KEY' "$f" && echo "$p $f";; esac
done

git log --all --diff-filter=A --name-only --pretty=format: \
| grep -E '\.(key|pem|pfx|p12|jks)$' | sort -u
```

O segundo comando merece ser rodado hoje. Chave privada que já esteve num commit continua no histórico mesmo depois de removida do diretório, e a única resposta correta é **considerá-la comprometida e rotacioná-la** — não apagar o arquivo.

6. Use ECDSA determinístico ou Ed25519 onde assinar for frequente. Se a plataforma só oferece ECDSA, verifique se a biblioteca implementa RFC 6979. Se assina em dispositivo embarcado, com entropia escassa no arranque, isso deixa de ser preferência e vira requisito — é exatamente o cenário em que o gerador falha.

7. Documente o inventário de chaves assimétricas com data de expiração. Um registro com titular, finalidade, algoritmo, tamanho, onde a privada vive, data de emissão e de validade. É o que evita a categoria de incidente mais evitável que existe: certificado expirado em produção num feriado. E é o insumo direto da migração pós-quântica, quando ela chegar.

8. Ao avaliar um fornecedor, pergunte onde a chave privada é gerada. “Nós geramos o par e enviamos por e-mail” descreve um sistema em que o não repúdio não existe, porque o controle nunca foi exclusivo. A resposta correta é que o par nasce no dispositivo do titular ou dentro do HSM, e que a privada nunca trafega.

18.9. Resumo

- A criptografia assimétrica resolveu o problema logístico que a simétrica não resolvia: **acordar chave entre partes que nunca se encontraram**, sem canal seguro prévio.
- Sua segurança é **conjectural** — repousa sobre fatoração e logaritmo discreto serem difíceis, o que ninguém provou e um computador quântico relevante desfaria.
- **Diffie-Hellman** estabelece um segredo por troca pública, e não autentica nada. Sem assinatura junto, entrega uma chave excelente ao atacante no meio.

- **RSA de livro-texto é inseguro**; use OAEP para cifrar e PSS para assinar. E RSA **não cifra dados**: cifra a chave simétrica que cifra os dados.
- A regra das chaves: **pública do destinatário para confidencialidade, privada do emissor para autenticidade**. A operação secreta é sempre a da chave privada.
- **Assinatura digital dá não repúdio; MAC não dá**, porque no MAC as duas partes têm a mesma chave. É essa diferença que sustenta a assinatura eletrônica juridicamente.
- **Curvas elípticas entregam a mesma segurança com chaves muito menores** — 256 bits de ECC equivalem a 3072 de RSA. Prefira X25519 e Ed25519 quando houver escolha; P-256 e P-384 quando houver exigência de compatibilidade ou regulatória.
- **ECDSA depende de um nonce por assinatura**, e reusá-lo, vazá-lo ou enviesá-lo revela a chave privada. Foi assim que a chave do PlayStation 3 caiu. Ed25519 é determinístico por projeto e remove a possibilidade do erro.
- **Sigilo encaminhado** exige acordo de chave efêmero: com ECDHE, comprometer o servidor amanhã não abre o tráfego de ontem. O TLS 1.3 removeu o transporte de chave por RSA justamente por isso.
- Todo canal seguro moderno combina **ECDHE + assinatura + KDF + AEAD**. Reconhecer as quatro peças torna legível qualquer protocolo — e a peça que falta explicar é como confiar na chave pública alheia, tema do capítulo seguinte.
- No Brasil, a ICP-Brasil dá base legal ao não repúdio, e a escolha entre **A1, A3 e nuvem** é, no fundo, uma decisão sobre onde a chave privada vive e quão defensável é a premissa de controle exclusivo.

19. PKI, certificados digitais e cadeias de confiança

Em agosto de 2011, um usuário iraniano do Gmail postou num fórum do Google uma pergunta simples: o navegador estava reclamando do certificado. Não era erro de configuração. Era um certificado válido para *.google.com, corretamente assinado, aceito sem alarme por qualquer sistema do planeta — e que o Google não havia solicitado.

A emissora era a DigiNotar, uma autoridade certificadora holandesa de porte modesto, que atendia principalmente o setor público dos Países Baixos. Seus servidores haviam sido comprometidos, e o atacante emitira mais de 500 certificados fraudulentos para domínios do Google, da Microsoft, da Mozilla, da CIA e do Mossad. O certificado do Google foi usado para interceptar o tráfego de aproximadamente 300 mil usuários iranianos por semanas.

O ponto que interessa é estrutural, não anedótico. A DigiNotar **jamaís** teve relação com o Google. Nunca hospedou nada do Google, nunca foi indicada por ele, nunca teve autorização para falar em seu nome. E, mesmo assim, seu certificado para *.google.com era tecnicamente perfeito — porque, no modelo de PKI da web, **qualquer autoridade em que o navegador confie pode emitir certificado para qualquer domínio do mundo**. A confiança não é por domínio; é global.

A DigiNotar foi removida dos armazéns de confiança em dias e decretou falência em menos de um mês. O modelo que a tornou perigosa continua em uso — com salvaguardas que este capítulo apresenta, e que existem por causa dela.

19.1. O problema que a PKI resolve

O Capítulo 18 terminou com uma pergunta em aberto. Diffie-Hellman estabelece um segredo com quem estiver do outro lado; a assinatura prova que quem assinou detém determinada chave privada. Falta o elo: **como saber que aquela chave pública pertence a quem se pretende alcançar?**

Sem resposta, todo o resto desmorona. Um atacante no caminho apresenta a **sua** chave pública, assina a troca com a **sua** privada, e tudo verifica corretamente. O canal é impecavelmente seguro — com a pessoa errada.

Três modelos de confiança foram propostos, e a web adotou o terceiro:

Confiança direta. As partes trocam chaves pessoalmente ou por canal fora de banda, e comparam impressões digitais. É o que o SSH faz na primeira conexão (*trust on first use*) e o que aplicativos de mensagem oferecem na verificação de código de segurança. É o modelo mais forte e o que menos escala.

Teia de confiança. Cada participante assina as chaves de quem conhece, e a confiança se propaga por caminhos. É o modelo do PGP. Funciona em comunidades técnicas pequenas, e falhou em escalar por uma razão social: exigir que usuários comuns raciocinem sobre grafos de confiança e assinaturas cruzadas não funcionou na prática.

Hierárquica. Um conjunto pequeno de **autoridades certificadoras** (ACs), previamente confiadas, atesta a ligação entre identidade e chave pública. É a PKI da web e da ICP-Brasil, e a razão de sua vitória é a escala: o navegador precisa confiar em algumas centenas de entidades para validar centenas de milhões de sites.

19.2. O certificado X.509

Um certificado é um documento estruturado que afirma “**esta chave pública pertence a esta identidade**”, assinado por uma autoridade. O formato é o X.509 (Cooper et al., 2008), e conhecer seus campos é o que separa quem lê um certificado de quem clica no cadeado.

Tabela 19.1.: Campos de um certificado X.509 que importam na prática.

Campo	O que contém	Por que importa
Subject	Identidade do titular	O CN está obsoleto para nomes de host
Subject Alternative Name	Lista de nomes válidos	É o campo que os navegadores usam. Sem SAN, o certificado é rejeitado
Issuer	Quem assinou	Aponta o próximo elo da cadeia
Validity	notBefore e notAfter	Fonte da falha operacional mais comum que existe
Public Key Info	Algoritmo e a chave	RSA-2048, P-256, Ed25519
Basic Constraints	CA: TRUE/FALSE	Certificado folha com CA:TRUE é catastrófico
Key Usage	Operações permitidas	Assinatura digital, cifragem de chave, assinatura de certificado
Extended Key Usage	Finalidade	Autenticação de servidor, de cliente, assinatura de código
CRL / OCSP	Onde checar revogação	Ponto de distribuição e respondedor
SCT	Provas de Certificate Transparency	Exigido pelos navegadores desde 2018
Signature	Assinatura da AC sobre tudo acima	O que amarra o conjunto

Três armadilhas merecem destaque. A primeira é o **CN obsoleto**: durante anos o nome do host ficava no **Common Name**, e ainda se vê certificado emitido só com ele. Navegadores modernos ignoram o CN por completo — se o nome não estiver no SAN, o certificado não vale, por mais correto que pareça. A segunda é o **Basic Constraints**: um certificado folha que declare **CA:TRUE** e seja aceito como tal permite ao titular assinar certificados para qualquer domínio. Foi exatamente a falha do *Basic Constraints* em navegadores antigos, e é o que o Flame, do Capítulo 17, obteve por outro caminho. A terceira é a **validade**: certificado expirado derruba serviço com a mesma eficiência de um ataque, e o setor caminha para prazos cada vez mais curtos — o que torna a automação obrigatória, não opcional.

19.3. A cadeia de confiança

Um certificado sozinho não prova nada; o que prova é a **cadeia** que leva dele até uma raiz previamente confiada. A Figura 19.1 mostra a estrutura e a validação.

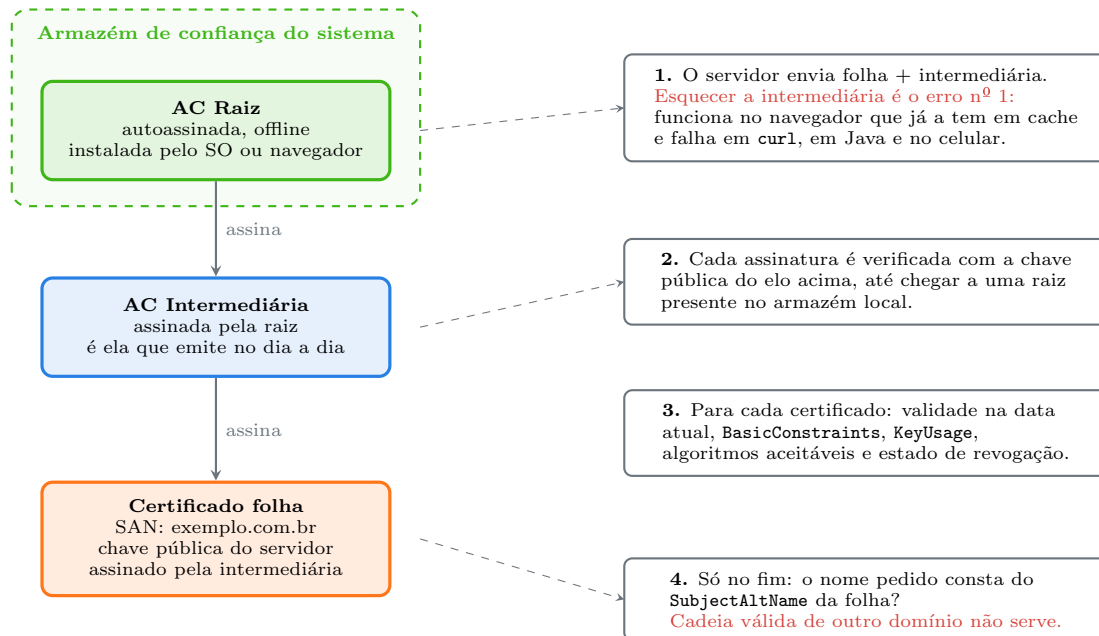


Figura 19.1.: Cadeia de confiança e as quatro verificações. A ausência da intermediária é a falha operacional mais frequente.

A **raiz** é autoassinada — ela afirma a própria identidade, e a confiança nela não vem da matemática, vem de estar no armazém do sistema. Por isso a chave privada da raiz fica em HSM, fisicamente isolada, acionada em cerimônias documentadas com múltiplas testemunhas: comprometê-la comprometeria tudo o que ela ancora, sem recurso.

A **intermediária** existe exatamente para proteger a raiz. É ela que emite no dia a dia, e seu comprometimento é grave mas recuperável: revoga-se a intermediária, emite-se outra, a raiz permanece intocada.

A **folha** é o certificado do serviço. Traz o SAN, a chave pública e a assinatura da intermediária.

O **erro operacional mais comum de todos** está no passo 1 da figura: o servidor precisa enviar a folha e a intermediária. Muitos navegadores contornam a omissão porque já viram a intermediária antes e a mantêm em cache, ou porque seguem a extensão AIA para buscá-la. O resultado é o pior cenário de diagnóstico: funciona no navegador do administrador e falha em curl, em clientes Java, em aplicativos móveis e em integrações entre servidores. Testar com `openssl s_client` a partir de uma máquina que nunca acessou aquele domínio é o que revela o problema.

19.4. Como um certificado é emitido

O processo tem quatro etapas, e a segunda é a única que envolve julgamento humano:

1. **Geração do par.** A chave privada nasce no servidor de destino, ou dentro de um HSM, e **nunca sai de lá**. Autoridade que se ofereça para gerar o par e enviá-lo é uma autoridade a evitar.
2. **CSR.** Uma requisição contendo a chave pública e os nomes pretendidos, assinada com a própria chave privada — o que prova posse.
3. **Validação.** A AC verifica que o solicitante controla os nomes. É aqui que está a diferença real entre os tipos.
4. **Emissão.** A AC assina e devolve o certificado, e o publica nos registros de Certificate Transparency.

Quanto aos tipos de validação: **DV** verifica apenas o controle do domínio, por registro DNS, arquivo em caminho conhecido ou e-mail; é automatizável e é o que o ACME (Barnes et al., 2019) fez virar rotina com o Let's Encrypt. **OV** verifica também a existência jurídica da organização. **EV** acrescenta verificação reforçada, e perdeu quase toda a relevância prática quando os navegadores removeram a barra verde — estudos mostraram que nenhum usuário reagia a ela. Para segurança do canal, **os três são idênticos**: a diferença está na identidade afirmada, não na criptografia.

O ACME merece destaque porque mudou a economia do problema. Certificado gratuito, emitido em segundos e renovado automaticamente por 90 dias tornou o HTTPS universal e — mais importante para este capítulo — tornou **a automação a norma**. Quem renova à mão descobre o custo num feriado.

19.5. A falha estrutural, e as três salvaguardas

Voltando à DigiNotar: o problema não foi um bug, foi o modelo. Com centenas de raízes no armazém de um sistema típico, **a segurança de qualquer domínio é a segurança da AC mais fraca do conjunto** — muitas delas sujeitas a jurisdições e pressões variadas.

Não é caso isolado. A Comodo foi comprometida em 2011. A TÜRKTRUST emitiu por engano, em 2013, um certificado intermediário que permitia assinar qualquer domínio. A Symantec, então a maior AC do mundo, emitiu certificados de teste para domínios do Google sem autorização, e após uma sucessão de descumprimentos teve toda a sua hierarquia distrustada pelos navegadores em 2018 — um evento que obrigou milhares de organizações a trocar certificados às pressas.

Três mecanismos foram criados em resposta.

Certificate Transparency (Laurie; Langley; Kasper, 2013) é o mais eficaz. Toda AC pública precisa publicar cada certificado emitido em registros públicos, apenas-anexação, estruturados como árvores de Merkle — a estrutura do Capítulo 17. O certificado carrega provas (SCTs) de ter sido publicado, e navegadores rejeitam quem não as tiver. O efeito prático: **emissão indevida não pode mais ser secreta**. Não impede a emissão; garante que ela seja detectável — e qualquer organização pode monitorar seus próprios domínios de graça.

CAA (Hallam-Baker; Stradling; Hoffman-Andrews, 2019) é um registro DNS que declara quais ACs estão autorizadas a emitir para aquele domínio. As ACs são obrigadas a consultá-lo e recusar emissão fora da lista. É uma linha de DNS, leva cinco minutos e reduz drasticamente o risco de emissão indevida por AC não relacionada.

Pinning amarra um serviço a chaves específicas. A versão baseada em cabeçalho HTTP, o HPKP, foi **abandonada** por ser perigosa demais: um erro de configuração tornava o site inacessível por meses, sem recurso, e a técnica passou a ser usada como extorsão. O que permanece é o *pinning* embarcado em aplicativos móveis, onde há controle sobre o cliente e sobre o ciclo de atualização

— e mesmo ali exige planejamento de rotação, sob pena de o aplicativo parar de funcionar na renovação do certificado.

19.6. Revogação: a parte que quase não funciona

Certificado comprometido antes do vencimento precisa ser invalidado. O mecanismo existe, e funciona mal.

CRL é uma lista assinada de números de série revogados, publicada pela AC. Cresce indefinidamente, exige download periódico e chega a dezenas de megabytes.

OCSP (Santesson et al., 2013) consulta a AC sobre um certificado específico. Resolve o tamanho e cria dois problemas: latência a cada conexão, e um vazamento de privacidade — a AC passa a saber quais sites cada cliente visita.

A Figura 19.2 mostra por que o mecanismo não entrega o que promete.

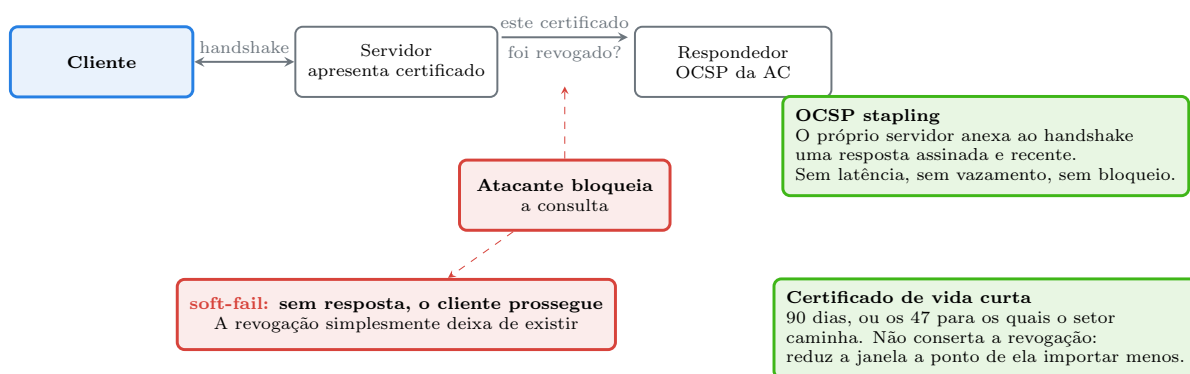


Figura 19.2.: O problema da revogação. O **soft-fail** a torna anulável por quem estiver no caminho; a resposta prática é encurtar a validade.

O problema fatal é o **soft-fail**. Se a consulta OCSP não responde — rede bloqueada, respondedor fora do ar, tempo esgotado —, o cliente **prossegue como se estivesse tudo bem**. A alternativa (**hard-fail**) tornaria a indisponibilidade de um respondedor equivalente à queda de todos os sites que ele atende, o que nunca foi aceitável. A consequência é dura de enunciar e importante de entender: **um atacante posicionado no caminho simplesmente bloqueia a consulta de revogação, e a revogação deixa de existir**.

As respostas práticas foram três. **OCSP stapling** faz o próprio servidor anexar ao handshake uma resposta OCSP recente e assinada, eliminando latência e vazamento — é a configuração correta e é de uma linha. **Listas agregadas nos navegadores** (CRLSets do Chrome, CRLite do Firefox) distribuem revogações de alto impacto diretamente na atualização do produto. E a resposta que efetivamente resolve: **certificados de vida curta**. Um certificado de 90 dias — ou dos 47 dias para os quais o setor caminha — reduz a janela de exposição a ponto de tornar a revogação um problema menor. Não por consertar o mecanismo, mas por reduzir a importância dele.

19.7. PKI interna e autenticação mútua

Nem toda PKI é pública. Organizações operam autoridades próprias para autenticar serviços internos, dispositivos, VPNs e usuários — e é aí que a autenticação mútua (**mTLS**) se torna

viável.

No TLS comum, só o servidor apresenta certificado; o cliente se autentica depois, por senha ou token. Em **mTLS**, o cliente também apresenta certificado, e a autenticação acontece **no handshake**, antes de qualquer dado de aplicação. É a base da comunicação entre microserviços em malha de serviço, do acesso a APIs de alto valor e — no Brasil — da integração com o Pix e com o Open Finance.

O ponto que costuma ser subestimado: **PKI interna é uma responsabilidade permanente, não um projeto**. Exige raiz offline, intermediária operacional, procedimento de emissão, revogação que efetivamente funcione (aqui **hard-fail** é viável, porque a escala é controlada), rotação e um plano para o dia em que a intermediária for comprometida. Uma raiz interna gerada com **openssl** numa tarde, sem procedimento por trás, é passivo: distribui-se por todo o parque e, quando sua chave vazar, todo o parque precisa ser reconfigurado. Ferramentas como **step-ca**, HashiCorp Vault e as ACs privadas de nuvem existem porque a operação manual não se sustenta.

19.8. Brasil em Foco

A ICP-Brasil é uma PKI hierárquica com raiz estatal única. A MP 2.200-2/2001 estabeleceu o ITI como **Autoridade Certificadora Raiz**, no topo de uma cadeia que se ramifica em ACs de primeiro nível (Serpro, Caixa, Serasa, Certisign, Valid, entre outras) e em Autoridades de Registro (ARs), responsáveis pela identificação presencial ou por videoconferência do titular. A diferença estrutural em relação à PKI da web é decisiva: **não há centenas de raízes concorrentes, há uma só**, e o risco DigiNotar — uma AC não relacionada emitindo em nome de outro — não se coloca da mesma forma. Em compensação, concentra o risco: comprometer a AC Raiz comprometeria a cadeia inteira, o que explica o rigor das cerimônias de chave do ITI (Brasil, 2001; Instituto Nacional de Tecnologia da Informação, 2024).

A validação presencial é a característica que define a ICP-Brasil. Enquanto a PKI da web caminhou para validação automatizada de domínio, a ICP-Brasil manteve a verificação de identidade da pessoa — presencial na AR ou por videoconferência com conferência de documentos. É mais cara e mais lenta, e é o que sustenta a presunção legal de autoria: a cadeia liga uma chave a um CPF ou CNPJ verificado por um humano, não a um controle de domínio.

Certificado ICP-Brasil e certificado de servidor web resolvem problemas diferentes. É uma confusão frequente e custosa. O e-CPF e o e-CNPJ servem para **assinar documentos e autenticar em sistemas de governo**; não são reconhecidos pelos navegadores para HTTPS público, porque as raízes da ICP-Brasil não estão nos armazéns do Chrome, do Firefox ou do Windows para essa finalidade. Um site brasileiro que precisa de HTTPS confiável no navegador usa uma AC pública comercial ou o Let's Encrypt. Já a integração com Receita Federal, e-Social, NF-e, PJe e Pix exige ICP-Brasil, e certificado de AC pública internacional não serve. **São dois parques de certificados distintos, com ciclos de renovação distintos**, e tratá-los como um só é fonte recorrente de indisponibilidade.

A NF-e e o Pix tornam a expiração um evento de faturamento. Certificado A1 vence em um ano, A3 em até três. Um e-CNPJ expirado interrompe a emissão de nota fiscal — o que para a operação da empresa, não apenas um serviço de TI. No Pix, o certificado usado na comunicação com o SPI tem seu próprio ciclo, e sua expiração derruba a liquidação. Nos dois casos, o controle correto é o mesmo: inventário com data de expiração, alerta com 60 e 30 dias de antecedência, e responsável nomeado — não “o pessoal do fiscal renova” (Banco Central do Brasil, 2021).

Monitorar Certificate Transparency é um controle gratuito e subutilizado no Brasil. Qualquer organização pode registrar alertas para seus domínios e receber aviso quando um certificado for emitido para eles. Serve para detectar emissão indevida e, com igual utilidade prática, para descobrir **shadow IT**: subdomínios que a equipe de segurança não sabia que existiam aparecem nos registros no momento em que alguém pede um certificado. Custa uma configuração e revela o parque real.

19.9. No Campo

1. Inspeção a cadeia como um cliente novo faria. O teste que revela a intermediária ausente:

```
# -showcerts lista a cadeia inteira que o servidor de fato envia
openssl s_client -connect exemplo.com.br:443 -servername exemplo.com.br
↪ -showcerts </dev/null

# Validacao explicita contra o armazem do sistema, sem cache de navegador
openssl s_client -connect exemplo.com.br:443 -servername exemplo.com.br \
  -verify_return_error -CApath /etc/ssl/certs </dev/null 2>&1 | grep -E 'Verify'
↪ return code|depth'
```

Verify return code: 0 (ok) é o único resultado aceitável. `unable to get local issuer certificate` quase sempre significa intermediária faltando na configuração do servidor — não certificado inválido.

2. Leia o certificado inteiro, não só o assunto. Os campos que decidem se ele serve:

```
openssl s_client -connect exemplo.com.br:443 -servername exemplo.com.br
↪ </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext
↪ subjectAltName,basicConstraints,keyUsage,extendedKeyUsage
```

3. Monitore expiração antes que ela monitore você. Um script no cron e um alerta valem mais que qualquer painel:

```
#!/usr/bin/env bash
# Alerta para certificados que vencem em menos de 30 dias
for host in exemplo.com.br api.exemplo.com.br pix.exemplo.com.br; do
  fim=$(openssl s_client -connect "$host":443 -servername "$host" </dev/null
↪ 2>/dev/null \
    | openssl x509 -noout -enddate | cut -d= -f2)
  restam=$(( ( $(date -d "$fim" +%s) - $(date +%s) ) / 86400 ))
  [ "$restam" -lt 30 ] && echo "ATENCAO: $host vence em $restam dias ($fim)"
done
```

Inclua no mesmo inventário os certificados **ICP-Brasil** — e-CNPJ, NF-e, Pix —, que não aparecem em varredura de porta 443 e são justamente os que param o faturamento.

4. Publique CAA hoje. Cinco minutos, e fecha a porta para emissão por AC não relacionada:

19. PKI, certificados digitais e cadeias de confiança

```
exemplo.com.br.  IN  CAA  0 issue "letsencrypt.org"
exemplo.com.br.  IN  CAA  0 issuewild ";"
exemplo.com.br.  IN  CAA  0 iodef "mailto:seguranca@exemplo.com.br"
```

```
dig +short CAA exemplo.com.br # confirmar publicacao
```

O `issuewild ";"` proíbe curinga, que é o formato preferido de quem emite indevidamente. O `iodef` faz a AC notificar sua equipe quando alguém tentar emitir fora da política.

5. Ligue OCSP stapling. Uma linha, e resolve latência e vazamento de privacidade de uma vez:

```
ssl_stapling on;
ssl_stapling_verify on;
ssl_trusted_certificate /etc/ssl/certs/cadeia-completa.pem;
resolver 1.1.1.1 9.9.9.9 valid=300s;
```

```
openssl s_client -connect exemplo.com.br:443 -status </dev/null 2>/dev/null \
| grep -A2 'OCSP Response Status'
```

6. Monitore Certificate Transparency para os seus domínios. Barato, e detecta tanto emissão indevida quanto shadow IT:

```
# Consulta pontual ao registro publico crt.sh
curl -s 'https://crt.sh/?q=%25.exemplo.com.br&output=json' \
| python -c 'import sys,json;[print(c["name_value"],c["not_before"]) for c in
↪ json.load(sys.stdin)]' \
| sort -u
```

Compare o resultado com o inventário. Nomes que você não reconhece são o achado — em qualquer das duas direções.

7. Automatize a renovação, sempre. Certificado renovado à mão é incidente agendado. Com ACME, a renovação e o recarregamento do serviço são uma unidade:

```
certbot renew --deploy-hook 'systemctl reload nginx'
systemctl list-timers | grep -i certbot # confirmar que o temporizador
↪ existe e roda
```

Teste a renovação **antes** de precisar dela: `certbot renew --dry-run`. Renovação que nunca foi exercitada é backup que nunca foi restaurado.

8. Trate PKI interna como serviço, com dono e procedimento. Raiz offline com chave em HSM ou, no mínimo, em mídia removível guardada em cofre; intermediária de vida curta para emissão; validade curta nos certificados de serviço, com emissão automatizada; revogação que funcione de verdade; e um exercício anual de comprometimento de intermediária. Uma AC criada com `openssl req -x509` numa tarde e distribuída para o parque inteiro é dívida técnica com data de vencimento desconhecida.

19.10. Resumo

- A PKI responde à pergunta que o Capítulo 18 deixou aberta: **como confiar que uma chave pública pertence a quem se pretende alcançar**. Sem ela, criptografia forte protege perfeitamente o canal com o atacante.
- Dos três modelos de confiança — direta, teia e hierárquica —, a web adotou o **hierárquico** por ser o único que escala para centenas de milhões de sites.
- No certificado X.509, o que decide é o **SubjectAltName** (o CN está obsoleto), o **BasicConstraints**, o **KeyUsage** e a validade. Ler certificado é ler esses campos.
- **A cadeia é o que prova, não o certificado**. Folha, intermediária e raiz no armazém local — e o erro operacional mais comum de todos é o servidor não enviar a intermediária, o que funciona no navegador do administrador e quebra em todo o resto.
- O modelo tem uma **falha estrutural**: qualquer AC confiada pode emitir para qualquer domínio. DigiNotar, Comodo, TÜRKTRUST e Symantec são as demonstrações, e a DigiNotar interceptou 300 mil usuários com um certificado tecnicamente perfeito.
- As salvaguardas são **Certificate Transparency** (torna a emissão indevida detectável), **CAA** (restringe quem pode emitir para o seu domínio) e *pinning* apenas onde há controle do cliente — o HPKP baseado em cabeçalho foi abandonado por ser perigoso demais.
- **A revogação quase não funciona**, por causa do **soft-fail**: quem bloqueia a consulta OCSP anula a revogação. O que resolve na prática é **certificado de vida curta** com renovação automatizada, mais OCSP stapling.
- **DV, OV e EV são criptograficamente idênticos**. A diferença está na identidade afirmada, e o EV perdeu relevância quando a barra verde saiu dos navegadores.
- **PKI interna é serviço permanente, não projeto**, e é o que viabiliza mTLS entre serviços — com a vantagem de que, em escala controlada, a revogação pode funcionar de verdade.
- No Brasil, a **ICP-Brasil é uma PKI de raiz estatal única** com validação presencial de identidade, e **não substitui certificado de AC pública para HTTPS** — são dois parques distintos, com dois ciclos de renovação, e confundi-los derruba tanto o site quanto o faturamento.

20. TLS na prática: handshake, versões e configuração segura

Em outubro de 2014, o ataque POODLE tornou pública uma lição que administradores de servidor levaram anos para absorver: **o que derruba um serviço não é o protocolo que ele usa, é o protocolo que ele ainda aceita.**

O cenário era banal. Servidores modernos negociavam TLS 1.2 com navegadores modernos, e mantinham SSL 3.0 habilitado “por compatibilidade com clientes antigos”. Ninguém o usava; ele estava apenas ligado. O ataque explorou exatamente isso: um adversário no caminho derrubava repetidamente as tentativas de handshake em TLS, e o navegador — seguindo uma lógica de degradação bem-intencionada, projetada para lidar com servidores mal implementados — tentava versões cada vez mais antigas até chegar ao SSL 3.0. Que o servidor aceitava. A partir daí, o preenchimento CBC do SSL 3.0, que não era coberto pelo MAC, permitia decifrar um cookie de sessão byte a byte.

O servidor usava TLS 1.2. O navegador usava TLS 1.2. E a sessão foi decifrada em SSL 3.0, porque ambos ainda o aceitavam.

Este capítulo trata do protocolo que carrega quase todo o tráfego sensível da internet, de como seu handshake funciona, e de por que configurá-lo é sobretudo um exercício de **remover** — não de adicionar.

20.1. O que o TLS garante, e onde ele fica

O TLS opera entre a camada de transporte e a de aplicação, como o Capítulo 8 posicionou. Para o HTTP, para o SMTP, para o LDAP ou para um protocolo próprio, ele oferece um soquete que se comporta como TCP e entrega três propriedades:

- **Confidencialidade** dos dados, por AEAD (Capítulo 16).
- **Integridade e autenticidade** de cada registro, pela mesma primitiva.
- **Autenticação do servidor** por certificado (Capítulo 19), e opcionalmente do cliente.

E é útil enunciar o que ele **não** garante, porque a confusão é frequente: não protege o dado antes de entrar no soquete nem depois de sair dele; não esconde **com quem** você fala — o nome do servidor viaja em claro no SNI, e o endereço IP sempre viaja; e não diz nada sobre a idoneidade do outro lado. Certificado válido prova identidade, não boa-fé: um site de golpe tem HTTPS impecável, e o cadeado nunca significou “seguro” no sentido que o público lhe atribui.

20.1.1. A linha do tempo

Tabela 20.1.: Versões de SSL e TLS e sua situação em 2026.

Versão	Ano	Situação
SSL 2.0	1995	Quebrado por projeto. Proibido
SSL 3.0	1996	POODLE. Proibido desde 2015 (RFC 7568)
TLS 1.0	1999	BEAST; aposentado pelos navegadores em 2020
TLS 1.1	2006	Sem AEAD; aposentado junto com o 1.0
TLS 1.2	2008	Aceitável , se bem configurado (Dierks; Rescorla, 2008)
TLS 1.3	2018	Preferencial . Só AEAD, só sigilo encaminhado (Rescorla, 2018)

A configuração correta hoje é curta: **TLS 1.2 e 1.3 habilitados, tudo abaixo desabilitado**. Se o parque não tiver clientes legados, apenas TLS 1.3.

20.2. O handshake do TLS 1.3

A Figura 20.1 mostra a troca completa. Vale segui-la mensagem a mensagem, porque cada uma resolve um problema discutido nos capítulos anteriores.

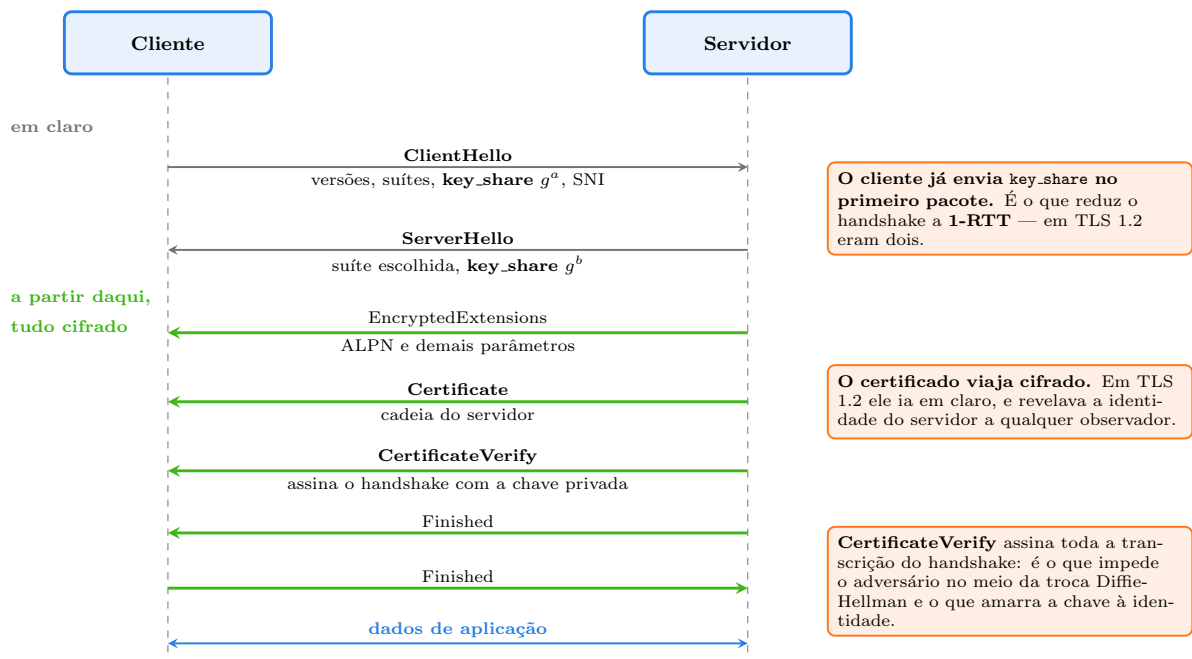


Figura 20.1.: Handshake TLS 1.3 completo em um round-trip. Tudo após o **ServerHello** já viaja cifrado.

O que a figura torna visível:

O cliente aposta na primeira mensagem. Junto do `ClientHello`, ele já envia sua parte pública de ECDHE (`key_share`) para os grupos que julga prováveis. Se acertar — e X25519 ou P-256 acertam quase sempre —, o servidor responde com a sua parte e ambos já derivam as chaves. É o que corta o handshake para **um round-trip**. Se o cliente errar o grupo, o servidor pede outro com `HelloRetryRequest`, e aí custa 2-RTT.

Tudo após o `ServerHello` já vai cifrado, inclusive o certificado. Em TLS 1.2 o certificado do servidor trafegava em claro, entregando a identidade a qualquer observador passivo. Continua sendo possível inferi-la pelo SNI e pelo IP, mas o vazamento direto acabou.

`CertificateVerify` é a peça que resolve o adversário ativo do Capítulo 18: o servidor assina toda a transcrição do handshake com a chave privada correspondente ao certificado. Só quem detém aquela chave produz essa assinatura, e ela cobre os `key_share` — de modo que um atacante que substituísse a troca ECDHE quebraria a assinatura.

As chaves saem de HKDF (Capítulo 17), derivadas do segredo ECDHE em uma cascata de rótulos: chaves de handshake, chaves de aplicação, chave de retomada, cada uma com propósito separado. Nenhuma chave é usada para duas coisas.

20.2.1. O que o TLS 1.3 removeu — e por que isso é a maior melhoria

A lista de remoções é mais importante que a de adições:

- **Transporte de chave por RSA.** Removido: todo handshake tem sigilo encaminhado, por construção.
- **Cifras CBC e todos os modos não autenticados.** Só AEAD. Padding oracle deixa de ser possível.
- **RC4, 3DES, MD5, SHA-1** em qualquer papel.
- **Renegociação e compressão** — origens de ataques como CRIME.
- **Grupos Diffie-Hellman customizados**, que permitiam ao servidor oferecer parâmetros fracos (Logjam).

O resultado é que a superfície de configuração encolheu drasticamente. Em TLS 1.2 é possível montar uma configuração formalmente válida e materialmente insegura; **em TLS 1.3 todas as opções disponíveis são seguras**. É o mesmo princípio do Capítulo 16 sobre APIs de alto nível: quando a interface não permite escolher errado, a documentação deixa de ser a última linha de defesa.

20.2.2. 0-RTT: o desconto que tem preço

O TLS 1.3 permite que um cliente que já conversou antes com o servidor envie dados **junto** com o `ClientHello`, usando uma chave pré-compartilhada da sessão anterior. Latência zero.

O preço é que **dados 0-RTT não têm proteção contra repetição**. Um atacante que capture aquele primeiro pacote pode reenviá-lo depois, e o servidor o processará de novo — sem saber que é repetição. Para um `GET` idempotente, é inofensivo. Para um `POST` que transfere dinheiro, não é.

A regra é simples e deve estar escrita na configuração: **0-RTT apenas para requisições comprovadamente idempotentes, ou desligado**. Na dúvida, desligado — o ganho é de alguns milissegundos, e o risco é de duplicar uma operação.

20.3. Como ler uma suíte de cifras

Nome de suíte parece ruído e é uma especificação completa. A Figura 20.2 decompõe.

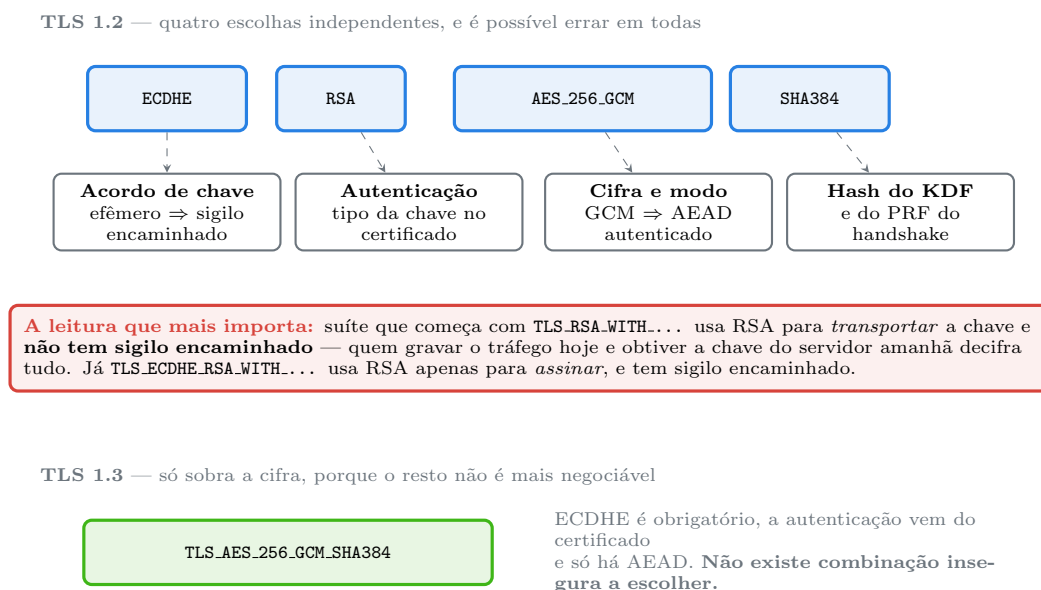


Figura 20.2.: Anatomia de uma suíte de cifras. Em TLS 1.3 sobrou apenas a cifra, porque o resto deixou de ser negociável.

A leitura prática que vale memorizar: **`TLS_RSA_WITH_*` não tem sigilo encaminhado; `TLS_ECDHE_*` tem.** Essa única distinção resolve a maior parte das revisões de configuração de TLS 1.2.

20.4. O que cada ataque histórico ensinou

Tabela 20.2.: Ataques contra TLS e o que cada um deixou como regra.

Ataque	Ano	Causa	Lição permanente
BEAST	2011	IV previsível em CBC no TLS 1.0	IV precisa ser imprevisível
CRIME / BREACH	2012	Compressão sobre dado secreto	Não comprimir o que mistura segredo e entrada do atacante
Lucky13	2013	Temporização no MAC-then-Encrypt	Verificação precisa ser em tempo constante
Heartbleed	2014	Falta de validação de tamanho no OpenSSL	Não é falha do protocolo; é do código. Rotacionar chave após comprometimento
POODLE	2014	Degradação para SSL 3.0 ainda aceito	Desligar o que não se usa
FREAK / Logjam	2015	Suítes de exportação e grupos DH fracos	Compatibilidade herdada é dívida de segurança

Ataque	Ano	Causa	Lição permanente
DROWN	2016	SSL 2.0 na mesma chave em outro serviço	Chave compartilhada propaga o comprometimento
ROBOT	2017	Bleichenbacher ressuscitado em PKCS#1 v1.5	Vulnerabilidade corrigida volta em nova implementação

Duas leituras transversais valem mais que a lista. A primeira: **quase todos foram viabilizados por compatibilidade retroativa** — suítes de exportação, SSL 3.0 habilitado, PKCS#1 v1.5 mantido. A superfície de ataque do TLS foi, historicamente, o que não se usava mas continuava ligado. A segunda: o **Heartbleed não foi falha do protocolo**, e por isso ilustra outra coisa: quando a chave privada pode ter vazado, revogar e reemitir é obrigatório — e o Capítulo 19 já explicou por que revogação em massa funciona mal.

20.5. Configuração segura, em concreto

A configuração correta hoje é curta o suficiente para caber numa página:

```
# nginx - perfil moderno com compatibilidade razoável
ssl_protocols          TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;                                # em TLS 1.3, o cliente
↳ escolhe melhor
ssl_ciphers
↳ ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305
ssl_ecdh_curve          X25519:prime256v1:secp384r1;
ssl_session_timeout     1d;
ssl_session_cache       shared:SSL:10m;
ssl_session_tickets     off;                                  # bilhete mal rotacionado
↳ anula o sigilo encaminhado

ssl_stapling            on;
ssl_stapling_verify     on;
ssl_trusted_certificate  /etc/ssl/certs/cadeia-completa.pem;

add_header Strict-Transport-Security "max-age=63072000; includeSubDomains"
↳ always;
```

Quatro decisões merecem justificativa:

ssl_session_tickets off não é preciosismo. O bilhete de sessão é cifrado com uma chave do servidor; se essa chave não for rotacionada com frequência, quem a obtiver decifra todas as sessões retomadas — **anulando o sigilo encaminhado que o ECDHE entregou**. Ou se desliga, ou se rotaciona a chave de bilhete a cada poucas horas, com um mecanismo que efetivamente funcione em todos os nós do balanceador.

HSTS (Hodges; Jackson; Barth, 2012) instrui o navegador a nunca mais acessar aquele domínio por HTTP, o que fecha a janela do primeiro acesso — o momento em que a degradação para HTTP é possível. **includeSubDomains** é potente e irreversível na prática: ativá-lo com um

subdomínio interno sem HTTPS o torna inacessível pelo prazo do `max-age`. Ativar primeiro com `max-age` curto, verificar, depois aumentar.

A ordem das suítes deixa de importar em TLS 1.3, e por isso `ssl_prefer_server_ciphers off`: o cliente sabe melhor que o servidor se tem aceleração AES em hardware. Um celular sem AES-NI escolherá ChaCha20 e ficará mais rápido e mais seguro contra canal lateral, exatamente como o Capítulo 16 descreveu.

As curvas devem incluir X25519 em primeiro lugar, pelas razões do Capítulo 18.

20.5.1. Autenticação mútua

Quando o cliente também apresenta certificado, a autenticação acontece no handshake, antes de qualquer dado de aplicação:

```
ssl_client_certificate /etc/ssl/ac-interna/ca.pem;    # AC que emite os
↪ certificados de cliente
ssl_verify_client      on;
ssl_verify_depth      2;
ssl_crl                /etc/ssl/ac-interna/revogados.crl;
```

Três observações que separam mTLS que funciona de mTLS decorativo. Primeira: `ssl_verify_client on` valida a **cadeia**, e não a identidade — qualquer certificado emitido por aquela AC entra. A autorização, isto é, *qual* cliente pode acessar *o quê*, precisa ser feita pela aplicação a partir do Subject, e é assunto do Volume 4. Segunda: a CRL precisa ser **atualizada**, ou a revogação é decorativa; em PKI interna o `hard-fail` é viável e deve ser usado. Terceira: certificado de cliente expira, e a renovação precisa ser automatizada com a mesma disciplina do certificado de servidor — com o agravante de que a falha aparece do lado do cliente, longe de quem opera o servidor.

20.6. Brasil em Foco

No Pix, mTLS não é boa prática — é condição de participação. A comunicação com o SPI exige autenticação mútua com certificado ICP-Brasil, e o manual de segurança do arranjo especifica versões de TLS e suítes admitidas. Instituições que integram por meio de prestador herdam a obrigação, e a homologação verifica a configuração efetiva, não a declarada. Reprovar por TLS 1.1 ainda habilitado ou por suíte sem AEAD é ocorrência comum e inteiramente evitável com uma varredura prévia (Banco Central do Brasil, 2021).

O Open Finance elevou o requisito e adicionou o cliente. O ecossistema brasileiro de compartilhamento de dados usa mTLS combinado com OAuth 2.0 e objetos de requisição assinados — o perfil FAPI, tratado no Volume 4. Do ponto de vista deste capítulo, o efeito prático é que cada participante opera **dois parques de certificados** com ciclos independentes: os de transporte e os de assinatura. Expirar qualquer um dos dois interrompe o compartilhamento, e o alerta precisa cobrir ambos.

A LGPD não exige TLS por nome, e a exigência decorre do art. 46. A obrigação de adotar medidas aptas a proteger os dados, avaliada à luz do estado da técnica, torna indefensável em 2026 trafegar dado pessoal sem TLS moderno. Na prática regulatória, o que se examina é a configuração efetiva no momento do incidente — o relatório de uma varredura pública, com data,

é evidência tanto a favor quanto contra. Manter esse registro periodicamente é barato e vale como prova de diligência (Brasil, 2018).

Certificado ICP-Brasil no servidor web público não funciona, e a confusão custa caro. Como o Capítulo 19 registrou, as raízes da ICP-Brasil não estão nos armazéns de confiança dos navegadores para autenticação de servidor. Um site público precisa de AC pública — Let's Encrypt resolve. A ICP-Brasil é para integração com sistemas de governo, assinatura de documentos e arranjos regulados como o Pix. Tentar usar e-CNPJ para HTTPS público resulta em erro de certificado para todos os visitantes.

Órgãos públicos e o gov.br operam sob exigências próprias. A conexão com serviços do governo federal, o peticionamento eletrônico e as integrações com Receita e e-Social especificam versões de TLS e, em muitos casos, exigem mTLS com ICP-Brasil. Como esses sistemas atualizam requisitos por norma técnica e não por aviso ao integrador, a prática que evita indisponibilidade é acompanhar as publicações e testar a integração num ambiente de homologação **antes** de a exigência entrar em vigor.

20.7. No Campo

1. Faça a varredura completa antes de mudar qualquer coisa. O `testssl.sh` é a ferramenta de referência, roda de dentro da rede e não depende de serviço externo:

```
testssl.sh --full --parallel --htmlfile relatorio.html https://exemplo.com.br

# Alternativas de bolso, sem instalar nada
nmap --script ssl-enum-ciphers -p 443 exemplo.com.br
openssl s_client -connect exemplo.com.br:443 -tls1_1 </dev/null # deve FALHAR
openssl s_client -connect exemplo.com.br:443 -tls1_3 </dev/null # deve
↪ funcionar
```

Guarde o relatório com data. É a linha de base para a próxima mudança e a evidência de diligência mencionada acima.

2. Desligue o que não se usa, e confirme que desligou. A lição do POODLE, em forma de comando. Depois de ajustar a configuração, o teste é tentar o protocolo antigo e **esperar a falha** — configuração recarregada nem sempre é configuração aplicada, especialmente atrás de balanceador ou CDN.

3. Verifique se o TLS realmente termina onde você pensa. Balanceador, CDN, WAF e malha de serviço frequentemente encerram o TLS antes da aplicação, e o trecho seguinte pode estar em claro:

```
# De dentro da rede, o trafego entre balanceador e aplicacao esta cifrado?
curl -sv http://backend-interno:8080/health 2>&1 | grep -i '^< HTTP'
ss -tlnp | grep -E ':(80|8080|8443)\b'
```

Descobrir que o tráfego interno entre o balanceador e a aplicação trafega em claro, atravessando três redes, é um achado comum — e é exatamente o que mTLS interno resolve.

4. Trate o bilhete de sessão como chave. Ou desligue, ou rotacione:

```
grep -rn 'ssl_session_tickets\|SSLSessionTickets' /etc/nginx /etc/apache2
↪ 2>/dev/null
```

Bilhete ligado com chave estática desde a instalação anula o sigilo encaminhado de todas as sessões retomadas — e é a configuração padrão de muitas distribuições.

5. Implante HSTS em etapas. Irreversível na prática, então a ordem importa:

```
1. max-age=300 # cinco minutos, verificar
2. max-age=86400 # um dia, verificar
3. max-age=63072000; includeSubDomains # dois anos
4. preload - só depois de garantir que TODO subdomínio tem HTTPS
```

O passo 4 é irreversível de verdade: sair da lista de precarga dos navegadores leva meses.

6. Monitore a expiração e a versão em um só lugar. O que derruba serviço é a expiração; o que reprova auditoria é a versão. Um único verificador cobre ambos:

```
for h in exemplo.com.br api.exemplo.com.br pix.exemplo.com.br; do
  info=$(openssl s_client -connect "$h":443 -servername "$h" </dev/null
  ↪ 2>/dev/null)
  ver=$(echo "$info" | grep -m1 'Protocol')
  fim=$(echo "$info" | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)
  echo "$h | $ver | expira: $fim"
done
```

7. Ligue 0-RTT apenas com conhecimento de causa. Se o serviço tem qualquer rota não idempotente, o ganho de latência não compensa. Se ligar, restrinja a métodos seguros na camada de aplicação e registre a decisão — inclusive porque ela precisa ser revisitada quando uma rota nova for criada.

8. Automatize a renovação e teste a rotação inteira. Renovação que nunca foi exercitada não é controle. Uma vez por trimestre, forçar a renovação num ambiente de homologação e verificar que o serviço recarrega, que a cadeia continua completa e que os clientes mTLS continuam autenticando. É o mesmo princípio de restaurar backup, aplicado a certificado.

20.8. Resumo

- **O que derruba um serviço é o que ele ainda aceita, não o que ele usa.** POODLE explorou SSL 3.0 mantido “por compatibilidade” em servidores que negociavam TLS 1.2 normalmente.
- A configuração correta em 2026 é **TLS 1.2 e 1.3 apenas**; só TLS 1.3 se não houver cliente legado. Tudo abaixo é proibido.
- O **handshake TLS 1.3 fecha em 1-RTT** porque o cliente já envia `key_share` no `ClientHello`, e tudo após o `ServerHello` — inclusive o certificado — viaja cifrado.
- **CertificateVerify é a peça que impede o adversário no meio**, assinando toda a transcrição do handshake com a chave privada do certificado.
- A maior melhoria do TLS 1.3 foi o que ele **removeu**: transporte de chave por RSA, CBC, renegociação, compressão e grupos DH customizados. Não há mais combinação insegura a escolher.

- **0-RTT não tem proteção contra repetição.** Só para requisições comprovadamente idempotentes, ou desligado.
- Na leitura de suítes, a distinção que mais importa: **TLS_RSA_WITH_* não tem sigilo encaminhado; TLS_ECDHE_* tem.**
- A história dos ataques a TLS é, em grande medida, a história da **compatibilidade retroativa**: suítes de exportação, versões antigas habilitadas, preenchimentos obsoletos mantidos.
- **Bilhete de sessão com chave estática anula o sigilo encaminhado.** Desligar ou rotacionar de verdade.
- **HSTS fecha a janela do primeiro acesso** e é irreversível na prática — implantar em etapas, e preload só com todos os subdomínios em HTTPS.
- No Brasil, **mTLS com ICP-Brasil é condição de participação no Pix e no Open Finance**, com dois parques de certificados a monitorar. E certificado ICP-Brasil **não serve** para HTTPS público — para isso, AC pública.

21. Erros clássicos: criptografia mal empregada

Em abril de 2020, com o mundo inteiro em reuniões remotas, pesquisadores do Citizen Lab publicaram uma análise do Zoom. A empresa anunciava “criptografia de ponta a ponta” no material de marketing e na interface do produto. A análise encontrou outra coisa.

O conteúdo das reuniões era protegido com AES-128 — em **modo ECB**. O mesmo modo do Capítulo 16, o mesmo que expôs 153 milhões de senhas da Adobe, o modo que preserva no texto cifrado toda a estrutura repetida do texto claro. Pior: as chaves eram geradas e distribuídas pelos **próprios servidores do Zoom**, alguns deles em Pequim, atendendo reuniões inteiramente norte-americanas. Ou seja, o operador tinha as chaves — o que, por definição, torna impossível chamar aquilo de ponta a ponta.

Três erros distintos e independentes, empilhados num produto usado por centenas de milhões de pessoas: **modo de operação errado, custódia de chave incompatível com a promessa e afirmação de segurança que a arquitetura não sustentava**. Nenhum deles é um problema de matemática. Todos os três aparecem, com nomes diferentes, em auditorias de sistemas brasileiros toda semana.

Este capítulo fecha o volume catalogando os erros que efetivamente acontecem. Ele é deliberadamente uma lista de verificação: cada item traz o que é, por que acontece, como detectar e como corrigir.

21.1. Onde a criptografia realmente falha

Antes do catálogo, vale olhar a distribuição. A Figura 21.1 organiza as camadas de um sistema criptográfico pela frequência com que cada uma é a causa raiz de um incidente.

A leitura é desconfortável e verdadeira: **a discussão sobre qual algoritmo usar é a menos importante das que precisam ser feitas**. AES-256 numa arquitetura em que a chave está no repositório do Git é pior do que AES-128 com a chave num cofre, e a primeira configuração passa em qualquer questionário de fornecedor que pergunte apenas o nome do algoritmo.

21.2. Categoria 1 — Escolha errada de primitiva

1.1 ECB. Já tratado no Capítulo 16 e ainda o achado mais comum. Aparece por omissão de configuração — `Cipher.getInstance("AES")` em Java seleciona ECB —, por exemplo copiado de fórum e por tutorial desatualizado. Detecta-se com `grep`; corrige-se migrando para AEAD, com uma janela de recifragem.

1.2 Hash rápido para senha. `SHA256(senha + sal)` é rápido, e essa é exatamente a propriedade errada. Uma GPU faz dezenas de bilhões por segundo. A correção é Argon2id, scrypt ou bcrypt, com o detalhe operacional que o Capítulo 17 registrou: **não é preciso conhecer a senha para migrar**. Aplica-se a nova função sobre o resumo antigo (`argon2(sha256(senha))`), registra-se o esquema, e na próxima autenticação bem-sucedida recalcula-se do zero.

21. Erros clássicos: criptografia mal empregada

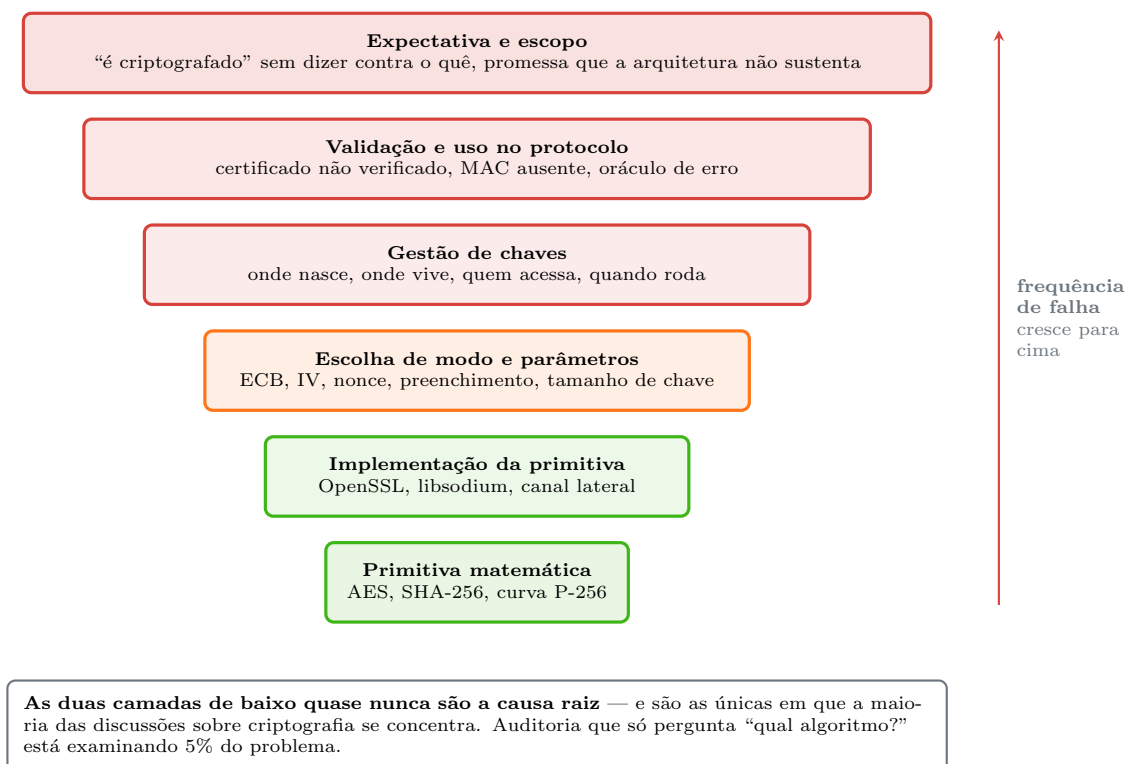


Figura 21.1.: Camadas de um sistema criptográfico. Quanto mais alto, mais perto do incidente real.

1.3 Codificação confundida com criptografia. Base64, hexadecimal, ROT13 e “ofuscação” não são criptografia: não há chave, e a reversão é pública. O campo `encrypted_token` que decodifica com uma linha é pior que o mesmo campo em claro, porque encerra a discussão de segurança. Detectável de forma quase automática:

```
# Campos que parecem cifrados e decodificam em texto legível
echo "$VALOR" | base64 -d 2>/dev/null | strings | head
```

1.4 Criptografia própria. “Nosso algoritmo proprietário” é o oposto do princípio de Kerckhoffs (Capítulo 15). Inclui variantes mais sutis: XOR com chave fixa, embaralhamento de bytes, “AES modificado para nosso caso”. A correção é sempre a mesma — substituir por primitiva padrão, com a nota de que a migração exige recifrar tudo o que foi protegido pelo esquema anterior, e que os dados devem ser tratados como se nunca tivessem sido protegidos.

21.3. Categoria 2 — Aleatoriedade e nonce

2.1 Gerador não criptográfico. `Math.random()`, `rand()`, `random.random()` gerando token, senha temporária, identificador de sessão ou chave. O estado interno é reconstruível a partir de saídas observadas. Corrige-se com o CSPRNG da plataforma, e a varredura está no Capítulo 15.

2.2 Reúso de IV ou nonce. Catastrófico em CTR e em GCM, onde permite recuperar a chave de autenticação. Causas típicas: IV fixo no código, IV derivado de dado previsível (identificador do registro, carimbo de tempo em segundos), contador reiniciado ao restaurar um instantâneo, ou o mesmo contador em duas instâncias de um serviço escalado horizontalmente. Detecção:


```
# IV ou nonce constante no código
grep -rnE
↪ 'iv\s*=\s*["\x27][0-9a-fA-F]{16,}|nonce\s*=\s*["\x27]|IvParameterSpec\(\new
↪ byte\[' \
    --include='*.java' --include='*.py' --include='*.go' --include='*.js' .
```

A correção estrutural é usar AES-GCM-SIV onde a unicidade não puder ser garantida, ou nonce aleatório de 96 bits com contador de volume por chave.

2.3 Nonce de ECDSA. O caso do PlayStation 3, do Capítulo 18. Corrige-se com ECDSA determinístico (RFC 6979) ou, melhor, migrando para Ed25519.

2.4 Semente previsível. Semear um gerador com o tempo atual, com o PID ou com um valor derivado do dado. Torna a saída enumerável mesmo que o gerador seja criptográfico. CSPRNG do sistema não precisa de semente — essa é uma das razões para usá-lo.

21.4. Categoria 3 — Integridade ausente ou mal feita

3.1 Cifrar sem autenticar. O erro conceitual mais caro, tratado no Capítulo 16. Sintoma na base de código: uso de AES/CBC ou AES/CTR sem nenhum MAC associado. Consequência: o atacante altera o texto cifrado de forma controlada e a aplicação decifra o resultado adulterado sem perceber. Correção: AEAD.

3.2 Ordem errada de MAC. MAC-then-Encrypt e Encrypt-and-MAC são frágeis; só Encrypt-then-MAC com chaves separadas é seguro. Se a construção é manual, é candidata a substituição por AEAD.

3.3 Comparação de MAC com ==. Vaza o MAC byte a byte por temporização. A varredura e a correção estão no Capítulo 17 — e vale notar que o mesmo erro aparece em comparação de token de API, de código de recuperação e de assinatura de webhook, não só de MAC.

3.4 MAC construído por concatenação. $H(\text{segredo} || \text{mensagem})$ sofre extensão de comprimento. Use HMAC.

21.5. Categoria 4 — Gestão de chaves

Esta categoria isolada responde por mais incidentes do que todas as outras somadas.

4.1 Chave no código-fonte. O clássico. Uma vez versionada, a chave permanece no histórico do Git para sempre, e a resposta correta é **rotacionar, não apagar o arquivo**. Serviços de varredura automatizada percorrem repositórios públicos continuamente: uma chave de nuvem exposta é explorada em minutos, não em dias.

```
# Chaves e segredos no historico completo, nao so na arvore atual
git log --all -p | grep -nE 'BEGIN (RSA |EC |OPENSSH )?PRIVATE
↪ KEY|AKIA[0-9A-Z]{16}|-----BEGIN'
git log --all --diff-filter=A --name-only --pretty=format: \
    | grep -E '\.(key|pem|pfx|p12|jks|env)$' | sort -u
```

4.2 Chave guardada junto do dado cifrado. Chave na mesma tabela, no mesmo bucket, no mesmo backup. Quem obtém o dado obtém a chave, e a cifragem vira decoração. É o padrão mais comum em “cifragem de campo” implementada às pressas para atender a uma exigência de auditoria.

4.3 Chave derivada de senha sem KDF. `chave = SHA256("senha-da-empresa")` produz uma chave de 256 bits com a entropia de uma senha — algo entre 20 e 40 bits. Se a derivação a partir de senha for inevitável, use Argon2id ou PBKDF2 com parâmetros altos, e com sal por instalação.

4.4 Ausência de rotação e de identificador de chave. Sistema que não previu o campo de identificador de chave no formato do texto cifrado (Capítulo 16) não consegue rotacionar sem indisponibilidade, e por isso nunca rotaciona. O erro se manifesta anos depois, no dia em que a rotação vira obrigatória.

4.5 Chave privada gerada fora do destino. Par gerado numa estação e enviado por e-mail ou por sistema de chamados. Destroí o não repúdio (Capítulo 18), porque o controle nunca foi exclusivo, e deixa cópia em todo lugar por onde passou.

4.6 Chave sem procedimento de comprometimento. Não é erro de código, é ausência de plano: quando a chave vaza, qual dado precisa ser recifrado, em que ordem, em quanto tempo, e quem decide. A hora de escrever isso não é durante o incidente.

21.6. Categoria 5 — Validação que não valida

Este é o erro que anula tudo o que os capítulos anteriores construíram: criptografia impecável, com a pessoa errada.

5.1 Verificação de certificado desabilitada. Aparece durante o desenvolvimento — para contornar um certificado autoassinado no ambiente de teste — e vai para produção. A varredura é a de maior retorno deste capítulo:

```
grep -rnE
↪ 'verify\s*=\s*False|InsecureSkipVerify\s*:\s*true|rejectUnauthorized\s*:\s*false|\
CURLOPT_SSL_VERIFYPEER\s*,\s*(0|false)|ALLOW_ALL_HOSTNAME_VERIFIER|NSAllowsArbitraryLoads|\
TrustAllCerts|X509TrustManager' \
--include='*.py' --include='*.go' --include='*.js' --include='*.ts'
↪ --include='*.java' \
--include='*.php' --include='*.cs' --include='*.swift' .

# curl -k e wget --no-check-certificate em scripts e pipelines
grep -rnE 'curl .*(-k|--insecure)|wget .*--no-check-certificate' \
--include='*.sh' --include='*.yaml' --include='*.yml'
↪ --include='Dockerfile*' .
```

Toda ocorrência é achado. A correção correta **não** é desabilitar em menos lugares: é instalar a AC interna no armazém de confiança do cliente, ou usar certificado válido no ambiente de teste — hoje trivial com ACME e validação por DNS.

5.2 Cadeia validada, nome não verificado. Erro mais sutil: a biblioteca valida a assinatura da cadeia mas não confere se o nome pedido consta do `SubjectAltName`. Qualquer certificado válido de qualquer domínio passa. Ocorre em código que usa APIs de baixo nível de TLS e monta a verificação à mão.

5.3 Aceitar certificado autoassinado em produção. Frequentemente justificado como “é rede interna”. Rede interna é exatamente onde a movimentação lateral acontece, e o Volume 9 mostra o que se faz a partir dela. A resposta é PKI interna (Capítulo 19), não exceção.

5.4 Confiar em identidade não verificada dentro do canal. Cabeçalho X-User-Id vindo de um serviço interno, ou Subject de certificado de cliente usado como autorização sem checagem de qual cliente pode fazer o quê. O canal está autenticado; a requisição, não.

21.7. Categoria 6 — Vazamento por canal lateral

6.1 Mensagem de erro que distingue. A base do padding oracle (Capítulo 15). Vale para toda a família: “usuário não existe” versus “senha incorreta”, “assinatura inválida” versus “carimbo expirado”, “preenchimento inválido” versus “conteúdo inválido”. Regra: **erro genérico para o cliente, detalhe completo no registro interno.**

6.2 Diferença de tempo. Comparação byte a byte, decifragem que só ocorre quando o formato é válido, consulta a banco que só acontece se o usuário existir. Corrige-se com comparação em tempo constante e, onde couber, com custo uniformizado — executar a verificação de senha mesmo quando o usuário não existe.

6.3 Dado sensível em registro. Chave, token, senha ou payload decifrado indo para o log em nível de depuração. Sobrevive por anos em sistema de agregação de logs, frequentemente com controle de acesso mais frouxo que o do banco original. É achado de auditoria recorrente e de correção barata.

21.8. Categoria 7 — Escopo e expectativa

7.1 “O banco é criptografado.” Cifragem em repouso protege contra roubo do disco e descarte de hardware. Com o sistema ligado, o dado está claro para todo processo autorizado, incluindo o invasor que obteve execução. A afirmação só tem significado acompanhada do cenário de ameaça.

7.2 Ponta a ponta que não é ponta a ponta. Se o operador do serviço pode ler o conteúdo — porque gera, distribui ou custodia as chaves —, não é ponta a ponta, qualquer que seja o material de marketing. Foi o caso do Zoom em 2020. A pergunta que decide: **quem detém as chaves?**

7.3 Criptografia usada como substituto de autorização. Cifrar não impede que a aplicação, com credencial legítima, devolva o registro de outro usuário. É controle de acesso, tema do Volume 4.

7.4 Conformidade confundida com segurança. “Somos criptografados, atendemos à LGPD.” Nenhuma norma exige um algoritmo específico; todas exigem adequação ao risco e ao estado da técnica. A conformidade é consequência de uma arquitetura defensável, não substituto dela.

21. Erros clássicos: criptografia mal empregada

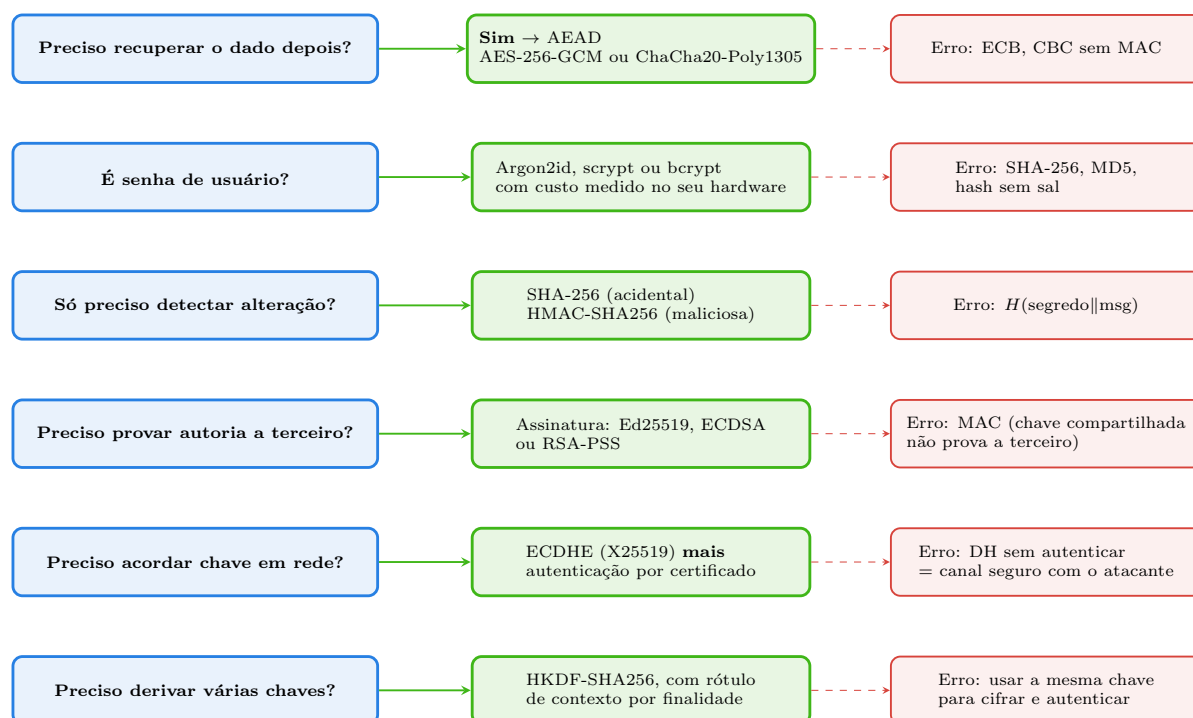


Figura 21.2.: Guia de decisão. A coluna da direita é o erro que corresponde a cada escolha correta.

21.9. O guia rápido de decisão

A Figura 21.2 condensa em uma imagem a resposta correta para as perguntas que aparecem no dia a dia.

E a regra que resume o volume inteiro em uma linha: **na dúvida, use libsodium ou a API de alto nível da sua plataforma, e não tome nenhuma dessas decisões.**

21.10. Brasil em Foco

A ANPD examina adequação, e adequação se demonstra com documento datado. Na apuração de um incidente, a autoridade avalia se as medidas eram apropriadas ao risco e ao estado da técnica (art. 46 da LGPD). Encontrar ECB, MD5 em senha ou verificação de certificado desabilitada torna a defesa insustentável — não porque a lei os proíba pelo nome, mas porque o padrão de mercado está documentado publicamente há mais de uma década. O que ajuda na defesa é o oposto do que costuma existir: registro de decisão de arquitetura com data, relatório de varredura periódica, plano de correção com prazos e evidência de execução (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

Chave vazada em repositório público é incidente de segurança, com dever de avaliação. Encontrar credencial ou chave privada no histórico do Git não é dívida técnica: é comprometimento presumido. A conduta correta é rotacionar imediatamente, investigar o uso indevido no período de exposição e avaliar, sob os arts. 48 e 50, se houve acesso a dado pessoal — o que dispara o dever de comunicação. Apagar o arquivo e seguir em frente é a resposta errada em três dimensões: técnica, jurídica e probatória.

“Criptografado” não afasta automaticamente a comunicação de incidente. A Resolução CD/ANPD 15/2024 orienta a avaliação de risco relevante ao titular, e cifragem adequada com chave comprovadamente não comprometida pesa a favor. Mas ECB, hash rápido de senha ou chave armazenada junto do dado não sustentam essa tese — nesses casos o dado deve ser tratado como exposto. A diferença entre comunicar e não comunicar um incidente pode se resumir a onde a chave estava (Autoridade Nacional de Proteção de Dados, 2024).

PCI-DSS v4 e o Pix transformam vários itens deste capítulo em reprovação objetiva.

O requisito 3 do PCI-DSS trata de proteção de dado de conta e do ciclo de vida das chaves; o requisito 4, de transmissão. No arranjo do Pix, a homologação verifica versões de TLS, suítes e autenticação mútua. São contextos em que os erros aqui catalogados não geram debate sobre razoabilidade: geram não conformidade documentada (Banco Central do Brasil, 2021; PCI Security Standards Council, 2024).

Assinatura eletrônica no Brasil depende de premissas que os erros deste capítulo destroem. A presunção de autoria da MP 2.200-2 pressupõe controle exclusivo da chave privada pelo titular. Chave A1 copiada para três servidores, compartilhada entre a equipe do fiscal ou guardada em rede compartilhada esvazia essa premissa — e a consequência não é técnica, é a fragilidade da assinatura quando ela for contestada. Quem opera assinatura em escala precisa tratar custódia como controle jurídico, não apenas operacional (Brasil, 2001).

21.11. No Campo

1. Rode a varredura completa hoje, e guarde o resultado com data. Os sete comandos deste capítulo, num único roteiro, produzem uma linha de base útil em qualquer conversa posterior:

```
#!/usr/bin/env bash
# auditoria-cripto.sh - varredura de base, para rodar no CI e trimestralmente
set -u
echo "== 1. ECB e modo implicito =="
grep -rniE 'MODE_ECB|AES|ECB|aes-[0-9]+--ecb|Cipher\.getInstance\("AES"\)'
  ↪ --include='*.*' . || echo " nenhum"
echo "== 2. Verificacao de certificado desabilitada =="
grep -rnE
  ↪ 'verify\s*=\s*False|InsecureSkipVerify|rejectUnauthorized\s*:\s*false|VERIFYPEER\s*,\s*'
  ↪ --include='*.*' . || echo " nenhum"
echo "== 3. Gerador nao criptografico =="
grep -rnE 'Math\.random|new Random\(|random\.random\(' --include='*.*' . | grep
  ↪ -viE 'test|spec|mock' || echo " nenhum"
echo "== 4. Hash rapido em senha =="
grep -rniE '(sha1|sha256|md5)\.{0,30}(senha|password|passwd|pwd)'
  ↪ --include='*.*' . || echo " nenhum"
echo "== 5. Comparacao de MAC insegura =="
grep -rniE '(hmac|mac|signature|token|digest)\s*==' --include='*.*' . || echo "
  ↪ nenhum"
echo "== 6. IV ou nonce constante =="
grep -rnE 'iv\s*=\s*["\x27][0-9a-fA-F]{16,}|IvParameterSpec\((new byte\[
  ↪ --include='*.*' . || echo " nenhum"
echo "== 7. Segredo versionado (historico completo) =="
git log --all --diff-filter=A --name-only --pretty=format: 2>/dev/null \
```

```
| grep -E '\.(key|pem|pfx|p12|jks|env)$' | sort -u || echo "nenhum"
```

2. Coloque as varreduras no pipeline, não só na auditoria anual. Achado que aparece no *pull request* custa minutos; o mesmo achado em produção custa uma janela de manutenção e uma recifragem. Ferramentas dedicadas — **gitleaks** e **trufflehog** para segredos, **semgrep** com as regras de criptografia, **bandit** no Python — cobrem mais que o **grep** e integram bem.

3. Classifique os achados por exploração, não por quantidade. Nem todo md5 é o mesmo achado. A ordem de prioridade que funciona:

Tabela 21.1.: Priorização de achados criptográficos.

Prioridade	Achado	Por quê
Crítica	Chave ou credencial versionada	Comprometimento presumido, exploração em minutos
Crítica	Verificação de certificado desabilitada em produção	Anula todo o resto
Alta	ECB, ou cifragem sem autenticação	Vazamento estrutural
Alta	Hash rápido em senha	Base vazada equivale a senhas expostas
Alta	Nonce ou IV reutilizado em CTR/GCM	Perda de confidencialidade e de autenticidade
Média	Comparação de MAC não constante	Explorável, exige acesso e volume
Média	TLS 1.0/1.1 ainda aceito	Exige adversário posicionado
Baixa	MD5 em verificação de download	Depende de propriedade não quebrada

4. Escreva a política de criptografia em uma página. Algoritmos aprovados por finalidade, tamanhos mínimos, onde as chaves vivem, período de rotação, o que fazer em caso de comprometimento, e a data da próxima revisão. Uma página que existe vale mais que trinta que ninguém lê — e é o documento que a ANPD, o auditor de PCI e o cliente corporativo pedem, nessa ordem.

5. Faça o exercício de comprometimento de chave antes de precisar dele. Escolher uma chave de produção não crítica, simular o vazamento e executar o procedimento inteiro: rotação, recifragem, invalidação de sessões, comunicação. Cronometrar. O número que sair é o seu tempo real de resposta, e ele costuma ser várias vezes maior que o estimado.

6. Padronize por biblioteca, não por documento. Em vez de um guia dizendo como usar AES corretamente, exponha uma biblioteca interna com duas funções — **cifrar(dado, contexto)** e **decifrar(pacote, contexto)** — que já embutem AEAD, nonce, identificador de chave e versão de formato. É a aplicação do princípio do Capítulo 16 em escala organizacional: **remover a decisão é mais eficaz que documentá-la.**

7. Reavalie anualmente, com data marcada. O inventário de algoritmos do Capítulo 15, a validade dos certificados do Capítulo 19 e os parâmetros de custo do Argon2 do Capítulo 17 envelhecem. Uma revisão anual agendada é o que impede a descoberta acidental de SHA-1 em produção durante uma auditoria — e é o insumo do inventário que a migração pós-quântica vai exigir.

8. Ao avaliar qualquer produto ou fornecedor, pergunte pelas camadas de cima. Não “qual algoritmo vocês usam”, mas: onde a chave é gerada, quem tem acesso a ela, com que frequência roda, o que acontece se vazarem, e — quando houver promessa de ponta a ponta — quem detém as chaves. As respostas a essas cinco perguntas discriminam fornecedores muito melhor que o nome do algoritmo, que todos acertam.

21.12. Resumo

- **A criptografia falha nas camadas de cima.** Primitiva e implementação quase nunca são a causa raiz; gestão de chaves, validação e expectativa são. Auditoria que só pergunta “qual algoritmo?” examina uma fração mínima do problema.
- **Escolha de primitiva:** ECB não tem uso legítimo, hash rápido não guarda senha, codificação não é criptografia, e algoritmo próprio é o oposto do princípio de Kerckhoffs.
- **Aleatoriedade e nonce:** gerador de propósito geral fora de caminho criptográfico, nonce único sem exceção, e ECDSA determinístico ou Ed25519 onde houver assinatura.
- **Integridade:** cifrar sem autenticar é erro de projeto; MAC comparado com == vazava byte a byte; `H(segredo || mensagem)` não é MAC.
- **Gestão de chaves responde por mais incidentes que todas as outras categorias somadas.** Chave versionada é comprometimento presumido — rotacionar, não apagar. Chave junto do dado cifrado torna a cifragem decorativa. Sem identificador de chave no formato, não há rotação possível.
- **Validação desabilitada anula tudo.** `verify=False`, `InsecureSkipVerify`, `curl -k` em pipeline: cada ocorrência é achado, e a correção é AC interna no armazém de confiança, não exceção no código.
- **Canal lateral:** erro genérico para o cliente e detalhe no registro interno; comparação em tempo constante; nada de dado sensível em log de depuração.
- **Escopo:** “é criptografado” só significa algo com o cenário de ameaça junto; ponta a ponta se define por quem detém as chaves; e criptografia não substitui autorização.
- **O guia de decisão** resolve a maioria dos casos, e a regra que o resume é não decidir: usar `libsodium` ou a API de alto nível da plataforma.
- No Brasil, os erros deste capítulo têm efeito jurídico direto: comprometem a defesa de adequação perante a ANPD, geram não conformidade objetiva em PCI-DSS e na homologação do Pix, e esvaziam a presunção de autoria da assinatura ICP-Brasil quando destroem o controle exclusivo da chave privada.

Parte IV.

**Volume 4 — Identidade, Autenticação e
Controle de Acesso**

22. Modelos de controle de acesso: DAC, MAC, RBAC e ABAC

Uma analista de suporte de um provedor regional recebe um chamado: cliente reclama que não consegue acessar o portal. Para investigar, ela abre a ferramenta interna de atendimento, digita o CPF e vê o cadastro completo — endereço, telefone, histórico de pagamento, contrato. Resolve o problema em três minutos e fecha o chamado.

No mesmo dia, sem nenhum chamado aberto, ela digita o CPF da ex-companheira. E o do vizinho. E o de uma pessoa pública que mora na área de cobertura.

Nenhum sistema foi invadido. Nenhuma senha foi quebrada. Nenhum controle técnico falhou, porque nenhum controle técnico existia para esse caso: o sistema perguntou “esta pessoa está autenticada e pertence ao grupo Suporte?”, recebeu “sim” e liberou tudo. A pergunta que ninguém fez foi “**esta pessoa tem motivo para ver este registro específico, agora?**”.

Esse é o problema do controle de acesso, e ele é diferente do problema da autenticação. Autenticar responde *quem é você*. Autorizar responde *o que você pode fazer com o quê, sob quais condições*. Os capítulos anteriores construíram a máquina de provar identidade; este volume começa perguntando o que fazer com a identidade depois de provada — e o Volume 3 inteiro não ajuda aqui, porque, como registrou o Capítulo 21, criptografia não substitui autorização.

22.1. O vocabulário mínimo

Todo modelo de controle de acesso, por mais elaborado que seja, é uma resposta a uma pergunta de três termos:

- **Sujeito** — quem age. Usuário, processo, serviço, dispositivo, chave de API. Não necessariamente uma pessoa.
- **Objeto** — sobre o que se age. Arquivo, registro, endpoint, fila, tabela, porta de rede, painel.
- **Operação** — o que se faz. Ler, escrever, executar, apagar, aprovar, exportar, delegar.

A soma dessas três coisas é uma **permissão**. Uma **política** é o conjunto de regras que decide quais permissões existem. E o componente que aplica a decisão em tempo de execução é o **monitor de referência**, conceito formulado por James Anderson em 1972 e que ainda define o padrão-ouro: ele precisa ser **inviolável** (não pode ser contornado ou adulterado), **sempre invocado** (nenhum acesso escapa dele) e **pequeno o bastante para ser verificado** (auditável na prática, não em tese).

A terceira propriedade é a que quase sempre se perde. Um monitor de referência espalhado por 4.000 verificações `if` no código da aplicação não é auditável — e é exatamente assim que a maioria dos sistemas corporativos implementa autorização.

Butler Lampson formalizou em 1974 a representação mais direta disso: a **matriz de acesso**, com sujeitos nas linhas, objetos nas colunas e operações nas células (Lampson, 1974). É uma

22. Modelos de controle de acesso: DAC, MAC, RBAC e ABAC

abstração excelente e uma implementação péssima — uma organização com 5.000 usuários e 200.000 objetos precisaria de um bilhão de células, quase todas vazias. Todos os modelos deste capítulo são, em algum sentido, estratégias diferentes para **comprimir essa matriz**.

	relatorio.pdf	folha.xls	/api/clientes
Ana	ler, escrever	—	ler
Bruno	ler	ler, escrever	—
backup-svc	ler	ler	—

Corte por linha → capacidade
“o que *este sujeito* carrega”
descriptor de arquivo,
token, chave assinada

Corte por coluna → ACL
“quem pode agir sobre *este objeto*”
Unix, NTFS, buckets, tabelas

A matriz completa é inviável: 5.000 sujeitos
× 200.000 objetos = 10^9 células, quase todas
vazias. Todo modelo deste capítulo é uma
estratégia de compressão dela.

Figura 22.1.: A matriz de acesso de Lampson e suas duas decomposições clássicas.

O corte por coluna produz a **lista de controle de acesso (ACL)**: cada objeto carrega a lista de quem pode fazer o quê com ele. É o modelo do Unix, do NTFS, das políticas de bucket. Responde bem a “quem tem acesso a este arquivo?” e mal a “a que tudo o Bruno tem acesso?”.

O corte por linha produz a **capacidade**: uma credencial não falsificável que o sujeito carrega e apresenta, e que já contém a autorização. Descriptor de arquivo no Unix, token de portador em HTTP, URL assinada de bucket. Responde bem a “o que este sujeito pode fazer?” e mal a “revogue já” — porque a autorização está distribuída, não centralizada. A dificuldade de revogação de capacidades vai reaparecer com força no Capítulo 25, quando tokens OAuth entrarem em cena.

22.2. DAC — o dono decide

No **controle de acesso discricionário**, quem cria o objeto vira seu proprietário e decide, a seu critério, quem mais pode acessá-lo. É o modelo mais familiar do mundo: `chmod` no Linux, aba Segurança no Windows, botão “Compartilhar” no Google Drive.

```
# DAC em Unix: o dono decide, e o dono pode delegar a decisao
$ ls -l relatorio.pdf
-rw-r----- 1 ana financeiro 84213 relatorio.pdf
#   ^^^ ^^^ ---
#   dono grupo outros

$ chmod o+r relatorio.pdf      # Ana, sozinha, expoe para todo o sistema
$ setfacl -m u:bruno:rw relatorio.pdf  # ACL POSIX: granularidade por usuario
```

A vantagem do DAC é a autonomia: não há gargalo administrativo, quem produz o dado gerencia o dado. A desvantagem é exatamente a mesma frase lida com outro olhar: **a política de segurança da organização fica na mão de cada usuário individual**, e não há como garantir globalmente nada.

O problema estrutural do DAC tem nome. Quando Ana concede leitura a Bruno, ela confia em Bruno — mas o modelo não distingue Bruno do *software que Bruno executa*. Um programa rodando com a identidade de Bruno herda todos os acessos de Bruno e pode copiar o conteúdo para onde quiser. É o **problema do cavalo de Troia**: o DAC controla quem lê, não o que acontece depois da leitura. Não existe, em DAC puro, o conceito de que um dado carrega restrições que o acompanham.

Esse é o ponto exato onde o modelo seguinte nasceu.

22.3. MAC — a organização decide, e ninguém pode contrariar

No **controle de acesso obrigatório**, a política é definida por uma autoridade central e é **imposta pelo sistema independentemente da vontade do dono**. Nem o criador do arquivo pode afrouxá-la. Objetos recebem **rótulos** de classificação, sujeitos recebem **habilitações**, e uma regra fixa compara os dois.

O modelo formal clássico é o de **Bell-LaPadula**, criado para o Departamento de Defesa dos EUA e voltado exclusivamente à confidencialidade (Bell; LaPadula, 1976). Suas duas regras são curtas e contraintuitivas:

- **Sem leitura para cima** (*no read up*): sujeito com habilitação “Confidencial” não lê documento “Secreto”. Óbvio.
- **Sem escrita para baixo** (*no write down*): sujeito com habilitação “Secreto” **não escreve** em documento “Confidencial”. Nada óbvio — e é a regra que mata o cavalo de Troia. Um processo que leu um dado secreto fica impedido, pelo sistema, de gravá-lo num destino de classificação inferior. A restrição acompanha o fluxo do dado, não a intenção de quem age.

Kenneth Biba propôs em 1977 o modelo espelhado para **integridade**, com as regras invertidas: sem leitura para baixo, sem escrita para cima (Biba, 1977). Um processo de alta integridade não consome entrada de fonte não confiável; um processo de baixa integridade não modifica dado de alta integridade. É a formalização da ideia de que dado que veio da internet não deve ser tratado como dado que veio do banco corporativo.

Quem administra Linux convive com MAC todo dia, ainda que sem esse nome:

```
# SELinux: rotulos MAC sobre objetos e processos
$ ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0    index.html

$ ps -eZ | grep nginx
system_u:system_r:httpd_t:s0      1284 ?    nginx: master process

# A politica permite httpd_t ler httpd_sys_content_t.
# Nao permite httpd_t ler shadow_t - mesmo que o processo rode como root,
# mesmo que as permissoes Unix permitam. Essa e a diferenca do MAC.

$ getenforce
Enforcing

# AppArmor faz o equivalente por caminho, e nao por rotulo
$ aa-status | head -3
apparmor module is loaded.
```

42 profiles are loaded.

O ganho concreto é enorme e mal aproveitado: quando um servidor web é comprometido, o MAC é a diferença entre o atacante ter o que o `nginx` precisa e o atacante ter o que o `root` pode. É por isso que a primeira instrução de tantos tutoriais — `setenforce 0` — é uma das piores linhas de comando do ecossistema Linux. Ela desliga a única camada que continua funcionando depois que todas as outras falharam. O Volume 6 volta a isso em detalhe, com a configuração completa.

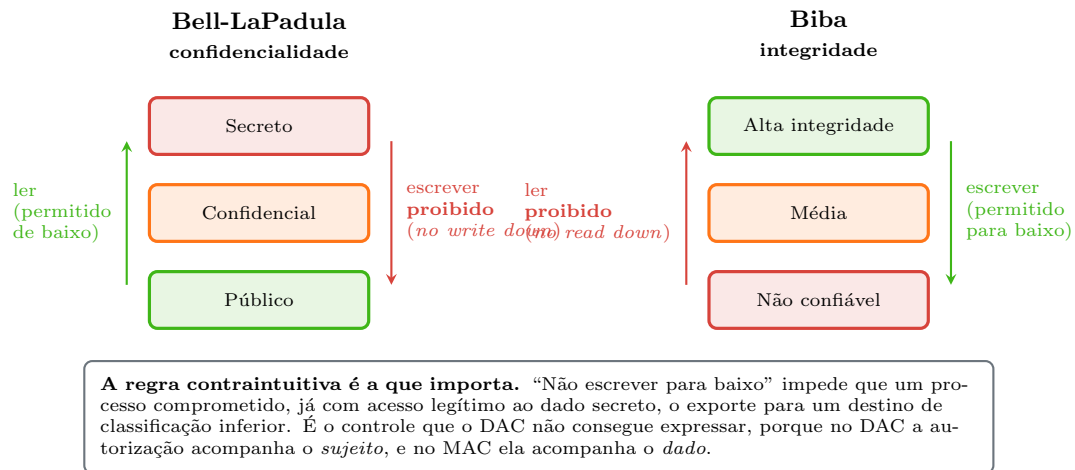


Figura 22.2.: Bell-LaPadula e Biba: a mesma estrutura formal aplicada a objetivos opostos.

O custo do MAC é operacional. Definir e manter rótulos exige disciplina, e uma política mal ajustada bloqueia trabalho legítimo de formas difíceis de diagnosticar. Fora de ambientes militares, ele raramente é a política principal — vive como camada de contenção sob os outros modelos.

22.4. RBAC — permissões viajam por papel, não por pessoa

O modelo dominante em sistemas corporativos surge de uma observação prática: **as permissões que uma pessoa precisa derivam da função que ela exerce, não de quem ela é.** Formalizado por Sandhu e colegas em 1996 e padronizado como ANSI/INCITS 359, o **controle de acesso baseado em papéis** insere uma camada entre sujeitos e permissões (InterNational Committee for Information Technology Standards, 2012; Sandhu et al., 1996).

Em vez de conceder permissões a usuários, você:

1. Define **papéis** que correspondem a funções reais (`analista-suporte-n1`, `dba-producao`, `aprovador-financeiro`).
2. Atribui **permissões a papéis** — decisão técnica, feita uma vez, revisada periodicamente.
3. Atribui **papéis a usuários** — decisão administrativa, feita a cada contratação, promoção e desligamento.

A compressão da matriz é dramática: com 5.000 usuários, 200.000 objetos e 80 papéis, você gerencia 5.000 atribuições de papel mais 80 conjuntos de permissão, em vez de um bilhão de células.

O padrão define quatro níveis cumulativos:

Tabela 22.1.: Os quatro níveis do modelo RBAC.

Nível	Nome	O que acrescenta
RBAC ₀	Núcleo	Usuários, papéis, permissões, sessões
RBAC ₁	Hierárquico	Herança: gerente-suporte herda de analista-suporte
RBAC ₂	Restrições	Segregação de funções, cardinalidade máxima por papel
RBAC ₃	Consolidado	RBAC ₁ + RBAC ₂

O nível RBAC₂ é o que raramente é implementado e o que mais evita fraude. **Segregação de funções** significa que dois papéis são mutuamente exclusivos porque a combinação permite fechar um ciclo sozinho: quem cadastra fornecedor não aprova pagamento; quem cria usuário não concede privilégio administrativo; quem desenvolve não faz o *deploy* em produção com poder de apagar log. A restrição pode ser **estática** (a pessoa não pode receber os dois papéis) ou **dinâmica** (pode ter os dois, mas não ativá-los na mesma sessão).

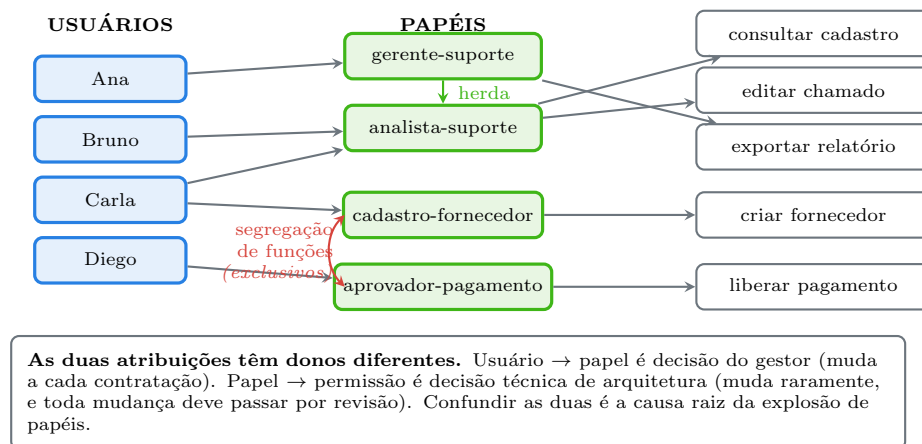


Figura 22.3.: Estrutura do RBAC: a camada de papéis é o que comprime a matriz de acesso.

O RBAC tem duas doenças conhecidas, e ambas aparecem em quase toda implantação madura.

A **explosão de papéis** ocorre quando cada exceção vira um papel novo: **analista-suporte-n1-filial-sul-sem-exportacao**. Uma organização com mais papéis que pessoas não tem RBAC — tem uma matriz de acesso com nomes bonitos. O sintoma é diagnóstico: se o número de papéis cresce proporcionalmente ao número de usuários, o modelo já falhou.

O **acúmulo de privilégios** é mais insidioso e mais universal: a pessoa muda de área e ganha o papel novo, mas ninguém remove o antigo. Depois de dez anos e quatro mudanças de área, ela tem acesso a tudo por onde passou. É a razão pela qual **revogação é mais importante que concessão** — e a razão pela qual campanhas de recertificação de acesso existem, tema que o Capítulo 26 desenvolve.

22.5. ABAC — a decisão vira uma expressão

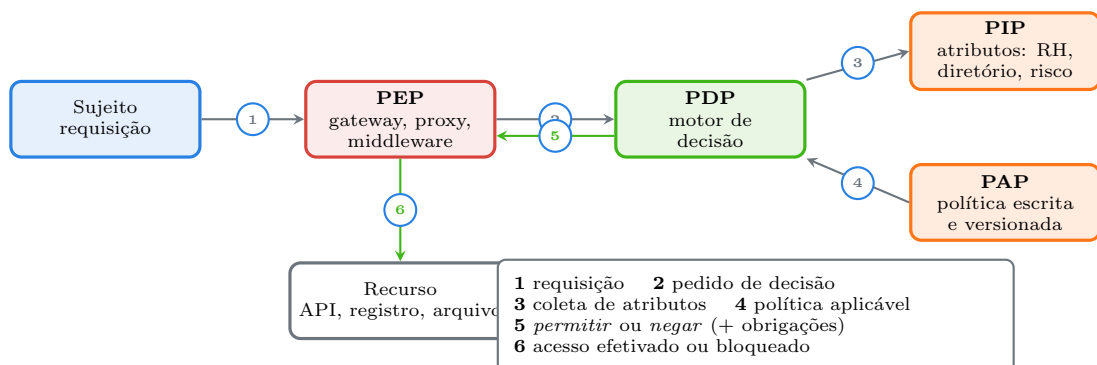
Nenhum dos modelos anteriores consegue expressar a regra que abriria este capítulo: “*analista de suporte pode ver o cadastro do cliente se houver chamado aberto atribuído a ele para esse cliente, em horário de expediente, a partir da rede corporativa*”.

Isso não é uma permissão sobre um objeto. É uma **função** que recebe atributos e devolve permitir ou negar. O **controle de acesso baseado em atributos**, especificado pelo NIST SP 800-162, torna a política uma expressão sobre quatro categorias de atributo (National Institute of Standards and Technology, 2014):

- **do sujeito** — cargo, departamento, nível de habilitação, tempo de casa, gestor;
- **do objeto** — classificação, dono, região de armazenamento, categoria de dado pessoal;
- **da operação** — ler, exportar, apagar, aprovar;
- **do ambiente** — horário, endereço IP, postura do dispositivo, nível de risco da sessão.

A arquitetura correspondente separa quatro componentes, e a separação é a parte que importa:

- **PEP** (*Policy Enforcement Point*) — onde o acesso é interceptado: o proxy, o gateway de API, o middleware.
- **PDP** (*Policy Decision Point*) — onde a decisão é calculada, com base na política.
- **PIP** (*Policy Information Point*) — de onde vêm os atributos: diretório, RH, CMDB, sinal de risco.
- **PAP** (*Policy Administration Point*) — onde a política é escrita, versionada e revisada.



O valor da arquitetura está na **separação**: a política deixa de estar espalhada pelo código da aplicação e passa a ser um artefato único, versionado e testável.

Figura 22.4.: Fluxo de decisão ABAC. O PDP é o monitor de referência tornado explícito.

Na prática moderna isso raramente se escreve em XACML (o padrão original, verboso a ponto de ser um obstáculo). Escreve-se em linguagem de política declarativa — Rego, Cedar, ou o motor embutido na plataforma:

```
# Rego (Open Policy Agent) - a regra que faltava na abertura do capítulo
package autorizacao.suporte

default permitir := false

permitir if {
  input.sujeito.papel == "analista-suporte"
```



```

input.operacao == "ler"
input.objeto.tipo == "cadastro-cliente"

# o vinculo que transforma "pode ver clientes" em "pode ver ESTE cliente"
some chamado in input.sujeito.chamados_abertos
chamado.cliente_id == input.objeto.cliente_id

input.ambiente.hora >= 8
input.ambiente.hora < 20
input.ambiente.origem == "rede-corporativa"
}

# Exportacao em massa exige justificativa e aprovacao - nunca so o papel
permitir if {
    input.operacao == "exportar"
    input.sujeito.papel == "analista-suporte"
    input.contexto.aprovacao_gestor == true
    input.contexto.justificativa != ""
    count(input.objeto.registros) <= 100
}

```

A política ganha uma propriedade que nenhum `if` espalhado pelo código tem: **pode ser testada**. Escrevem-se casos — “analista sem chamado é negado”, “analista com chamado de outro cliente é negado” — e eles rodam no pipeline a cada alteração. É o monitor de referência recuperando sua terceira propriedade, a de ser pequeno o bastante para ser verificado.

O custo do ABAC também é real. Uma política expressiva o suficiente para tudo é complexa o suficiente para ninguém entender: quando ninguém consegue responder “quem tem acesso a este recurso?” sem rodar simulações, a auditabilidade se perdeu por outro caminho. E o PDP vira dependência crítica no caminho de cada requisição — com as consequências óbvias de latência e disponibilidade.

22.6. Os princípios que atravessam todos os modelos

Saltzer e Schroeder publicaram em 1975 os princípios de projeto que envelheceram melhor que qualquer modelo específico (Saltzer; Schroeder, 1975). Três deles governam controle de acesso diretamente.

Privilegio mínimo. Cada sujeito opera com o conjunto mínimo de permissões necessário para a tarefa, pelo menor tempo necessário. A segunda metade da frase é a esquecida: privilégio permanente para tarefa eventual é o padrão em quase toda organização, e é o que a elevação temporária resolve (Capítulo 26).

Padrão à prova de falhas. A decisão padrão é negar. O sistema concede acesso por regra explícita, e a ausência de regra é negação — não permissão. Sistema que libera quando o serviço de autorização está fora do ar tem uma condição de falha que é também um vetor de ataque: derrubar o PDP vira escalonamento de privilégio.

Mediação completa. Toda requisição passa pela verificação, sempre. O erro clássico é verificar na tela e não na API: o botão some da interface, mas o `POST` direto ao endpoint continua

funcionando. Esse achado tem nome próprio e capítulo próprio — é o IDOR, tema do Volume 7.

A esses três a prática acrescentou o **princípio da necessidade de saber**, distinto do privilégio mínimo: ter habilitação para o nível não implica ter motivo para o registro. É precisamente o que faltava na abertura deste capítulo. A analista tinha o privilégio correto para sua função; o que ela não tinha era motivo para aqueles três CPFs.

22.7. Como os modelos convivem de verdade

A pergunta “qual modelo adotar?” quase sempre está mal formulada. Sistemas reais empilham modelos, cada um resolvendo o que os outros não expressam:

Tabela 22.2.: Convivência dos modelos em uma arquitetura real.

Camada	Modelo	Exemplo concreto
Sistema operacional	MAC	SELinux confina o processo do nginx
Sistema de arquivos	DAC	permissões Unix e ACL POSIX
Aplicação corporativa	RBAC	papéis do ERP, grupos do Active Directory
Decisão sensível ao contexto	ABAC	horário, dispositivo, vínculo com chamado
Nuvem	ABAC sobre RBAC	políticas IAM com condições sobre etiquetas

A recomendação prática que sobrevive ao contato com a realidade: **RBAC como espinha dorsal, ABAC nas bordas**. Papéis dão a estrutura auditável — respondem “quem pode o quê” de forma legível por um gestor não técnico. Atributos tratam as exceções e o contexto sem multiplicar papéis. Inverter essa ordem, começando por ABAC puro, produz sistemas em que a resposta a “quem tem acesso?” exige rodar o motor de políticas contra todas as combinações possíveis.

22.8. Brasil em Foco

A LGPD exige controle de acesso em dois artigos, com naturezas diferentes. O art. 46 impõe medidas de segurança aptas a proteger dado pessoal de acessos não autorizados — inclusive os internos, o que abrange exatamente o cenário da abertura. Já o art. 6º, I, consagra a **finalidade**: o tratamento deve ocorrer para propósitos legítimos, específicos e informados. Um sistema que permite a qualquer analista consultar qualquer CPF sem vínculo com um atendimento viola o princípio da finalidade mesmo que nenhum dado vaze para fora — a irregularidade está no acesso interno sem propósito, não no vazamento (Brasil, 2018).

Acesso indevido de funcionário é o incidente que mais aparece e o que menos se investiga. Provedores de internet, operadoras, planos de saúde e órgãos públicos concentram dados de milhões de titulares em ferramentas de atendimento com autorização por papel amplo. Quando a apuração chega, a pergunta da ANPD não é se havia senha: é se havia **controle de finalidade, registro de acesso e revisão periódica**. Log que registra apenas autenticação,

sem registrar qual registro foi consultado e sob qual chamado, não sustenta a apuração — e a ausência de trilha vira, ela própria, um agravante (Autoridade Nacional de Proteção de Dados, 2024).

O sigilo de dados cadastrais e de conexão tem regime legal próprio. O Marco Civil (arts. 10 e 22) e a Lei 9.296/1996 delimitam quem pode acessar registros de conexão, de acesso a aplicações e conteúdo de comunicação, e sob qual ordem judicial. Para um provedor, isso significa que o controle de acesso à base de registros não é escolha de arquitetura: é o mecanismo que materializa uma obrigação legal. Consulta a registro de conexão sem ordem judicial correspondente expõe a empresa e a pessoa que consultou (Brasil, 1996, 2014).

Acesso não autorizado a sistema de terceiro é crime tipificado. A Lei 12.737/2012, com a redação atualizada pela Lei 14.155/2021, criminaliza a invasão de dispositivo informático — e a pena é agravada quando há obtenção de conteúdo de comunicações privadas ou segredos comerciais. O ponto que costuma escapar: o funcionário que usa sua credencial legítima para acessar dado fora da finalidade autorizada não está imune. A depender do caso, a conduta atrai enquadramentos de violação de sigilo e as sanções trabalhistas e civis correspondentes (Brasil, 2012, 2021).

A segregação de funções é exigência explícita no setor regulado. Nas normas do Banco Central sobre política de segurança cibernética e igualmente na conformidade PCI-DSS, a separação entre quem cadastra, quem aprova e quem executa não é boa prática opcional: é requisito verificado em auditoria. O requisito 7 do PCI-DSS trata de restrição de acesso por necessidade de saber; o requisito 8, de identificação individual — que é o que impede a conta compartilhada de destruir a atribuição de responsabilidade (Banco Central do Brasil, 2021; PCI Security Standards Council, 2024).

22.9. No Campo

1. Faça o inventário do que existe, antes de desenhar o que deveria existir. A maioria dos projetos de RBAC falha por começar pelo desenho ideal. Comece extraindo a realidade:

```
# Active Directory: usuarios por grupo, e grupos aninhados (a fonte das
↪  surpresas)
```

```
Get-ADGroup -Filter * -Properties Members |
  Select-Object Name, @{n='Membros';e={$_.Members.Count}} |
  Sort-Object Membros -Descending | Select-Object -First 30
```

```
# Quem esta em grupos privilegiados, incluindo por aninhamento
```

```
Get-ADGroupMember -Identity "Domain Admins" -Recursive |
  Select-Object name, objectClass
```

```
# Linux: contas com shell valido, sudo irrestrito e grupos sensiveis
```

```
awk -F: '$3>=1000 && $7 !~ /nologin|false/' {print $1}' /etc/passwd
grep -rE '(ALL\s*:\s*ALL|NOPASSWD:\s*ALL)' /etc/sudoers /etc/sudoers.d/
getent group sudo wheel docker adm
```

```
-- PostgreSQL: privilegios amplos concedidos diretamente a usuario,
-- isto e, fora de qualquer role - a origem silenciosa do acumulo
```

```
SELECT grantee, table_schema, privilege_type, count(*)
```

```
FROM information_schema.role_table_grants
WHERE grantee <> 'PUBLIC'
GROUP BY 1,2,3 ORDER BY 4 DESC LIMIT 20;
```

O resultado costuma ser desconfortável: grupos com centenas de membros, **Domain Admins** com contas de serviço, gente no grupo **docker** — que é equivalente a root, porque permite montar o sistema de arquivos do hospedeiro dentro de um contêiner.

2. Descubra papéis a partir dos dados, não da imaginação. *Role mining* é o processo de agrupar usuários com conjuntos de permissão semelhantes e nomear os agrupamentos. Um exercício simples e revelador: exportar a matriz usuário × permissão, agrupar por similaridade e comparar o resultado com o organograma. As divergências são o achado — pessoas com acesso que sua função não justifica, e funções que precisam de acesso que não têm (e por isso compartilham credencial).

3. Escreva a matriz de segregação de funções e implemente-a como bloqueio. Uma tabela de pares proibidos, curta e específica:

Tabela 22.3.: Pares de papéis que devem ser mutuamente exclusivos.

Papel A	Papel B	Risco da combinação
Cadastrar fornecedor	Aprovar pagamento	Fornecedor fictício com pagamento próprio
Criar usuário	Conceder privilégio administrativo	Conta oculta com poder total
Desenvolver	Publicar em produção sem revisão	Código não revisado em produção
Operar backup	Apagar log de auditoria	Rastro removível pelo próprio operador

Documento que descreve a segregação sem impedi-la tecnicamente é decoração. A regra precisa estar no fluxo de concessão, rejeitando a atribuição na hora do pedido.

4. Trate o padrão de negação como decisão de arquitetura, não como configuração. Verifique explicitamente o que acontece quando o serviço de autorização não responde. Em muitos sistemas a resposta é “libera” — e derrubar o PDP vira o caminho mais barato de escalonamento. Teste isso: pare o PDP em homologação e observe se o acesso é negado. Se não for, você tem um achado crítico.

5. Registre a decisão de autorização, não só a autenticação. O log que sustenta uma apuração precisa responder: quem, qual registro específico, qual operação, sob qual justificativa ou chamado, permitido ou negado, e por qual regra. Negativas são especialmente valiosas — uma sequência de negações é o sinal mais limpo de que alguém está sondando os limites do próprio acesso. O Volume 12 constrói a detecção sobre esse insumo.

6. Instrumente detecção de acesso sem finalidade. O controle preventivo ideal exige o vínculo (consulta só com chamado aberto). Onde ele ainda não existe, a alternativa detectiva é barata e eficaz: alertar sobre consulta a CPF sem chamado associado, volume de consultas muito acima da média do time, consulta a registros de pessoas com o mesmo sobrenome do operador e consulta a titulares marcados como sensíveis. É a mesma lógica dos controles bancários contra consulta indevida.

7. Recertifique acessos com periodicidade fixa e efeito real. Campanha trimestral para acesso privilegiado, semestral para o restante, com uma regra que dá sentido ao exercício: **o que o gestor não confirmar ativamente é revogado**. Recertificação em que a inação significa manter o acesso mede apenas quanto tempo os gestores gastam clicando.

8. Ative o MAC antes de precisar dele. SELinux em Enforcing ou AppArmor com perfis ativos é a camada que ainda funciona depois que a aplicação foi comprometida. Se algo quebrar, o caminho correto é ajustar a política — `audit2allow` lê as negações e gera o módulo correspondente —, nunca `setenforce 0`:

```
# Diagnostico correto de uma negacao SELinux
ausearch -m AVC -ts recent | audit2why
ausearch -m AVC -ts recent | audit2allow -M meu_modulo
semodule -i meu_modulo.pp

# Verificacao periodica: nenhuma maquina fora de Enforcing
ansible all -m shell -a 'getenforce' | grep -v Enforcing
```

9. Externalize a política quando ela começar a se repetir. Enquanto a autorização couber em poucas verificações consistentes, mantenha-a no código. Quando a mesma regra aparecer em três serviços com três implementações ligeiramente diferentes, é hora de OPA, Cedar ou equivalente — com a política versionada no Git, testada no pipeline e implantada como qualquer outro artefato.

22.10. Resumo

- **Autenticar e autorizar são problemas distintos.** Provar quem é você não diz nada sobre o que você pode fazer, e a maioria dos incidentes de acesso indevido acontece com credencial legítima.
- Todo modelo é uma **estratégia de compressão da matriz de acesso** de Lampson. O corte por coluna gera ACL (bom para “quem acessa este objeto”); o corte por linha gera capacidade (bom para portabilidade, ruim para revogação).
- **DAC** dá autonomia ao dono e não consegue impor política organizacional; é vulnerável ao cavalo de Troia porque a autorização acompanha o sujeito, não o dado.
- **MAC** impõe rótulos que o dono não pode afrouxar. Bell-LaPadula protege confidencialidade (*no read up, no write down*); Biba espelha o modelo para integridade. Na prática cotidiana isso é SELinux e AppArmor — e `setenforce 0` desliga a camada que sobrevive ao comprometimento.
- **RBAC** insere papéis entre usuários e permissões e é o padrão corporativo. Suas duas doenças são a explosão de papéis e o acúmulo de privilégios; o nível RBAC₂, com segregação de funções, é o menos implementado e o que mais evita fraude.
- **ABAC** transforma a decisão numa expressão sobre atributos de sujeito, objeto, operação e ambiente, com PEP, PDP, PIP e PAP separados. Ganha expressividade e política testável; perde legibilidade se aplicado sozinho.
- A arquitetura que funciona é **RBAC como espinha dorsal e ABAC nas bordas**, sobre MAC no sistema operacional.
- Os princípios de Saltzer e Schroeder atravessam todos os modelos: **privilégio mínimo** (inclusive no tempo), **padrão à prova de falhas** (negar por omissão) e **mediação completa** (verificar na API, não só na tela). A **necessidade de saber** é o princípio adicional que separa ter o nível de ter motivo.

22. Modelos de controle de acesso: DAC, MAC, RBAC e ABAC

- No Brasil, acesso interno sem finalidade viola o art. 6º da LGPD independentemente de vazamento; registros de conexão têm regime próprio no Marco Civil e na Lei 9.296; e a segregação de funções é requisito auditado no setor financeiro e em PCI-DSS.

23. Senhas: armazenamento, políticas e ataques

Em 2012, o LinkedIn sofreu um vazamento. A empresa comunicou 6,5 milhões de resumos de senha expostos, forçou a troca dos usuários afetados e o assunto saiu do noticiário. Quatro anos depois, em 2016, o conjunto completo apareceu à venda: **167 milhões de contas**, não 6,5 milhões. O que a empresa achava que era um incidente contido tinha sido, o tempo todo, a base inteira.

Duas decisões técnicas transformaram o vazamento em catástrofe. As senhas estavam protegidas com **SHA-1**, uma função projetada para ser rápida. E estavam **sem sal**. A combinação significa que resumos idênticos correspondem a senhas idênticas — dá para ordenar por frequência e quebrar as mais comuns uma vez, para milhões de contas simultaneamente. Em poucos dias, mais de 90% do conjunto estava em texto claro.

O detalhe que dói: quebrar aquelas senhas não exigiu nenhuma técnica nova. A recomendação de usar função lenta e sal por usuário já tinha, em 2012, mais de uma década de idade e estava em qualquer manual.

Este capítulo trata do que fazer com senhas em três frentes que se determinam mutuamente: como armazená-las, que política impor a quem as escolhe e o que exatamente acontece quando a base vaza.

23.1. Por que a senha ainda existe

A senha tem defeitos conhecidos: é reutilizada, é adivinhável, é entregue por phishing, é anotada. E continua sendo o fator dominante de autenticação em praticamente todos os sistemas do planeta. A razão é econômica, não técnica — ela é o único fator com custo marginal zero de distribuição, funciona em qualquer dispositivo e não exige nenhuma logística de provisionamento.

Compreender a senha como o **fator do conhecimento** é o que estrutura o assunto: ela prova identidade porque, em tese, só o titular conhece o segredo. Toda falha de senha é uma falha dessa premissa — o segredo deixou de ser exclusivo, ou nunca foi difícil o bastante para permanecer secreto. O Capítulo 24 trata dos fatores que compensam essa fragilidade, e o caminho de longo prazo é substituí-la. Enquanto isso não acontece, ela precisa ser bem feita.

23.2. Armazenamento: o único jeito certo

A regra é curta: **nunca armazene a senha; armazene o resultado de uma função de derivação de chave lenta, com sal único por usuário**. Cada palavra dessa frase corrige um erro histórico específico.

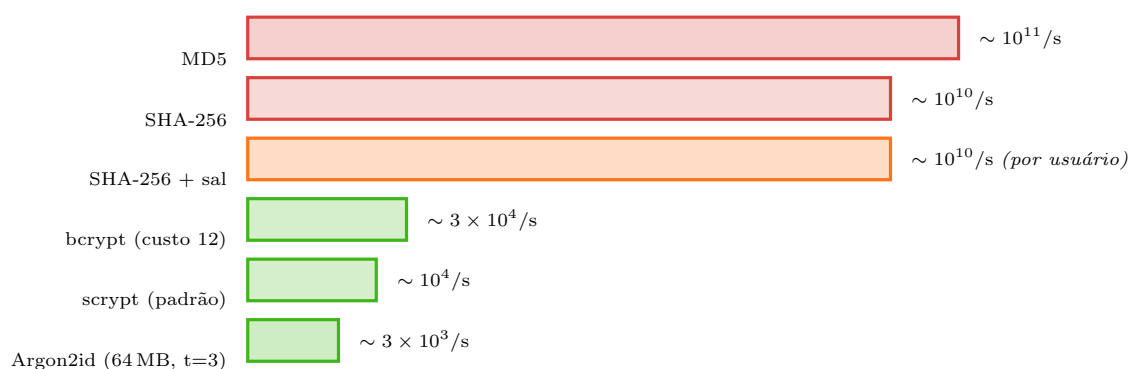
O Capítulo 17 já estabeleceu o mecanismo. Aqui interessa a aplicação e a operação.

23.2.1. A escada dos erros

Tabela 23.1.: A escada dos esquemas de armazenamento de senha.

Esquema	Por que falha	Custo para o atacante
Texto claro	Vazou, acabou	Zero
Cifrado com chave da aplicação	Chave vive junto do dado (Capítulo 21)	Baixo
MD5(senha)	Rápida e com tabelas prontas na internet	Consulta em tabela
SHA-256(senha)	Rápida; sem sal, quebra em lote	Bilhões de tentativas por segundo em GPU
SHA-256(sal + senha)	Sal impede lote e tabela, mas a função continua rápida	Ainda bilhões por segundo, agora por usuário
bcrypt / scrypt / Argon2id	Lenta e parametrizável por projeto	Dezenas de milhares por segundo, no máximo

O salto que importa está na última linha, e ele não é sobre qual algoritmo é “mais forte”: é sobre **custo por tentativa**. O sal resolve um problema (ataque em lote e tabelas pré-computadas); a lentidão resolve outro (força bruta contra um alvo). São defesas diferentes contra ataques diferentes, e as duas são necessárias.



Ordens de grandeza aproximadas, para uma GPU de mercado. A diferença entre a primeira e a última linha é de **sete a oito ordens de grandeza**. Uma senha que cai em 3 segundos contra MD5 resiste por anos contra Argon2id com os mesmos parâmetros de ataque — e o sal é o que impede que esse trabalho seja feito *uma vez* para a base inteira.

Figura 23.1.: Custo por tentativa: a única métrica que importa no armazenamento de senha.

23.2.2. O que usar hoje

Argon2id é a recomendação padrão, vencedora da Password Hashing Competition e especificada na RFC 9106 (Biryukov et al., 2021). Sua característica distintiva é a **dureza de memória**: exigir memória, e não só tempo, é o que reduz a vantagem de GPUs e ASICs, cuja força está no paralelismo massivo com pouca memória por núcleo. A variante **id** combina resistência a canal lateral (do Argon2i) com resistência a compromisso tempo-memória (do Argon2d).

scrypt é a alternativa madura e igualmente dura em memória. **bcrypt** continua aceitável e amplamente disponível, com duas ressalvas operacionais que precisam ser conhecidas: ele **trunca**

a entrada em 72 bytes e algumas implementações lidam mal com o byte nulo. **PBKDF2** só se houver exigência regulatória de FIPS, com contagem de iterações alta — ele não é duro em memória e envelheceu pior que os demais.

```
# Argon2id na pratica - a biblioteca faz sal, parametros e formato
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError, InvalidHashError

ph = PasswordHasher(
    time_cost=3,          # numero de passagens
    memory_cost=65536,    # 64 MiB - o parametro que atrapalha GPU
    parallelism=4,
    hash_len=32,
    salt_len=16,
)

registro = ph.hash("senha-do-usuario")
#
↪ $argon2id$v=19$m=65536,t=3,p=4$c29tZXNhbmHQ...$RdescudvJCsgt3ub+b+dWRWJTmaaJObG
# ~~~~~
# algoritmo ver    parametros          sal          resumo
```

O formato do registro é uma decisão de projeto tão importante quanto os parâmetros. Guardar algoritmo, versão, parâmetros e sal **junto do resumo** é o que torna possível trocar de esquema sem indisponibilidade — a mesma lição do identificador de chave do Capítulo 16.

```
def verificar(registro_armazenado: str, senha_digitada: str) -> tuple[bool, str
↪ | None]:
    """Retorna (autenticado, novo_registro_se_precisa_atualizar)."""
    try:
        ph.verify(registro_armazenado, senha_digitada)
    except (VerifyMismatchError, InvalidHashError):
        return (False, None)

    # Parametros defasados? Recalcula agora, que a senha esta em maos.
    # E assim que a migracao acontece: gradual, sem pedir nada ao usuario.
    if ph.check_needs_rehash(registro_armazenado):
        return (True, ph.hash(senha_digitada))
    return (True, None)
```

23.2.3. Calibração dos parâmetros

Parâmetro copiado de tutorial está errado por definição — ele foi medido em outro hardware, em outro ano. A regra prática: **ajuste o custo até que a verificação leve entre 250 e 500 ms no seu servidor de produção**, sob a carga de pico esperada.

```
# Calibracao: medir, nao copiar
import time
from argon2 import PasswordHasher
```

```
for mem_mib in (16, 32, 64, 128, 256):
    ph = PasswordHasher(time_cost=3, memory_cost=mem_mib * 1024, parallelism=4)
    inicio = time.perf_counter()
    ph.hash("referencia-de-calibracao")
    print(f"{mem_mib:4d} MiB -> {(time.perf_counter() - inicio) * 1000:6.1f}
        ↪ ms")
```

Há um cuidado de disponibilidade que costuma ser descoberto tarde: cada verificação concorrente aloca `memory_cost` bytes. Com 64 MiB e 200 autenticações simultâneas, são 12,8 GiB de pico. Uma rajada de tentativas de autenticação vira, sem limitação de taxa, uma negação de serviço contra o próprio serviço de identidade.

23.2.4. Pimenta: a defesa que o vazamento de banco não derruba

O sal é público e fica junto do resumo. A **pimenta** é um segredo global que **não** fica no banco: vive em variável de ambiente, em cofre ou, no melhor caso, dentro de um HSM. Aplicada como HMAC antes ou depois da derivação, ela garante que quem obtiver apenas um *dump* de SQL — por injeção, por backup exposto, por réplica mal protegida — não consiga sequer começar a quebrar as senhas.

```
import hmac, hashlib, os

PIMENTA = os.environ["PIMENTA_SENHA"] # nunca no banco, nunca no repositório

def proteger(senha: str) -> str:
    temperada = hmac.new(PIMENTA.encode(), senha.encode(),
        ↪ hashlib.sha256).hexdigest()
    return ph.hash(temperada)
```

O custo é operacional e precisa ser encarado de frente: rotacionar a pimenta exige recifrar tudo ou versioná-la por registro, e perdê-la invalida todas as senhas. É uma decisão de arquitetura com plano de continuidade associado, não uma linha extra de código.

23.3. Ataques: o que realmente acontece com a base vazada

23.3.1. As modalidades

Força bruta online tenta senhas contra o sistema em produção. É a menos eficaz — limitação de taxa, bloqueio progressivo e MFA a contêm — e a mais barulhenta.

Preenchimento de credenciais (*credential stuffing*) é o ataque que domina o cenário atual. Não adivinha nada: usa pares usuário-senha já vazados de **outros** serviços, apostando na reutilização. A taxa de sucesso fica tipicamente entre 0,1% e 2%, o que parece pouco até multiplicar por dez milhões de tentativas. Do ponto de vista do sistema atacado, cada requisição isolada é uma autenticação perfeitamente legítima — a defesa não pode ser “senha mais forte”, porque a senha está correta.

Pulverização de senha (*password spraying*) inverte o laço: uma senha muito provável (Brasil@2026, Empresa123) contra milhares de usuários. É a forma de contornar bloqueio

por tentativas, já que cada conta recebe uma ou duas tentativas apenas. É o vetor mais comum contra Active Directory exposto e portais de webmail.

Quebra offline ocorre depois do vazamento da base. Sem limitação de taxa, sem log, sem pressa. É contra ela que Argon2id e pimenta trabalham.

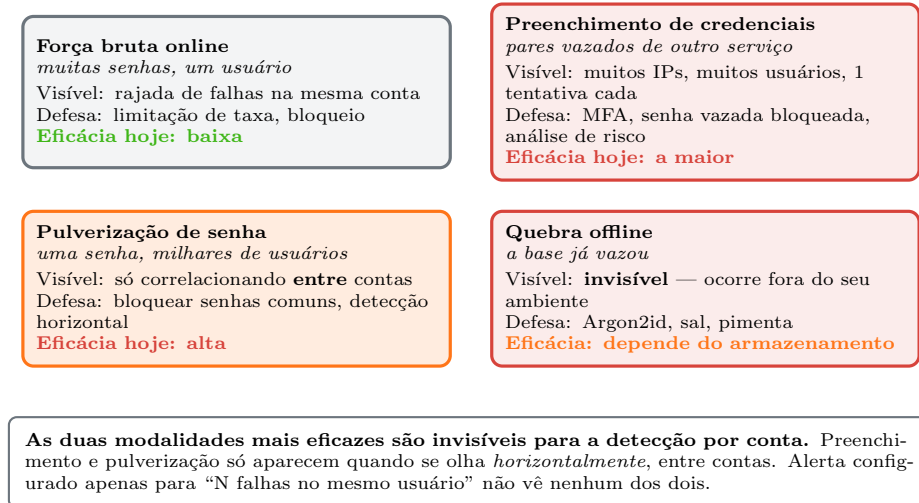


Figura 23.2.: As quatro modalidades, e por que a detecção por conta não é suficiente.

23.3.2. Como a quebra offline funciona de verdade

Ataque real não é força bruta exaustiva. É **ataque por dicionário com regras de mutação**, porque senhas humanas seguem padrões humanos com fidelidade quase decepcionante.

```
# Regras de mutacao (formato Hashcat) aplicadas sobre um dicionario.
# Cada linha transforma cada palavra da lista, multiplicando a cobertura.
c          # Capitaliza: senha -> Senha
$1 $2 $3   # Sufixo comum: senha -> senha123
c $@ $2 $0 $2 $6 # Padrao brasileiro tipico: senha -> Senha@2026
sa@ se3 si1 so0  # Substituicao "leet": seguranca -> s3gur@nc@
```

A consequência para a política de senha é direta e frequentemente mal compreendida: **S3nh@2026!** **satisfaz qualquer regra de complexidade e está entre as primeiras milhões de tentativas de qualquer ataque sério.** A regra de complexidade não mediu a resistência da senha — mediu a conformidade dela com um formato que o atacante também conhece.

O material do ataque é abundante: `rockyou.txt` com 14 milhões de senhas, listas por país e por idioma, e as compilações de vazamentos. Para o português brasileiro, os padrões previsíveis incluem nome de time, @2026, nome próprio com sufixo numérico e as sequências de teclado (`qwerty`, `asdf`, `1q2w3e`).

23.4. Política de senha: o que a evidência mudou

Por vinte anos a política padrão foi: mínimo de 8 caracteres, maiúscula, minúscula, número, símbolo, troca a cada 90 dias, sem repetir as últimas cinco. Bill Burr, autor do documento do

23. Senhas: armazenamento, políticas e ataques

NIST que popularizou essas regras em 2003, declarou publicamente em 2017 que se arrependia delas.

A revisão veio na série NIST SP 800-63B, e inverte quase tudo (National Institute of Standards and Technology, 2024a):

Tabela 23.2.: O que mudou na política de senha, e a razão de cada mudança.

Regra antiga	Recomendação atual	Por quê
Complexidade obrigatória	Não exigir	Produz Senha@2026 , previsível e difícil de lembrar
Troca periódica	Só ao suspeitar de comprometimento	Troca forçada gera incremento: Senha1 → Senha2
Mínimo de 8	Mínimo de 8, recomendado 15+	Comprimento supera composição
Máximo de 16	Aceitar 64 caracteres	Limite baixo impede frases-senha
Bloquear colagem	Permitir colagem	Bloquear inviabiliza gerenciadores de senha
Perguntas de recuperação	Não usar	A resposta está no perfil público da pessoa
—	Verificar contra lista de vazadas	O único controle que ataca o problema real

A mudança conceitual é abandonar regras de *forma* em favor de verificação de *fato*. Não importa se a senha tem símbolo; importa se ela já apareceu num vazamento. Essa é a verificação que barra o preenchimento de credenciais na origem, e ela pode ser feita sem enviar a senha a lugar nenhum:

```
import hashlib, requests

def senha_ja_vazou(senha: str) -> int:
    """Consulta por k-anonimato: envia so os 5 primeiros digitos do SHA-1.
    O servico nunca recebe a senha nem o resumo completo."""
    sha1 = hashlib.sha1(senha.encode()).hexdigest().upper()
    prefixo, sufixo = sha1[:5], sha1[5:]

    resp = requests.get(f"https://api.pwnedpasswords.com/range/{prefixo}",
                        timeout=3)
    for linha in resp.text.splitlines():
        h, contagem = linha.split(":")
        if h == sufixo:
            return int(contagem)
    return 0

# No cadastro e na troca - nao no login, para nao criar dependencia externa
# no caminho critico de autenticacao.
if senha_ja_vazou(nova_senha) > 0:
    recusar("Esta senha aparece em vazamentos públicos. Escolha outra.")
```

Para quem prefere não depender de serviço externo, a mesma verificação roda localmente contra uma lista baixada — filtro de Bloom ou tabela indexada resolvem o volume sem custo de latência.

23.4.1. Comprimento versus composição

A aritmética resolve a discussão. Uma senha aleatória de 8 caracteres sobre um alfabeto de 95 símbolos tem cerca de 52 bits de entropia. Quatro palavras aleatórias de uma lista de 7.776 (o método *diceware*) têm cerca de 51 bits — equivalente. Cinco palavras chegam a 64 bits, superior a qualquer senha de 8 caracteres, e é incomparavelmente mais fácil de memorizar.

A ressalva essencial: essa conta só vale para escolha **aleatória**. Frase-senha escolhida por uma pessoa — um verso conhecido, um bordão, o nome do time com o ano do título — tem uma fração dessa entropia, porque o espaço real de busca é o das frases que humanos escolhem, não o das combinações possíveis.

23.5. O ciclo de vida completo

Senha não é só o campo do banco. Todo ponto do ciclo de vida é um ponto de ataque.

Criação. Verificar contra lista de vazadas, comparar com dados do próprio usuário (nome, e-mail, CPF, nome da empresa), medir força com estimador realista — **zxcvbn** avalia padrões e não conta símbolos — e nunca impor complexidade cega.

Autenticação. Comparação em tempo constante (o **verify** da biblioteca já faz), mensagem de erro genérica que não distingue “usuário inexistente” de “senha errada” (Capítulo 21), e **custo uniforme**: se a verificação só roda quando o usuário existe, a diferença de tempo enumera a base de usuários. A correção é executar uma verificação descartável contra um resumo fixo quando o usuário não existir.

Recuperação. Este é, com folga, o elo mais fraco — e o mais negligenciado. Todo o investimento em Argon2id é irrelevante se o fluxo de “esqueci minha senha” pode ser abusado. O que precisa estar certo:

```
import secrets, hashlib
from datetime import datetime, timedelta, timezone

def gerar_token_recuperacao(usuario_id: int) -> str:
    token = secrets.token_urlsafe(32)          # CSPRNG, nunca Math.random

    # Armazena o RESUMO do token, não o token: vazamento do banco de
    # recuperacao não pode dar acesso as contas.
    registro = {
        "usuario_id": usuario_id,
        "token_hash": hashlib.sha256(token.encode()).hexdigest(),
        "expira_em": datetime.now(timezone.utc) + timedelta(minutes=15),
        "usado": False,
    }
    salvar(registro)
    return token    # só este valor vai para o e-mail
```

```
# Regras que acompanham:  
# - uso unico, invalidado na primeira utilizacao  
# - resposta identica para e-mail existente e inexistente (nao enumerar)  
# - limitacao de taxa por conta e por IP de origem  
# - ao concluir a troca: invalidar TODAS as sessoes ativas do usuario  
# - notificar o titular por e-mail, com horario e origem
```

A última regra é a que mais falta na prática: trocar a senha sem invalidar as sessões existentes deixa o invasor conectado depois da “resposta ao incidente”. O Capítulo 26 desenvolve a invalidação em escala.

Rotação e comprometimento. Não force troca periódica sem motivo. Force troca quando houver evidência: a senha apareceu num vazamento, houve autenticação suspeita confirmada, ou o incidente exige. E aí a troca precisa vir acompanhada de invalidação de sessão e de tokens de recuperação pendentes.

23.6. Gerenciadores de senha

Toda a discussão anterior pressupõe que a pessoa escolha e memorize senhas. O gerenciador remove essa premissa: senhas longas, aleatórias e diferentes por serviço, sem esforço de memória. É a recomendação mais eficaz e a menos adotada.

A objeção óbvia — “concentra tudo num único ponto” — merece resposta honesta em vez de descarte. Sim, concentra. Mas a alternativa real não é “senhas fortes e distintas memorizadas”: é a mesma senha em quarenta serviços, que é a condição que torna o preenchimento de credenciais lucrativo. Trocar quarenta pontos de falha correlacionados por um ponto de falha protegido por senha-mestra forte e MFA é uma melhora clara de postura.

O que exigir na escolha: cifragem de ponta a ponta com derivação da chave a partir da senha-mestra **no cliente** (o operador nunca deve conseguir ler o cofre), MFA na conta, código auditado ou aberto, e exportação possível — para que a migração seja viável. Em ambiente corporativo, um cofre com compartilhamento por equipe elimina a planilha de senhas compartilhadas, que é o achado mais comum de qualquer auditoria interna.

23.7. Brasil em Foco

Base de senhas vazada é incidente com dever de avaliação sob a LGPD, e o esquema de armazenamento decide o desfecho. Se as senhas estavam protegidas com Argon2id, sal e pimenta, é defensável sustentar que o risco ao titular é reduzido. Se estavam em MD5 sem sal, ou pior, o dado deve ser tratado como exposto — e a comunicação aos titulares e à ANPD passa a ser praticamente inevitável, sob os arts. 48 e 50. A Resolução CD/ANPD 15/2024 estrutura essa avaliação, e a decisão sobre comunicar frequentemente se resume à qualidade do esquema escolhido anos antes do incidente (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

Senha é dado pessoal, e reutilização entre serviços amplia o dano de um vazamento seu. Quando a senha vazada da sua aplicação abre a conta bancária do titular, o nexo causal não desaparece porque o titular reutilizou. A adoção de verificação contra listas de vazamento e a oferta de MFA são, nesse contexto, medidas de mitigação com valor probatório — demonstram

diligência sob o art. 46, e essa demonstração vale na dosimetria de eventual sanção (Brasil, 2018).

Golpes de conta com senha vazada alimentam a fraude do Pix. O ecossistema brasileiro de pagamento instantâneo, com liquidação irreversível em segundos, transforma o acesso indevido a conta em prejuízo consumado. Por isso as normas do Banco Central sobre segurança cibernética e o arranjo do Pix exigem autenticação robusta, controles antifraude e mecanismos de bloqueio — e por isso o MEDs e os limites por horário existem. Senha isolada não é controle suficiente em nenhum ponto desse arranjo (Banco Central do Brasil, 2021).

A Lei 14.155/2021 endureceu a resposta penal ao uso de credencial alheia. Ela agravou as penas da invasão de dispositivo informático e criou o tipo de fraude eletrônica com aumento quando praticada por meio de dispositivo eletrônico, com agravamento adicional se a vítima for idosa. Para quem opera sistemas, a implicação prática é sobre a preservação de evidência: registros de autenticação com data, hora, endereço de origem e agente de usuário são o insumo do inquérito, e a retenção precisa ser compatível com o prazo de apuração (Brasil, 2012, 2021).

Contas gov.br concentram acesso a serviços públicos e elevam a aposta. Os níveis bronze, prata e ouro correspondem a graus distintos de confiabilidade da identidade, e o nível ouro exige validação biométrica ou certificado ICP-Brasil. Para o cidadão, uma senha fraca ali expõe declaração de imposto, benefícios e assinatura de documentos; para o gestor público, a comunicação sobre elevação de nível e sobre MFA é medida de segurança, não campanha institucional (Instituto Nacional de Tecnologia da Informação, 2024; Secretaria de Governo Digital, Ministério da Gestão e da Inovação, 2024).

23.8. No Campo

1. Descubra qual esquema está realmente em uso — em todos os sistemas, não só no principal. A resposta costuma variar entre aplicações da mesma empresa:

```
# 0 prefixo do registro identifica o esquema
#  $2a$ $2b$ $2y$  -> bcrypt          $argon2id$ -> Argon2id
#  $6$              -> SHA-512 crypt  $scrypt$   -> scrypt
#  32 hex           -> provavel MD5   40 hex      -> provavel SHA-1
grep -rhoE '^\$[a-z0-9]+\$$' dump_usuarios.txt | sort | uniq -c | sort -rn

# Nenhum prefixo e comprimento fixo = hash cru, sem sal: achado critico
awk -F'\t' '{print length($2)}' dump_usuarios.txt | sort | uniq -c
```

```
-- Distribuicao de esquemas na tabela de usuarios (PostgreSQL)
SELECT
  CASE
    WHEN senha_hash LIKE '$argon2%' THEN 'argon2'
    WHEN senha_hash LIKE '$2%'      THEN 'bcrypt'
    WHEN senha_hash LIKE '$6$%'     THEN 'sha512crypt'
    WHEN length(senha_hash) = 32    THEN 'md5 (CRITICO)'
    WHEN length(senha_hash) = 40    THEN 'sha1 (CRITICO)'
    ELSE 'desconhecido'
  END AS esquema,
  count(*), max(ultimo_login) AS login_mais_recente
FROM usuarios GROUP BY 1 ORDER BY 2 DESC;
```


2. Migre sem pedir nada ao usuário. O algoritmo de migração é o mesmo do Capítulo 17 e não exige conhecer as senhas: aplique a função nova **sobre o resumo antigo**, registre o esquema composto e, na próxima autenticação bem-sucedida, recalcule do zero.

```
# Migracao em duas fases, sem janela de indisponibilidade
# Fase 1 (lote, hoje): argon2(md5_existente) - a base deixa de ser
#                               quebravel em massa imediatamente
# Fase 2 (gradual):    no login correto, recalcula argon2(senha) puro
def verificar_migrado(reg: dict, senha: str) -> bool:
    if reg["esquema"] == "argon2(md5)":
        if not ph_verifica(reg["hash"], md5(senha)):
            return False
        reg.update(esquema="argon2", hash=ph.hash(senha)) # fase 2, agora
        salvar(reg)
        return True
    return ph_verifica(reg["hash"], senha)
```

Contas que nunca mais autenticam permanecem na fase 1 — protegidas, ainda que não ideais. Contas inativas há mais de um ano são candidatas a desativação, o que resolve o resíduo.

3. Meça os parâmetros no seu hardware e dimensione a memória de pico. Alvo de 250 a 500 ms por verificação, e cálculo explícito do consumo: `memory_cost` × autenticações simultâneas. Sem limitação de taxa na frente, esse número vira o vetor de negação de serviço mais barato contra seu serviço de identidade.

4. Bloqueie senhas vazadas no cadastro e na troca. É o controle de melhor relação custo-benefício deste capítulo. Consulta por k-anonimato à API pública, ou lista local com filtro de Bloom. Faça no cadastro e na troca, não no login — dependência externa no caminho crítico de autenticação é risco de disponibilidade.

5. Configure a detecção horizontal, não só a vertical. Alerta de “5 falhas na mesma conta” não vê pulverização nem preenchimento. As consultas que veem:

```
-- Pulverizacao: muitos usuarios distintos falhando, poucas tentativas cada
SELECT ip_origem, count(DISTINCT usuario) AS alvos, count(*) AS tentativas
FROM log_autenticacao
WHERE sucesso = false AND horario > now() - interval '1 hour'
GROUP BY ip_origem HAVING count(DISTINCT usuario) > 20
ORDER BY alvos DESC;

-- Preenchimento de credenciais: taxa de sucesso anomala no conjunto
SELECT date_trunc('minute', horario) AS minuto,
       count(*) FILTER (WHERE sucesso) * 100.0 / count(*) AS pct_sucesso,
       count(*) AS total, count(DISTINCT ip_origem) AS ips
FROM log_autenticacao
WHERE horario > now() - interval '6 hours'
GROUP BY 1 HAVING count(*) > 100 AND
       count(*) FILTER (WHERE sucesso) * 100.0 / count(*) < 5
ORDER BY 1 DESC;
```

Taxa de sucesso muito **baixa** com volume alto e muitos IPs é a assinatura clássica do preenchimento de credenciais.

6. Prefira bloqueio progressivo a bloqueio duro. Bloquear a conta após N tentativas transfere ao atacante o poder de causar indisponibilidade em massa: basta errar cinco vezes em cada conta. Atraso crescente, desafio adicional após a terceira falha e bloqueio por origem em vez de por conta preservam a disponibilidade e continuam elevando o custo do ataque.

7. Audite o fluxo de recuperação com o mesmo rigor do fluxo de login. Faça o roteiro completo: o token é gerado por CSPRNG? Expira em minutos? É de uso único? O banco guarda o resumo do token ou o token? A resposta é idêntica para e-mail existente e inexistente? Há limitação de taxa? A troca invalida sessões e tokens pendentes? O titular é notificado? Cada “não” é um achado, e é comum encontrar quatro deles em sistemas cujo armazenamento de senha está impecável.

8. Elimine contas compartilhadas e senhas padrão de fábrica. admin/admin em equipamento de rede, credencial padrão de câmera IP, conta suporte usada por seis pessoas. A conta compartilhada destrói a atribuição de responsabilidade — a trilha de auditoria passa a apontar para um cargo, não para uma pessoa. Onde a credencial compartilhada for tecnicamente inevitável, ela precisa ir para um cofre com registro de retirada e rotação automática (Capítulo 26).

9. Trate a senha como controle de transição. O destino é a autenticação sem senha, com passkeys — assunto do Capítulo 24. A senha bem armazenada é o que sustenta a operação enquanto essa migração acontece, e sustentá-la bem é a diferença entre um vazamento contornável e um vazamento como o do LinkedIn.

23.9. Resumo

- **O que vaza não é a senha, é o esquema de armazenamento.** SHA-1 sem sal transformou o incidente do LinkedIn em 167 milhões de senhas em texto claro, quatro anos depois do fato.
- **Armazene com Argon2id** (ou scrypt, ou bcrypt), com sal único por usuário e parâmetros medidos no seu hardware — alvo de 250 a 500 ms por verificação. Guarde algoritmo, versão e parâmetros junto do resumo, para poder migrar sem indisponibilidade.
- **Sal e lentidão defendem de ataques diferentes:** o sal impede lote e tabelas pré-computadas; a lentidão impede força bruta contra um alvo. As duas são necessárias.
- A **pimenta** é o segredo fora do banco que torna inútil o *dump* isolado — com custo operacional de rotação e continuidade que precisa ser planejado.
- **Preenchimento de credenciais e pulverização são os ataques dominantes**, e ambos são invisíveis para alertas por conta. A detecção precisa ser horizontal, entre contas e por origem.
- A política moderna (NIST SP 800-63B) **abandona complexidade obrigatória e troca periódica** e adota comprimento maior, aceitação de frases-senha, colagem permitida e **verificação contra listas de senhas vazadas** — o único controle que ataca a reutilização.
- A **recuperação é o elo mais fraco**: token por CSPRNG, resumo armazenado em vez do token, expiração curta, uso único, resposta não enumerável, limitação de taxa e invalidação de todas as sessões ao concluir.
- **Gerenciador de senha é a recomendação mais eficaz e menos adotada**; em ambiente corporativo, o cofre com compartilhamento por equipe elimina a planilha de senhas.
- No Brasil, o esquema de armazenamento decide o desfecho da avaliação de incidente sob a LGPD; a irreversibilidade do Pix eleva o custo do acesso indevido; e o log de autenticação é o insumo probatório da apuração criminal sob a Lei 14.155.

24. Autenticação multifator e fatores de autenticação

Em agosto de 2022, a Twilio informou que um grupo de atacantes havia comprometido contas de funcionários. O método não tinha nada de sofisticado: SMS dizendo que a senha expirara, com link para um domínio parecido com o da empresa. A página falsa pedia usuário, senha e **o código de autenticação de dois fatores** — e repassava tudo em tempo real para o site verdadeiro. A vítima recebia o código no celular, digitava na página do atacante e a sessão legítima abria do outro lado.

A mesma campanha atingiu mais de cem organizações. Empresas com MFA obrigatório, treinamento de conscientização e política escrita foram comprometidas exatamente como as que não tinham nada disso.

Duas semanas depois, a Cloudflare foi alvo da mesma campanha, com a mesma isca. Três funcionários entregaram credencial e código. **Nenhuma conta foi comprometida.** A diferença: a Cloudflare usava chaves de segurança FIDO2, e a chave simplesmente não produz nada utilizável para um domínio que não é o legítimo. Não houve decisão humana envolvida — o protocolo se recusou a assinar.

Esses dois incidentes, separados por semanas e com a mesma isca, resumem o capítulo: **MFA não é uma coisa só, e a diferença entre suas formas é a diferença entre ser comprometido e não ser.**

24.1. Os fatores, e por que a categoria importa

A autenticação combina evidências de três categorias clássicas:

- **Algo que você sabe** — senha, PIN, resposta secreta. Barato, replicável sem limite, e transferível: se você sabe, pode contar (ou ser enganado a contar).
- **Algo que você tem** — chave de segurança, celular com aplicativo, cartão inteligente, certificado. Exige posse; roubá-lo à distância é substancialmente mais difícil.
- **Algo que você é** — impressão digital, face, íris, padrão de digitação. Conveniente, não secreto (você deixa impressões digitais em tudo) e, crucialmente, **não revogável**.

Duas categorias adicionais aparecem na prática moderna e são melhor entendidas como **sinais de contexto** do que como fatores: **onde você está** (endereço IP, geolocalização, rede) e **o que você faz** (padrão de comportamento). Não autenticam sozinhas, mas ajustam a exigência — é a base da autenticação adaptativa.

Multifator exige categorias diferentes. Senha mais pergunta secreta são dois conhecimentos: um fator só, com dois campos. Senha mais código do aplicativo autenticador são conhecimento mais posse: genuinamente dois fatores. A distinção não é acadêmica — ela determina se um único comprometimento derruba tudo.

O ganho é grande e vale registrar sem exagero: **MFA elimina a esmagadora maioria dos ataques de credencial**, porque preenchimento de credenciais, pulverização e quebra offline

(Capítulo 23) todos terminam com o atacante de posse de uma senha válida — e a senha válida deixa de bastar. O que MFA **não** elimina sozinho é o phishing em tempo real, que foi o que aconteceu na Twilio.

24.2. A hierarquia real de segurança

Nem todo segundo fator resiste às mesmas coisas. A ordem abaixo é por resistência ao ataque que hoje mais importa: o phishing com retransmissão em tempo real.

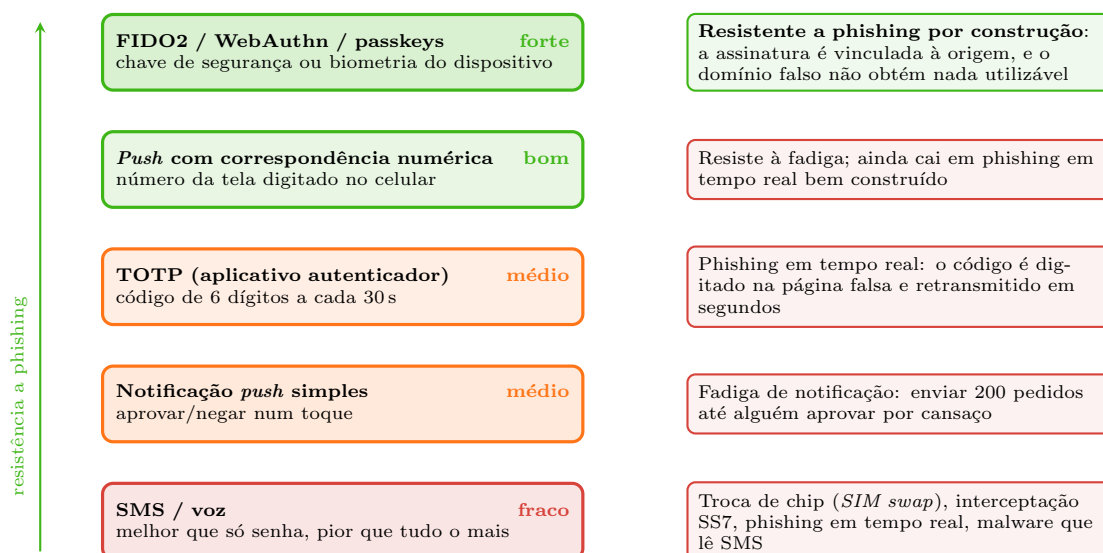


Figura 24.1.: Hierarquia dos segundos fatores. O degrau que separa o topo do resto é qualitativo, não gradual.

O salto entre os quatro primeiros níveis e o quinto não é de grau: é de natureza. Os quatro primeiros dependem de a pessoa **transmitir um segredo** — e qualquer segredo transmissível pode ser transmitido ao destinatário errado. O quinto não transmite segredo nenhum.

24.3. TOTP: como funciona e onde falha

O código de seis dígitos do aplicativo autenticador é um HMAC truncado, com o tempo como contador. A especificação está na RFC 6238, sobre a base da RFC 4226 (M’Raihi et al., 2005, 2011).

O servidor e o aplicativo compartilham um segredo, estabelecido uma única vez no cadastro (o QR Code). A cada intervalo de 30 segundos, ambos calculam a mesma função sobre esse segredo e o carimbo de tempo. O código é o resultado, truncado.

```
import hmac, hashlib, struct, time, base64

def totp(segredo_b32: str, passo: int = 30, digitos: int = 6) -> str:
    chave = base64.b32decode(segredo_b32, casefold=True)
    contador = int(time.time()) // passo

    resumo = hmac.new(chave, struct.pack(">Q", contador),
        ↪ hashlib.sha1).digest()
```

```
# Truncagem dinamica (RFC 4226): o ultimo nibble escolhe o deslocamento
desloc = resumo[-1] & 0x0F
trecho = struct.unpack(">I", resumo[desloc:desloc + 4])[0] & 0x7FFFFFFF
return str(trecho % (10 ** digitos)).zfill(digitos)
```

Três detalhes de implementação separam um TOTP correto de um TOTP frágil:

Janela de tolerância. Relógios divergem. Aceitar o intervalo anterior e o seguinte (± 1 passo) é o padrão razoável. Janela maior é ganho de conveniência com perda direta de segurança: cada passo extra multiplica a superfície de adivinhação.

Uso único obrigatório. Um código válido por 30 segundos que pode ser reapresentado nesse intervalo permite reprodução. Registre o último contador aceito por usuário e recuse contadores iguais ou anteriores.

Limitação de taxa. Seis dígitos são um milhão de possibilidades. Sem limite de tentativas, adivinhar é viável — especialmente com janela de tolerância ampla. Máximo de cinco tentativas por janela, com atraso progressivo.

E, sobretudo: **o segredo TOTP é um segredo de longo prazo**, armazenado no servidor. Se ele vazar, o atacante gera códigos válidos indefinidamente. Ele precisa ser cifrado em repouso com chave que não esteja no mesmo banco — a lição do Capítulo 21 aplicada aqui.

A limitação fundamental do TOTP não é implementação, porém. É que **ele é digitável**, e o que se digita numa página legítima também se digita numa página falsa.

24.4. Por que SMS é o pior fator que ainda vale a pena ter

O NIST desaconselha SMS desde 2016 (National Institute of Standards and Technology, 2024a), e as razões se acumularam desde então:

Troca de chip. O atacante convence a operadora a portar o número para um chip sob seu controle — com engenharia social no atendimento, com documento falsificado, ou com um funcionário cooptado. É um vetor com histórico documentado e recorrente no Brasil, envolvendo tanto contas de aplicativo de mensagem quanto contas bancárias.

Interceptação na rede de sinalização. O protocolo SS7 permite, a quem tem acesso à rede de sinalização, redirecionar mensagens. Esse acesso é mais disponível do que se imagina.

O SMS não é cifrado ponta a ponta e aparece na tela bloqueada, visível a quem estiver por perto.

Encaminhamento e sincronização. Mensagem espelhada no computador, no relógio, no aplicativo desktop — cada espelho é uma superfície nova.

Dito isso, e sem ambiguidade: **SMS é muito melhor que nada**. Ele derruba preenchimento de credenciais, pulverização e uso de senha vazada — os ataques de maior volume. Onde a alternativa realista for “SMS ou nenhum segundo fator”, o SMS entra. O erro é tratá-lo como equivalente aos demais, e sobretudo mantê-lo como **caminho de recuperação universal**: se a conta pode ser recuperada por SMS, a segurança da conta é a segurança do SMS, por mais forte que seja o fator principal.

24.5. Fadiga de notificação: o ataque contra o fator “bom”

O *push* simples parecia a solução ideal — sem código para digitar, sem erro de digitação. Até que os atacantes perceberam que podiam simplesmente insistir.

Com a senha em mãos, o atacante dispara pedidos de autenticação repetidamente. Às três da manhã, depois da quinquagésima notificação, alguém aprova para o celular parar de vibrar. Foi assim que a Uber foi comprometida em setembro de 2022 — combinando as notificações com uma mensagem no WhatsApp se passando por alguém do time de TI, pedindo que a pessoa aprovasse para “resolver o problema”.

A correção é a **correspondência numérica**: a tela de autenticação mostra um número de dois dígitos, e o celular pede que a pessoa o digite. Aprovar por engano deixa de ser possível — quem não está diante da tela legítima não sabe o número. Complementos que ajudam: mostrar origem geográfica e aplicativo no pedido, limitar pedidos por minuto, bloquear a conta após várias negações e alertar o time de segurança quando o padrão aparecer.

24.6. FIDO2, WebAuthn e passkeys

Este é o degrau que muda a natureza do problema. A especificação WebAuthn, do W3C, com o protocolo CTAP da FIDO Alliance para a comunicação com o autenticador, substitui o segredo compartilhado por **criptografia de chave pública** — exatamente o material do Capítulo 18.

No registro, o autenticador gera um par de chaves **exclusivo para aquele site**. A chave privada nunca sai do dispositivo; o site guarda apenas a pública. Na autenticação, o site envia um desafio aleatório, o autenticador assina e o site verifica com a chave pública que já tem.

O detalhe que derrota o phishing está numa linha da estrutura assinada: o autenticador inclui a **origem** (o domínio que efetivamente fez a requisição, informado pelo navegador, não pela página) no que ele assina, e usa a chave correspondente àquele domínio. Numa página em **contoso-sso.com**, o navegador informa **contoso-sso.com**; o autenticador não tem chave registrada para esse domínio e não produz nada. Se produzisse, a assinatura conteria a origem errada e o servidor legítimo a rejeitaria.

Não há decisão humana no caminho. É por isso que os três funcionários da Cloudflare que caíram na isca não comprometeram nada.

```
// Registro de uma credencial WebAuthn (lado do navegador)
const credencial = await navigator.credentials.create({
  publicKey: {
    challenge: desafioDoServidor,           // aleatorio, uso unico
    rp: { name: "Manual de Segurança", id: "exemplo.com.br" },
    user: { id: idUsuario, name: "ana@exemplo.com.br",
            displayName: "Ana Souza" },
    pubKeyCredParams: [
      { type: "public-key", alg: -7 },      // ES256 (P-256)
      { type: "public-key", alg: -257 },    // RS256
    ],
    authenticatorSelection: {
      userVerification: "required",         // biometria ou PIN: 2FA num gesto
      residentKey: "preferred",             // credencial descobrível = passkey
    }
  }
});
```

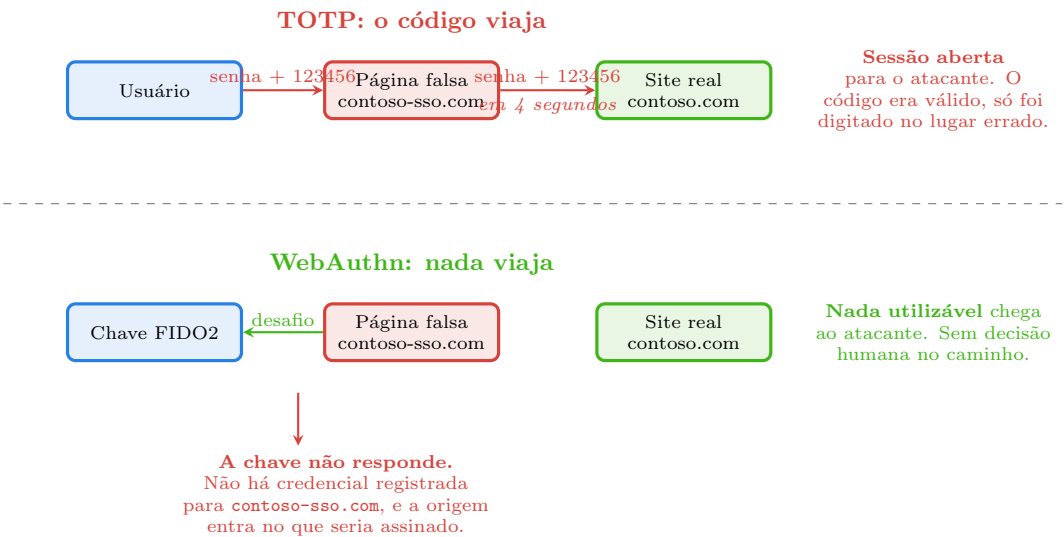


Figura 24.2.: A diferença estrutural: o TOTP transmite um segredo; o WebAuthn assina uma origem.

```
    timeout: 60000,
    attestation: "none",
  },
});
// O servidor armazena a chave publica, o ID da credencial e o contador.
// A chave privada nunca sai do autenticador.
```

Um ponto de terminologia que confunde muita gente: **passkey é uma credencial FIDO2 sincronizada** entre os dispositivos da pessoa pelo ecossistema (Apple, Google, Microsoft, gerenciador de senha). A criptografia é a mesma; o que muda é onde a chave privada vive e como ela é reposta (FIDO Alliance, 2024).

Tabela 24.1.: Chave física e passkey: a mesma criptografia, decisões de custódia diferentes.

	Chave de segurança física	Passkey sincronizada
Onde vive a chave privada	Só no dispositivo	No dispositivo e no cofre sincronizado
Perda do dispositivo	Perde a credencial	Recupera pela conta do ecossistema
Superfície adicional	Nenhuma	A conta do ecossistema vira alvo
Adoção	Exige comprar e distribuir	Já embutida em celulares e navegadores
Uso típico	Administrador, acesso privilegiado	Usuário final em escala

A recomendação prática que decorre da tabela: **passkeys para o usuário em geral** — resolvem o phishing em escala com adoção viável — e **chave física para acesso privilegiado**, onde a garantia de que a chave privada não é sincronizável importa mais que a conveniência.

Um efeito colateral importante: com verificação de usuário exigida, a passkey já é dois fatores num gesto — posse do dispositivo mais biometria ou PIN. Isso permite eliminar a senha, e não apenas somar um fator a ela. É a diferença entre autenticação **sem senha** e autenticação **de dois fatores**, e a primeira é o destino.

24.7. Biometria: entender o que ela é

Biometria não é segredo. Sua face está em fotos públicas; suas digitais ficam em tudo que você toca. O que a biometria oferece é **conveniência com vinculação ao corpo**, e o modelo de segurança correto é: a biometria destrava o autenticador local; o autenticador faz a criptografia.

É assim que funciona no celular. A digital não vai para o servidor: ela libera o acesso à chave privada guardada no enclave seguro do aparelho, e é a chave que autentica. Essa arquitetura tem uma consequência forte — **a biometria correspondida no dispositivo é aceitável; a biometria correspondida no servidor é um banco de dados de características biométricas irrevogáveis**, com a implicação óbvia de que um vazamento não tem remediação: não se troca de rosto.

As métricas que importam ao avaliar um sistema: taxa de falsa aceitação (impostor aceito), taxa de falsa rejeição (legítimo recusado), e **detecção de vivacidade** — a resistência a foto, máscara, molde de silicone ou vídeo gerado. Sistemas sem vivacidade caem com uma foto impressa, e a qualidade dos vídeos sintéticos atuais tornou a verificação facial remota sem vivacidade robusta um controle fraco.

24.8. Recuperação: onde o MFA quase sempre vaza

O padrão de falha é sempre o mesmo. A organização implanta chaves FIDO2, treina todo mundo, comemora — e mantém, no fluxo de “perdi meu dispositivo”, a possibilidade de a central de atendimento redefinir o MFA depois de confirmar nome, CPF e data de nascimento. **A segurança do sistema passa a ser a do atendimento telefônico.**

Regra geral: **a recuperação não pode ser mais fraca que o fator que ela recupera.** Na prática:

- **Registre mais de um autenticador desde o início.** Duas chaves de segurança, ou uma chave e uma passkey. Isso resolve a maior parte dos casos de perda sem nenhum fluxo excepcional.
- **Códigos de recuperação de uso único**, gerados por CSPRNG, exibidos uma única vez, armazenados como resumo (não em claro), e invalidados na primeira utilização.
- **Verificação de identidade compatível com o risco.** Para conta privilegiada: videoconferência com documento, ou presença física, ou aprovação de duas pessoas. Nunca só dados que constam de um cadastro vazado.
- **Período de espera para mudanças sensíveis.** Trocar o MFA com efeito imediato é o alvo do atacante; efeito em 24 horas, com notificação ao titular e possibilidade de cancelamento, converte o ataque numa janela de detecção.

24.9. Autenticação adaptativa

Nem toda operação merece a mesma fricção. A autenticação adaptativa (ou baseada em risco) calcula um escore a partir do contexto e ajusta a exigência: dispositivo conhecido e rede habitual passam com o fator principal; país novo, dispositivo desconhecido ou operação sensível exigem um fator adicional.

Os sinais que costumam entrar no escore são conhecidos: dispositivo reconhecido, faixa de rede, geografia e viagem impossível (autenticação em São Paulo e, vinte minutos depois, em Lisboa), horário atípico para aquela pessoa, reputação do endereço de origem, e sensibilidade da operação pretendida.

Dois cuidados. Primeiro, **a autenticação adaptativa complementa e não substitui o MFA em operação sensível** — reduzir fricção não pode virar dispensa de fator para transferência financeira ou alteração de cadastro. Segundo, o escore precisa ser explicável: quando alguém for barrado, o time de suporte tem de conseguir dizer por quê. Modelo opaco gera chamado insolúvel e, invariavelmente, uma exceção permanente que anula o controle.

24.10. Brasil em Foco

A troca de chip é o ataque que torna o SMS particularmente frágil no Brasil. A portabilidade numérica e os canais de atendimento das operadoras foram repetidamente explorados por engenharia social para assumir o número da vítima e, com ele, o segundo fator. A Anatel regula portabilidade e atendimento, e as operadoras adotaram controles adicionais, mas o vetor persiste. Para quem projeta autenticação de serviço brasileiro, a conclusão prática é direta: **SMS pode ser um fator adicional, nunca o caminho de recuperação de conta de valor** (Agência Nacional de Telecomunicações, 2019).

As normas do Banco Central exigem autenticação compatível com o risco da operação. A Resolução BCB 4.893/2021 estrutura a política de segurança cibernética das instituições financeiras, e o arranjo do Pix trouxe exigências específicas de autenticação, limites e mecanismos antifraude. A irreversibilidade da liquidação em segundos é o que muda a matemática: no cartão há estorno, no Pix não há. Isso empurra o setor para autenticação de posse vinculada ao dispositivo — que é, no fundo, o mesmo princípio criptográfico do FIDO2 (Banco Central do Brasil, 2021).

A LGPD trata dado biométrico como sensível, com regime mais restrito. O art. 5º, II, classifica biometria como dado pessoal sensível; o art. 11 restringe as hipóteses de tratamento. Na prática isso significa que implantar reconhecimento facial exige base legal específica, avaliação de impacto e — o ponto mais frequentemente ignorado — preferência arquitetural pela **correspondência no dispositivo**, que evita a formação de um banco central de características biométricas. Vazamento de biometria é irremediável por natureza, e essa característica pesa na análise de proporcionalidade (Brasil, 2018).

O gov.br usa níveis de confiabilidade, e o nível ouro é ancorado em biometria ou ICP-Brasil. Bronze, prata e ouro correspondem a graus crescentes de garantia sobre a identidade, com o nível mais alto exigindo validação biométrica contra as bases da Justiça Eleitoral ou da CNH, ou certificado digital ICP-Brasil. É a materialização brasileira da distinção que o NIST SP 800-63 faz entre nível de garantia de identidade e nível de garantia de autenticação — o primeiro sobre quem a pessoa é, o segundo sobre a força da autenticação a cada acesso (Instituto Nacional de Tecnologia da Informação, 2024; National Institute of Standards and Technology, 2024b; Secretaria de Governo Digital, Ministério da Gestão e da Inovação, 2024).

A assinatura eletrônica avançada e a qualificada dependem da força do segundo fator. A Lei 14.063/2020 organiza os níveis de assinatura no relacionamento com o poder público, e a MP 2.200-2 sustenta a presunção de autoria da assinatura ICP-Brasil no controle exclusivo da chave. Autenticação por SMS não sustenta a mesma qualidade probatória que posse de chave criptográfica com verificação de usuário — e essa diferença aparece quando a assinatura é contestada em juízo (Brasil, 2001, 2020).

24.11. No Campo

1. Levante a cobertura real antes de discutir tecnologia. A conversa produtiva não é “qual MFA adotar”, é “quais contas hoje não têm nenhum”:

```
# Entra ID / Microsoft 365: usuarios sem metodo forte registrado
Get-MgReportAuthenticationMethodUserRegistrationDetail -All |
  Where-Object { -not $_.IsMfaRegistered } |
  Select-Object UserPrincipalName, IsAdmin, LastUpdatedDateTime |
  Export-Csv .\sem-mfa.csv -NoTypeInfoInformation -Encoding utf8

# O achado que importa mais: administradores sem MFA
Get-MgReportAuthenticationMethodUserRegistrationDetail -All |
  Where-Object { $_.IsAdmin -and -not $_.IsMfaRegistered }
```

```
# Google Workspace
gam report users parameters accounts:is_2sv_enrolled | grep -i false

# SSH: chave publica sim, senha nao (o "MFA" do administrador de Linux)
grep -E '^((PasswordAuthentication|PubkeyAuthentication|AuthenticationMethods)'
↪ \
    /etc/ssh/sshd_config
# Alvo:
# PasswordAuthentication no
# AuthenticationMethods publickey,keyboard-interactive    (chave + TOTP)
```

2. Priorize por valor da conta, não por facilidade de implantação. A ordem que reduz risco mais rápido: administradores de domínio e de nuvem, acesso remoto e VPN, e-mail (que é o caminho de recuperação de todo o resto), sistemas financeiros, e só então o restante. Administrador sem MFA é achado crítico independentemente do tamanho da organização.

3. Elimine as exceções, porque elas concentram o ataque. Toda implantação acumula dispensas: a conta de serviço que “não suporta”, o executivo que reclamou, o sistema legado. Essas exceções são precisamente onde o atacante vai bater. Faça o inventário, dê prazo a cada uma e trate conta de serviço com o mecanismo adequado — certificado, identidade gerenciada, chave com escopo restrito — em vez de dispensa.

4. Desligue os protocolos de autenticação legados. Este é o passo que anula o MFA sem que ninguém perceba: IMAP, POP3, SMTP AUTH e outros protocolos antigos não sabem apresentar segundo fator, e por isso o ignoram. Enquanto estiverem habilitados, existe um caminho com senha apenas. Bloqueá-los costuma reduzir mais risco que qualquer outra medida isolada da implantação.

5. **Migre de *push* simples para correspondência numérica.** Configuração de baixo custo e efeito imediato contra fadiga de notificação. Some contexto no pedido (aplicativo, localização, endereço), limite de pedidos por minuto e alerta ao SOC quando um usuário acumular negações — esse é o sinal de que a senha dele já está em mãos erradas.
6. **Implante FIDO2 pelo topo.** Chaves físicas para quem tem acesso privilegiado, duas por pessoa (uma em uso, uma de reserva em local seguro), com registro presencial. Depois passkeys para a população geral. Comece por um piloto com o time de segurança, que produzirá a documentação de recuperação antes de a escala tornar cada caso um chamado.
7. **Audite o fluxo de recuperação como se fosse um ataque.** Ligue para a sua própria central de atendimento fingindo ser um usuário que perdeu o celular e veja o que basta para redefinir o MFA. Se nome, CPF e data de nascimento resolverem, seu MFA vale exatamente isso — todos esses dados estão em vazamentos públicos. É o teste mais barato e mais revelador de todo este capítulo.
8. **Registre e detecte os eventos de MFA, não só os de senha.** Sinais que merecem alerta: múltiplas negações de *push* na mesma conta (senha comprometida em uso), registro de novo método logo após uma autenticação (persistência do atacante), remoção de método forte, e uso de código de recuperação — que deve ser evento raro e sempre investigado.
9. **Teste a recuperação antes de precisar dela.** Escolha uma conta privilegiada não crítica, simule a perda do autenticador e execute o procedimento completo, cronometrando. O número que sair é o seu tempo real de recuperação, e ele é o insumo do plano de continuidade — porque uma organização inteira travada fora do provedor de identidade é um incidente de disponibilidade, não de segurança.
10. **Documente o caminho para sem senha.** MFA sobre senha é a etapa intermediária, não o destino. Passkey com verificação de usuário já entrega dois fatores num gesto e elimina a senha do fluxo — o que remove, de uma vez, o phishing de credencial, o preenchimento de credenciais e a quebra offline do Capítulo 23.

24.12. Resumo

- **MFA exige categorias diferentes de fator** — conhecimento, posse, inerência. Senha mais pergunta secreta é um fator só com dois campos.
- **A hierarquia de segundos fatores é qualitativa, não gradual.** SMS, TOTP e *push* simples transmitem um segredo digitável; FIDO2 assina uma origem. É por isso que a Twilio caiu e a Cloudflare, com a mesma isca, não.
- **TOTP é HMAC truncado sobre o tempo:** exige janela de tolerância curta, uso único por contador e limitação de taxa — e o segredo compartilhado precisa ser cifrado em repouso com chave fora do banco.
- **SMS é o fator mais fraco e ainda vale mais que nada.** Troca de chip e SS7 o comprometem; o erro grave é usá-lo como caminho universal de recuperação.
- **Fadiga de notificação** derrubou o *push* simples; a correção é correspondência numérica, com contexto no pedido e limite de tentativas.
- **FIDO2 e WebAuthn são resistentes a phishing por construção:** a chave é exclusiva por domínio e a origem entra no material assinado, sem depender de decisão humana. Passkeys são a mesma criptografia com chave sincronizada — escala melhor; chave física serve melhor ao acesso privilegiado.
- **Biometria não é segredo.** Ela deve destravar o autenticador local, com correspondência no dispositivo; base central de biometria é dado sensível e irrevogável.

24. Autenticação multifator e fatores de autenticação

- **A recuperação define o piso de segurança de todo o esquema.** Registre dois autenticadores, use códigos de uso único armazenados como resumo, verifique identidade de forma compatível com o risco e aplique espera para mudanças sensíveis.
- **Protocolo legado habilitado anula o MFA:** IMAP, POP3 e SMTP AUTH ignoram o segundo fator, e desligá-los costuma ser a medida isolada de maior efeito.
- No Brasil, a troca de chip torna o SMS particularmente frágil, a irreversibilidade do Pix empurra o setor financeiro para posse criptográfica, a biometria é dado sensível sob a LGPD e os níveis do gov.br materializam a distinção entre garantia de identidade e garantia de autenticação.

25. Federação de identidade: SAML, OAuth 2.0 e OpenID Connect

Toda organização de porte médio chega ao mesmo impasse. São quarenta sistemas — ERP, CRM, folha, chamados, monitoramento, três SaaS de marketing, o portal do cliente. Cada um com seu cadastro de usuários, sua política de senha, seu próprio conceito de quem é administrador. Quando alguém é desligado, o RH avisa a TI, a TI desativa o Active Directory, e as outras trinta e nove contas continuam existindo. Ninguém sabe exatamente quantas são, porque o inventário nunca esteve completo.

A federação de identidade resolve esse problema movendo a autenticação para **um lugar só**. As aplicações deixam de guardar senhas e passam a confiar num provedor central de identidade. Desligar alguém vira uma operação, não quarenta. A política de MFA do Capítulo 24 é definida uma vez e vale para tudo.

E cria um problema novo, que é a contrapartida honesta: **quem compromete o provedor de identidade compromete tudo o que confia nele**. A federação concentra o risco na mesma medida em que concentra o controle. Entender os protocolos é entender exatamente onde essa concentração pode ser explorada.

Este capítulo trata dos três padrões que dominam o campo — SAML, OAuth 2.0 e OpenID Connect — e do que costuma dar errado em cada um.

25.1. Os papéis e o vocabulário

Antes dos protocolos, os papéis. Eles são os mesmos nos três, com nomes diferentes:

- **Provedor de identidade** (IdP) — quem autentica a pessoa e emite a afirmação sobre ela. Entra ID, Keycloak, Okta, gov.br, Google.
- **Provedor de serviço** (SP), ou **parte confiante** (RP), ou **cliente** — a aplicação que confia no IdP em vez de autenticar por conta própria.
- **Sujeito** — a pessoa, ou o serviço, sobre quem a afirmação fala.
- **Afirmação** ou **token** — o documento assinado que carrega o resultado da autenticação e os atributos.

E uma distinção que atravessa todo o capítulo, porque confundi-la é a origem de uma classe inteira de vulnerabilidades:

***Autenticação** responde *quem é esta pessoa*. **Autorização delegada** responde *o que este aplicativo pode fazer em nome dela*.*

SAML e OpenID Connect fazem autenticação. **OAuth 2.0, sozinho, não faz** — ele é um protocolo de autorização delegada, e usá-lo como se fosse de autenticação produz falhas reais, não teóricas. Voltaremos a isso.

25.2. SAML 2.0: XML, assinatura e o navegador como intermediário

SAML é de 2005, é XML, é verboso — e é o padrão que sustenta a integração corporativa até hoje, porque funciona e porque está em toda parte (OASIS, 2005).

O fluxo mais usado é o *Web Browser SSO Profile*, iniciado pelo provedor de serviço:

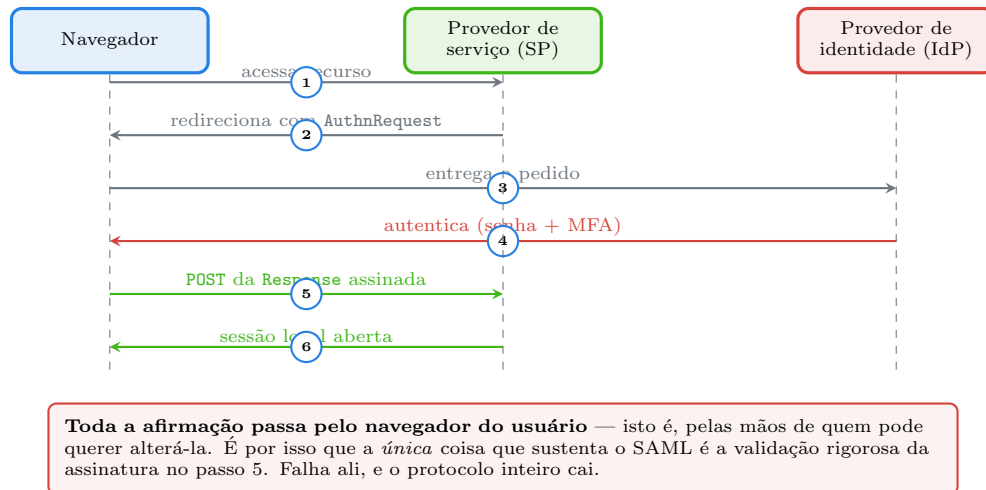


Figura 25.1.: Fluxo SAML Web SSO. A afirmação trafega pelo agente do usuário, e a assinatura é a única defesa.

A afirmação é um documento XML assinado, e as partes que importam:

```

<saml:Assertion ID="_a1b2c3" IssueInstant="2026-07-30T14:22:11Z">
  <saml:Issuer>https://idp.exemplo.com.br</saml:Issuer>
  <ds:Signature>...</ds:Signature>          <!-- a única defesa real -->
  <saml:Subject>
    <saml:NameID Format="...emailAddress">ana@exemplo.com.br</saml:NameID>
    <saml:SubjectConfirmation Method="...bearer">
      <saml:SubjectConfirmationData
        NotOnOrAfter="2026-07-30T14:27:11Z"
        Recipient="https://app.exemplo.com.br/saml/acs"
        InResponseTo="_pedido123"/>          <!-- vincula ao pedido enviado -->
      </saml:SubjectConfirmationData>
    </saml:SubjectConfirmation>
  </saml:Subject>
  <saml:Conditions NotBefore="2026-07-30T14:22:11Z"
    NotOnOrAfter="2026-07-30T14:27:11Z">
    <saml:AudienceRestriction>
      <saml:Audience>https://app.exemplo.com.br</saml:Audience>
    </saml:AudienceRestriction>          <!-- para QUEM esta afirmacao vale -->
  </saml:Conditions>
  <saml:AttributeStatement>
    <saml:Attribute Name="grupos">
      <saml:AttributeValue>financeiro</saml:AttributeValue>
    </saml:Attribute>
  </saml:AttributeStatement>
</saml:Assertion>
  
```

Cada campo existe por causa de um ataque. **Audience** impede que uma afirmação emitida para o sistema A seja apresentada ao sistema B. **NotOnOrAfter** limita a janela de reprodução. **InResponseTo** amarra a resposta ao pedido específico. **Recipient** fixa o destino. **Validar a assinatura e ignorar esses campos é uma implementação incompleta**, e implementações incompletas são a norma.

25.2.1. As armadilhas do SAML

Envelopamento de assinatura (XSW). É a família de ataques mais característica do SAML, e ela explora uma dissociação: o verificador confere a assinatura de um elemento, mas a aplicação lê os dados de **outro** elemento. O atacante mantém a afirmação original assinada — válida, mas em posição ou identificador diferente — e insere uma afirmação forjada que o processador de aplicação acaba lendo. A biblioteca diz “assinatura válida”; a aplicação lê o nome do atacante.

A correção não é conferir a assinatura com mais cuidado: é garantir que **o objeto verificado e o objeto consumido sejam o mesmo**, rejeitar documentos com múltiplas afirmações, processar apenas afirmações cobertas pela assinatura e usar uma biblioteca madura em vez de montar a verificação com um analisador de XML genérico.

XXE e expansão de entidade. Processar XML de fonte externa com resolução de entidades habilitada permite leitura de arquivos do servidor e requisições internas (SSRF). Desabilitar DTD e entidades externas no analisador é obrigatório — não é endurecimento opcional.

Aceitar afirmação não solicitada. O perfil permite fluxo iniciado pelo IdP, em que a aplicação recebe uma afirmação sem ter pedido nada. É conveniente e amplia a superfície: sem **InResponseTo** para comparar, a defesa contra reprodução fica só nos carimbos de tempo. Onde a operação não exigir, desabilite.

Metadados e certificado desatualizados. A confiança está no certificado do IdP configurado no SP. Certificado vencido derruba a integração; certificado antigo mantido aceito depois da rotação prolonga a validade de material que deveria ter saído de circulação. Rotação de certificado de assinatura precisa estar no calendário — é o mesmo problema de ciclo de vida do Capítulo 19.

25.3. OAuth 2.0: delegação, não autenticação

OAuth 2.0 resolve um problema diferente. Você quer que um aplicativo de contabilidade leia suas notas fiscais num sistema, sem entregar a ele sua senha. A solução: o aplicativo recebe um **token de acesso** com escopo limitado e validade curta, emitido pelo servidor de autorização depois que **você** aprovou (Hardt, 2012).

Os papéis: **dono do recurso** (você), **cliente** (o aplicativo), **servidor de autorização** (emite tokens) e **servidor de recursos** (a API que valida o token).

O fluxo correto — e hoje o único recomendado para praticamente todos os casos — é o **código de autorização com PKCE**:

```
import secrets, hashlib, base64

# Cliente, antes de iniciar o fluxo (RFC 7636)
verificador =
    ↪ base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
desafio = base64.urlsafe_b64encode(
```

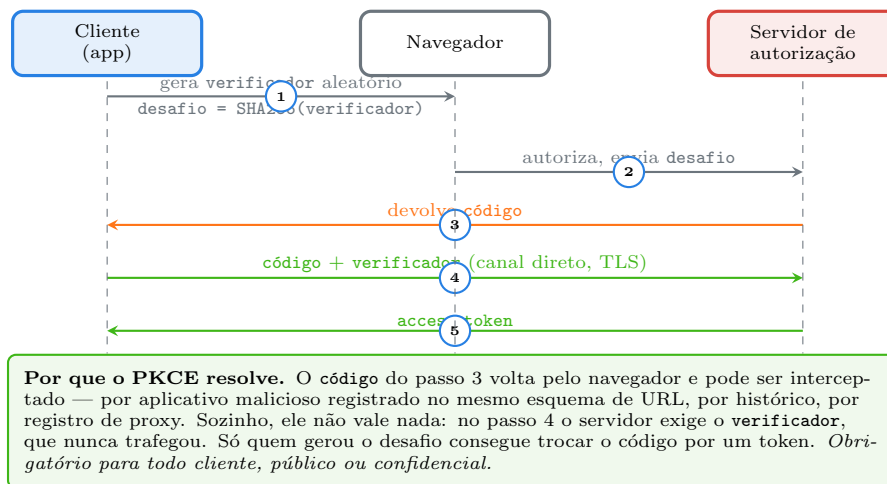


Figura 25.2.: Código de autorização com PKCE. O verificador é o que torna o código interceptado inútil.

```
hashlib.sha256(verificador.encode()).digest()).rstrip(b"=").decode()

url = (f"{AS}/authorize?response_type=code&client_id={CLIENT_ID}"
      f"&redirect_uri={REDIRECT}&scope=nfe.leitura"
      f"&state={secrets.token_urlsafe(16)}"          # anti-CSRF, obrigatorio
      f"&code_challenge={desafio}&code_challenge_method=S256")

# Depois de receber o codigo: troca no canal direto, com o verificador
resp = requests.post(f"{AS}/token", data={
    "grant_type": "authorization_code",
    "code": codigo_recebido,
    "redirect_uri": REDIRECT,
    "client_id": CLIENT_ID,
    "code_verifier": verificador,          # o que o interceptador nao tem
}, timeout=10)
```

25.3.1. Os fluxos que morreram, e por quê

Tabela 25.1.: Situação atual dos fluxos OAuth 2.0, conforme a BCP de segurança.

Fluxo	Situação	Motivo
Código de autorização + PKCE	Recomendado	Código interceptado é inútil sem o verificador
Credenciais de cliente	Recomendado (máquina a máquina)	Não há usuário envolvido
Implícito	Obsoleto	Token no fragmento da URL: histórico, log, Referer
Senha do dono do recurso	Obsoleto	A aplicação vê a senha — anula o propósito do OAuth
Dispositivo	Uso específico	TV, console, CLI sem navegador

A BCP de segurança do OAuth consolidou essas decisões: PKCE obrigatório sempre, `redirect_uri` comparado por **igualdade exata** (nada de correspondência por prefixo ou curinga), implícito e senha do dono aposentados (Denniss; Bradley, 2017; Lodderstedt et al., 2025).

25.3.2. As armadilhas do OAuth

Redirecionamento aberto. Se o servidor de autorização aceita `redirect_uri` por prefixo, `https://app.exemplo.com.br.atacante.com` ou `https://app.exemplo.com.br/redir?next=//atacante.com` desviam o código. É a vulnerabilidade mais explorada da família, e a defesa é uma lista branca com comparação exata.

Ausência de state. O parâmetro `state` é aleatório, ligado à sessão do usuário e conferido no retorno. Sem ele, o atacante induz a vítima a completar um fluxo que vincula a conta dela à conta dele no serviço externo — CSRF de autorização.

Confusão de destinatário. Um token emitido para a API A apresentado à API B, que o aceita sem verificar o campo `aud`. Toda API tem de validar emissor, audiência, validade e escopo antes de qualquer coisa.

Escopos amplos e permanentes. `scope=full_access` para quem só precisa ler é a versão OAuth do excesso de privilégio do Capítulo 22. E o **token de renovação** é a credencial de vida longa que costuma ser esquecida: sem rotação e sem detecção de reutilização, ele é a chave permanente que sobrevive a toda a política de expiração.

Revogação. Esta é a fragilidade estrutural do modelo de capacidade que o Capítulo 22 antecipou: token de acesso autocontido (JWT) é validado por assinatura, sem consulta ao emissor — o que é ótimo para desempenho e péssimo para revogação. Desativar o usuário no IdP **não invalida** um token já emitido; ele vale até expirar. Daí a regra prática: tokens de acesso com vida curta (5 a 15 minutos), revogação real no token de renovação, e introspecção para operações sensíveis.

25.4. OpenID Connect: a camada que faltava

OIDC é uma camada de **autenticação** sobre o OAuth 2.0, e existe precisamente porque tanta gente usou OAuth para autenticar sem perceber a diferença (OpenID Foundation, 2014).

O acréscimo central é o **token de identidade** (`id_token`): um JWT assinado que afirma *quem* autenticou, *quem* emitiu, *para quem* vale e *quando* aconteceu.

```
{
  "iss": "https://idp.exemplo.com.br", // emissor: confirme que é o esperado
  "sub": "a1b2c3d4",                 // identificador ESTÁVEL do usuário
  "aud": "cliente-abc",               // para você - verifique
  "exp": 1785412800,                  // expira
  "iat": 1785412500,
  "nonce": "n-OS6_WzA2Mj",           // amarra ao SEU pedido:
  ↪ anti-reprodução
  "auth_time": 1785412480,
  "amr": ["pwd", "hwk"],              // como autenticou: senha + chave
  "email": "ana@exemplo.com.br",
  "email_verified": true               // sem isto, o e-mail não prova nada
}
```

A diferença prática entre OAuth e OIDC merece ser dita sem meias palavras. Com OAuth puro, a aplicação recebe um token de acesso e conclui “a autenticação funcionou”. Mas o token de acesso é opaco para o cliente e **não é destinado a ele** — é destinado à API. Um aplicativo malicioso pode obter um token legítimo de um usuário para *seus próprios* fins e apresentá-lo a um terceiro site que o aceite como prova de identidade. É o **problema do confuso deputado**, e produziu falhas reais em implementações de “entrar com” no início da década de 2010.

O `id_token` corrige isso porque é assinado, **destinado explicitamente ao cliente** (`aud`) e amarrado ao pedido específico (`nonce`).

A validação mínima é inegociável:

```
import jwt    # PyJWT
from jwt import PyJWKClient

jwks = PyJWKClient(f"{ISSUER}/.well-known/jwks.json")    # rotação automática

def validar_id_token(token: str, nonce_enviado: str) -> dict:
    chave = jwks.get_signing_key_from_jwt(token).key
    dados = jwt.decode(
        token,
        chave,
        algorithms=["RS256"],          # LISTA FIXA - nunca ler do cabeçalho
        audience=CLIENT_ID,           # aud
        issuer=ISSUER,                 # iss
        options={"require": ["exp", "iat", "aud", "iss", "sub"]},
    )
    if dados.get("nonce") != nonce_enviado:
        raise ValueError("nonce divergente: possível reprodução")
    return dados
```

25.4.1. As armadilhas do JWT

alg: none. A falha histórica: a biblioteca lia o algoritmo do cabeçalho do próprio token e, se ele dizia `none`, aceitava sem assinatura. Bibliotecas atuais corrigiram, mas o princípio permanece — **o algoritmo aceito é decisão do verificador, nunca do token.**

Confusão de algoritmo. Token emitido com RS256 (assimétrico) e apresentado como HS256 (simétrico), usando a **chave pública** como se fosse segredo HMAC. A chave pública é pública; o atacante forja livremente. A defesa é a mesma: lista fixa de algoritmos no verificador.

Usar o e-mail como identificador. Endereços mudam de dono e são reatribuídos. O identificador estável é **sub combinado com iss**. Ancorar contas no e-mail cria a situação em que o novo titular de um endereço herda a conta do anterior — e, em federação com múltiplos IdPs, permite que um IdP afirme e-mails de domínio que não controla.

email_verified ignorado. Aceitar `email` sem checar `email_verified` permite que alguém cadastre no IdP um e-mail alheio e assuma a conta correspondente na sua aplicação.

Token no armazenamento local do navegador. `localStorage` é legível por qualquer script da página: um XSS vira roubo de token direto. Cookie `HttpOnly`, `Secure` e `SameSite` é a alternativa correta. O Volume 7 desenvolve isso.

25.5. Comparação e escolha

Tabela 25.2.: SAML, OAuth 2.0 e OIDC lado a lado.

	SAML 2.0	OAuth 2.0	OIDC
Propósito	Autenticação	Autorização delegada	Autenticação
Formato	XML assinado	Opaco ou JWT	JWT
Idade	2005	2012	2014
Ponto forte	Onipresente no corporativo	Delegação com escopo	Simples, nativo para móvel e SPA
Fraqueza típica	Complexidade do XML, XSW	Não autentica sozinho	Herda as armadilhas do JWT
Onde usar	SaaS corporativo, integração legada	API, acesso de terceiro	Aplicação nova, móvel, SPA

Regra de decisão que resolve a maioria dos casos: **aplicação nova usa OIDC**; **SaaS corporativo usa o que ele suporta** (frequentemente só SAML); **API para terceiro usa OAuth 2.0 com PKCE**. E, sempre: nunca OAuth puro como prova de identidade.

25.6. Os riscos que a federação cria

Vale enfrentar de frente o que a concentração traz:

O IdP vira alvo único. Comprometer o provedor de identidade é comprometer tudo. Consequência prática: o IdP recebe o tratamento de sistema mais crítico da organização — MFA resistente a phishing para seus administradores, acesso privilegiado com elevação temporária, log completo e monitoramento dedicado.

Roubo de token de sessão contorna todo o resto. O atacante que rouba o cookie de sessão federada — por malware, por XSS, por proxy de phishing — está autenticado sem passar por senha nem MFA. Ataques de retransmissão em tempo real terminam exatamente aqui. As defesas: vinculação do token ao dispositivo (mTLS ou DPoP), sessões curtas para operação sensível, e detecção de mudança abrupta de dispositivo ou geografia dentro da mesma sessão (Campbell et al., 2020; Fett et al., 2023).

Afirmação forjada. Quem obtém a chave de assinatura do IdP emite afirmações para qualquer usuário, em qualquer aplicação, sem tocar no IdP. Foi o padrão do incidente Golden SAML da campanha SolarWinds. A defesa é custódia da chave de assinatura em HSM, rotação periódica e alerta sobre qualquer emissão que não corresponda a uma autenticação registrada.

Confiança transitiva mal delimitada. Federação entre organizações — matriz e filial, empresa e parceiro — significa aceitar as decisões de autenticação de outra organização. Se o parceiro não exige MFA, seu sistema não exige MFA para os usuários dele. A federação precisa de contrato explícito sobre nível de garantia, e o `amr` do token é o que permite verificar isso em tempo de execução.

Disponibilidade. IdP fora do ar é a organização inteira sem acesso. Isso muda a classificação do serviço no plano de continuidade do Volume 14 e exige, no mínimo, um caminho de contingência para os administradores.

25.7. Brasil em Foco

O gov.br é o maior provedor de identidade federado do país, e usa OIDC. Milhares de serviços públicos federais, estaduais e municipais autenticam por ele, com os níveis bronze, prata e ouro sinalizando a confiabilidade da identidade. Para quem integra, dois pontos merecem atenção: o nível exigido deve ser compatível com a sensibilidade do serviço — nível bronze para consulta simples, ouro para assinatura ou movimentação — e a validação do `id_token` precisa seguir todas as verificações deste capítulo, especialmente `iss`, `aud` e `nonce` (OpenID Foundation, 2014; Secretaria de Governo Digital, Ministério da Gestão e da Inovação, 2024).

O Open Finance brasileiro é construído sobre OAuth 2.0 e OIDC com perfil endurecido. O arranjo regulado pelo Banco Central adota o perfil FAPI, que acrescenta exigências acima do padrão: autenticação mútua por TLS ou assinatura de cliente com chave privada, requisição de autorização assinada, vinculação do token ao certificado e consentimento com escopo e prazo explícitos. É o exemplo mais maduro de federação regulada no Brasil, e o consentimento granular com prazo é a materialização técnica do princípio da finalidade da LGPD (Banco Central do Brasil; Conselho Monetário Nacional, 2020; Brasil, 2018; Campbell et al., 2020).

Federação é compartilhamento de dado pessoal e precisa de base legal. Quando o IdP envia atributos ao provedor de serviço — nome, CPF, e-mail, grupos, cargo —, há tratamento por ambos. Isso exige base legal, minimização (enviar só os atributos necessários àquela aplicação, e não o diretório inteiro) e clareza sobre os papéis de controlador e operador. Configuração de SAML que despeja todos os atributos do usuário para todo SP integrado viola a minimização do art. 6º, III, mesmo que ninguém use os campos extras (Brasil, 2018).

Desligamento e o dever de encerrar o acesso. A promessa da federação — desativar num lugar e cortar em todos — só se cumpre se o encerramento de sessão for propagado. Token de acesso já emitido continua válido até expirar, e a integração precisa combinar tempo de vida curto com revogação efetiva. Para dado pessoal sob a LGPD, manter acesso ativo de ex-funcionário é falha de medida de segurança sob o art. 46, e a ausência de log dessa transição inviabiliza a demonstração do contrário (Brasil, 2018).

Provedores de conexão e aplicação têm dever de guarda de registros. O Marco Civil (arts. 13 e 15) impõe a guarda de registros de conexão e de acesso a aplicações, com prazos definidos. Em arquitetura federada, é preciso decidir explicitamente quem registra o quê: o IdP tem o evento de autenticação, o SP tem o evento de acesso ao recurso. Sem correlação entre os dois — por identificador de sessão ou de requisição —, a resposta a uma ordem judicial fica incompleta (Brasil, 2014).

25.8. No Campo

1. Inventarie as integrações existentes antes de criar a próxima. Toda organização com federação madura tem aplicações registradas que ninguém reconhece:

```
# Entra ID: aplicativos empresariais e quando foram usados pela ultima vez
Get-MgServicePrincipal -All |
    Select-Object DisplayName, AppId, SignInAudience, ServicePrincipalType |
    Export-Csv .\apps-federados.csv -NoTypeInfo -Encoding utf8

# Consentimentos amplos concedidos a aplicativos de terceiros - o achado
Get-MgOAuth2PermissionGrant -All |
    Where-Object { $_.Scope -match 'Mail.Read|Files.ReadWrite|Directory.Read' } |
```

```
Select-Object ClientId, ConsentType, Scope
```

Consentimento do tipo `AllPrincipals` (para toda a organização) concedido a um aplicativo desconhecido é achado crítico: é o mecanismo do phishing de consentimento, em que a vítima autoriza um aplicativo malicioso e o atacante obtém acesso persistente sem nunca tocar na senha.

2. Desligue o consentimento livre de usuário. Por padrão, muitos ambientes permitem que qualquer pessoa autorize um aplicativo de terceiro a ler seu e-mail. Restrinja a consentimento administrativo, ou a um conjunto de escopos de baixo risco, com fluxo de aprovação para o restante.

3. Faça a auditoria de configuração de cada integração. A lista de verificação que pega a maior parte dos problemas:

```
# SAML - no metadado e na configuracao do SP
# [ ] assinatura da Response E da Assertion exigidas
# [ ] AudienceRestriction confere com a entidade correta
# [ ] NotOnOrAfter respeitado, com tolerancia de relógio <= 3 min
# [ ] InResponseTo validado (e fluxo iniciado pelo IdP desabilitado)
# [ ] certificado do IdP com validade > 60 dias e rotacao agendada
# [ ] analisador XML com DTD e entidades externas DESABILITADAS
# [ ] biblioteca madura, nao verificacao manual de assinatura

# OAuth / OIDC
# [ ] PKCE (S256) obrigatorio, inclusive em cliente confidencial
# [ ] redirect_uri por igualdade exata, sem curinga
# [ ] state gerado por CSPRNG e conferido no retorno
# [ ] nonce gerado, enviado e conferido no id_token
# [ ] algoritmos em lista fixa no verificador
# [ ] access_token com vida de 5 a 15 min
# [ ] refresh_token com rotacao e deteccao de reuso
# [ ] token em cookie HttpOnly+Secure+SameSite, nunca em localStorage
```

4. Verifique o `redirect_uri` com um teste ativo. Em homologação, tente registrar e usar variações: subdomínio adicional, sufixo no domínio, caminho extra, // no início, codificação de caractere. Se alguma passar, você tem um redirecionamento aberto — a vulnerabilidade mais explorada do OAuth.

5. Reduza a vida do token e implemente rotação de renovação. Acesso de 5 a 15 minutos, renovação rotacionada a cada uso, com **detecção de reutilização**: se um token de renovação já usado reaparecer, é sinal de roubo, e toda a família de tokens daquele usuário deve ser invalidada imediatamente.

6. Minimizar os atributos enviados a cada provedor de serviço. Faça o inventário de quais atributos cada SP realmente consome e corte o resto. A configuração padrão que envia o diretório inteiro é comum, viola a minimização e amplia o dano de qualquer comprometimento do SP.

7. Proteja a chave de assinatura do IdP como o ativo mais crítico que ela é. HSM ou serviço gerenciado de chaves, rotação periódica com sobreposição planejada, e alerta sobre emissão de afirmação sem evento de autenticação correspondente — o sinal de Golden SAML.

8. Registre e correlacione os dois lados. O IdP registra a autenticação; o SP registra o acesso. Sem um identificador comum de sessão ou requisição atravessando os dois, a investigação de “quem acessou o quê e como autenticou” não fecha. Defina esse identificador na integração, não durante o incidente.

9. Monitore os eventos que indicam abuso de federação. Registro de novo aplicativo ou de nova credencial de cliente, alteração de configuração de confiança, consentimento amplo concedido, emissão de token para escopo administrativo, e uso do mesmo token a partir de origens geograficamente incompatíveis.

10. Planeje a indisponibilidade do IdP. Defina o caminho de contingência para administradores — contas locais de emergência, com credencial forte, guardadas em cofre físico, monitoradas em uso e testadas semestralmente. Sem isso, uma falha do provedor de identidade tranca a organização fora da própria infraestrutura no exato momento em que ela precisa agir.

25.9. Resumo

- **Federação concentra controle e risco na mesma medida.** Um lugar para desligar acesso, um lugar para comprometer tudo — o IdP passa a ser o sistema mais crítico da organização.
- **SAML autentica, OAuth delega, OIDC autentica sobre OAuth.** Usar OAuth puro como prova de identidade é a confusão que produz falhas reais, e é exatamente o que o `id_token` do OIDC corrige.
- Em **SAML**, tudo depende da validação da assinatura, porque a afirmação passa pelo navegador. Envelopamento de assinatura, XXE, afirmação não solicitada e certificado desatualizado são as armadilhas; validar `Audience`, `NotOnOrAfter`, `InResponseTo` e `Recipient` é tão obrigatório quanto validar a assinatura.
- Em **OAuth 2.0**, o único fluxo recomendado para usuário é **código de autorização com PKCE**. Implícito e senha do dono do recurso estão obsoletos; `redirect_uri` compara-se por igualdade exata; `state` é obrigatório.
- Em **OIDC**, o `id_token` precisa ser validado com lista fixa de algoritmos, `iss`, `aud`, `exp` e `nonce` conferidos. O identificador estável é `sub + iss`, nunca o e-mail — e `email_verified` não é opcional.
- **Revogação é a fragilidade estrutural do modelo de token:** desativar o usuário não invalida token já emitido. A compensação é vida curta no acesso, rotação com detecção de reuso no renovação e introspecção em operação sensível.
- **Roubo de token de sessão contorna senha e MFA.** Vinculação ao dispositivo (mTLS, DPoP), sessão curta em operação sensível e detecção de anomalia dentro da sessão são as defesas.
- **Comprometer a chave de assinatura do IdP permite forjar identidade sem tocar no IdP** (Golden SAML). HSM, rotação e alerta sobre emissão sem autenticação correspondente.
- No Brasil, o gov.br federa serviços públicos por OIDC com níveis de confiabilidade; o Open Finance usa OAuth com perfil FAPI e consentimento granular; e a federação é compartilhamento de dado pessoal, sujeito a base legal e minimização de atributos.

26. Gestão de identidades e de acessos privilegiados (IAM e PAM)

Toda auditoria de acesso encontra a mesma pessoa. Ela entrou como estagiária no suporte, virou analista, passou pela equipe de infraestrutura, foi para o financeiro e hoje é coordenadora de projetos. Em doze anos, acumulou os acessos de cinco funções — e nunca perdeu um. Tem permissão de leitura no banco de produção que ninguém mais do time dela tem, ainda administra três máquinas virtuais que sobraram do tempo de infra, e continua no grupo que aprova pagamento.

Ninguém errou. Cada concessão foi legítima no dia em que aconteceu. O que faltou foi o outro lado da operação: **remover**. E a soma de doze anos de concessões legítimas sem nenhuma revogação produz um acesso que nenhum gestor teria aprovado se lhe pedissem tudo de uma vez.

Esse é o problema que a gestão de identidades resolve. O Capítulo 22 tratou de *como decidir* quem pode o quê; este capítulo trata de *como operar essa decisão ao longo do tempo*, em escala, com rotatividade — e do subconjunto onde o erro custa mais caro: o acesso privilegiado.

26.1. O ciclo de vida da identidade

Uma identidade corporativa nasce, muda e morre. A disciplina de IAM é fazer com que os três eventos aconteçam no sistema no mesmo ritmo em que acontecem na realidade.

A regra estruturante é que **a fonte da verdade é o RH, não a TI**. Quem sabe que alguém foi contratado, mudou de área ou foi desligado é o sistema de gestão de pessoas. Todo processo que depende de um e-mail avisando a TI vai falhar — não por má vontade, mas porque processos manuais têm taxa de erro e o desligamento costuma acontecer numa sexta-feira à tarde.

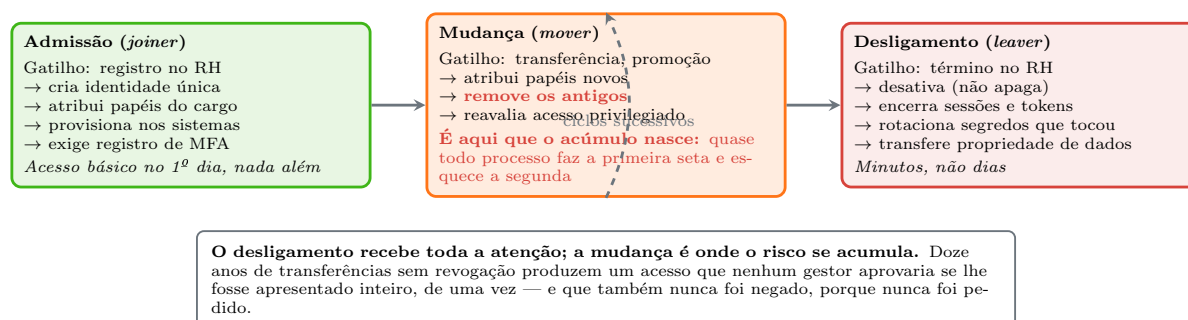


Figura 26.1.: As três fases do ciclo de vida. A do meio é a que quebra na prática.

26.1.1. Admissão

O objetivo é que a pessoa tenha, no primeiro dia, exatamente o acesso do cargo — e nada mais. O padrão que funciona é o **acesso por nascimento** (*birthright access*): um conjunto derivado automaticamente de cargo, departamento e localidade. Tudo além disso passa por solicitação com aprovação.

O erro mais comum aqui tem nome: **clonagem de acesso**. “Dê ao novo analista o mesmo acesso do Bruno.” É rápido, é tentador, e propaga em uma operação todo o acúmulo de privilégios que o Bruno juntou em oito anos. Uma vez adotada, a clonagem faz o excesso crescer geometricamente.

26.1.2. Mudança

É a fase que quase todo mundo implementa pela metade. O gatilho da transferência concede o acesso novo — porque a pessoa reclama se não tiver — e não remove o antigo, porque ninguém reclama do que sobra.

A implementação correta trata a mudança como **desligamento seguido de admissão**: revoga tudo o que derivava do cargo anterior e concede o que deriva do novo, com um período de sobreposição curto e explícito onde a transição de trabalho exigir. E toda exceção a essa regra recebe prazo de validade.

26.1.3. Desligamento

O relógio importa. Entre a decisão de desligar e a desativação efetiva existe uma janela, e é nela que ocorrem a cópia de base de clientes, a exclusão de arquivos e o acesso de retaliação. O alvo é **minutos**, com o gatilho vindo do RH.

E desativar a conta principal não basta. O que sobrevive a um desligamento mal feito:

- Tokens de API e chaves de acesso emitidas em nome da pessoa
- Chaves SSH espalhadas por `authorized_keys` de servidores
- Sessões ativas em aplicações federadas (que valem até o token expirar, Capítulo 25)
- Contas locais em equipamentos de rede e appliances, fora do diretório
- Acesso a repositórios de código, registros de contêiner e ferramentas de CI/CD
- Credenciais compartilhadas que a pessoa conhecia — que precisam ser **rotacionadas**, não apenas revogadas
- Encaminhamento de e-mail e regras de caixa postal configuradas para endereço externo

Essa última merece destaque porque é usada tanto por ex-funcionário quanto por atacante: uma regra de encaminhamento continua exfiltrando mensagens depois que a conta foi desativada, se ela existir no nível do servidor.

26.2. Governança: além do provisionamento

Provisionar é operar. Governar é responder — a qualquer momento, com evidência — a três perguntas: **quem tem acesso a quê? por quê? ainda deveria?**

26.2.1. Recertificação

A campanha de revisão periódica é o instrumento principal, e a maioria delas é teatro. O sintoma é a taxa de revogação: campanhas que revogam menos de 5% dos acessos revisados estão medindo a velocidade de clique dos gestores, não a adequação dos acessos.

O que faz a diferença:

- **Padrão de exclusão, não de manutenção.** O acesso que o gestor não confirmar ativamente até o prazo é revogado. Inverte o ônus e é a única mudança que muda o resultado.
- **Contexto na tela.** Mostre a data da última utilização daquele acesso. Gestor que vê “não usado há 14 meses” revoga; gestor que vê apenas `SAP_FI_DISPLAY_ALL` aprova.
- **Escopo por risco.** Acesso privilegiado a cada trimestre; acesso comum a cada semestre; acesso de baixo risco anualmente ou nunca — revisar tudo com a mesma frequência dilui a atenção onde ela importa.
- **Granularidade útil.** Revisar 400 permissões atômicas é impossível; revisar 12 papéis com descrição em linguagem de negócio é viável.

26.2.2. Solicitação e aprovação

O acesso além do básico passa por um fluxo com quatro elementos: **justificativa** escrita pelo solicitante, **aprovação** de quem tem responsabilidade (gestor e, para dado sensível, o dono do dado), **prazo** de validade — acesso permanente é a exceção, não o padrão — e **registro** completo de quem pediu, quem aprovou, quando e por quê.

O prazo é o item mais transformador e o menos adotado. Acesso concedido “até 31/12” desaparece sozinho; acesso permanente exige que alguém se lembre de removê-lo, o que nunca acontece.

26.2.3. Análise de acesso não utilizado

O relatório mais barato e mais eficaz de toda a disciplina: permissões concedidas e nunca exercidas. As nuvens públicas oferecem isso pronto — o IAM Access Analyzer da AWS gera política com base no uso real, e há equivalentes no Azure e no GCP. A conversa com o gestor muda completamente quando a pergunta deixa de ser “ele precisa disso?” e passa a ser “ele não usou isso em 18 meses; podemos remover?”.

26.3. Acesso privilegiado: onde o erro custa mais

Conta privilegiada é a que pode alterar a configuração do sistema, acessar dado de todos os usuários, apagar registro de auditoria ou conceder privilégio a outros. Elas são uma fração pequena do total e concentram quase todo o risco — motivo pelo qual têm disciplina própria.

O inventário costuma ser maior do que se imagina: administradores de domínio, `root` e `Administrator` locais, contas de serviço (numerosas, esquecidas e frequentemente com privilégio excessivo), administradores de banco, credenciais de equipamento de rede, chaves raiz de nuvem, credenciais de pipeline de CI/CD e contas de emergência.

26. Gestão de identidades e de acessos privilegiados (IAM e PAM)

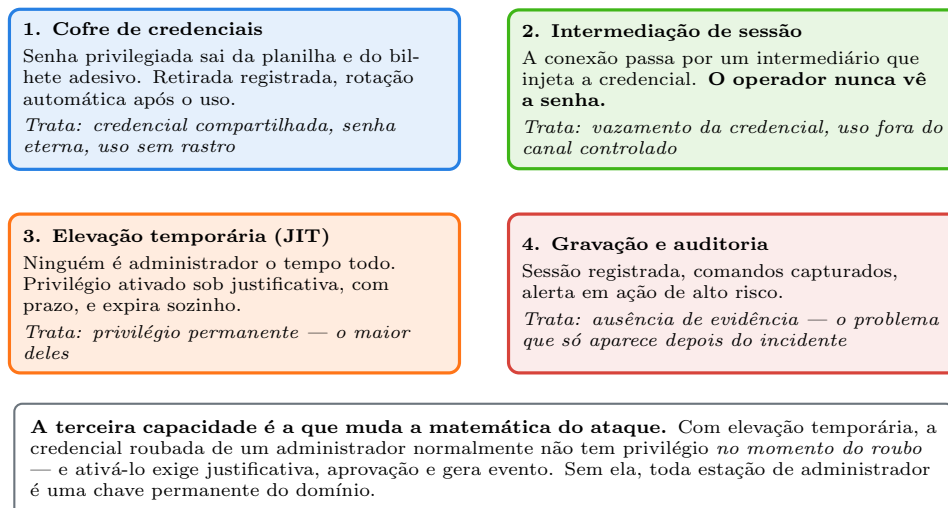


Figura 26.2.: As quatro capacidades de PAM e o risco específico que cada uma endereça.

26.3.1. As quatro capacidades do PAM

Elevação temporária merece o destaque que a figura lhe dá. O modelo tradicional — a pessoa é administradora do domínio, permanentemente — significa que comprometer a estação dela em qualquer momento é comprometer o domínio. Com acesso *just-in-time*, o privilégio existe por 60 minutos, mediante justificativa, com registro e alerta. Um roubo de credencial fora dessa janela rende uma conta comum.

26.3.2. Contas de serviço

São o problema mais persistente do PAM, porque não têm dono natural, não trocam de senha, frequentemente têm privilégio muito além do necessário e ninguém sabe quais sistemas quebram se elas mudarem.

O caminho é, sempre que possível, **eliminá-las**: identidade gerenciada em nuvem, autenticação por certificado, conta de serviço gerenciada por grupo no Active Directory — todas trocam a senha estática por credencial rotacionada pela plataforma. Onde a senha estática for inevitável, ela vai para o cofre, com rotação automatizada, dono nomeado e privilégio revisado.

```
# Contas de servico com senha que nunca expira e privilegio de dominio
Get-ADUser -Filter { PasswordNeverExpires -eq $true -and Enabled -eq $true } `
  -Properties PasswordLastSet, MemberOf, ServicePrincipalName |
Where-Object { $_.ServicePrincipalName -or $_.MemberOf -match 'Admin' } |
Select-Object Name, PasswordLastSet,
  @{n='DiasSemTroca';e={ (New-TimeSpan -Start $_.PasswordLastSet).Days }} |
Sort-Object DiasSemTroca -Descending
```

Encontrar contas com `DiasSemTroca` na casa dos milhares é rotina. Cada uma é uma credencial que sobreviveu a todos os desligamentos do período.

26.3.3. Kerberoasting: por que a conta de serviço fraca é um caminho para o domínio

Vale explicar o mecanismo, porque ele conecta o Capítulo 23 a este capítulo e explica por que “senha de conta de serviço” não é detalhe.

No Active Directory, qualquer usuário autenticado pode solicitar um tíquete de serviço para uma conta que tenha SPN registrado. Parte desse tíquete é cifrada com uma chave derivada da **senha da conta de serviço**. O atacante com acesso comum ao domínio solicita o tíquete, leva-o para fora e tenta quebrar a senha offline — sem limitação de taxa, sem gerar falhas de autenticação, sem alarme.

Conta de serviço com senha de 10 caracteres criada em 2014 cai em horas. Se ela for administradora do domínio — situação lamentavelmente comum —, o comprometimento do domínio inteiro decorre de uma senha fraca escolhida uma década antes.

As defesas são específicas: senha de 25 caracteres ou mais gerada aleatoriamente (o que torna a quebra offline inviável), preferência por contas gerenciadas por grupo (a plataforma gera e rotaciona senha de 240 bits), remoção de contas de serviço dos grupos administrativos, e detecção de solicitação anômala de tíquetes de serviço (evento 4769 com cifra RC4). O Volume 6 desenvolve o endurecimento do Active Directory em detalhe.

26.3.4. Modelo em camadas

O princípio que organiza o acesso administrativo é impedir que credencial de alto privilégio apareça em máquina de baixa confiança:

- **Camada 0** — controladores de domínio, IdP, PAM, autoridade certificadora. Administra-se apenas a partir de estações dedicadas, sem e-mail e sem navegação.
- **Camada 1** — servidores e aplicações. Administradores próprios, que não têm privilégio na camada 0.
- **Camada 2** — estações de trabalho e suporte ao usuário.

A regra: credencial de uma camada nunca é usada em máquina de camada inferior. Administrador de domínio que abre sessão RDP numa estação de usuário para resolver um chamado deixa material de autenticação naquela máquina — e um atacante já presente ali o coleta. É assim que a maioria dos comprometimentos de domínio efetivamente acontece.

26.4. Métricas que revelam a saúde do programa

Tabela 26.1.: Métricas de saúde de um programa de IAM e PAM.

Métrica	Alvo	O que revela quando ruim
Tempo de desligamento (RH → acesso cortado)	< 1 hora	Janela de risco pós-demissão
Contas ativas sem uso há 90 dias	~ 0	Órfãs, ex-funcionários, contas esquecidas
Contas privilegiadas permanentes	Tendendo a zero	Ausência de elevação temporária
Taxa de revogação na recertificação	> 5%	Abaixo disso, a revisão é carimbo

Métrica	Alvo	O que revela quando ruim
Contas de serviço sem rotação há 1 ano	~ 0	Superfície de Kerberoasting e credencial eterna
Permissões concedidas e nunca usadas	$< 10\%$	Excesso de privilégio estrutural
Acessos com prazo de validade	Crescente	Maturidade do modelo de concessão

A segunda linha costuma ser a mais reveladora numa primeira medição. Contas ativas sem uso são, quase sempre, uma mistura de ex-funcionários que escaparam do processo, contas de serviço abandonadas e acessos criados para projetos que terminaram.

26.5. Brasil em Foco

Acesso de ex-funcionário ativo é falha de medida de segurança sob o art. 46 da LGPD. A obrigação de adotar medidas aptas a proteger dados pessoais de acessos não autorizados abrange o controle de ciclo de vida das contas. Numa apuração, a ANPD examina se havia processo de desligamento com gatilho automático, qual o tempo real entre o término e o corte, e se há registro dessa transição. Conta ativa meses depois do desligamento é achado que compromete a defesa de adequação, independentemente de ter havido uso indevido (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

A segregação de funções e o controle de privilégio são requisitos auditados no setor regulado. A Resolução BCB 4.893/2021 exige política de segurança cibernética com controles de acesso proporcionais ao risco; o PCI-DSS trata explicitamente de restrição por necessidade de saber (requisito 7), identificação individual (requisito 8) e rastreabilidade de acesso a dado de conta (requisito 10). Conta administrativa compartilhada é não conformidade objetiva em ambos — não porque a senha seja fraca, mas porque destrói a atribuição individual de responsabilidade (Banco Central do Brasil, 2021; PCI Security Standards Council, 2024).

A trilha de auditoria precisa existir antes do incidente para servir de prova. Registro de quem acessou o quê, quando e sob qual justificativa é o insumo tanto da apuração administrativa quanto de eventual ação judicial. Para provedores, o Marco Civil impõe guarda de registros com prazos definidos; para tratamento de dado pessoal, a demonstração de conformidade sob o art. 6º, X, depende de evidência. Log de acesso privilegiado que o próprio administrador pode apagar não sustenta nenhuma das duas funções — daí a exigência de destino de log fora do alcance de quem é auditado (Brasil, 2014, 2018).

Uso de credencial legítima fora da finalidade tem consequência jurídica. A Lei 12.737/2012, com o agravamento da Lei 14.155/2021, alcança o acesso não autorizado a dispositivo informático; e o funcionário que usa acesso legítimo fora do escopo autorizado responde nas esferas trabalhista e civil, além de expor a organização perante o titular do dado. Do ponto de vista operacional, o que separa uma apuração conclusiva de uma inconclusiva é a existência de gravação de sessão privilegiada e de log de decisão de autorização (Brasil, 2012, 2021).

Terceirização não transfere responsabilidade. Provedores brasileiros operam com técnicos terceirizados, integradores e fornecedores com acesso remoto à infraestrutura. Sob a LGPD, o controlador responde pelo tratamento realizado por seus operadores (arts. 39 e 42), e contrato de prestação de serviço não afasta essa responsabilidade. Na prática isso exige, para todo acesso de terceiro: identidade nominal (nunca conta compartilhada do fornecedor), acesso intermediado

e gravado, prazo de validade vinculado ao contrato e revogação automática no término (Brasil, 2018).

26.6. No Campo

1. Comece pelo relatório de contas órfãs — é o diagnóstico mais rápido. Cruze o diretório com a base ativa do RH:

```
# Contas habilitadas sem autenticacao ha mais de 90 dias
$corde = (Get-Date).AddDays(-90)
Get-ADUser -Filter { Enabled -eq $true } -Properties LastLogonDate, Title,
↪ Department |
  Where-Object { $_.LastLogonDate -lt $corde -or -not $_.LastLogonDate } |
  Select-Object Name, SamAccountName, LastLogonDate, Department |
  Sort-Object LastLogonDate |
  Export-Csv .\contas-inativas.csv -NoTypeInfoation -Encoding utf8

# Cruzamento com o RH: quem esta ativo no AD e desligado na folha
$rh = Import-Csv .\ativos-rh.csv # matricula, cpf, situacao
$ad = Get-ADUser -Filter { Enabled -eq $true } -Properties EmployeeID
$ad | Where-Object { $_.EmployeeID -notin $rh.matricula } |
  Select-Object Name, SamAccountName, EmployeeID
```

Toda linha do segundo comando é uma conta ativa de alguém que já saiu. Esse relatório costuma abrir o orçamento do projeto de IAM sozinho.

2. Automatize o desligamento com gatilho no RH. O fluxo mínimo, disparado pelo evento de término:

```
def desligar(matricula: str) -> None:
    ident = identidade_por_matricula(matricula)

    diretorio.desativar(ident) # desativa, nao apaga
    idp.encerrar_sesoes(ident) # sesoes federadas ativas
    idp.revogar_refresh_tokens(ident) # o que sobrevive a expiracao
    for chave in api.chaves_de(ident):
        api.revogar(chave)
    ssh.removever_chaves_publicas(ident) # authorized_keys em todo o
↪ parque
    email.removever_regras_encaminhamento(ident) # exfiltracao silenciosa
    email.converter_em_compartilhada(ident, destino=gestor_de(ident))

    # Credenciais compartilhadas que a pessoa conhecia: ROTACIONAR
    for segredo in cofre.segredos_acessados_por(ident, dias=180):
        cofre.agendar_rotacao(segredo, prioridade="alta")

    auditoria.registrar("desligamento", ident, origem="rh")
```

A rotação dos segredos compartilhados é o passo mais esquecido, e é o único que trata do que a pessoa **sabe** — o resto trata do que ela **tem**.

3. Elimine o privilégio permanente antes de comprar ferramenta. A elevação temporária existe nativamente em várias plataformas, sem projeto de PAM:

```
# Linux: sudo com escopo estreito, em vez de ALL:ALL
# /etc/sudoers.d/operacao
%operacao ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx, \
                                /usr/bin/systemctl status nginx
# Registro completo do que foi executado
Defaults log_input, log_output, iolog_dir=/var/log/sudo-io
```

```
# Entra ID PIM: quem tem papel PERMANENTE (o que se quer zerar)
Get-MgRoleManagementDirectoryRoleAssignment -All |
    Select-Object PrincipalId, RoleDefinitionId, DirectoryScopeId
```

4. Reduza Domain Admins a um número que caiba na mão. A pergunta a fazer para cada membro: esta pessoa precisa disso **agora**, ou precisou uma vez em 2019? A maioria dos administradores de servidor não precisa de privilégio de domínio — precisa de privilégio local nos servidores que administra, que é coisa diferente. Contas de serviço nesse grupo são achado crítico com correção obrigatória.

5. Endureça as contas de serviço contra Kerberoasting. Inventarie SPNs, meça o comprimento e a idade das senhas, migre para contas gerenciadas por grupo onde a aplicação suportar, e remova do grupo administrativo o que não precisar estar lá:

```
# Todas as contas com SPN - o universo exposto ao Kerberoasting
Get-ADUser -Filter { ServicePrincipalName -like '*' } `
    -Properties ServicePrincipalName, PasswordLastSet, MemberOf |
    Select-Object Name, PasswordLastSet,
        @{n='Admin';e={ [bool]($_.MemberOf -match 'Domain Admins|Enterprise
↵ Admins') }},
        @{n='SPNs';e={ $_.ServicePrincipalName -join '; ' }}
```

6. Reformule a recertificação para que ela revogue. Padrão de exclusão por inação, data da última utilização visível na tela, escopo por risco, e uma métrica publicada: taxa de revogação por campanha. Se ela ficar abaixo de 5%, o problema não são os acessos — é a campanha.

7. Coloque prazo em toda concessão fora do básico. Acesso a projeto expira com o projeto; acesso de fornecedor expira com o contrato; acesso emergencial expira em horas. O sistema deve exigir uma data no formulário, sem opção de “permanente” para quem não for aprovador de exceção.

8. Proteja a conta de emergência e teste-a. Duas contas de vidro quebrado, com senha longa dividida entre dois responsáveis e guardada em cofre físico, excluídas das políticas de acesso condicional (para não travarem junto com o IdP), com alerta imediato em qualquer uso e teste semestral documentado. Sem elas, uma falha do provedor de identidade tranca todo mundo do lado de fora.

9. Envie o log privilegiado para fora do alcance do privilegiado. Destino de log em conta ou domínio separado, com escrita permitida e exclusão negada para os administradores auditados. Log que o administrador pode apagar não é evidência — é conveniência.

10. Trate acesso de terceiro como categoria própria. Identidade nominal por técnico (nunca conta genérica do fornecedor), acesso intermediado e gravado, janela de tempo aprovada, e revogação automática vinculada à vigência do contrato. É o caminho mais comum de comprometimento por cadeia de suprimentos, tema do Volume 16.

26.7. Resumo

- **O ciclo de vida da identidade tem três eventos** — admissão, mudança e desligamento — e a fonte da verdade é o RH. A fase de **mudança** é onde o acúmulo de privilégios nasce, porque o processo concede o novo e não remove o antigo.
- **Clonagem de acesso** (“dê o mesmo que o Bruno”) propaga o excesso acumulado de uma pessoa para toda a organização. Use acesso por nascimento derivado do cargo.
- **Desligar não é desativar a conta principal:** tokens, chaves SSH, sessões federadas, contas locais em appliances, regras de encaminhamento de e-mail e — sobretudo — as **credenciais compartilhadas que a pessoa conhecia**, que precisam ser rotacionadas.
- **Recertificação só vale se revogar.** Padrão de exclusão por inação, última utilização visível na tela, escopo por risco. Taxa de revogação abaixo de 5% indica carimbo, não revisão.
- **Prazo de validade em toda concessão** é a mudança de maior efeito: acesso com data desaparece sozinho; acesso permanente depende de alguém lembrar.
- **PAM tem quatro capacidades** — cofre, intermediação de sessão, elevação temporária e gravação. A elevação temporária é a que muda a matemática do ataque, porque a credencial roubada normalmente não carrega privilégio no momento do roubo.
- **Contas de serviço são o problema mais persistente:** sem dono, sem rotação, com excesso de privilégio. O Kerberoasting transforma uma senha fraca de conta de serviço em comprometimento de domínio, e a defesa é senha longa aleatória, conta gerenciada por grupo e remoção dos grupos administrativos.
- O **modelo em camadas** impede que credencial de alta camada apareça em máquina de baixa confiança — é o controle que quebra o caminho mais comum de comprometimento de domínio.
- No Brasil, conta de ex-funcionário ativa é falha sob o art. 46 da LGPD; conta administrativa compartilhada é não conformidade objetiva em PCI-DSS e nas normas do BCB; e a responsabilidade pelo acesso de terceirizados permanece com o controlador.

27. Arquitetura Zero Trust

Em dezembro de 2009, o Google detectou uma intrusão em sua rede corporativa. A operação, que ficaria conhecida como Aurora, começou com um funcionário abrindo um link, seguiu com a exploração de uma falha do navegador e terminou com o atacante circulando pela rede interna por meses. A investigação revelou o que investigações desse tipo sempre revelam: o perímetro tinha sido atravessado numa única máquina, e a partir dali quase tudo estava acessível, porque quase tudo confiava na rede interna.

A resposta do Google não foi comprar um firewall melhor. Foi uma decisão de arquitetura que levou anos para ser implementada e que se chamou **BeyondCorp**: remover completamente o privilégio associado à localização de rede. Estar dentro do escritório deixou de significar coisa alguma. Todo acesso, de qualquer lugar, passou a exigir prova de identidade e prova de que o dispositivo estava em condições aceitáveis (Ward; Beyer, 2014).

Essa é a ideia inteira do Zero Trust, e ela cabe numa frase: **a rede não é uma credencial**.

Este capítulo fecha o Volume 4 porque é onde tudo o que ele construiu se encontra. Modelos de decisão (Capítulo 22), autenticação de fatores (Capítulo 24), federação (Capítulo 25) e ciclo de vida de identidade (Capítulo 26) são as peças; Zero Trust é a arquitetura que as organiza.

27.1. O que o modelo de perímetro pressupunha

A arquitetura tradicional é fácil de descrever: um firewall separa o “dentro” do “fora”. Quem está dentro é confiável. Acesso remoto é resolvido com VPN, que **coloca você dentro**.

Essa arquitetura funcionou por décadas porque suas premissas eram verdadeiras: os funcionários trabalhavam no escritório, os servidores ficavam no data center da empresa, os dispositivos eram corporativos e o “fora” era genuinamente separado do “dentro”.

Nenhuma dessas premissas sobreviveu. Trabalho remoto e híbrido é a norma; aplicações estão em SaaS e em nuvem pública; dispositivos pessoais acessam recursos corporativos; fornecedores e integradores precisam de acesso; e os dados que importam frequentemente nunca estiveram no data center.

O problema não é apenas que o perímetro encolheu. É que ele **nunca foi uma boa defesa contra o que efetivamente acontece**: o phishing entrega ao atacante uma posição dentro, e a partir dela o modelo não oferece resistência nenhuma. A metáfora conhecida — casca dura e miolo mole — descreve exatamente o incidente Aurora e a maioria dos que vieram depois.

27.2. O que Zero Trust é, e o que não é

O NIST SP 800-207 dá a definição de referência e — importante — deixa claro que se trata de um **conjunto de princípios de arquitetura**, não de um produto (National Institute of Standards and Technology, 2020b).

Os princípios centrais:

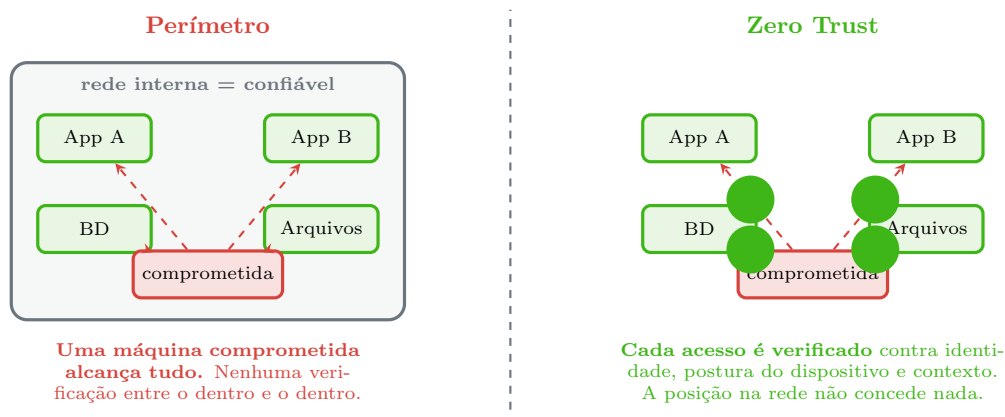


Figura 27.1.: A diferença essencial: no perímetro, o acesso interno não é verificado; no Zero Trust, todo acesso é.

1. **Toda fonte de dados e serviço é um recurso.** Não há categoria “interno” que dispense controle.
2. **Toda comunicação é protegida, independentemente da localização.** Cifragem e autenticação valem igualmente dentro do data center.
3. **O acesso é concedido por sessão.** Autenticar uma vez não concede acesso permanente a tudo; cada sessão é avaliada.
4. **A decisão é dinâmica e baseada em política.** Identidade, postura do dispositivo, sensibilidade do recurso, comportamento observado e outros sinais entram na conta.
5. **A organização monitora a integridade e a postura de todos os ativos.** Nenhum dispositivo é confiável por natureza.
6. **Autenticação e autorização são dinâmicas e estritamente aplicadas antes de cada acesso.**
7. **A organização coleta informação sobre o estado atual e usa isso para melhorar a postura.**

E a lista do que Zero Trust **não** é merece o mesmo espaço, porque o termo foi apropriado pelo marketing a ponto de perder sentido:

- **Não é um produto.** Nenhuma caixa “implementa Zero Trust”. Fornecedores vendem componentes que ajudam.
- **Não é “confiar em nada”** — o nome é infeliz. É *não conceder confiança implícita por localização*; confiança explícita, verificada e revalidada continua existindo.
- **Não é o fim do firewall.** Segmentação continua valendo; ela deixa de ser a defesa principal.
- **Não é substituir a VPN por um portal** e declarar vitória, embora seja assim que a maioria dos projetos começa e termina.
- **Não é um projeto com data de conclusão.** É uma mudança arquitetural incremental, medida em anos.

27.3. Os componentes

A arquitetura do NIST descreve um **plano de controle** que decide e um **plano de dados** que aplica — a mesma separação PDP/PEP do Capítulo 22, agora em escala de infraestrutura.

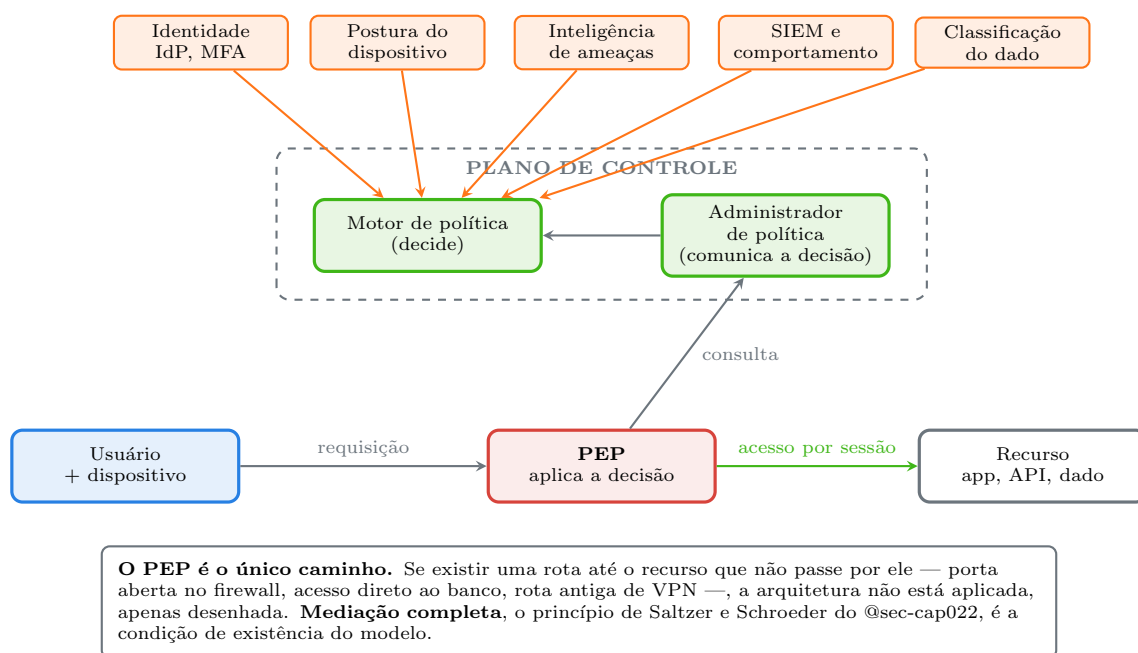


Figura 27.2.: Arquitetura de referência do NIST SP 800-207 e os sinais que alimentam a decisão.

O sinal que distingue Zero Trust de “SSO com MFA” é a **postura do dispositivo**. Verificar quem é a pessoa é metade; a outra metade é verificar de onde ela está agindo. Os atributos que costumam entrar:

- dispositivo registrado e gerenciado pela organização;
- disco cifrado, firewall local ativo;
- sistema operacional dentro da janela de atualização aceita;
- EDR instalado, ativo e sem detecção pendente;
- ausência de indicadores de comprometimento.

E o resultado da avaliação não precisa ser binário. Acesso pode ser concedido em modo restrito — somente leitura, sem download, com sessão mais curta — quando a postura for parcial. Essa gradação é o que torna a política aplicável sem parar o trabalho.

27.4. Os pilares e um caminho de implantação

O modelo de maturidade da CISA organiza o trabalho em cinco pilares, sobre três capacidades transversais — visibilidade, automação e governança (Cybersecurity and Infrastructure Security Agency, 2023):

Tabela 27.1.: Os cinco pilares do modelo de maturidade da CISA e onde este livro os desenvolve.

Pilar	Pergunta central	Base construída em
Identidade	Quem está pedindo?	Capítulo 24, Capítulo 25, Capítulo 26
Dispositivo	De onde, e em que estado?	Volume 6
Rede	O tráfego está segmentado e cifrado?	Volume 5

Pilar	Pergunta central	Base construída em
Aplicação e carga de trabalho	O acesso é mediado e autorizado por sessão?	Volume 7
Dado	O que é sensível, e onde está?	Volume 3, Volume 15

A pergunta prática é por onde começar. A resposta que a experiência sustenta: **identidade primeiro**, porque nenhum outro pilar funciona sem ela — política de dispositivo sem identidade confiável não tem a quem se aplicar.

Uma sequência que produz redução de risco em cada etapa, em vez de só na última:

Etapla 1 — inventário e visibilidade. Quais aplicações existem, quem as acessa, por quais caminhos, e quais dados elas guardam. Não é possível mediar o que não se conhece, e essa etapa costuma revelar aplicações e caminhos de acesso que ninguém sabia que existiam.

Etapla 2 — consolidar identidade. Federar as aplicações num IdP único, MFA resistente a phishing para administradores, ciclo de vida automatizado. É o Volume 4 inteiro, aplicado.

Etapla 3 — postura de dispositivo. Gerenciamento, cifragem de disco, EDR, e o sinal de conformidade alimentando a decisão de acesso.

Etapla 4 — acesso a aplicação sem VPN de rede. Substituir o túnel que coloca a pessoa “dentro” por acesso mediado por aplicação (ZTNA): a pessoa alcança o sistema autorizado, não a sub-rede onde ele mora.

Etapla 5 — microssegmentação. Restringir a comunicação entre cargas de trabalho, começando pelos ativos mais críticos. É a etapa mais trabalhosa e a que mais reduz movimentação lateral.

Etapla 6 — política dinâmica e resposta automática. Escore de risco alimentando reavaliação contínua, com revogação de sessão ao detectar anomalia.

O erro mais comum de sequenciamento é começar pela etapa 5 — comprar segmentação e tentar aplicá-la sobre uma base de identidade frágil. Sem saber quem é quem, a política de rede vira lista de endereços IP, que é exatamente o modelo que se queria abandonar.

27.5. ZTNA: o que muda em relação à VPN

A VPN tradicional resolve conectividade: ela coloca o dispositivo remoto **na rede**. A consequência é que o acesso é concedido por sub-rede, o dispositivo comprometido varre tudo o que a rota alcança, e o túnel permanece aberto sem reavaliação.

O acesso de rede Zero Trust inverte a granularidade: a pessoa recebe acesso **a uma aplicação específica**, mediado por um intermediário, com identidade e postura verificadas a cada sessão, e com reavaliação contínua. As aplicações não ficam expostas na internet — elas são alcançadas por um conector que estabelece a conexão de dentro para fora.

Tabela 27.2.: VPN e ZTNA lado a lado.

	VPN tradicional	ZTNA
Unidade de acesso	Sub-rede	Aplicação
Verificação	Na conexão	Contínua, por sessão

	VPN tradicional	ZTNA
Dispositivo comprometido	Varre tudo que a rota alcança	Alcança o que foi explicitamente autorizado
Exposição da aplicação	Porta aberta ou rota interna	Conector de dentro para fora
Postura do dispositivo	Raramente considerada	Insumo da decisão

A ressalva honesta: ZTNA resolve o acesso do usuário à aplicação. Ele **não** resolve a comunicação entre cargas de trabalho — servidor falando com banco, serviço falando com serviço — que é onde a movimentação lateral acontece depois do comprometimento de um servidor. Essa parte é microsegmentação, e é a etapa que a maioria dos projetos adia indefinidamente.

27.6. Microsegmentação e identidade de carga de trabalho

Segmentar por IP e VLAN funciona até a infraestrutura virar dinâmica — contêineres com endereços efêmeros, escalonamento automático, múltiplas nuvens. O modelo que sobrevive a isso é dar **identidade criptográfica à carga de trabalho**, e não ao endereço.

É o que faz uma malha de serviços com mTLS: cada serviço tem um certificado de curta duração, a autenticação é mútua, e a política é escrita em termos de identidade — **serviço-pagamento pode chamar serviço-cobrança**, e não 10.2.3.4 pode falar com 10.2.9.7:5432. O material criptográfico é o do Capítulo 19 e do Capítulo 20, aplicado ao tráfego interno.

A abordagem prática para não travar a operação: começar em modo de observação, mapear os fluxos reais durante semanas, escrever a política a partir do que se observou, aplicar primeiro em modo de alerta e só então bloquear. Escrever política de segmentação a partir da documentação — que está desatualizada — é o caminho conhecido para uma interrupção em produção.

27.7. Os riscos que o modelo cria

Um capítulo honesto sobre Zero Trust precisa dizer o que ele custa.

A concentração no plano de controle. O motor de política e o IdP passam a decidir todo acesso da organização. Comprometê-los é comprometer tudo; derrubá-los é parar tudo. Isso exige alta disponibilidade real, proteção de nível máximo para seus administradores e um caminho de contingência testado (Capítulo 26).

Complexidade que vira exceção permanente. Política granular demais gera chamados, e chamados geram exceções. A exceção criada “temporariamente” para o time comercial em 2024 é a porta que continua aberta em 2027. Exceção precisa de prazo e de revisão, como qualquer concessão.

Custo e prazo subestimados. O BeyondCorp do Google levou anos e consumiu equipe dedicada. Projeto vendido como implantação de seis meses entrega, na prática, um portal de acesso — o que é uma melhora, mas não é a arquitetura.

Falso senso de conclusão. Trocar a VPN e declarar “somos Zero Trust” com o tráfego entre servidores ainda plano deixa intacta a movimentação lateral, que era o problema original do incidente Aurora.

Dependência da nuvem do fornecedor. Grande parte das soluções de ZTNA é entregue como serviço. Isso significa que a disponibilidade do acesso corporativo passa a depender de

um terceiro, e que o tráfego passa por ele — decisão que merece análise explícita de risco e de conformidade, não adoção por inércia.

27.8. Brasil em Foco

Zero Trust responde diretamente à exigência de medidas técnicas adequadas da LGPD. O art. 46 fala em medidas aptas a proteger dado pessoal de acessos não autorizados e situações acidentais ou ilícitas. Uma arquitetura em que cada acesso a dado pessoal é autenticado, autorizado e registrado produz exatamente o tipo de evidência que a ANPD examina — e, quando o incidente ocorre, permite delimitar com precisão o que foi acessado. Delimitar o alcance é o que separa uma comunicação específica de uma comunicação a toda a base de titulares (Autoridade Nacional de Proteção de Dados, 2024; Brasil, 2018).

O trabalho remoto tornou o modelo de perímetro incompatível com a realidade brasileira. A adoção acelerada do teletrabalho, com uso intenso de dispositivos pessoais, tornou comum o acesso a sistemas corporativos a partir de máquinas sobre as quais a organização não tem controle. Sob a LGPD, isso é tratamento de dado pessoal em ambiente não gerenciado, e a resposta técnica adequada é verificação de postura com acesso gradual — não a proibição formal em política que ninguém consegue fazer cumprir.

O setor financeiro caminha nessa direção por exigência regulatória. A Resolução BCB 4.893/2021 exige controles proporcionais ao risco, autenticação robusta e capacidade de detecção; o Open Finance opera com consentimento granular, escopo explícito e prazo — que é a materialização do “acesso por sessão” do princípio 3 do NIST. Instituições que implementaram os requisitos do arranjo já construíram boa parte dos pilares de identidade e de aplicação sem chamar isso de Zero Trust (Banco Central do Brasil, 2021; Banco Central do Brasil; Conselho Monetário Nacional, 2020).

Para provedores de internet, a arquitetura tem efeito direto sobre a guarda de registros. O Marco Civil impõe guarda de registros de conexão e de acesso a aplicações, e o acesso às bases que contêm esses registros é ele próprio uma operação sensível, sujeita a ordem judicial. Mediar cada acesso a essas bases por um ponto de aplicação com registro é o que permite responder, com evidência, quem consultou o quê — inclusive internamente. Sem essa mediação, a resposta a uma requisição judicial sobre acesso indevido é uma estimativa (Brasil, 1996, 2014).

Órgãos públicos têm referências nacionais aplicáveis. O governo federal publica guias de segurança da informação e requisitos de privacidade para a administração pública, e a Estratégia Nacional de Segurança Cibernética estabelece diretrizes que convergem com os princípios aqui descritos. Para gestor público, a implantação começa igualmente pela identidade — com a integração ao gov.br e seus níveis de confiabilidade como base do pilar (Capítulo 25) (CERT.br, 2024; Secretaria de Governo Digital, Ministério da Gestão e da Inovação, 2024).

27.9. No Campo

1. Meça sua maturidade antes de comprar qualquer coisa. Percorra os cinco pilares e responda com evidência, não com impressão:

IDENTIDADE

[] IdP único federando as aplicações principais

- [] MFA resistente a phishing para administradores
- [] Ciclo de vida automatizado com gatilho no RH
- [] Privilégio administrativo permanente tendendo a zero

DISPOSITIVO

- [] Inventário completo do que acessa recursos corporativos
- [] Disco cifrado e EDR ativo, com medição de cobertura
- [] Sinal de conformidade alimentando a decisão de acesso

REDE

- [] Segmentação além de "interno x externo"
- [] Tráfego entre servidores cifrado e autenticado
- [] Acesso remoto por aplicação, não por sub-rede

APLICAÇÃO

- [] Autenticação centralizada, sem base de senha local
- [] Autorização verificada na API, não só na interface
- [] Log de decisão de autorização, não só de autenticação

DADO

- [] Classificação aplicada e conhecida
- [] Dado sensível identificado e localizado
- [] Acesso a dado sensível registrado individualmente

Cada quadrado vazio é uma etapa do plano. Essa lista vale mais que qualquer avaliação de fornecedor, porque ela é sobre a sua realidade.

2. Descubra os caminhos que contornam o ponto de aplicação. É a verificação que decide se a arquitetura existe ou está apenas desenhada:

```
# Servicos expostos que nao passam pelo PEP
nmap -sS -p- --open -T4 10.0.0.0/24 -oG - | grep '/open/'

# Regras de firewall permissivas herdadas de projetos antigos
iptables -S | grep -E 'ACCEPT.*(0\.0\.0\.0/0|ANY)'

# Bancos aceitando conexao direta de fora do host da aplicacao
ss -tlnp | grep -E ':(3306|5432|1433|27017|6379)\b'
```

```
# Regras de firewall do dominio com escopo amplo demais
Get-NetFirewallRule -Enabled True -Direction Inbound |
  Get-NetFirewallAddressFilter |
  Where-Object { $_.RemoteAddress -eq 'Any' }
```

Rota antiga de VPN, banco acessível diretamente, porta aberta “para um teste” em 2022 — cada uma anula a mediação completa naquele caminho.

3. Comece pela identidade, e comece pelos administradores. Chaves FIDO2 para quem tem acesso privilegiado, elevação temporária substituindo privilégio permanente, e MFA obrigatório sem exceções — com atenção especial ao desligamento dos protocolos legados do Capítulo 24, que criam um caminho só com senha.

4. Construa o sinal de postura antes de escrever política que o use. Sem inventário e sem gerenciamento de dispositivo, a política de conformidade não tem insumo. A ordem é: inventariar, gerenciar, medir cobertura de cifragem e EDR, e só então usar o sinal na decisão de acesso — em modo de alerta antes de bloquear.

5. Substitua a VPN por aplicação, uma de cada vez. Escolha uma aplicação interna de uso amplo e baixo risco, publique-a por acesso mediado, meça a experiência real dos usuários, ajuste, e siga. A migração de tudo de uma vez transforma um projeto de segurança num incidente de disponibilidade.

6. Faça microssegmentação em modo de observação primeiro. Semanas coletando os fluxos reais entre cargas de trabalho, política escrita a partir do observado, aplicação em modo de alerta, e bloqueio apenas depois de um ciclo completo sem falso positivo. Comece pelos ativos mais críticos — controladores de domínio, bancos com dado pessoal, sistemas financeiros.

7. Trate exceção como dívida, com prazo e responsável. Registre cada exceção com justificativa, dono, data de expiração e revisão agendada. Publique a contagem de exceções abertas como métrica do programa: ela cresce silenciosamente e é o principal caminho de erosão do modelo.

8. Garanta a disponibilidade do plano de controle. IdP e motor de política em alta disponibilidade real, com teste de falha executado, e caminho de contingência para administradores em cofre físico (Capítulo 26). Uma arquitetura que verifica tudo depende inteiramente de o verificador estar disponível.

9. Meça resultado, não adoção de ferramenta. As métricas que dizem se o modelo está funcionando: percentual de aplicações atrás do ponto de aplicação, percentual de acessos com postura de dispositivo verificada, tempo entre desligamento e corte efetivo, número de caminhos que contornam o PEP (alvo: zero) e alcance de movimentação lateral medido em exercício de time vermelho. “Número de licenças implantadas” não é métrica de segurança.

10. Aceite que é um caminho longo, e mostre valor a cada etapa. MFA resistente a phishing para administradores reduz risco no primeiro mês. Desligamento automatizado reduz no segundo. Segmentação dos ativos críticos reduz no sexto. Um programa que só entrega valor no final não sobrevive à primeira revisão de orçamento — e, nesse caso, a organização fica com o custo sem a arquitetura.

27.10. Resumo

- **Zero Trust é uma frase: a rede não é uma credencial.** Nasceu da resposta do Google ao incidente Aurora, em que uma máquina comprometida deu acesso a quase tudo porque quase tudo confiava na rede interna.
- **É um conjunto de princípios de arquitetura, não um produto.** Não é “confiar em nada”, não é o fim do firewall, não é trocar a VPN por um portal, e não é um projeto com data de conclusão.
- Os princípios do NIST SP 800-207: todo serviço é recurso, toda comunicação é protegida, **acesso por sessão**, decisão dinâmica baseada em política, monitoramento contínuo da postura dos ativos e aplicação estrita antes de cada acesso.
- A arquitetura separa **plano de controle** (motor e administrador de política) de **plano de dados** (ponto de aplicação) — e o **PEP precisa ser o único caminho**. Qualquer rota que o contorne anula o modelo naquele caminho.

- **A postura do dispositivo é o que distingue Zero Trust de “SSO com MFA”**, e a decisão não precisa ser binária: acesso restrito para postura parcial mantém a política aplicável.
- A sequência que funciona é **identidade primeiro**, depois dispositivo, depois acesso por aplicação, depois microssegmentação. Começar pela segmentação sobre identidade frágil reproduz o modelo baseado em endereço IP que se queria abandonar.
- **ZTNA troca acesso por sub-rede por acesso por aplicação**, com verificação contínua — mas não resolve a comunicação entre cargas de trabalho, que exige microssegmentação com identidade criptográfica (mTLS), sempre implantada em modo de observação antes de bloquear.
- Os riscos reais do modelo: **concentração no plano de controle**, exceções que viram permanentes, custo e prazo subestimados, falso senso de conclusão e dependência da nuvem do fornecedor.
- No Brasil, a arquitetura responde diretamente ao art. 46 da LGPD e melhora a delimitação de escopo de incidente; o setor financeiro já construiu parte dos pilares por exigência do BCB e do Open Finance; e para provedores ela é o que permite responder, com evidência, quem consultou registros protegidos pelo Marco Civil.

Parte V.

Volume 5 — Segurança de Redes

28. Arquitetura segura e segmentação de redes

(em elaboracao)

29. Firewalls: tipos, políticas e boas práticas

(em elaboracao)

30. VPNs: IPsec, WireGuard e acesso remoto seguro

(em elaboracao)

31. IDS, IPS e detecção em rede

(em elaboracao)

32. Segurança de DNS: ataques, DNSSEC e DoH

(em elaboracao)

33. Segurança do roteamento e BGP: sequestros, vazamentos e RPKI

(em elaboracao)

34. Segurança em redes sem fio e enlaces de rádio

(em elaboracao)

35. Ataques de negação de serviço e estratégias de mitigação

(em elaboracao)

Parte VI.

**Volume 6 — Endpoints e Sistemas
Operacionais**

36. Hardening de servidores Linux

(em elaboracao)

37. Hardening de Windows e Active Directory

(em elaboracao)

38. Antivírus, EDR e XDR

(em elaboracao)

39. Gestão de vulnerabilidades e ciclo de patches

(em elaboracao)

40. Segurança de dispositivos móveis e BYOD

(em elaboracao)

41. Criptografia de disco e proteção de dados em repouso

(em elaboracao)

Parte VII.

Volume 7 — Segurança de Aplicações

42. Ciclo de desenvolvimento seguro e DevSecOps

(em elaboracao)

43. OWASP Top 10: panorama e como usar

(em elaboracao)

44. Injeção: SQL, NoSQL e comandos de sistema

(em elaboracao)

45. Cross-site scripting e ataques no lado do cliente

(em elaboracao)

46. Falhas de autenticação e gestão de sessão

(em elaboracao)

47. Falhas de controle de acesso: IDOR, SSRF e escalonamento

(em elaboracao)

48. Segurança de APIs REST e GraphQL

(em elaboracao)

49. SAST, DAST, SCA e automação de testes de segurança

(em elaboracao)

Parte VIII.

Volume 8 — Nuvem e Contêineres

50. Modelo de responsabilidade compartilhada

(em elaboracao)

51. IAM em nuvem e os erros de configuração mais caros

(em elaboracao)

52. Segurança de contêineres e de imagens

(em elaboracao)

53. Segurança em Kubernetes

(em elaboracao)

54. Infraestrutura como código e segurança de pipeline

(em elaboracao)

55. Segredos, cofres e gestão de chaves

(em elaboracao)

Parte IX.

Volume 9 — Testes de Invasão

56. Metodologias e definição de escopo: PTES, OSSTMM e NIST SP 800-115

(em elaboracao)

57. Reconhecimento passivo e OSINT

(em elaboracao)

58. Varredura e enumeração

(em elaboracao)

59. Análise de vulnerabilidades

(em elaboracao)

60. Exploração: conceitos e anatomia de uma falha

(em elaboracao)

61. Pós-exploração, persistência e movimentação lateral

(em elaboracao)

62. Teste de invasão em aplicações web na prática

(em elaboracao)

63. Relatório técnico e comunicação de achados

(em elaboracao)

Parte X.

Volume 10 — Fator Humano

64. Engenharia social: os princípios psicológicos da persuasão

(em elaboracao)

65. Phishing, spear phishing e fraude do e-mail corporativo

(em elaboracao)

66. Fraudes digitais no Brasil: Pix, clonagem e golpes por mensageria

(em elaboracao)

67. Segurança física e ataques presenciais

(em elaboracao)

68. Cultura de segurança e programas de conscientização

(em elaboracao)

Parte XI.

**Volume 11 — Malware e Inteligência de
Ameaças**

69. Taxonomia de malware

(em elaboracao)

70. Anatomia de um ataque de ransomware

(em elaboracao)

71. Análise estática em laboratório

(em elaboracao)

72. Análise dinâmica e sandboxing

(em elaboracao)

73. MITRE ATT&CK e o mapeamento de TTPs

(em elaboracao)

74. Inteligência de ameaças e indicadores de comprometimento

(em elaboracao)

Parte XII.

**Volume 12 — Blue Team, SOC e
Detecção**

75. Estrutura e operação de um SOC

(em elaboracao)

76. Logging: o que registrar, por quanto tempo e por quê

(em elaboracao)

77. SIEM: arquitetura, ingestão e correlação

(em elaboracao)

78. Escrita de regras de detecção com Sigma e YARA

(em elaboracao)

79. Threat hunting: caçando o que a regra não pegou

(em elaboracao)

80. Deception: honeypots, honeytokens e canários

(em elaboracao)

81. Métricas e maturidade da detecção

(em elaboracao)

Parte XIII.

**Volume 13 — Resposta a Incidentes e
Forense Digital**

82. O ciclo de resposta a incidentes

(em elaboracao)

83. Preparação: plano, papéis e playbooks

(em elaboracao)

84. Contenção, erradicação e recuperação

(em elaboracao)

85. Fundamentos de forense digital e cadeia de custódia

(em elaboracao)

86. Forense de disco e sistemas de arquivos

(em elaboracao)

87. Forense de memória

(em elaboracao)

88. Forense de rede e análise de tráfego

(em elaboracao)

Parte XIV.

Volume 14 — Continuidade e Resiliência

89. Backup: estratégias e a regra 3-2-1-1-0

(em elaboracao)

90. Recuperação de desastres e continuidade de negócio

(em elaboracao)

91. Testes de continuidade e exercícios de mesa

(em elaboracao)

92. Alta disponibilidade e resiliência de infraestrutura

(em elaboracao)

93. Recuperação após um ataque de ransomware

(em elaboracao)

Parte XV.

**Volume 15 — Governança, Risco e
Conformidade**

94. Governança de segurança e o papel do CISO

(em elaboracao)

95. ISO/IEC 27001 e 27002

(em elaboracao)

96. NIST Cybersecurity Framework e CIS Controls

(em elaboracao)

97. LGPD na prática: bases legais, incidentes e a ANPD

(em elaboracao)

98. Marco Civil, Lei 12.737 e o direito penal digital brasileiro

(em elaboracao)

99. Obrigações regulatórias de provedores de internet

(em elaboracao)

100. PCI-DSS e requisitos setoriais

(em elaboracao)

101. Auditoria, conformidade e gestão de riscos de terceiros

(em elaboracao)

Parte XVI.

Volume 16 — Fronteira e Carreira

102. Segurança de sistemas de IA e modelos de linguagem

(em elaboracao)

103. Criptografia pós-quântica

(em elaboracao)

104. Segurança de OT, IoT e sistemas embarcados

(em elaboracao)

105. Segurança da cadeia de suprimentos de software

(em elaboracao)

106. Privacidade, anonimato e vigilância

(em elaboracao)

107. Carreira, certificações e aprendizado contínuo

(em elaboracao)

Referências

AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES. Resolução n. 717, de 23 de dezembro de 2019. Regulamento de Qualidade dos Serviços de Telecomunicações — RQUAL., 2019.

AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES. Resolução n. 740, de 21 de dezembro de 2020. Regulamento de Segurança Cibernética Aplicada aos Produtos de Telecomunicações., 2020.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. Guia orientativo: segurança da informação para agentes de tratamento de pequeno porte., 2023.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. Resolução CD/ANPD n. 15, de 24 de abril de 2024. Aprova o Regulamento de Comunicação de Incidente de Segurança., 2024.

BANCO CENTRAL DO BRASIL. Resolução CMN n. 4.893, de 26 de fevereiro de 2021. Política de segurança cibernética das instituições financeiras., 2021.

BANCO CENTRAL DO BRASIL; CONSELHO MONETÁRIO NACIONAL. Resolução Conjunta n. 1/2020 — Open Banking (Open Finance) no Brasil., 2020.

BARNES, Richard *et al.* Automatic Certificate Management Environment (ACME)., 2019.

BELL, D. Elliott; LAPADULA, Leonard J. Secure Computer System: Unified Exposition and Multics Interpretation., 1976.

BIBA, Kenneth J. Integrity Considerations for Secure Computer Systems., 1977.

BIRYUKOV, Alex *et al.* Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications., 2021.

BLEICHENBACHER, Daniel., 1998.

BRASIL. Lei n. 9.296, de 24 de julho de 1996. Regulamenta o inciso XII do art. 5º da Constituição Federal., 1996.

BRASIL. Medida Provisória n. 2.200-2, de 24 de agosto de 2001. Institui a Infraestrutura de Chaves Públicas Brasileira — ICP-Brasil., 2001.

BRASIL. Lei n. 12.737, de 30 de novembro de 2012. Tipificação criminal de delitos informáticos., 2012.

BRASIL. Lei n. 12.965, de 23 de abril de 2014. Marco Civil da Internet., 2014.

BRASIL. Lei n. 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais., 2018.

BRASIL. Lei n. 14.063, de 23 de setembro de 2020. Assinaturas eletrônicas em interações com entes públicos., 2020.

BRASIL. Lei n. 14.155, de 27 de maio de 2021. Agrava as penas dos crimes de invasão de dispositivo informático, furto e estelionato cometidos de forma eletrônica., 2021.

BRASIL. Decreto n. 11.491, de 12 de abril de 2023. Promulga a Convenção sobre o Crime Cibernético (Convenção de Budapeste)., 2023.

CAMPBELL, Brian *et al.* OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens., 2020.

CENTER FOR INTERNET SECURITY. CIS Benchmarks., 2024.

CERT.BR. Práticas de Segurança para Administradores de Redes Internet., 2024.

COOPER, David *et al.* Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile., 2008.

CROCKER, Dave; HANSEN, Tony; KUCHERAWY, Murray. DomainKeys Identified Mail (DKIM) Signatures., 2011.

CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY. Zero Trust Maturity Model, Version 2.0., 2023.

DENNISS, William; BRADLEY, John. OAuth 2.0 for Native Apps., 2017.

DIERKS, Tim; RESCORLA, Eric. The Transport Layer Security (TLS) Protocol Version 1.2., 2008.

DIFFIE, Whitfield; HELLMAN, Martin E. New Directions in Cryptography. IEEE Transactions on Information Theory, v. 22, n. 6, p. 644–654, 1976.

FERGUSON, Paul; SENIE, Daniel. Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing., 2000.

FETT, Daniel *et al.* OAuth 2.0 Demonstrating Proof of Possession (DPoP)., 2023.

FIDO ALLIANCE. Passkeys: Multi-Device FIDO Credentials., 2024.

FIELDING, Roy T.; NOTTINGHAM, Mark; RESCHKE, Julian. HTTP Semantics., 2022.

FORUM OF INCIDENT RESPONSE AND SECURITY TEAMS. Common Vulnerability Scoring System version 4.0: specification document., 2023.

FREUND, Jack; JONES, Jack. Measuring and Managing Information Risk: A FAIR Approach. [S.l.]: Butterworth-Heinemann, 2014.

- FULLER, Vince; LI, Tony. **Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan.**, 2006.
- HALLAM-BAKER, Phillip; STRADLING, Rob; HOFFMAN-ANDREWS, Jacob. **DNS Certification Authority Authorization (CAA) Resource Record.**, 2019.
- HARDT, Dick. **The OAuth 2.0 Authorization Framework.**, 2012.
- HODGES, Jeff; JACKSON, Collin; BARTH, Adam. **HTTP Strict Transport Security (HSTS).**, 2012.
- HOFFMAN, Paul; MCMANUS, Patrick. **DNS Queries over HTTPS (DoH).**, 2018.
- HUBBARD, Douglas W.; SEIERSEN, Richard. **How to Measure Anything in Cybersecurity Risk.** [S.l.]: Wiley, 2016.
- INSTITUTO NACIONAL DE TECNOLOGIA DA INFORMAÇÃO. **Infraestrutura de Chaves Públicas Brasileira (ICP-Brasil): documentos principais.**, 2024.
- INTERNATIONAL COMMITTEE FOR INFORMATION TECHNOLOGY STANDARDS. **Information Technology — Role Based Access Control.**, 2012.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 27000: information technology — security techniques — information security management systems — overview and vocabulary.** GenebraISO, a2018.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 31000: risk management — guidelines.** GenebraISO, b2018.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 29147: information technology — security techniques — vulnerability disclosure.** GenebraISO, c2018.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 30111: information technology — security techniques — vulnerability handling processes.** GenebraISO, 2019.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 27005: information security, cybersecurity and privacy protection — guidance on managing information security risks.** GenebraISO, 2022.
- ISC2. **ISC2 Code of Ethics.**, 2024.
- IYENGAR, Jana; THOMSON, Martin. **QUIC: A UDP-Based Multiplexed and Secure Transport.**, 2021.
- JOSEFSSON, Simon; LIUSVAARA, Ilari. **Edwards-Curve Digital Signature Algorithm (EdDSA).**, 2017.
- KERCKHOFFS, Auguste. La cryptographie militaire. **Journal des sciences militaires**, v. 9, p. 5–38, 1883.

- KITTERMAN, Scott. **Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1.**, 2014.
- KRAWCZYK, Hugo; BELLARE, Mihir; CANETTI, Ran. **HMAC: Keyed-Hashing for Message Authentication.**, 1997.
- KRAWCZYK, Hugo; ERONEN, Pasi. **HMAC-based Extract-and-Expand Key Derivation Function (HKDF).**, 2010.
- KUCHERAWY, Murray; ZWICKY, Elizabeth. **Domain-based Message Authentication, Reporting, and Conformance (DMARC).**, 2015.
- KUROSE, James F.; ROSS, Keith W. **Redes de Computadores e a Internet: uma abordagem top-down.** 8. ed. [S.l.]: Pearson, 2021.
- LAMPSON, Butler W. Protection. **ACM SIGOPS Operating Systems Review**, v. 8, n. 1, p. 18–24, 1974.
- LANGLEY, Adam; HAMBURG, Mike; TURNER, Sean. **Elliptic Curves for Security.**, 2016.
- LAURIE, Ben; LANGLEY, Adam; KASPER, Emilia. **Certificate Transparency.**, 2013.
- LODDERSTEDT, Torsten *et al.* **Best Current Practice for OAuth 2.0 Security.**, 2025.
- M'RAIHI, David *et al.* **HOTP: An HMAC-Based One-Time Password Algorithm.**, 2005.
- M'RAIHI, David *et al.* **TOTP: Time-Based One-Time Password Algorithm.**, 2011.
- MITRE CORPORATION. **MITRE ATT&CK: adversarial tactics, techniques and common knowledge.**, 2024.
- MORIARTY, Kathleen *et al.*, 2016.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Advanced Encryption Standard (AES).**, 2001.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.**, 2007.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Guide to Security for Full Virtualization Technologies.**, 2011.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Guide for Conducting Risk Assessments.**, 2012.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Guide to Attribute Based Access Control (ABAC) Definition and Considerations.**, 2014.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.**, b2015.

- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Recommendation for Random Number Generation Using Deterministic Random Bit Generators.**, a2015.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Transitioning the Use of Cryptographic Algorithms and Key Lengths.**, 2019.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Zero Trust Architecture.**, b2020.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Recommendation for Key Management: Part 1 — General.**, a2020.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Digital Identity Guidelines: Authentication and Authenticator Management.**, a2024.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Digital Identity Guidelines: Identity Proofing and Enrollment.**, b2024.
- NIR, Yoav; LANGLEY, Adam. **ChaCha20 and Poly1305 for IETF Protocols.**, 2018.
- OASIS. **Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0.**, 2005.
- OPENID FOUNDATION. **OpenID Connect Core 1.0.**, 2014.
- PARKER, Donn B. **Fighting Computer Crime: A New Framework for Protecting Information.** [S.l.]: Wiley, 1998.
- PCI SECURITY STANDARDS COUNCIL. **Payment Card Industry Data Security Standard v4.0.1.**, 2024.
- REKHTER, Yakov *et al.* **Address Allocation for Private Internets.**, 1996.
- RESCORLA, Eric. **The Transport Layer Security (TLS) Protocol Version 1.3.**, 2018.
- RIVEST, Ronald L.; SHAMIR, Adi; ADLEMAN, Leonard. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems. **Communications of the ACM**, v. 21, n. 2, p. 120–126, 1978.
- SALTZER, Jerome H.; SCHROEDER, Michael D. The Protection of Information in Computer Systems. **Proceedings of the IEEE**, v. 63, n. 9, p. 1278–1308, 1975.
- SANDHU, Ravi S. *et al.* Role-Based Access Control Models. **IEEE Computer**, v. 29, n. 2, p. 38–47, 1996.
- SANTESSON, Stefan *et al.* **X.509 Internet Public Key Infrastructure Online Certificate Status Protocol — OCSP.**, 2013.
- SCHNEIER, Bruce. Attack Trees. **Dr. Dobbs's Journal**, v. 24, n. 12, p. 21–29, 1999.
- SCHNEIER, Bruce. **Secrets and Lies: Digital Security in a Networked World.** [S.l.]:

Referências

Wiley, 2000.

SECRETARIA DE GOVERNO DIGITAL, MINISTÉRIO DA GESTÃO E DA INOVAÇÃO. **Login Único gov.br: níveis de confiabilidade da conta (bronze, prata e ouro).**, 2024.

SHANNON, Claude E. Communication Theory of Secrecy Systems. **Bell System Technical Journal**, v. 28, n. 4, p. 656–715, 1949.

SHOSTACK, Adam. **Threat Modeling: Designing for Security**. *[S.l.]*: Wiley, 2014.

STEVENS, Marc *et al.* **The First Collision for Full SHA-1.**, 2017.

VAUDENAY, Serge. **Security Flaws Induced by CBC Padding: Applications to SSL and IPsec.**, 2002.

WARD, Rory; BEYER, Betsy. **BeyondCorp: A New Approach to Enterprise Security.**, 2014.